
The Traveling Thief Problem: Per-Paper Metadata Fact-Sheets

September 22, 2026

Companion reference for the SLR & Taxonomy: one landscape table per variant. Each paper is a fact-sheet — title bar (\cite + title), structured metadata rows, the abstract (reused from the reference table / narrative review, verified against the source PDF), and a critical appraisal (contribution, authors' claims, evidence base, stated limitations, strengths, weaknesses).**Synopsis.** This document consolidates **93**

primary TTP research studies (2013–2026) into per-paper fact-sheets, one table per problem variant, plus a dedicated fact-sheet for the single existing systematic review of the field ([Sarkar and Rajendran, 2024](#)), given in Table 1 below. Within each table the papers are ordered by year of publication (then alphabetically by citation key), and the entry numbers match the NN_YYYY prefixes of the corresponding source PDF files. The per-table breakdown is:

Table	Content	# papers
1	Existing systematic review of the TTP	1
2	Single-objective TTP (TTP1) and Packing While Travelling (PWT)	62
3	Bi-objective TTP (TTP2)	16
4	Multiple TTP (MTTP) and Dynamic TTP (DTTP)	4
5	Thief Orienteering Problem (ThOP)	8
6	TTP with Time Windows (TTPTW)	1
7	Chance-Constrained TTP (CCTTP)	2
Total	primary research studies	93

Table 1 — Existing Systematic Review of the TTP (1 paper)

[1] Sarkar and Rajendran (2024) — Travelling thief problem: a survey of recent variants, solution approaches and future directions			
Variant	Survey	Objective / #obj	— / —
Route	—	Timing	—
Agents	—	Capacity	—
Uncertainty	—	Dynamics	—
Value model	—	Speed / Cost	— / —
Algorithm	Survey	Family (L1/L2)	Survey / PRISMA-based systematic literature review
Benchmark	n/a (survey)	Size	—
Metric	n/a (narrative synthesis; no computational experiments)	Stat. test	n/a
Key result	68 articles reviewed through mid-2023; TTP1/TTP2 solution methods classified into exact/heuristic/metaheuristic/hyper-heuristic; research gaps and future directions identified, incl. proposing TTPTW, ThOPTW and MTTPTW as future variants.		
Competition	N	Code	n/a
Budget / Runs	— / —	Hardware	—
Abstract. The first systematic literature review of the Traveling Thief Problem: a PRISMA-based survey of 68 articles (identified from 113) covering TTP1, PWT, TTP2, MTTP, DTTP and the Thief Orienteering Problem, with re-stated problem definitions and mathematical models, a chronological reference table, a classification of TTP1/TTP2 solution methods into exact / heuristic / metaheuristic / hyper-heuristic, and a research-gap and future-directions analysis (which itself proposes TTPTW, ThOPTW and MTTPTW as future variants).			
Contribution	The first systematic literature review of the Traveling Thief Problem: a PRISMA-based survey of 68 articles (identified from 113) covering TTP1, PWT, TTP2, MTTP, DTTP and the Thief Orienteering Problem, with re-stated problem definitions and mathematical models, a chronological reference table, a classification of TTP1/TTP2 solution methods into exact / heuristic / metaheuristic / hyper-heuristic, and a research-gap and future-directions analysis (which itself proposes TTPTW, ThOPTW and MTTPTW as future variants).		
Authors' claims	It is the first SLR on the TTP; TTP and ThOP are increasingly relevant to pick-up-and-delivery, waste-management and recyclable-goods-collection logistics; the field's literature is growing rapidly and needs consolidation; several research gaps remain (RL for TTP, real-world validation, exact/metaheuristic ThOP methods, time-window variants).		
Evidence base	Narrative synthesis; no computational experiments and no re-implementation of any method. Databases: Scopus (primary), Google Scholar, Web of Science, ScienceDirect. Search strings: (travelling AND thief AND problem), (packing AND while AND travelling), (thief AND orienteering AND problem). 113 records identified, 68 analysed. Received August 2023, so the effective search cutoff is mid-2023.		

Stated limitations The authors note it is the first-ever SLR on the TTP so no prior review could be referenced, and frame the solution-method classification as covering TTP1 and TTP2 where enough papers exist.

Strengths. ✓ The first systematic (PRISMA) review of the TTP, filling a real consolidation gap • ✓ Broad variant coverage for its time (TTP1, PWT, TTP2, MTTP, DTTP, ThOP) with re-stated formal models • ✓ A useful chronological reference table and a clear exact/heuristic/metaheuristic/hyper-heuristic method classification for TTP1/TTP2 • ✓ Explicit research-gap and future-directions section that correctly anticipated the time-window variants and the reinforcement-learning direction • ✓ Grounds the problem in concrete logistics applications (waste collection, courier/postal pickup)

Weaknesses. ✗ Mid-2023 search cutoff: misses about two dozen subsequent papers, including 2023 state-of-the-art work (namazi2023solving CoCoP, nguyen2023simulated SAVI) and four ThOP papers, plus all of the TTPTW and CCTTP literature and the reinforcement-learning, multi-objective-RL and quantum-annealing paradigms • ✗ 68 papers vs the present corpus of over 90; TTPTW and CCTTP are entirely absent (proposed only as future work) • ✗ No unified, like-for-like cross-paper performance comparison (no RDI/hypervolume synthesis, no re-run of any method) • ✗ No formal taxonomy of the variant design space: variants are surveyed in a flat list rather than organised along the structural, objective, constraint and dynamic dimensions that separate them • ✗ The solution-method classification is confined to TTP1/TTP2 and does not cover MTTP, DTTP or the newer variants • ✗ Published in a logistics/operations journal outside the evolutionary-computation venues where most TTP work appears, and has had limited uptake (1 citing article at the time of writing)

Remark. *The single existing systematic review of the TTP; this fact-sheet compendium is positioned as its updated and extended successor.*

Table 2 — Literature Review of TTP1 and PWT (62 papers)

[1] Bonyadi et al. (2013) — <i>The travelling thief problem: The first step in the transition from theoretical problems to realistic problems</i>			
Variant	TTP1, TTP2	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	RLS/EA baseline (TTP intro)	Family (L1/L2)	Problem def./benchmark/analysis / Problem definition
Benchmark	—	Size	—
Metric	—	Stat. test	—
Competition	N	Code	Unknown
Budget / Runs	— / —	Hardware	—
Abstract. Bonyadi et al. (2013) introduced the Traveling Thief Problem (TTP) as a formal benchmark for studying interdependent multi-component optimization. Motivated by real-world systems such as mine supply chains in which routing, production scheduling, and delivery cannot be decoupled, the authors abstracted the interdependence into a combination of the Traveling Salesman Problem (TSP) and the 0-1 Knapsack Problem (KP). The key coupling mechanism is that the thief’s speed decreases non-linearly with the weight of the collected items, making the tour cost a function of the packing plan and vice versa. The paper formally defines the problem, introduces two families of test instances derived from existing TSP and KP benchmarks, and provides three baseline algorithms—a simple constructive heuristic, a random local search, and an evolutionary algorithm—that serve as foundational reference points for all subsequent work. The central insight is that solving each subproblem independently and then merging solutions does not yield high-quality integrated solutions, establishing the interdependence challenge as the central research question of the field.			
Contribution	Problem-definition paper (IEEE CEC 2013): introduces the TTP as a benchmark class capturing two features of real-world problems, combination and interdependence; defines TTP1 (renting-rate coupling) and TTP2 (time-decay coupling); gives an instance-generation procedure and public objective-evaluation code.		
Authors’ claims	Standard benchmarks omit interdependence; solving each sub-problem to optimality in isolation does not give the overall optimum; the two coupling mechanisms yield two distinct models TTP1 and TTP2.		
Evidence base	No computational study and no algorithm is benchmarked. The suboptimality-of-decomposition claim is supported only by a hand-worked n=4, m=5 example (optimal-in-isolation objective -30 vs true optimum 50). Instances only sketched; the widely used 9720-instance suite comes later from polyakovskiy2014comprehensive.		

Stated limitations Authors note it is unclear which approaches suit which interdependency type; future work is a general graph-based model for different interdependency types and constraints such as unplanned maintenance / breakdown probabilities.

Strengths. ✓ First formal definition of the TTP and clear articulation of the interdependence principle • ✓ Concrete counterexample showing decomposition is provably suboptimal • ✓ Public instance generator and objective-evaluation code • ✓ Separates two coupling mechanisms into TTP1 and TTP2

Weaknesses. ✗ No computational experiments and no algorithm proposed or benchmarked; only $n=4$ hand examples • ✗ Strong NP-hardness only asserted by composition, not proven (proof left to polyakovskiy2014comprehensive) • ✗ The TTP2 value-decay model here (Dr^T/C) differs from the formulation adopted by most later TTP2 work, a source of downstream inconsistency • ✗ Instance generator has many free parameters chosen by rule of thumb with no sensitivity analysis • ✗ Only two instances are sketched; no benchmark suite is actually delivered

[2] Bonyadi et al. (2014) — *Socially inspired algorithms for the travelling thief problem*

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	CoSolver, DH	Family (L1/L2)	Population-based metaheuristic / Cooperative (CoSolver)
Benchmark	—	Size	Cities: 3 to 33; Items: 10 to 146
Metric	% gap from exact solution, $gap=100(z^*-z)/z^*$ (exhaustive search baseline);	Stat. test	none
Baselines	Density Heuristic (DH);		
Key result	CoSolver avg gap 5.16 (std 11.76) vs. DH 13.70 (std 20.03); CoSolver advantage grows with item count, items/city ratio, and rent rate;		
Competition	N	Code	No
Budget / Runs	— / —	Hardware	—

Abstract. Bonyadi et al. (2014) introduced **CoSolver**, the first cooperative co-evolutionary framework for the TTP that explicitly models subproblem communication. The system consists of two specialized modules—one solving the TSP component and one solving the KP component—that exchange solution information at regular intervals. This is contrasted with a **Density Heuristic (DH)** baseline that processes the two components sequentially and ignores interdependency. Using self-generated instances, the study shows that CoSolver consistently outperforms DH on large instances, quantitatively demonstrating that frequent bidirectional communication between subproblem solvers is necessary for high-quality integrated solutions. The work established cooperative co-evolution as a principled approach to multi-component optimization and motivated numerous subsequent decomposition-based TTP solvers.

Contribution	CoSolver, a decomposition-and-negotiation framework: splits TTP into TSKP (TSP-with-knapsack) and KRP (knapsack-on-route), solves KRP exactly by dynamic programming and TSKP by Chained LK on a relaxed-profit instance, and iterates with node weight/profit adjustment until no improvement; plus DH, a sequential baseline, and a generic instance generator.
Authors' claims	CoSolver outperforms DH in most cases, especially when the rent rate is positive (components not decomposable), with the advantage growing with item count, items-per-city ratio and small city counts.
Evidence base	45 self-generated instances only, 3-33 cities, 10-146 items, 6 generation classes. Metric is percentage gap from the exact (exhaustive-search) optimum: CoSolver mean gap 5.16 (std 11.76) vs DH 13.70 (std 20.03), plus scatter plots. No statistical test; number of runs not reported despite randomised LK; no runtime or hardware reported; no results on the standard 9720-instance suite.
Stated limitations	Future work: other negotiation schemes, other decompositions, and replacing the exact sub-solvers with approximation algorithms; instances kept small because exhaustive search is needed for the reference optimum.

Strengths. ✓ First explicit cooperative negotiation decomposition for TTP with a principled TSKP + KRP split • ✓ Compares against the true optimum (exhaustive search) rather than a heuristic baseline • ✓ Analyses when decomposition helps (rent rate, ratio) rather than only reporting averages • ✓ Reusable generic instance generator with a controllable greedy-proof KP class

Weaknesses. ✗ Evaluated only on 45 tiny self-generated instances (≤ 33 cities); no large-scale test and nothing on the standard suite • ✗ Exact DP for the KRP sub-problem does not scale, and CoSolver's scalability is never demonstrated • ✗ No statistical test, no runtime/hardware, and no run count despite randomised LK • ✗ Very high variance (std 11.76 on a 5.16 mean) is left unanalysed • ✗ DH baseline is deliberately naive; no comparison with SH/RLS/EA or MATLS • ✗ Self-generated instances were not adopted by the community, limiting comparability

[3] Mei et al. (2014) — Improving Efficiency of Heuristics for the Large Scale Traveling Thief Problem

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting

Algorithm	MATLS	Family (L1/L2)	Population-based metaheuristic / Memetic algorithm
Benchmark	—	Size	Cities: 76; 100; 130; 159; 280; 574; 724; 1000; 1304; 1577; 2103; 3038; 4461; 7397; 11849; 13509; 14051; 15112; 18512; 33810 Items: 1x; 5x; 10x KP-Types: bsc; uco; usw
Metric	mean and std of objective value (10 runs implied);	Stat. test	none
Baselines	RLS, EA polyakovskiy2014comprehensive;		
Key result	MATLS positive on all instances while RLS/EA negative, e.g. brd14051-bsc MATLS $2.66!x!10^7$ vs. RLS $-5.50!x!10^7$, EA $-6.30!x!10^7$;		
Competition	N	Code	No
Budget / Runs	10min / —	Hardware	—

Abstract. [Mei et al. \(2014\)](#) proposed the **Memetic Algorithm with Two-Stage Local Search (MATLS)**, the first algorithm specifically engineered for large-scale TTP instances containing up to 33,810 cities. The key algorithmic challenge addressed is the prohibitive $O(n^2)$ complexity of standard TSP neighborhood evaluation, which the authors reduced to $O(n)$ by restricting the 2-opt search to edges derived from the **Delaunay triangulation** of the city set. The item-selection stage employs three **fitness approximation schemes**—empty-tour, worst-case, and expected increased travel time—that estimate the marginal cost of carrying each item without performing a full objective evaluation, enabling rapid filtering of unpromising items. Tested on the benchmark introduced by [Polyakovskiy et al. \(2014\)](#), MATLS outperforms the constructive heuristics of [Bonyadi et al. \(2013\)](#) by a wide margin, demonstrating that approximate but computationally tractable subproblem evaluations can drive effective population-based search in the TTP.

Contribution	First computational-complexity analysis of TTP local search; complexity-reduction strategies (Delaunay-triangulation neighbourhood built in $O(n \log n)$, incremental $O(n)$ evaluation, splay-tree $O(\log n)$ tour edits); a two-stage local search using three analytically derived fitness-approximation schemes; the MATLS memetic algorithm for large-scale TTP.
Authors' claims	Full local-search cost $O(T(n^3+n^2 m+m^2))$ is reduced to $O(T(n^2+mn))$; two-stage LS practical cost $T1.O(n)+O(nm)$; MATLS gives positive profit on every tested large instance while RLS and EA give negative profit.
Evidence base	Tested on a subset of the polyakovskiy2014 suite: only ~6 city instances (brd14051, d15112, d18512, pla33810, rl11849, usa13509), all > 10,000 cities / 100,000 items, each x 3 KP types (Tables 1-3). Baselines RLS and EA re-run by the authors under an identical 10-min budget, population 30. Number of independent runs not stated; mean/std only; no statistical test; no new best-known reported; no code released.
Stated limitations	Authors note the two compared algorithms are deliberately simple because TTP research is 'in its infancy', that the strategy 'may vary from problem to problem', and that the reduction relies on exploiting domain knowledge.

Strengths. ✓ First rigorous complexity analysis of TTP local search with concrete, reusable reduction strategies • ✓ Targets the > 10,000-city regime largely ignored by later work • ✓ Approximation schemes derived analytically with a proven lower/upper-bound sandwich • ✓ Baselines re-run under an identical time budget

Weaknesses. ✗ Very narrow empirical base: about 6 city instances, all extremely large; no small or medium instances and no categorised subset • ✗ Baselines limited to the two weakest heuristics (RLS, EA) of polyakovskiy2014comprehensive • ✗ Number of independent runs not stated; no statistical significance testing • ✗ No new best-known solutions; the 'competitive' claim rests on beating known-weak baselines • ✗ Source code and solutions not released • ✗ Delaunay-triangulation neighbourhood pruning has no guarantee and its effect on missed improving moves is not quantified

[4] Polyakovskiy et al. (2014) — *A comprehensive benchmark set and heuristics for the traveling thief problem*

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	SH, S1-S5, C1-C6, RLS, EA	Family (L1/L2)	Problem def./benchmark/analysis / Benchmark + scoring heuristics
Benchmark	—	Size	Heuristics (SH, RLS, EA). Introduces a comprehensive benchmark suite for the TTP, a multi-component optimization problem combining the TSP and KP, with the goal of promoting research on interactions between interdependent NP-hard sub-problems. The benchmark suite is constructed from established benchmark instances of the two underlying problems, enabling systematic investigation of the additional complexity introduced by their integration. In addition to the benchmark design, the study presents several simple heuristic approaches for the TTP and reports their performance on the proposed benchmark suite, providing initial baselines for future research on multi-component optimization.& Cities: 76; 100; 130; 159; 280; 574; 724; 1000; 1304; 1577; 2103; 3038; 4461; 7397; 11849; 13509; 14051; 15112; 18512; 33810 Items: 1x; 5x; 10x KP-Types: bsc; uco; usw
Metric	—	Stat. test	Matlab decision-tree analysis
Key result	EA best on small instances, RLS on mid/large, SH on the very largest ($\geq 85,900$)		
Competition	N	Code	Unknown

Budget / Runs — / —

Hardware —

Abstract. Polyakovskiy et al. (2014) established the **comprehensive benchmark suite** that became the standard testbed for TTP research. The benchmark systematically combines 81 TSP instances from the TSPLIB library (EUC_2D / CEIL_2D, spanning 51 to 85,900 cities) with four item factors—1, 3, 5, or 10 items per city—and three KP correlation types: uncorrelated (**uco**), uncorrelated with similar weights (**usw**), and bounded strongly correlated (**bsc**)—across ten capacity categories, yielding $81 \times 3 \times 4 \times 10 = 9,720$ distinct instances. The paper also proposes three baseline methods: a simple constructive heuristic (**SH**), a random local search (**RLS**), and a standard (1+1) evolutionary algorithm (**EA**). By providing both the instance generator and the baseline performance data, this work created the shared evaluation platform that enables direct comparison across virtually all subsequent TTP algorithms.

Contribution	The canonical TTP benchmark suite: 9720 instances (81 TSPLIB graphs 51-85,900 cities x 3 KP types x 4 item factors x 10 capacity categories), with a per-instance renting rate calibrated so that an optimal-in-isolation solution scores zero; plus three baseline heuristics (deterministic SH, RLS, (1+1) EA) and public instances and objective code.
Authors' claims	Instances systematically span a wide feature range; SH is best on very large many-item instances, EA on small instances, RLS on mid/large; the packing plan strongly influences the objective even under a near-optimal LK tour; much room remains for smarter algorithms.
Evidence base	All 9720 instances run; iterative heuristics stopped at 10,000 non-improving iterations or 10 min, 30 independent repetitions. Analysis via a pruned decision tree over all runs plus rescaled [0,1] aggregate plots and a single 20-instance results table. No statistical hypothesis test. Baselines are the three heuristics themselves (first benchmark paper); a PackNone lower bound is reported.
Stated limitations	Authors expect small instances to be solved to optimality soon but large ones to stay open 'for years'; state it is unknown by how much the scores can be improved; future work is instance-hardness feature analysis and problem-specific algorithms; note the ~0.8 relative-performance plateau is 'currently unclear'.

Strengths. ✓ Establishes a single, large, systematically constructed benchmark that unified the whole field • ✓ Per-instance renting rate calibrated so decomposition scores zero, making interdependence measurable • ✓ Large-scale study: all 9720 instances, 30 runs, 10-min cap • ✓ Honest that optimal values are unknown and much improvement is possible • ✓ Instances and objective code released publicly

Weaknesses. ✗ No recommended representative subset, which forced ad-hoc subsetting across the later literature • ✗ No optimal or best-known reference values, so quality is only relative to three weak baselines • ✗ Only one detailed results table (20 instances); the rest is aggregate plots and a decision tree • ✗ No statistical testing • ✗ The three heuristics are deliberately simple, so the benchmark's power to discriminate strong algorithms is not demonstrated • ✗ Decision-tree analysis conflates factors; the 'number of cities plays no role' conclusion is acknowledged as possibly an encoding artefact

[5] Beham et al. (2015) — *Optimization Strategies for Integrated Knapsack and Traveling Salesman Problems*

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single

Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	HeuristicLab MA	Family (L1/L2)	Single-solution metaheuristic / Framework/local search
Benchmark	—	Size	Cities: 52; Items: 51
Metric	avg objective (StdDev, sec) over 10 runs.	Stat. test	none
Key result	variegation significantly improves TTP - bsc =15,269.5 vs. 14,246.9; usw =7,934.2 vs. 5,476.9; uncorr =8,777.0 vs. 5,559.1.		
Competition	N	Code	No
Budget / Runs	1s/run / 10	Hardware	Intel Core i7 (3rd gen) 2.6GHz, single core

Abstract. Beham et al. (2015) investigated strategies for **combining existing algorithms** for the KP and TSP into an integrated solver for multi-component problems, using the TTP as a motivating application. A key methodological contribution is the application of **Lagrangian decomposition** to the Knapsack-Constrained Profitable Tour Problem (KCPTP) as a blueprint for structuring the interaction between subproblem solvers. The paper discusses how the relaxed Lagrangian subproblems can be solved independently and how the dual information can be used to coordinate the two solvers. Applied to the berlin52 TTP instance (alongside KCPTP and orienteering instances), the framework introduces the important conceptual point that the interaction pattern between subproblem algorithms—not just the algorithms themselves—largely determines the quality of the integrated solution.

Contribution	A methodology paper for integrated problems (KCPTP, OP, TTP): Lagrange decomposition of the knapsack-constrained profitable tour problem, then a generalisation to ‘sub-problem input variegation’ where a coordinating agent alters sub-problem inputs (e.g. profit vectors via CMA-ES) so sub-problem optima align with the master objective; sequential and cooperative solver-orchestration schemes.
Authors’ claims	Input variegation is beneficial for the search when the same algorithm is used; for TTP it makes a significant difference; for KCPTP results are less convincing; constraint variegation still needs improvement.
Evidence base	TTP results are on a single 52-city instance (berlin52_n51) in 3 KP-type configurations, 10 runs, avg/std, single-core i7 laptop; with vs without variegation only. No standard benchmark, no comparison with any TTP solver, no statistical test. The TTP variant tested is relaxed to visit-only-stolen cities.
Stated limitations	Authors state constraint variegation needs improvement, KCPTP results are unconvincing, and that adoption depends on the method being considerably simpler than developing a new algorithm.

Strengths. ✓ Frames a general idea (align sub-problem landscapes with the master via input adjustment) linking Lagrangian decomposition and heuristic coordination • ✓ Applies to three related integrated problems, not only TTP • ✓ Cautious, honest claims that report where the method underperforms

Weaknesses. ✗ TTP evaluation is a single 52-city instance in 3 configurations, giving no real support for the ‘significant difference’ claim • ✗ The TTP variant tested is a relaxed, cities-skippable version, not standard TTP1 • ✗ No comparison with any established TTP algorithm; only self-comparison with/without variegation • ✗ No statistical test; 10 runs; tiny instance • ✗ The CMA-ES outer loop for variegation is an expensive overhead whose cost/benefit is not analysed at scale • ✗ Largely conceptual proof-of-concept with no scalability evidence; not adopted by the later literature

[6] El Yafrani and Ahiod (2015) — *Cosolver2B: an efficient local search heuristic for the travelling thief problem*

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	CoSolver2B	Family (L1/L2)	Single-solution metaheuristic / Local search
Benchmark	—	Size	Cities: 52 to 33,810; Items: 5x KP-Types: usw
Metric	objective, time (s).	Stat. test	none
Baselines	EA, RLS polyakovskiy2014comprehensive; CoSolver2B in firstfit and bestfit variants.		
Key result	CoSolver2B-bestfit best on most instances (e.g. berlin52_n255 =26,658 vs. EA 20,591, RLS 20,591; ts225 =89,079 vs. EA 84,907); runtime hits the 600s cap on the largest instances.		
Competition	N	Code	No
Budget / Runs	600s / 1 (deterministic; EA/RLS 30 runs on separate machine)	Hardware	Java, Intel Core i3-2370M 2.40GHz, 4GB, Linux

Abstract. El Yafrani and Ahiod (2015) extended the cooperative co-evolutionary paradigm by proposing **CoSolver2B**, which provides a rigorous mathematical formulation of the communication protocol between the TSP and KP modules. The paper formalizes the interaction rules that govern when and how partial solutions are exchanged between the two solvers, offering stronger theoretical grounding than the original CoSolver framework. Evaluated on TTP instances with five items per city of the uncorrelated similar-weights type (**5x, usw**), CoSolver2B consistently discovers better solutions than both RLS and the standard EA from Polyakovskiy et al. (2014). The explicit modeling of information exchange lays groundwork for more disciplined decomposition frameworks in subsequent research.

Contribution Cosolver2B: a concrete deterministic local-search realisation of the CoSolver framework (LK tour + insertion/elimination packing init, then alternating Delaunay-restricted 2-opt on TSKP and bit-flip on KRP until no improvement), with time/weight accumulator-and-register vectors for cheap incremental objective recovery; best-fit and first-fit variants.

Authors' claims	Cosolver2B surpasses EA and RLS on various TTP sizes, finds new better solutions on most tested instances, gains mostly on the KRP sub-problem, is deterministic (unlike RLS/EA), and best-fit beats first-fit on small/mid instances.
Evidence base	15 instances from polyakovskiy2014, all of a single KP type (uncorrelated-similar-weights) and a single capacity category (05). Baselines RLS and EA are quoted from an online repository (not re-run); protocol is inconsistent (best-of-30 for RLS/EA vs one deterministic run for Cosolver2B). Different hardware for the compared methods (acknowledged; 'runtimes for guidance'). 600 s cap; no statistical test. On the largest instances Cosolver2B times out and is beaten by RLS on several (e.g. rl11849, pla33810).
Stated limitations	Authors note runtime is high for very large instances and propose better space-exploration and more sophisticated search heuristics as future work.

Strengths. ✓ Turns the abstract CoSolver into a working, deterministic solver with reproducible operators • ✓ Accumulator/register data structures make 2-opt and bit-flip evaluation cheap • ✓ Uses the standard polyakovskiy2014 instances (unlike bonyadi2014socially) • ✓ Openly reports the hardware mismatch and the large-instance timeouts

Weaknesses. ✗ Only 15 instances, all one KP type and one capacity category; not the categorised subset • ✗ Baselines quoted from a repository, not re-run; best-of-30 vs single-run comparison is not like-for-like • ✗ Different hardware for the compared methods; runtimes not comparable • ✗ On large instances Cosolver2B times out and is clearly beaten by RLS, yet the abstract claims it surpasses RLS/EA • ✗ No statistical testing; single run; no variance reported • ✗ No comparison with MATLS despite citing it and claiming complementary strengths

[7] Faulkner et al. (2015) — *Approximate Approaches to the Traveling Thief Problem*

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	S1-S5, C1-C6	Family (L1/L2)	Constructive/iterative heuristic / Scoring heuristic
Benchmark	—	Size	Cities: 76; 100; 130; 159; 280; 574; 724; 1000; 1304; 1577; 2103; 3038; 4461; 7397; 11849; 13509; 14051; 15112; 18512; 33810 Items: 1x; 5x; 10x KP-Types: bsc; uco; usw
Metric	approximation ratio (mean over 30 runs / best-found), %; Key results (avg over top 60 instances, n<85900): S5 = 98.3, C3 = 97.5, C4 = 97.4, C5 = 97.0, C6 = 96.8, S1 = 95.0, MIP = 98.1, MATLS = 97.0; avg over all 72: S5 = 98.2, S1 = 95.2;	Stat. test	none

Competition	N	Code	No
Budget / Runs	10min/instance (8h for MIP on n 33810,85900) / 30	Hardware	Intel Xeon E5430 2.66GHz, Java SE RE 1.7 (MIP: AMD Opteron 6348 2.8GHz, CPLEX 12.6)

Abstract. Faulkner et al. (2015) conducted a comprehensive study of packing strategies applied on top of near-optimal TSP tours generated by the **Chained Lin-Kernighan (CLK)** heuristic. The paper introduces five simple strategies (**S1–S5**) and six combination strategies (**C1–C6**) that differ in how items are selected, with S5 applying a distance-to-end-weighted scoring to rank item attractiveness along the tour. A defining contribution is the empirical observation that **S5 dominates in Category A** instances: because Category A features small capacities and strongly correlated items, the restricted packing space makes simple scoring-based greedy heuristics more effective than complex iterative operators that require substantial exploration budget. Multi-start variants C3 and C4 achieve the best overall performance, and the paper introduces the influential **Category A/B/C** difficulty classification that has since become a standard reporting framework for TTP results.

Contribution	A family of packing-focused heuristics on Chained-LK tours: PACK (exponent-weighted scoring), PACKITERATIVE (auto-tuned exponent), BITFLIP and INSERTION operators, the simple heuristics S1-S5 and complex heuristics C1-C6, plus a piecewise-linear MIP packing; establishes S5 (tour resampling) as the strong simple baseline and introduces the 72-instance representative subset.
Authors' claims	S5 outperforms all the more complex heuristics C1-C6 and the MIP approach on average despite low cost; the initial TSP tour is the dominant factor for the final objective; the best packing exponent depends on the KP type; MATLS beats their methods on a few instances but is not among the best overall; the MIP approach is second-best on average.
Evidence base	72 instances selected from the 9720 suite (6 city sizes x item factors 3,10 x 3 KP types); 10-min budget (8 h for MIP on the largest); 30 repetitions; metric is approximation ratio against the best-found value across all methods, plus podium-count bar charts. No statistical hypothesis test. Different hardware/solvers for heuristics (Java, Xeon) vs MIP (CPLEX 12.6, Opteron).
Stated limitations	Authors state there is no single best paradigm, interdependence is rarely exploited, problem-specific local search performs best but needs faster particular operators, and suggest instance-hardness and complexity analyses; note tour generation is slow for very large instances and pre-computing tours is an obvious improvement.

Strengths. ✓ Introduces PackIterative and the S1-S5 taxonomy adopted as baseline by the whole later literature • ✓ The 72-instance representative subset is a concrete, reusable evaluation design (precursor to the categorised 60) • ✓ 30 repetitions and a consistent 10-min budget across heuristics • ✓ Well-supported counter-intuitive finding that tour resampling beats sophisticated local search under a time budget • ✓ Compares against MIP and MATLS, not only weak baselines

Weaknesses. ✗ No statistical significance testing; conclusions rest on ratio averages and podium counts • ✗ The 'best-found' pseudo-optimum is the union of the compared methods, so approximation ratios are only relative • ✗ Different hardware and solvers for heuristics vs MIP, and the MIP is given 8 h against 10 min • ✗ Exponent tuning in PackIterative adds instance-specific calibration; the exponent behaviour on bsc instances 'varies significantly' and is unexplained • ✗ BitFlip and Insertion run single-pass per iteration for time reasons; their full potential is not evaluated • ✗ The MATLS comparison has many missing table entries

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	G1/G2	Family (L1/L2)	Constructive/iterative heuristic / Greedy
Benchmark	—	Size	Cities: 51 to 85,900; Items: 1x; 3x; 5x; 10x KP-Types: bsc; uco; usw
Metric	total profit;	Stat. test	none
Baselines	SH polyakovskiy2014comprehensive		
Key result	both beat SH on almost all instances; G1 best on uco/usw, G2 best on bsc; e.g. uco-Cat10-F10 pla85900: $G1 = 1.24!x!10^8$, $G2 = 7.78!x!10^7$, $SH = 5.04!x!10^7$;		
Competition	N	Code	No
Budget / Runs	— / 1 (deterministic)	Hardware	Intel Core i7, 8GB DDR3, Windows 8.1

Abstract. Gupta and Prakash (2015) proposed two **greedy heuristics** for the TTP that prioritize items based on a profit-to-weight ratio modified by the remaining tour distance. The heuristics differ in how they account for the interaction between item weight and future travel cost: the first applies a fixed scoring rule, while the second dynamically adjusts item priorities as items are collected along the route. Both methods are evaluated on the full benchmark set of Polyakovskiy et al. (2014) and consistently outperform the well-known reference heuristic of Bonyadi et al. (2013) in both solution quality and computation time. The study demonstrates that even straightforward greedy strategies can be highly competitive when the scoring function correctly captures the nonlinear cost of carrying weight over the remaining tour.

Contribution	Two greedy packing heuristics on a fixed LKH tour: G1 (classic profit/weight-ratio KP greedy) and G2 (fill from the last city backward, delaying pickups to travel faster longer), each with two tuning factors KF and WF swept on a 0.05 grid, returning the best objective found.
Authors' claims	G1 and G2 consistently outperform the SH heuristic of polyakovskiy2014 on both solution quality and computation time on almost all 9720 instances; G1 suits uncorrelated types, G2 suits bounded-strongly-correlated; G1 is much faster than G2.
Evidence base	All 9720 instances (aggregate scatter plots only). Detailed tables cover a single capacity category (10) and single item factor (10), i.e. 30 instances x 3 KP types x 10 large graphs. Only baseline is SH (the weakest polyakovskiy heuristic). No comparison with S5, MATLS or CoSolver despite citing them. No statistical test; single run; $\delta = 0.05$ 'empirically set'; no time budget stated. C on core i7.
Stated limitations	Minimal: future work is effective search operators and metaheuristics that build on these heuristic ideas.

Strengths. ✓ Simple, fast, deterministic; $O(n + m \log m)$ for G1 • ✓ G2's delay-packing-to-travel-faster idea is a clean insight echoed by later Insertion operators • ✓ Full 9720-instance coverage in aggregate • ✓ Analyses which KP type suits which heuristic

Weaknesses. ✗ Only compared against SH, the weakest baseline; S5, MATLS and CoSolver are ignored despite being cited • ✗ Detailed tables cover only 1 capacity category and 1 item factor; the '9720' claim rests on plots without numbers • ✗ The iterative sweep over KF and WF (step 0.05) is an undisclosed overhead that makes 'greedy' a misnomer; sensitivity to the step delta is not studied • ✗ No statistical testing and no robustness analysis • ✗ Beating SH is a low bar (SH can be worse than picking nothing) • ✗ Very short paper; little analysis of why G1/G2 beat SH beyond aggregate observations

[9] Mei et al. (2015) — *Heuristic evolution with genetic programming for traveling thief problem*

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	GAIN/PICKFUNC (GP)	Family (L1/L2)	Hyper-heuristic / GP-based
Benchmark	—	Size	Cities: 51, Items: 50 (GP); Cities: 11,849 to 33,810 (MA); Items: 10x (MA)
Metric	mean objective (std).	Stat. test	significance tests (TSMA-GAIN/PICKFUNC = TSMA; both RLS/EA)
Baselines	RLS, EA polyakovskiy2014comprehensive, original TSMA.		
Key result	TSMA-GAIN and TSMA-PICKFUNC are statistically indistinguishable from the hand-designed TSMA, and all three significantly outperform RLS and EA on every test instance; GP heuristics generalise from small training to large test instances (GAIN training fitness 0.347, PICKFUNC 0.414).		
Competition	N	Code	No
Budget / Runs	— / —	Hardware	—

Abstract. Mei et al. (2015) proposed the **Two-Stage Memetic Algorithm (TSMA)**, which couples tour improvement using standard TSP local search with an item-selection stage whose heuristic is evolved using **Genetic Programming (GP)**. Rather than manually designing the item-scoring function, the GP automatically evolves both a **gain function** (evaluating the marginal value of adding an item to the packing plan) and a **picking function** (deciding which uncollected items to include), allowing the algorithm to discover problem-specific heuristics that exploit structural properties of the TTP objective. The evolved heuristics achieve competitive performance against prior state-of-the-art methods, demonstrating the promise of automated heuristic design through GP as a systematic methodology for multi-component combinatorial problems.

Contribution	Uses genetic programming to evolve two item-selection heuristics for the two-stage memetic algorithm (MATLS): GAIN (a real-valued gain function replacing the hand-designed priority) and PICKFUNC (a picking-decision function replacing the sign test); a study of whether the hand-designed heuristic is actually good.
Authors' claims	GP-evolved heuristics generalise from 51-city training instances to > 10,000-city test instances; TSMA-GAIN and TSMA-PICKFUNC are statistically indistinguishable from the hand-designed TSMA and significantly better than RLS/EA; the evolved gain function is intuitively reasonable.
Evidence base	Training on 3 instances (eil51, one per KP category); testing on 18 instances = 6 large graphs (> 10,000 cities) x 3 KP types; 30 independent runs per algorithm, 10-min budget, results 'compared statistically' (test not named on the reported pages). GP: population 100, 2-hour training, no parameter tuning. Baselines RLS/EA quoted from polyakovskiy2014; hand-designed TSMA re-run.
Stated limitations	Authors call it 'merely a preliminary work'; note only one tour per training instance was used, GP was untuned, better GAIN training fitness did not yield better test performance (fitness measure 'may not be proper'), and selecting proper training tours is 'another issue to be addressed'.

Strengths. ✓ Systematically asks whether the hand-designed item heuristic is good, using GP as a yardstick • ✓ Demonstrates small-to-large generalisation (51 to > 14,000 cities) • ✓ Reports honest negative results: evolved heuristics only match, never beat, the hand-designed one • ✓ Analyses the interpretable evolved gain function

Weaknesses. ✗ Evolved heuristics do not outperform the existing hand-designed heuristic; net practical gain is zero • ✗ Training set is 3 instances with a single tour each; acknowledged as inadequate • ✗ Test set is 18 very large instances only; no small/medium and no categorised subset • ✗ Statistical test not named or detailed on the reported pages; only mean/std tables • ✗ Baselines RLS/EA quoted, not re-run; only the weak baselines are beaten • ✗ Minimal GP setup (no tuning, single elitist) leaves open whether a stronger GP would beat the hand heuristic

[10] Polyakovskiy and Neumann (2015) — Packing While Traveling: Mixed Integer Programming for a Class of Nonlinear Knapsack Problems

Variant	PWT	Objective / #obj	Single-objective / 1
Route	Fixed route	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	PWT MIP	Family (L1/L2)	Exact methods / MIP (PWT)
Benchmark	—	Size	Cities: 51; 76; 101; Items: 1x; 5x; 10x KP-Types: bsc; uco; usw
Metric	relative gap %, runtime (s), ratio $\wedge$, aux-variable rate %.	Stat. test	none
Key result	exact NKP ^a c solves only small instances to optimality within the time limit (b-s-corr hardest, not reducible by pre-processing); NKP ^a a_ is very fast and near-optimal even at =100 (median ~45.3%), solving pla33810 in <40min and pla85900 within 2-5.5h.		

Competition	N	Code	No
Budget / Runs	1 day (NKP ^c small), 2h / 6h (NKP ^a large) / 1 (deterministic exact)	Hardware	Java, CPLEX 12.6, cluster 128GB RAM, 2.8GHz 48-core AMD Opteron

Abstract. Polyakovskiy and Neumann (2015) introduced the **Packing While Traveling Problem (PWT)** as a dedicated combinatorial subproblem, isolating the nonlinear knapsack component of the TTP by fixing the route and focusing exclusively on item selection. The paper establishes that both the capacity-constrained and unconstrained variants of PWT are **NP-hard**, providing the first formal complexity analysis of this subproblem. To solve it, the authors develop a preprocessing scheme that eliminates dominated items, an **exact Mixed Integer Programming (MIP)** formulation, and a faster **approximate MIP** based on a linearization technique. Tested on selected instances from the TTP benchmark with 51 to 101 cities and all three KP types, the approximate MIP achieves near-optimal solutions in significantly less computation time than the exact approach, making it a practical subroutine for exact or heuristic TTP solvers.

Contribution	Formalises Packing While Travelling (PWT), the packing sub-problem of TTP on a fixed route: constrained NKP ^c and unconstrained NKP ^u . Proves NKP ^u is NP-hard (reduction from Subset-Sum). Provides a provably correct compulsory/unprofitable-item pre-processing scheme, an exact MIP (RLT linearisation plus valid inequalities) and a fast piecewise-linear approximate MIP.
Authors' claims	NKP ^u is NP-hard; only small instances (eil51/76/101) are solved exactly within the time limit; the approximate MIP with tau=100 is near-optimal (ratio approx 1) and very fast, handling pla33810 in under 40 min and pla85900 in under 5.5 h; unconstrained cases are easier; bounded-strongly-correlated instances do not reduce under pre-processing and have much larger gaps.
Evidence base	Benchmark set B of polyakovskiy2014 with the route fixed to a Chained-LK tour. Exact MIP on eil51/76/101 families; approximate MIP on pla33810/pla85900. CPLEX 12.6, Java, 48-core cluster; time limits 1 day (exact), 2 h and 6 h (approximate). Metrics: optimality gap, ratio of approximate lower bound to exact, runtime, auxiliary-variable usage. Deterministic. No comparison with heuristic packing methods.
Stated limitations	Authors note the exact method solves only small instances and that bsc instances resist pre-processing; future work is allowing the city sequence to change (full TTP), skippable cities and pickup-and-delivery.

Strengths. ✓ First rigorous complexity result for the packing sub-problem (NKP^u NP-hard proof) • ✓ Clearly isolates PWT as a sub-problem worth studying on its own • ✓ Compulsory/unprofitable pre-processing is provably correct and effective, and is reused by later exact work • ✓ The approximate MIP is genuinely strong: near-optimal packing, scaling to 85,900 cities • ✓ Deterministic exact and approximate methods with reported optimality gaps, rare in the TTP literature

Weaknesses. ✗ No comparison with existing heuristic packing (PackIterative, G1/G2) on speed or quality • ✗ Exact NKP^e closes only instances up to 101 cities; most of the benchmark stays open exactly • ✗ Approximate quality is measured against the exact model, which is itself unsolved for larger instances, so large-instance quality is not verified against ground truth • ✗ The method degrades exactly on bounded-strongly-correlated instances, the hardest and most studied category • ✗ The route is fixed to a single CLK tour; interaction with tour optimisation is not addressed • ✗ Heavy CPLEX dependence and a 48-core cluster with multi-hour runs limit accessibility

[11] Strzeżek et al. (2015) — DiverGene: Experiments on controlling population diversity in genetic algorithm with a dispersion operator

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	DiverGene	Family (L1/L2)	Population-based metaheuristic / EA (diversity)
Benchmark	—	Size	Presents diverGene, a diversity-aware population selection operator for Genetic Algorithms designed for complex multi-criteria optimization problems. The method extends the standard selection mechanism by explicitly controlling the level of population diversity to balance exploration and exploitation during the search process. The paper discusses the theoretical foundation of the operator, including its computational complexity, and proposes an efficient computation strategy. The approach is evaluated on the KP, TSP, and the TTP, a composite problem combining both. Experimental results indicate that diverGene has potential to improve solution quality for difficult optimization problems by enhancing population diversity control within the evolutionary search process. Only one instance is selected from the TTP benchmark. No explicit results are reported.& Cities: 52; Items: 1x KP-Types:
Metric	best objective $f(S)$ swept over dispersion parameters k , $[0,1]$ and population size $ S $ 10,25,50,100,150,200.	Stat. test	none
Key result	dissimilarity-aware selection improves the TTP objective, with the gain growing for larger populations, and the improvement is largest on the hardest problem (TTP > TSP > KP).		
Competition	N	Code	No
Budget / Runs	— / parameter-sweep plots (no single best-known reported)	Hardware	—

Abstract. [Strzeżek et al. \(2015\)](#) presented **DiverGene**, a diversity-aware population selection operator for Genetic Algorithms designed to prevent premature convergence in complex multi-criteria optimization problems. The operator extends the standard fitness-based selection by explicitly penalizing the selection of individuals that are too similar to others already chosen, using a **pairwise dissimilarity metric** to quantify structural distance between solutions. The computational complexity of the operator is analyzed and an efficient approximation is proposed. When applied to the TTP—tested on a single representative instance of 52 cities—the operator demonstrates potential for improving solution quality by maintaining population diversity throughout the evolutionary search, motivating its application to more complex combinatorial landscapes where diverse exploration is essential.

Contribution	diverGene, a diversity-aware GA selection operator that combines roulette selection with a ‘dissimilarity selection’ derived from the Max-Sum Facility Dispersion problem (using its 2-approximation for tractable computation); applied to KP, TSP and TTP1.
Authors’ claims	Controlling population diversity via diverGene is a promising technique; for TTP the positive impact is strongest (about 40-45 percent improvement) and grows with problem hardness; for hard problems diversity almost never worsens quality; the best-solution peak shifts toward about 70-75 percent of individuals selected by dissimilarity for TTP.
Evidence base	TTP is tested on a single instance (Berlin52, bounded-strongly-correlated, capacity 01), 52 cities / 51 items, from the CEC2014 dataset. Parameter sweeps over the dissimilarity fraction k , the fitness/dissimilarity balance α and population size; median of 10 runs. No comparison with any TTP solver or even a plain GA baseline; only with vs without the operator. No statistical test. TTP objective values are large and negative.
Stated limitations	Authors state more tests should be performed, other dissimilarity implementations and settings examined, the near-zero effect of α investigated, and propose self-adaptation of k and α and a convergence-rate study as future work.

Strengths. ✓ Diversity operator grounded in Max-Sum Dispersion theory with a 2-approximation for efficient computation • ✓ Studies the operator across three problems with parameter-sensitivity sweeps • ✓ Observes the operator helps more on harder compound problems

Weaknesses. ✗ TTP evaluation is a single 52-city instance, so the 40-45 percent improvement and harder-problem claims have essentially no support for TTP • ✗ No comparison with any TTP algorithm or even a plain GA baseline; only self-comparison with/without the operator • ✗ Large negative TTP objective values indicate the underlying GA is very weak; the operator improves a poor solver • ✗ No statistical significance testing; median of 10 runs only • ✗ The base GA ignores TTP interdependence and uses naive item selection • ✗ Not adopted by the later literature; a diversity-operator experiment that happens to include TTP

[12] El Yafrani and Ahiod (2016) — *Population-based vs. Single-solution Heuristics for the Travelling Thief Problem*

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting

Algorithm	MA2B	Family (L1/L2)	Population-based metaheuristic / Memetic algorithm
Benchmark	—	Size	Cities: 76; 100; 130; 159; 280; 574; 724; 1000; 1304; 1577; 2103; 3038; 4461; 7397; 11849; 13509; 14051; 15112; 18512; 33810 Items: 1x; 5x; 10x KP-Types: bsc; uco; usw
Metric	objective, RSD (%), time (s);	Stat. test	none
Baselines	MATLS, S5;		
Key result	MA2B best on small/mid instances, S5 best on Cat1, CS2SA/S5 best on large; e.g. Cat1 eil76 MA2B =4290 vs. S5 3742, CS2SA 3906, MATLS 3705;		
Competition	N	Code	Yes
Budget / Runs	600s / 10	Hardware	CS2SA on Core i3-2370M 2.40GHz/4GB; MA2B/S5 on Core i7-5700HQ 2.70GHz/8GB

Abstract. El Yafrani and Ahiod (2016) conducted the first systematic comparison of **population-based versus single-solution metaheuristic paradigms** for the TTP. The paper proposes two new algorithms: **MA2B**, a Memetic Algorithm employing Maximal Preservative Crossover (MPX) with a 2-opt local search, and **CS2SA**, a single-solution solver that alternates between a 2-opt steepest-ascent hill-climber for the TSP component and **Simulated Annealing (SA)** for the KP component. Extensive experiments on the full benchmark reveal a critical scalability pattern: population-based methods excel on small instances but are overwhelmed on large ones because population initialization consumes the majority of the available 10-minute time budget, leaving insufficient time for actual optimization. Single-solution heuristics, by contrast, scale gracefully because their per-iteration overhead is lower, and the stochastic exploration of SA allows the solver to escape local optima even without a population.

Contribution	Two TTP1 algorithms and a population-based vs single-solution study: CS2SA (single-solution CoSolver-style decomposition with 2-opt hill-climb for the tour and simulated annealing with bit-flip for packing) and MA2B (memetic algorithm with 2-opt + bit-flip local search and crossover); three competition-style difficulty categories C1/C2/C3.
Authors' claims	CS2SA and MA2B achieve competitive performance and surpass MATLS and S5 on different instances; the honest conclusion is that S5 is best on average for all three categories, memetic algorithms (MA2B, MATLS) do better on small/mid instances, single-solution methods (CS2SA, S5) do better on large instances, and CS2SA does best at high knapsack capacities; results are 'still not very conclusive'.
Evidence base	The categorised 60-instance benchmark: 20 base graphs (eil76 to pla33810) evaluated at the three fixed category points (Cat1 T3/C1/F1, Cat2 T2/C5/F5, Cat3 T1/C10/F10). Experiment 1: 6 large graphs (> 10,000 cities) x 3 categories; Experiment 3 (main comparison): 20 graphs x 3 categories, MATLS/S5/CS2SA/MA2B, 10 runs, objective + relative standard deviation + time, 600 s. Different hardware for the compared algorithms (i3 vs i7) and different languages (MATLS in C++, the rest in Java); 'runtimes for guidance'. No statistical test.
Stated limitations	Authors state memetic algorithms are difficult to adapt for large TTP instances, results are 'still not very conclusive', the runtime limit and TSP base have important impact, and suggest fitness-landscape study and a hyper-heuristic direction.

Strengths. ✓ Systematic population-vs-single-solution comparison, a useful methodological question • ✓ Three competition-style categories, an early step toward the categorised subset • ✓ Compares against the two established strong baselines (S5, MATLS), re-run • ✓ 10 runs with relative standard deviation and per-instance tables • ✓ Identifies the memetic weakness under tight budgets

Weaknesses. ✗ Different hardware and languages for the compared algorithms, so runtime comparisons are unreliable and quality comparisons are confounded by implementation • ✗ No statistical significance testing despite 10 runs and relative standard deviation • ✗ The main comparison uses only 3 fixed category points per graph; the wider T/C/F grid is explored on just 4 graphs and only as a heatmap without numbers • ✗ The abstract's 'surpass MATLS and S5' claim is contradicted by the paper's own conclusion that S5 is best on average for all three categories • ✗ CS2SA is essentially Cosolver2B with SA for packing, incremental over el2015cosolver2b • ✗ SA parameters fixed, not tuned per instance

[13] Lourenço et al. (2016) — An Evolutionary Approach to the Full Optimization of the Traveling Thief Problem

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	EA	Family (L1/L2)	Population-based metaheuristic / Evolutionary algorithm
Benchmark	—	Size	Cities: 51; 76; 100; Items: 1x; 3x KP-Types:
Metric	best fitness, mean best fitness (MBF std) over 30 runs.	Stat. test	none
Baselines	DH heuristic (decompose: solve TSP then fix tour and pack)		
Key result	EA finds better best solutions than DH on 4/6 instances (e.g. eil51-1: EA 4706.54 vs. DH 2591.95; kroA100-3: EA 25,923 vs. DH 25,923 tie), within 9%/13% on the larger kroA100; the best TTP tours differ from TSP-optimal tours in 31.4-71.1% of edges (>50% in all but one) and are 10-29% longer than the TSP optimum.		
Competition	N	Code	No
Budget / Runs	— / 30	Hardware	—

Abstract. Lourenço et al. (2016) challenged the prevalent approach of fixing the TSP component before optimizing the KP by proposing a **unified Evolutionary Algorithm** that simultaneously represents and evolves both the tour and the packing plan within each individual. Variation operators are designed to modify both subcomponents jointly, guided by a packing heuristic that repairs infeasible packing plans after crossover or mutation. The key experimental finding—validated on three small instances with 51, 76, and 100 cities—is that **fixing one subproblem creates a strong bias** that systematically excludes large regions of the search space. Jointly optimizing both components, even in a simple EA framework, leads to solutions of higher quality, because the algorithm can discover favorable trade-offs between tour length and item collection that are invisible to methods that pre-fix the tour.

Contribution	An ‘unbiased’ EA that co-evolves the tour and the packing plan simultaneously (individual = permutation + binary string, adapted PMX + inversion, per-tour score-based packing, bit-flip local search); plus a structural analysis comparing best-TTP tours with best-TSP tours.
Authors’ claims	Solving TTP as a whole creates conditions for better solutions; the EA beats DH on 4 of 6 instances; best-TTP tours differ from best-TSP tours in 31-71 percent of edges and are 10-29 percent longer than the optimal TSP tour, showing that fixing the tour compromises global optimality.
Evidence base	6 instances from 3 small graphs (eil51, eil76, kroA100), all strongly correlated, 1 or 3 items/city. Population 100, 250,000 evaluations (fixed budget), 30 runs, mean+/-std for the EA. Only baseline is DH (density heuristic), for which only best solutions are available; no statistical analysis (explicitly declined). No comparison with S5, MATLS, CoSolver2B or MA2B. Structural analysis on the same 3 graphs.
Stated limitations	Authors explicitly note the study considers only a subset of instances and focuses only on the influence of fixed TSP-optimal tours, and that the EA has scalability issues on larger instances; future work includes fixing items and evolving the tour, and alternative symbiotic representations.

Strengths. ✓ Clear empirical demonstration that the TTP-optimal tour is not the TSP-optimal tour (31-71 percent edge difference), supporting the non-separability thesis • ✓ Co-evolves both components, a genuinely different design from the fix-one-component mainstream • ✓ Uses a fixed evaluation budget rather than wall-clock for reproducibility • ✓ 30 runs with mean+/-std for the EA • ✓ Insightful, reusable structural analysis (edge overlap, tour-length inflation)

Weaknesses. ✗ Only 6 instances from 3 graphs (≤ 100 cities), all strongly correlated; no medium/large and no categorised subset • ✗ Only baseline is DH, one of the weakest heuristics; no comparison with S5, MATLS, CoSolver2B or MA2B • ✗ No statistical significance testing (explicitly declined) • ✗ The EA is beaten by DH on the larger (kroA100) instances, so the ‘enhanced quality’ claim holds only on the smallest instances • ✗ PMX + inversion is a weak TSP recombination; no edge-recombination tried • ✗ The per-tour packing heuristic is essentially standard; novelty is the co-evolution framing

[14] Mei et al. (2016) — On investigation of interdependence between sub-problems of the Travelling Thief Problem

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static

Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	Cooperative co-evolution	Family (L1/L2)	Population-based metaheuristic / Cooperative co-evolution
Benchmark	—	Size	Investigates the interdependence between sub-problems in the TTP, a benchmark combining the TSP and KP. Shows through mathematical analysis that the problem cannot be decomposed into independent sub-problems due to the non-linear coupling in the objective function, making separate optimization of TSP and KP insufficient. Proposes two metaheuristic approaches: a Cooperative Co-evolution (CC) method that optimizes sub-problems separately while exchanging information between them, and a Memetic Algorithm (MA) that directly solves the full TTP as a unified problem. Experimental comparisons show that MA consistently outperforms both standard and dynamic CC variants under similar computational budgets, highlighting the importance of explicitly modeling sub-problem interdependence in the TTP. The dataset utilized in the experiments consists of a representative subset of 39 instances from the TTP instances generated by Bonyadi et al. (2013) . For the sake of simplicity, for each parameter setting with $10 \leq n, m \leq 100$, only the first instance is chosen from the 10 generated instances as a representative. The selected subset consists of 39 instances.& Cities: 10 to 100; Items: 10 to 150
Metric	mean benefit (SD) and average number of fitness evaluations over 30 runs.	Stat. test	significance (best marked bold; Wilcoxon)
Baselines	cooperative-coevolution variants SCC and DCC, each with k 1,3 collaborators, vs. the proposed Memetic Algorithm (MA).		
Key result	MA is best on all 39 instances; among CC variants SCC-k=1 wins 25/39; k=3 better on small/medium, k=1 better on large; DCC converges faster but to worse optima - confirming the sub-problems are too interdependent for population-level decomposition.		
Competition	N	Code	No
Budget / Runs	— / 30	Hardware	—

Abstract. Mei et al. (2016) provided the first rigorous **mathematical analysis of interdependence** in the TTP, showing through the structure of the nonlinear objective function that it is impossible to decompose TTP into independent subproblems. The paper proposes and compares two metaheuristic strategies: a **Cooperative Co-evolution (CC)** approach that solves TSP and KP in separate populations with periodic information exchange, and a **Memetic Algorithm (MA)** that treats the TTP as a holistic problem. Tested on a representative subset of 39 instances spanning 10 to 100 cities, MA consistently and substantially outperforms both the standard and a dynamic variant of CC under identical computational budgets. This result provides strong empirical evidence that the nonlinear coupling in the TTP objective is too tight for population-level decomposition to be effective, and that treating the problem as a unified entity is the correct algorithmic direction.

Contribution	A theoretical and empirical investigation of TTP interdependence: shows the TTP objective is not additively separable (nonlinear cross-terms in x and z), and develops cooperative co-evolution (static SCC, dynamic DCC) and a memetic algorithm (MA) that solves TTP as a whole, then compares them.
Authors' claims	The TTP objective is not additively separable, so solving TSP and KP sub-problems individually can hardly give high-quality solutions; the whole-problem MA significantly beats both SCC and DCC on all 39 instances (Wilcoxon rank-sum, $\alpha=0.05$) with larger mean and smaller std; $k=3$ collaborators suit small/medium instances, $k=1$ suits large-scale.
Evidence base	39 instances only, n and m in $[10,100]$, from the bonyadi2013 generator (not the standard 9720 suite). 30 runs, Wilcoxon rank-sum test, mean+/-std and benefit/evaluation-count tables, convergence curves. Fixed generation budget (100 for MA, 100/ k for CC). Standard weak operators (OX, one-point crossover, 2-opt). No comparison with any external TTP algorithm.
Stated limitations	Future work: more sophisticated operators (3-opt, LK) and measures that reflect interdependence / 'direction' so frameworks can be designed systematically rather than heuristically; the significance 'may go beyond just TTP'.

Strengths. ✓ The separability analysis is a clear, widely cited argument that the TTP objective is not additively separable • ✓ Both theoretical and empirical treatment, with a complexity analysis • ✓ 30 runs plus a Wilcoxon rank-sum test, proper statistical testing for its time • ✓ Detailed convergence analysis and study of collaborator count k and static vs dynamic updates

Weaknesses. ✗ Uses only tiny instances ($n, m \leq 100$) from the bonyadi2013 generator, not the standard 9720 suite; results do not transfer to the benchmark the field uses • ✗ No comparison with any established TTP solver; MA only beats the authors' own CC variants, not the state of the art • ✗ The non-separability argument is often over-read: it does not show coordinated decomposition cannot work well (later CoCo-type methods became state of the art) • ✗ Fixed-generation budget with different per-generation cost for CC vs MA; the 100/ k adjustment is approximate • ✗ Weak operators (OX, one-point crossover, 2-opt), so absolute solution quality is low • ✗ Many reported benefits are negative, so 'better' often means 'less bad'

[15] Wagner (2016) — Stealing Items More Efficiently with Ants: A Swarm Intelligence Approach to the Travelling Thief Problem

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single

Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	MMAS / ACO	Family (L1/L2)	Population-based metaheuristic / Swarm (ACO)
Benchmark	—	Size	Cities: 51 to 1000; Items: 3x KP-Types: uco; usw
Metric	average approximation ratio (trend lines over 1100 instances);	Stat. test	—
Baselines	S1, S5, C3-C6 faulkner2015approximate, MATLS;		
Key result	MMAS best for ≤ 250 cities and ≤ 800 items (up to $>10\%$ gain); C3-C6 best for 250-500 cities; S5 dominates on larger; e.g. eil51-uco MMAS is 4 obj = 11763 (ratio 0.997) vs. S1/C6 = 10079;		
Competition	N	Code	Unknown
Budget / Runs	— / —	Hardware	—

Abstract. Wagner (2016) proposed a **MAX-MIN Ant System (MMAS)** approach that fundamentally reframes the TTP optimization objective: rather than constructing near-optimal TSP tours and then optimizing packing, the ant colony directly targets **high-quality TTP tours** in which the combined routing and packing objective is maximized. Each ant constructs a complete tour guided by pheromone trails updated based on TTP—not TSP—objective scores, while packing plans are computed heuristically for each tour. An additional hill-climber further refines the best solution after each iteration. Evaluated on instances with up to 250 cities and 2000 items of uncorrelated and uncorrelated-similar-weights types, MMAS outperforms all contemporary state-of-the-art approaches, sometimes by more than 10%, confirming that decoupling TSP tour quality from TTP solution quality is a fundamental design principle for effective TTP solvers.

Contribution	The first swarm-intelligence approach for TTP: a MAX-MIN Ant System for the tour, where each ant's tour is scored by its TTP objective (via PackIterative) rather than tour length, with optional 'boosting' hill-climbers ((1+1)-EA on packing, Insertion, BitFlip) on the best solutions per iteration.
Authors' claims	MMAS is best on instances up to about 250 cities and 2000 items, sometimes by more than 10 percent over MATLS and C3-C6; for 250-500 cities C3-C6 lead; for larger instances S5 dominates; exploring longer non-TSP tours is beneficial if done efficiently; boosting helps but is too slow on large instances.
Evidence base	108 instances from the 9720 suite (9 city sizes 51-1000, 2 item factors, 3 KP types, 2 capacities). 10-min budget, 30 repetitions, public code and results. Results reported as fitted degree-6 polynomial trend lines; only Table 1 (2 instances) has per-instance numbers. Approximation ratio against a self-referential best-found value. No statistical test. No MMAS parameter tuning (ACOTSPjava defaults).
Stated limitations	Authors state MMAS is 'most definitely not one approach to rule them all', boosting is too time-consuming on larger instances, the TTP search space is 'incredibly hard to navigate', no parameter tuning was done, and suggest focusing on small instances and instance-feature analysis.

Strengths. ✓ First ACO / swarm approach for TTP, conceptually motivated (a distribution of tours vs a single current tour) • ✓ Reasonably broad instance set (108, 9 city sizes, full KP-type coverage) • ✓ 30 repetitions, consistent 10-min budget, public code and results • ✓ Honest, well-scoped claims about where MMAS wins and loses • ✓ Confirms the longer-non-TSP-tours-help thesis with a working method

Weaknesses. ✗ Results reported only as fitted degree-6 polynomial trend lines; almost no per-instance data, so the 'beats MATLS by >10 percent' claim is hard to verify or reuse • ✗ No statistical significance testing • ✗ Approximation ratio uses a self-referential best-found baseline • ✗ No MMAS parameter tuning • ✗ The win region is narrow (up to ~250 cities) and the method loses badly on large instances • ✗ Boosting, the component that makes it competitive, does not scale

[16] Wu et al. (2016) — On the Impact of the Renting Rate for the Unconstrained Nonlinear Knapsack Problem

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	renting-rate analysis	Family (L1/L2)	Problem def./benchmark/analysis / Analysis
Benchmark	—	Size	Cities: 51; Items: 1x KP-Types: bsc; uco; usw
Metric	non-trivial items rate (instance hardness proxy), and failure rate of the (1+1) EA vs. exact MIP.	Stat. test	none
Key result	only 3 of 12 R values ($10^1, 10^2, 10^3$) yield hard instances (high ~ 0.8 , EA failure rate >0 , peak MIP runtime); a theoretically constructed instance is given on which the (1+1) EA fails with constant probability (exponential expected optimisation time).		
Competition	N	Code	No
Budget / Runs	— / 5 (EA), 20 (EA); max iters 10^4 - 10^5	Hardware	—

Abstract. Wu et al. (2016) investigated the structural role of the **renting rate** R in the TTP, analyzing how its value modulates instance difficulty for the PWT subproblem. The study demonstrates theoretically that the packing subproblem remains NP-hard even when the knapsack capacity constraint is removed, and provides a preprocessing scheme based on R that eliminates items whose inclusion is guaranteed to reduce profit regardless of the tour. By varying R systematically on a single small instance (51 cities, three KP types), the paper shows how the renting rate controls which items can be profitably included and thereby determines the effective size of the feasible packing space. The construction of instances particularly resistant to simple evolutionary algorithms further highlights how parameter design can be used to generate benchmark instances of controlled hardness.

Contribution	A theoretical and empirical study of the renting rate R in the unconstrained NKP (PWT with non-binding capacity): analytic upper/lower bounds R_L, R_U bounding a 'non-trivial range', a 'non-trivial items rate' λ as an instance-hardness measure, an EA to evolve high- λ instances, a theoretically constructed instance on which (1+1) EA fails with constant probability, and a delta-EA evolving instances that maximise (1+1)-EA failure vs MIP.
Authors' claims	Hard NKP instances lie in R in (R_L, R_U) and specifically where λ is large; only R in $1e1, 1e2, 1e3$ give $\lambda \geq 0.8$ and non-zero (1+1)-EA failure for the tested instance; the constructed instance gives (1+1) EA exponential expected optimisation time (argued, not fully proven); MIP runtime also spikes at those R values.
Evidence base	Analysis plus a single base instance: eil151 (151 cities), one fixed LK tour, $m=250$. λ -EA / delta-EA run 5 times per R value; failure rate over 20 (1+1)-EA restarts vs the MIP result; a 1-million-random-instance comparison. All on this one eil151-derived instance. No standard benchmark and no solver comparison beyond (1+1) EA vs MIP.
Stated limitations	Authors state a formal (1+1)-EA proof would need multi-bit-flip analysis, the general non-trivial range is 'tremendous without much guarantee', and frame the λ threshold as a future mechanism to filter easy instances from benchmarks.

Strengths. ✓ Rigorous, useful contribution: analytic R bounds and the λ hardness measure give a principled way to identify trivial NKP/PWT instances • ✓ Combines theory, instance evolution and runtime experiments • ✓ Directly relevant to benchmark quality: many R settings make instances trivial • ✓ Honest that the (1+1)-EA hardness argument is not a full proof

Weaknesses. ✗ All experiments on a single eil151-derived instance with one fixed tour; the curves are not shown to generalise • ✗ The (1+1)-EA hardness result is asserted, not proven • ✗ Only the unconstrained NKP (non-binding capacity) is studied, a restricted sub-case • ✗ Only (1+1) EA and MIP considered; no practical packing heuristics, so 'hard for simple EAs' says little about real solvers • ✗ The proposed λ -threshold benchmark-filtering mechanism is suggested but never implemented or validated

[17] El Yafrani and Ahiod (2017) — A local search based approach for solving the Travelling Thief Problem: The pros and cons

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	JNB / J2B	Family (L1/L2)	Single-solution metaheuristic / Local search
Benchmark	—	Size	Cities: 51 to 1304; Items: 1x; 10x KP-Types: bsc; uco; usw
Metric	best objective, time (s).	Stat. test	none

Baselines	EA, RLS polyakovskiy2014comprehensive.		
Key result	JNB/J2B beat EA and RLS on many small/medium instances (J2B better quality on small, JNB better quality/runtime ratio on large); performance degrades on large instances (premature convergence or 600s timeout, occasionally negative gains); Lin-Kernighan tour init far outperforms random init, while initial picking-plan choice is negligible (obj std 0.23%).		
Competition	N	Code	Yes
Budget / Runs	600s / 1 (JNB/J2B deterministic), 30 (EA/RLS)	Hardware	Java, Intel Core i3-2370M 2.40GHz, 4GB, Linux
Abstract. El Yafrani and Ahiod (2017) introduced a formal local search framework for the TTP and instantiated it as two iterative neighborhood algorithms. The framework systematically defines neighborhood structures over the joint solution space of tours and packing plans, ensuring that moves are evaluated with respect to the full TTP objective rather than approximations. The paper also conducts a detailed analysis of initialization strategies —comparing random, greedy, and heuristic starts—and shows that the choice of initial solution has a significant impact on the final solution quality, particularly on small and medium instances. Tested on 180 TTP instances ranging from 51 to 1,304 cities with 1x and 10x item densities across all KP types, the proposed solvers find improved solutions for several benchmark instances, confirming the effectiveness of structured local search when the neighborhood is defined over the coupled TTP decision space.			
Contribution	Two deterministic neighbourhood local-search heuristics for TTP1, JNB (adjacent-swap x bit-flip) and J2B (2-opt x bit-flip), with an accumulator/register ‘backup’ technique to partially recover the objective after a move; plus an analysis of why the O(1) delta trick fails for TTP and an initialisation ablation (tour vs packing). Public code.		
Authors’ claims	The LK initial tour matters a lot while the initial packing plan has very insignificant influence (objective std 0.23 percent); JNB and J2B surpass RLS and EA on many instances, especially small ones; J2B gives better quality on small instances, JNB a better objective/runtime ratio on large; both degrade on mid/large instances through premature convergence or time-out.		
Evidence base	180 instances (13 TSP bases eil51..rl1304, 3 KP types, 2 capacities, 2 item factors). Baselines RLS and EA run on different hardware and software (i7 / Matlab, best-of-30) vs deterministic single runs of JNB/J2B (i3 / Java); 600 s cap; per-instance objective + time tables; negative gains highlighted. No statistical test. On large instances JNB/J2B time out and are matched or beaten by RLS/EA.		
Stated limitations	Authors explicitly frame the paper as ‘pros and cons’: local search exploits rather than explores and suffers premature convergence on large instances; the naive item-picking should be replaced; future work is better item selection, neighbourhood reduction and memetic hybridisation. They state the main purpose is to provide a starting point for future neighbourhood heuristics.		
Strengths. ✓ Explicit, honest ‘pros and cons’ framing; the analysis shows where local search fails • ✓ Clear explanation of why the O(1) delta trick works for TSP/KP but not TTP • ✓ The accumulator/register recovery technique is a concrete, reusable engineering contribution • ✓ 180 instances across a decent parameter grid; public code • ✓ Useful initialisation ablation: tour initialisation dominates packing initialisation			

Weaknesses. ✗ Baselines RLS/EA run on different hardware and software, best-of-30 vs single deterministic run; not like-for-like • ✗ Only RLS and EA as baselines, the weakest in the field; no S5, MATLS, CoSolver2B, MA2B or MMAS • ✗ No statistical significance testing • ✗ The methods are matched or beaten by even RLS/EA on large instances (timeouts, negative objectives), so the claim holds only for small instances • ✗ $O(m n^3)$ worst case for J2B makes the 2-opt variant impractical at scale • ✗ Incremental: the neighbourhoods are standard; the novelty is the recovery bookkeeping and the framing

[18] Karder et al. (2017) — Solving the Traveling Thief Problem Using Orchestration in Optimization Networks

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	Switchover LS	Family (L1/L2)	Single-solution metaheuristic / Local search
Benchmark	—	Size	Cities: 52; Items: 1x; 3x
Metric	best objective, relative distance to best-known.	Stat. test	none
Baselines	best-known qualities faulkner2015approximate, plus SASEGASA.		
Key result	Loot First finds new best on 29/60 (48.3%); Switchover and SASEGASA find new best on 59/60 (98.3%), with Switchover occasionally beating SASEGASA.		
Competition	N	Code	Unknown
Budget / Runs	2h / 1 (best-found)	Hardware	—

Abstract. Karder et al. (2017) proposed **cooperative optimization networks** as a general framework for solving the TTP by decomposing it into interacting specialized solvers. Unlike monolithic metaheuristics, the framework delegates the TSP component to a dedicated tour-optimization solver and the KP component to a specialized packing solver, with an **orchestration mechanism** managing information exchange and solution consistency between the two. The network architecture allows each solver to operate at its best performance level while the orchestrator enforces TTP feasibility and objective improvement. Applied to berlin52 instances with 1x and 3x item densities, the cooperative network discovers new best-known solutions on nearly all tested configurations, providing compelling evidence that structured solver cooperation—when properly orchestrated—can outperform both pure decomposition and monolithic approaches on small-to-medium TTP instances.

Contribution Applies HeuristicLab optimization networks and orchestration to TTP: four network variants (Loot First / Tour First / Loot and Tour, with a CMA-ES meta-optimizer adapting item values or city coordinates and P3 / local search as sub-solvers; and Switchover, with an OSGA meta-optimizer alternating loot and tour optimization).

Authors' claims	New best solutions are found for all selected instances; Loot First finds new best on 29/60, Switchover and SASEGASA both on 59/60; Switchover sometimes beats SASEGASA; problem-instance adaptation 'does not scale well with dimensionality'.
Evidence base	60 instances from the berlin52 subset only: 30 berlin52_n51 (1 item/city) + 30 berlin52_n153 (3 items/city), all about 52 cities. Baselines: best-known values from faulkner2015 plus SASEGASA (re-run). 2-hour budget. Table 1 reports best achieved values (rounded); no mean/std and no run count. No statistical test. Tour First and Loot-and-Tour 'did not perform well in preliminary tests' and were dropped.
Stated limitations	Authors explicitly state problem-instance adaptation does not scale with dimensionality, the meta-optimizer struggles to evolve good adaptation parameters for city coordinates, and Loot-and-Tour is 'even more difficult'; future work is dynamic parameter adjustment, on-the-fly algorithm exchange and dynamic resource allocation.

Strengths. ✓ New conceptual framing: orchestration lets existing solvers (P3, OSGA, local search) cooperate without a bespoke TTP algorithm • ✓ Reports new best-known solutions on the berlin52 subset • ✓ Includes a strong monolithic baseline (SASEGASA) alongside published bests • ✓ Honest that two of the four proposed networks failed and were dropped

Weaknesses. ✗ Evaluated only on about 52-city berlin52 instances; no medium or large instances at all • ✗ No statistical testing; number of runs not reported; Table 1 is 'best achieved' only • ✗ SASEGASA (a general GA) matches or beats the networks on 59/60, so the orchestration machinery is not shown to add value over a good monolithic GA • ✗ Two of the four proposed networks abandoned without analysis • ✗ A 2-hour budget for about 52-city instances is very generous; efficiency not addressed • ✗ Heavy HeuristicLab-specific infrastructure; limited portability

[19] Martins et al. (2017) — HSEDA: a heuristic selection approach based on estimation of distribution algorithm for the travelling thief problem

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	HSEDA	Family (L1/L2)	Hyper-heuristic / EDA-based
Benchmark	—	Size	Cities: 51; 100; 280; 439; 783; Items: 1x; 5x; 10x KP-Types: bsc; uco; usw
Metric	objective, approximation ratio, Vargha-Delaney A-measure.	Stat. test	Friedman + Dunn-Sidak post-hoc (=0.05), Shapiro-Wilk, A-test
Baselines	MMAS, MA2B, S5.		
Key result	HSEDA competitive overall and its edge grows with city count - beats MMAS on a280/pr439/rat783 (worst on small eil51 where MMAS wins), beats S5 on most kroA100; MA2B strong on small instances.		

Competition	N	Code	Unknown
Budget / Runs	600s / 30	Hardware	Matlab, Phoenix HPC, University of Adelaide
Abstract. Martins et al. (2017) proposed HSEDA, an Estimation of Distribution Algorithm-based hyper-heuristic for the TTP that learns to combine low-level heuristics for tour improvement and item selection. The method maintains a probabilistic model over the space of heuristic sequences and uses it to sample promising operator combinations during the search. To reduce the cost of evaluating each new combination, a surrogate model approximates solution quality from structural features of the heuristic sequence, allowing the algorithm to explore a large number of combinations without full TTP evaluations. Comparative experiments on selected instances with 51 to 783 cities and all item densities show that HSEDA is competitive with three state-of-the-art TTP algorithms, and achieves superior performance on most medium-sized instances, demonstrating that automatic combination of domain-specific heuristics via probabilistic modeling is a viable approach for multi-component problems.			
Contribution	HSEDA, an estimation-of-distribution hyper-heuristic that evolves length-6 sequences of 7 known low-level heuristics (KP bit-flip, KP simulated annealing, TSP 2-opt, and disruptive TSP/KP moves). It learns a Bayesian network over heuristic positions and uses a radial-basis-function surrogate to approximate the fitness of sampled combinations and save evaluations.		
Authors' claims	HSEDA is competitive with and outperforms MMAS, MA2B and S5 on most medium-sized instances, and its relative performance improves as the number of cities grows; the Bayesian network, the two operator types and the surrogate are the three key ingredients.		
Evidence base	135 instances from 5 TSP base graphs (eil51, kroA100, a280, pr439, rat783; max 783 cities), 3 KP types, item factors and capacities in 1,5,10. 10-min budget, 30 runs. Parameters tuned offline by irace on a disjoint small-instance set. Statistics: Shapiro-Wilk, Friedman + Dunn-Sidak, and Vargha-Delaney A-test per instance. Aggregate comparison shown as degree-6 polynomial trend lines. Public source code and raw results. Per Table 2, HSEDA beats MMAS/MA2B on the larger graphs (a280, pr439, rat783) but loses to S5 and MMAS on eil51 and kroA100.		
Stated limitations	Authors state experiments were limited to small and mid-size instances because HSEDA trains online (per instance, not transferable) and the 10-min limit makes larger instances hard 'even for S5 and MA2B'; the processes 'have been simplified'; future work is probabilistic inference with prior causal knowledge and stronger ML in the surrogate.		
Strengths. ✓ Proper statistical rigour: Shapiro-Wilk, Friedman + Dunn-Sidak, and A-test, with 30 runs • ✓ irace offline tuning on an instance set disjoint from the test set • ✓ Public source code and raw results • ✓ Novel angle: a Bayesian network over heuristic positions plus an RBF surrogate to cut evaluation cost • ✓ Honest about the online per-instance nature and the limited instance range			
Weaknesses. ✗ Only 5 TSP base graphs (max 783 cities); no large instances and not the categorised 60-subset • ✗ HSEDA loses to S5 and MMAS on the small instances; the improvement over MA2B/S5 in the mid-size range is often only 'comparable' • ✗ Aggregate results shown as degree-6 polynomial trend lines • ✗ Heavy machinery (EDA + Bayesian network + RBF surrogate + irace) for gains that are marginal and confined to the mid-size range • ✗ Online per-instance tuning consumes much of the 10-min budget on learning rather than solving • ✗ No comparison with CS2SA, MATLS or CoCo-style coordination methods			

[20] Moeini et al. (2017) — A Hybrid Evolutionary Approach for Solving the Traveling Thief Problem

Variant	TTP1	Objective / #obj	Single-objective / 1
----------------	------	-------------------------	----------------------

Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	hybrid heuristic	Family (L1/L2)	Single-solution metaheuristic / Hybrid local search
Benchmark	—	Size	Cities: 51; 76 Items: 1x; 3x
Metric	best objective, % change vs. best-known, avg over 10 runs.	Stat. test	none
Baselines	best-known benchmark values wagner2018case		
Key result	hybrid EA improves best-known on eil51_n50 (e.g. +10.4% on 50_07: 9306 vs. 8427; +12.4% on 50_10) and several eil51_n150, but is mostly worse on eil76.		
Competition	N	Code	No
Budget / Runs	10min / 10	Hardware	Matlab, 2.5GHz dual-core

Abstract. Moeini et al. (2017) investigated a **hybrid evolutionary algorithm** for the TTP that combines a genetic search process with local refinement operators tailored to the joint TSP-KP structure. The algorithm is designed to exploit the interdependence between routing and packing by applying item-selection refinement after each tour modification, ensuring that packing plans remain adapted to the current tour throughout the evolutionary search. Evaluated on small instances with 51 and 76 cities and item densities of 1x and 3x, the method improves upon several best-known solution values from the literature on medium-scale instances, demonstrating that careful coordination of evolutionary and local-search components can yield incremental but consistent quality gains, particularly when the instance is small enough for the population to converge on high-quality solutions within the available computational budget.

Contribution	A hybrid EA for TTP combining a GA (PMX, exponential ranking selection, steady-state), 2-opt + bit-flip local search, and variable neighbourhood search with shaking, plus dynamic adjustment of local-search iteration limits and population shaking. VNS with shaking is presented as new for TTP.
Authors' claims	The hybrid improves some best-known solutions and performs 'significantly well' on eil51_n50 instances, while for larger instances it gives 'competitive results that need to be improved'; a secondary analysis relates the used capacity and the constant-speed tour length to the capacity factor.
Evidence base	40 instances from only 2 TSP base graphs (eil51, eil76), item factors 1,3, 10 capacity factors. MATLAB, 2.5 GHz dual-core, 10-min budget, 10 runs. The only baseline is best-known values quoted from Wagner et al. (2017); no head-to-head with any solver. No statistical test. On eil51_n150/n225 and eil76_n225 the EA is 5-13 percent worse than best-known.
Stated limitations	Authors state the algorithm must be improved for large-scale instances and that 'research in this direction is in progress'.

Strengths. ✓ Introduces VNS with shaking as a diversification component for TTP • ✓ Reports improved best-known solutions on small eil51 instances • ✓ Insightful secondary analysis of used capacity and tour length vs the capacity factor • ✓ Dynamic iteration-limit adjustment and population shaking are sensible anti-stagnation mechanisms

Weaknesses. ✗ Only 2 TSP base graphs (≤ 76 cities); 40 tiny instances; nothing medium or large; not the categorised subset • ✗ No comparison with any actual TTP algorithm; only against quoted best-known values • ✗ On the larger instances the EA is 5-13 percent worse than best-known; 'significantly well' holds only for eil51_n50 • ✗ No statistical testing; 10 runs • ✗ MATLAB on a dual-core machine; the 10-min budget is not comparable to the C/C++/Java competitors • ✗ PMX is a weak TSP crossover; the GA core is not strong

[21] Polyakovskiy and Neumann (2017) — *The Packing While Traveling Problem*

Variant	PWT	Objective / #obj	Single-objective / 1
Route	Fixed route	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	PWT exact (NP-hardness)	Family (L1/L2)	Exact methods / ILP / branch-and-bound
Benchmark	—	Size	Cities: 51; 76; 101; Items: 1x; 5x; 10x KP-Types: bsc; uco; usw
Metric	relative gap %, runtime (s), ratio;	Stat. test	—
Key result	pre-processing removes avg 31.6% items (uco), 18.5% (usw); =1000 best balance; MIP [^] best on b-s-corr small-cap, BIB [^] 500 best on uco/usw large-cap; LB [^] 100 solves pla33810 in 30min, pla85900 in 4h with tiny ;		
Competition	N	Code	No
Budget / Runs	— / 1 (exact/deterministic)	Hardware	128GB RAM, 2.8GHz 48-core AMD Opteron cluster, CPLEX

Abstract. Polyakovskiy and Neumann (2017) provided a full theoretical treatment and exact solution methodology for the **Packing While Traveling Problem (PWT)**, extending the preliminary MIP study of Polyakovskiy and Neumann (2015). The paper introduces tighter **lower and upper bounds** using a piecewise linear approximation of the nonlinear objective, and proposes two complementary exact methods: an enhanced MIP with the linearized bounds embedded as cutting planes, and a **Branch-Infer-and-Bound (BIB)** algorithm that combines the MIP upper bound with a constraint programming model enhanced by custom propagators for the nonlinear travel-cost function. Applied to PWT instances with 51 to 101 cities and all three KP types, BIB solves larger instances within the time limit than the pure MIP, demonstrating the value of integrating constraint reasoning into the bounding procedure for nonlinear combinatorial optimization.

Contribution	A definitive journal treatment of Packing While Travelling: NP-hardness proofs for the constrained and unconstrained variants, provably correct compulsory/unprofitable-item pre-processing with dummy-profit propagation and a fast sequencing-constraint acceleration, MIP-based lower and upper bounds via piecewise-linear approximation, and two exact approaches (MIP ^{lambda} and a branch-infer-and-bound hybrid).
Authors' claims	Both PWT variants are NP-hard; pre-processing removes on average 31.6 percent (uncorr) / 13.2 percent (uncorr-similar-weight) of items but fails for bounded-strongly-correlated; MIP ^{lambda} (lambda=1000) now closes small instances on an ordinary PC; the lower bound LB ¹⁰⁰ is near-optimal (ratio approx 1) and fast, handling pla33810 in under 30 min and pla85900 in under 4 h; uncorr-similar-weight instances are hardest for the MIP, unlike classical KP.
Evidence base	Benchmark set B with a fixed CLK tour: eil51/76/101 (m up to 1000) small, pla33810/pla85900 large; IBM CPLEX / CP Optimizer 12.6.2; time limits up to 1 day; small experiments on a commodity PC, large on a 48-core cluster. Metrics: runtime, LB/OPT ratio, auxiliary-variable usage, branches/fails, speed-up. Deterministic. No comparison with heuristic packing on solution quality.
Stated limitations	Authors note pre-processing can 'stultify all benefits' if slow (hence the acceleration), bsc instances resist pre-processing and are hardest, and larger lambda gives 'very limited improvement' at higher time and memory cost; future work is variable-route TTP, skippable cities and pickup-and-delivery.

Strengths. ✓ Rigorous journal treatment: NP-hardness proofs for both variants, provably correct pre-processing, tunable-precision MIP bounds • ✓ MIP^{lambda} now closes small instances on a commodity PC, a major practicality gain over the 2015 version • ✓ LB¹⁰⁰ is a fast near-optimal packing routine scaling to hundreds of thousands of items • ✓ Extensive, careful experiments with per-KP-type breakdown and a large-instance study • ✓ Insightful finding that uncorr-similar-weight (not bsc) is hardest for the MIP

Weaknesses. ✗ No head-to-head comparison with heuristic packing (PackIterative, G1/G2) on quality or speed • ✗ Exact optimality still limited to small instances; large-instance results are approximate with quality judged only by a near-1 ratio, not against true optima • ✗ bsc instances (the hardest, heavily used KP type) are exactly where pre-processing fails • ✗ Heavy dependence on commercial solvers and, for large instances, a 48-core cluster • ✗ Route fixed to a single CLK tour; interaction with tour choice is explicitly deferred • ✗ Very notation-heavy; the many bound and constraint variants make practical guidance hard to extract

[22] Vieira et al. (2017) — A Genetic Algorithm for Multi-component Optimization Problems: The Case of the Travelling Thief Problem

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	MCGA	Family (L1/L2)	Population-based metaheuristic / Memetic algorithm

Benchmark	—	Size	Cities: 195; 783; 3038; Items: 3x; 10x KP-Types: bsc; uco; usw
Metric	mean objective (std) over 30 runs.	Stat. test	none
Baselines	S5 faulkner2015approximate, MIP.		
Key result	MCGA best on 55.56% (20/36) instances (dominant on rat-based 195/783, 20/24), S5 best on 36.11% (13/36, mainly n=3038), MIP best on 8.33% (3/36); MCGA underperforms on n=3038 (time-limited convergence).		
Competition	N	Code	No
Budget / Runs	— / 30	Hardware	2xIntel Xeon E5-2630v3 (32 cores), 128GB RAM, Ubuntu 14.04

Abstract. [Vieira et al. \(2017\)](#) proposed the **Multi-Component Genetic Algorithm (MCGA)**, which begins from a high-quality CLK tour and then applies a genetic evolutionary process with dedicated crossover and mutation operators for both the TSP and KP components. Unlike prior methods that treat the two subproblems sequentially, MCGA evolves complete TTP solutions in which both the tour and the packing plan co-adapt during the genetic search. Evaluated on 36 benchmark instances with 195, 783, and 3,038 cities—spanning two item factors ($F \in \{3, 10\}$), all three KP types, and two capacities ($C \in \{3, 7\}$)—MCGA produces the best mean objective on 20 of 36 instances, with the greatest relative gains on the small/medium ($n \in \{195, 783\}$) problems where the genetic operators have sufficient population diversity and iteration budget to explore the combined solution space effectively.

Contribution	MCGA (Multi-Component Genetic Algorithm), a whole-problem GA with component-specific encodings and operators: permutation tour (CLK-initialised) with order-based crossover + 2-opt mutation, binary packing with N-point crossover + bit-flip, tournament selection, elitism and a p_i/w_i feasibility repair; public GitHub code.
Authors' claims	MCGA obtains competitive solutions on 20 of 24 instances with 195 and 783 cities, and better averages on 55.56 percent (20/36) of instances; it underperforms clearly on 3038-city instances, which is linked to TSP-component characteristics; S5 is best on 13/36, MIP on 3/36.
Evidence base	36 instances (subset of faulkner's 72): 3 city sizes 195, 783, 3038, item factors 3,10, 3 KP types, 2 capacities. Baselines S5 and MIP quoted from faulkner2015, not re-run. C1 config, 10-min runtime, parameters from 'a simple empirical study', 30 runs (mean+/-std) for MCGA, single values for S5/MIP. No statistical test. Xeon E5-2630 v3 (1 core).
Stated limitations	Authors state MCGA clearly underperforms on 3038-city instances, its performance is visibly linked to TSP-component characteristics, time is not the only feature needing improvement, and propose new evaluation strategies, component-specific operators and a parameter study.

Strengths. ✓ Whole-problem GA with thoughtfully component-specific encodings and operators • ✓ Public source code (GitHub), above the norm for reproducibility • ✓ 30 runs with standard deviation for MCGA • ✓ Honest about the 3038-city failure and its link to TSP structure

Weaknesses. ✗ Only 36 instances, max 3038 cities; the ‘competitive’ claim really holds only for the 195/783-city rat instances • ✗ Baselines S5/MIP quoted, not re-run; only 2 baselines; no MATLS, CoSolver, MMAS or MA2B • ✗ No statistical significance testing despite 30 runs • ✗ Parameters from ‘a simple empirical study’, not systematic tuning; only 2 configurations • ✗ 2-opt used as a low-probability mutation rather than a proper local search; the GA is weak on the TSP component, exactly its failure mode • ✗ Strongly negative objectives on 3038-city uncorrelated instances indicate poor absolute quality

[23] Wu et al. (2017) — *Exact Approaches for the Travelling Thief Problem*

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	B&B, DP, CP, DP-S1/S5	Family (L1/L2)	Exact methods / Branch-and-bound / DP / CP
Benchmark	—	Size	Cities: 5 to 20; Items: 4 to 19 KP-Types: bsc; uco; usw
Metric	optimality gap (%) to the exact optimum, and runtime (s).	Stat. test	none (gap/#opt summary)
Baselines	MA2B, CS2B, CS2SA, S1, S5, C5, DP-S1, DP-S5.		
Key result	MA2B closest to optimal (avg gap 0.3%, optimal on 312/432, within 1% on 265); DP-S5 avg gap 3.3% (245 within 1%); multiple-strongly-correlated instances are hardest and the only ones where heuristics rarely reach the optimum; a small fraction of instances show a gap >10%.		
Competition	N	Code	No
Budget / Runs	24h for the exact-vs-exact comparison, 10 min for the DP-vs-approximate comparison / 10 (approximate algorithms)	Hardware	Phoenix HPC, University of Adelaide

Abstract. Wu et al. (2017) developed **three exact algorithms and a hybrid solver** for small TTP instances to establish a ground-truth reference for evaluating the accuracy of heuristic methods. The exact methods employ branch-and-bound enumeration, dynamic programming, and a hybrid that combines both, applied to adapted instances of the eil51 TSP benchmark reduced to 5 to 20 cities. By comparing heuristic solutions from the literature against these provably optimal results, the paper quantifies the **optimality gap** of state-of-the-art TTP heuristics and identifies instances on which existing methods perform particularly poorly. The work emphasizes that without exact reference values, the TTP community cannot assess how much algorithmic improvement remains achievable, especially for structured instances with strong correlations between item weights and values.

Contribution	Three exact TTP algorithms (Held-Karp-based dynamic programming with an inner PWT DP and an E_U pruning bound, branch-and-bound, and a constraint-programming model) plus DP-S1/DP-S5 hybrids; the purpose is to produce optimal reference values to audit heuristic accuracy on small instances.
Authors' claims	DP scales furthest, BnB solves up to n about 14 and CP up to n about 11 in 24 h; MA2B dominates the heuristics (avg gap 0.3 percent, optimal on 312/432, within 1 percent on 265/432); C5 beats S1/S5; DP-S5 packing beats PackIterative; the multiple-strongly-correlated KP config is hardest; most heuristics reach near-optimal but a few exceed 10 percent gap.
Evidence base	A new eil51-subsample set (n=5..20, m=n-1, k in 1,5,10, capacity in 1,6,10, 3 KP types); 432 instances for the heuristic-vs-optimum comparison, 24-h exact limit, 10-min / 10-repetition heuristics, gap = (OPT-Obj)/OPT. Three exact methods cross-validate the optima. No statistical test (gaps to a true optimum). DP runs out of memory on some n=19-20 instances.
Stated limitations	Authors note DP memory is $O(2^n nC)$, exact methods work only for small instances, and that for a small fraction of instances the heuristic gap exceeds 10 percent.

Strengths. ✓ Provides genuine optimal reference values for TTP, enabling absolute (not relative) accuracy assessment • ✓ Three independent exact methods cross-validate the optima • ✓ Audits 8 heuristic solvers against the optima on 432 instances, a rare like-for-like check • ✓ The E_U bound and DP-Sx hybrids are reusable • ✓ Honest scatter plot showing heuristics can exceed 10 percent gap on some instances

Weaknesses. ✗ Optima only for very small instances (n ≤ 20, mostly ≤ 14); says nothing about accuracy at benchmark scale • ✗ The eil51-subsample instances are non-standard and were not adopted • ✗ Heuristic 'average of 10 repetitions' compared to the optimum as a point estimate; per-instance heuristic reliability not reported • ✗ No statistical testing of the heuristic ranking • ✗ CP and BnB are quickly dominated by DP and add little; the 'three exact approaches' framing oversells two of them • ✗ The 'MA2B dominates' conclusion is on tiny instances only and conflicts with later large-instance findings favouring coordination methods

[24] Alharbi (2018a) — *The Design and Development of a Modified Artificial Bee Colony Approach for the Traveling Thief Problem*

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	mod-ABC	Family (L1/L2)	Population-based metaheuristic / Swarm (ABC)
Benchmark	—	Size	Cities: 51; 76; Items: 1x; 3x KP-Types: bsc; uco; usw
Metric	best objective.	Stat. test	none

Baselines	DH, SH, RLS, EA, MATLS (benchmarks from wagner2018case).		
Key result	ABC beats DH and SH on all instances and RLS/EA on most (all 1-item instances; only 1-2 of the 3-item BSC/USW); beats MATLS on some (e.g. eil51_01_50_1); best on USW (best on 3, 2nd-best on 5); degrades with multiple items per city.		
Competition	N	Code	No
Budget / Runs	— / 30	Hardware	Intel Core i7-4790s 3.2GHz, 12GB, MATLAB R2014a, Win64

Abstract. Alharbi (2018a) proposed a **modified Artificial Bee Colony (ABC)** algorithm for the TTP that adapts the swarm intelligence framework to handle the joint permutation-and-binary decision space. Employee bees exploit the neighborhood of the current tour and packing plan simultaneously, and onlooker bees select promising solutions with probability proportional to their TTP objective value. Scout bees introduce diversity when solutions stagnate. The modification addresses the standard ABC's limitation in handling permutation-based problems by introducing TTP-specific neighborhood operators for both the TSP and KP components. Evaluated on instances with 51 and 76 cities across all item densities and KP types, the algorithm achieves competitive performance against recent state-of-the-art methods, demonstrating that swarm intelligence frameworks can be effectively adapted to multi-component combinatorial problems with appropriate problem-specific modifications.

Contribution	A modified artificial bee colony algorithm for TTP: CLK + SH initialisation, employed bees do 2-opt tour mutation, onlooker bees do insertion crossover (delaying pickups) plus PackIterative re-packing, scout bees restart with a new CLK tour + SH + BitFlip.
Authors' claims	The modified ABC is competitive with the state of the art, especially on small and medium instances; it beats DH and SH on all tested instances, beats RLS and EA on most, and beats MATLS on several one-item-per-city instances; performance degrades on larger multi-item instances because the hill-climb packing needs longer.
Evidence base	Only eil51 and eil76 base graphs (≤ 76 cities, ≤ 225 items); 12 instances $\times$ 3 KP types (Tables 2-4). All baselines (MATLS, DH, RLS, EA, SH) quoted from Wagner et al. (2017), none re-run. 30 runs, i7-4790s, MATLAB R2014a, colony 100, 200 cycles. Only best objective reported; no mean/std; no statistical test. MATLS wins most of the multi-item and eil76 rows.
Stated limitations	Authors state performance is reduced on larger multi-item instances because of the larger search space and the hill-climb packing, and propose a parameter-tuning study, a better packing mechanism and hybridisation with tabu search or simulated annealing as future work.

Strengths. ✓ First (modified) bee-colony approach for TTP, with the three bee roles adapted sensibly to TTP structure • ✓ 30 runs on standard hardware • ✓ Honest about the degradation on multi-item / large instances and its cause • ✓ Detailed problem-representation section

Weaknesses. ✗ Only eil51 and eil76 (≤ 76 cities); 12 instances; no medium/large; not the categorised subset • ✗ All baselines quoted from Wagner et al. (2017), none re-run; best-vs-quoted comparison is not like-for-like • ✗ Only best objective reported; no mean/std despite 30 runs; no variance • ✗ No statistical test • ✗ ABC clearly beats only the weak baselines and only the smallest instances; MATLS wins most rows, so 'competitive with state-of-the-art' overstates it • ✗ MATLAB implementation; budget/runtime not reported or compared • ✗ Parameters not tuned (acknowledged)

[25] Alharbi (2018b) — A Hybrid Genetic Algorithm with Tabu Search for Optimization of the Traveling Thief Problem

Variant	TTP1	Objective / #obj	Single-objective / 1
----------------	------	-------------------------	----------------------

Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	GATS	Family (L1/L2)	Population-based metaheuristic / Hybrid (GA+TS)
Benchmark	—	Size	Cities: 51; 52; 76; 100; 150; Items: 1x; 3x; 5x KP-Types: bsc; uco; usw
Metric	best objective (normalized).	Stat. test	none
Baselines	EA, RLS (also DH/SH); CLK vs. random tour init.		
Key result	GATS beats EA and RLS at all capacities on the 51/76/100-city U instances; weak only on the 52-city (berlin52) set (better on just 2, medium-capacity); CLK init yields tours >50% shorter than random.		
Competition	N	Code	No
Budget / Runs	10min / 10	Hardware	Intel Core i7-4790S 3.20GHz, 12GB, MATLAB R2014a, Win8

Abstract. Alharbi (2018b) proposed **GATS**, a hybrid algorithm combining a **Genetic Algorithm (GA)** with **Tabu Search (TS)** for the TTP. Unlike decomposition approaches that apply GA to one component and TS to the other, GATS applies all operators—including crossover, mutation, and tabu moves—to the joint tour-and-packing representation simultaneously, ensuring that genetic variation always produces TTP-feasible offspring that fully reflect the interdependence between routing and collection. Different initialization strategies are analyzed on 540 TTP instances spanning 51 to 150 cities with 1x, 3x, and 5x item densities across all three KP types. GATS outperforms several state-of-the-art approaches on numerous benchmark configurations, confirming the benefit of **coordinated joint operator application** over sequential subproblem optimization when the genetic diversity is maintained and the tabu tenure is appropriately calibrated.

Contribution	GATS, a GA + tabu-search hybrid for TTP: a GA with OX crossover, opt-mutation/insertion and roulette selection handles the tour (with PackIterative per tour), and a tabu search over the packing plan (bit-flip neighbourhood, tabu list of visited plans) is called after each generation; also a random-vs-CLK tour-initialisation study.
Authors' claims	GATS surpasses EA and RLS on various instances, mainly with large-capacity knapsacks; CLK initialisation is significantly better than random (tour > 50 percent shorter); GATS 'struggles' on berlin52 and its performance decreases with more items and cities, particularly at small capacity.
Evidence base	540 instances = 5 city groups (max 150 cities) x item factors 1,3,5 x 3 KP types x 10 capacities. Baselines EA and RLS only, quoted from Wagner et al. (2017). 10-min budget, 10 runs, i7-4790S, MATLAB R2014a. Tables IV-XII report best objective only; no mean/std; no statistical test (superiority stated qualitatively, e.g. 'only 1 percent higher'). No comparison with S5, MATLS, CS2SA, MA2B, CoSolver2B or MMAS.

Stated limitations Authors explicitly state GATS' performance significantly decreased with more items and cities and small capacity, that it struggled on berlin52 'in almost all instances', and that these issues and larger instance sets should be investigated in future.

Strengths. ✓ First GA + tabu-search hybrid for TTP, with tabu search specialised to the packing plan • ✓ Large instance count (540) across a systematic $5 \times 3 \times 3 \times 10$ grid • ✓ Clearly reported CLK-vs-random tour-initialisation ablation confirming tour init dominates • ✓ 10 runs on standard hardware

Weaknesses. ✗ Only EA and RLS as baselines, both among the weakest in the field; no S5, MATLS, CS2SA, MA2B or MMAS, so 'surpasses the state-of-the-art' is unsupported • ✗ All instances ≤ 150 cities; no medium/large; not the categorised 60-subset • ✗ Only best objective reported across 540 instances; no mean/std despite 10 runs • ✗ No statistical testing; superiority claims are qualitative ('only 1 percent higher', 'not substantial') • ✗ GATS is beaten by EA/RLS on medium-capacity, on berlin52 and on multi-item uncorrelated instances; wins are confined to large-capacity, ≤ 100 -city instances • ✗ MATLAB implementation; 10-min budget not comparable to the C/Java competitors whose values are quoted • ✗ Enormous results tables with no aggregation, statistics or win/loss summary

[26] Araujo et al. (2018) — *A novel List-Constrained Randomized VND approach in GPU for the Traveling Thief Problem*

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	LCRVND (GPU)	Family (L1/L2)	Single-solution metaheuristic / Local search (GRASP/VND, GPU)
Benchmark	—	Size	Cities: 51 to 100; Items: 1x; 3x; 5x; 10x KP-Types: bsc; uco; usw
Metric	avg objective (3 runs), time (s), gap% vs. baseline.	Stat. test	none
Baselines	Oliveira et al. tabu search		
Key result	LCRVND beats the literature on most instances at lower runtime - reached the target faster on 54/60 (avg 3.84min vs. 10min); premature convergence on rat99n98/rat99n294.		
Competition	N	Code	No
Budget / Runs	— / 3	Hardware	Intel Core i7-4820K 3.70GHz, NVIDIA GeForce GTX 780 (2304 CUDA cores), C++ CUDA+OpenMP

Abstract. Araujo et al. (2018) proposed a **hybrid GRASP** approach for the TTP that incorporates a novel **list-constrained local search** inspired by Variable Neighborhood Descent (VND), in which multiple neighborhood structures are explored in a fixed order and the search moves to the next structure only when the current one is locally optimal. A key innovation is the use of **GPU parallelism** to evaluate large numbers of neighboring solutions simultaneously during the local search phase, while the GRASP construction and path-relinking phases are parallelized on a multi-core CPU. Tested on instances with 51 to 150 cities and all KP types, the method achieves promising solution quality with reduced computation times compared to sequential implementations, illustrating the potential of high-performance computing architectures for scaling TTP local search to larger instances.

Contribution	LCRVND, a list-constrained randomized variable-neighbourhood-descent local search over 8 neighbourhoods (4 tour, 4 item) that keeps a size-limited list of best solutions and expands their neighbours in parallel on a GPU (CUDA dynamic parallelism), wrapped in a GRASP metaheuristic.
Authors' claims	LCRVND reaches better solutions than the literature (a tabu search by Oliveira et al.) on most instances in lower computational time; it converges in under 1.5 minutes on rat99 instances but 'not with better results, indicating premature convergence'; the list size is a very sensitive parameter; parameters were not fine-tuned (prototype).
Evidence base	60 instances from the Bonyadi combinatorial set (mixed bases, about 50-1000 cities). The only baseline is one non-mainstream Portuguese-language SBPO 2015 tabu-search paper (Oliveira et al.), values quoted. 3 executions per instance, average value + time. GTX 780 GPU vs an i7 CPU tabu search. Percentage gap vs Oliveira only. No comparison with S5, MATLS, CS2SA, MA2B or CoCo. No statistical test. Table 1 shows large positive gaps (up to +43 percent) on several mid/large instances.
Stated limitations	Authors state it is a prototype with untuned parameters (list size), that premature convergence occurs on rat99 instances, and that future work is improving search efficiency and better GPU/multi-core use.

Strengths. ✓ Novel angle: GPU-parallel simultaneous exploration of multiple neighbourhoods via CUDA dynamic parallelism • ✓ Reports faster convergence (avg 3.84 vs 10 min) on 54/60 instances • ✓ Honest that it is an untuned prototype and that premature convergence occurs • ✓ 8 neighbourhood structures over both components

Weaknesses. ✗ The only baseline is a single non-mainstream tabu-search paper; no comparison with any standard TTP solver despite availability • ✗ Results are mixed: LCRVND loses to even that one baseline on several mid/large instances by +13 to +43 percent, contradicting the 'better on most instances' framing • ✗ Only 3 runs per instance; no statistical test; no variance reported • ✗ List size acknowledged as highly sensitive but never tuned or swept • ✗ GPU-vs-CPU speed comparison on different hardware is not a controlled comparison • ✗ Very short paper; the method and results are underdeveloped • ✗ No absolute quality context (best-known gaps)

[27] El Yafrani and Ahiod (2018) — *Efficiently solving the Traveling Thief Problem using hill climbing and simulated annealing*

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single

Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	CS2SA / CS2SA*	Family (L1/L2)	Single-solution metaheuristic / Simulated annealing
Benchmark	—	Size	Cities: 76; 100; 130; 159; 280; 574; 724; 1000; 1304; 1577; 2103; 3038; 4461; 7397; 11849; 13509; 14051; 15112; 18512; 33810 Items: 1x; 5x; 10x KP-Types: bsc; uco; usw
Metric	mean objective, relative std (rsd %), time (s);	Stat. test	Vargha-Delaney A-measure (Table~8: CS2SA* vs. S5 =0.1/0.6/0.6 on Cat1/2/3; CS2SA* vs. MATLS =0.5/0.6/0.6)
Baselines	MATLS, S5, CS2SA-R;		
Key result	CS2SA* best on most mid/large instances (m>1000, Cat2/Cat3), S5 best on Cat1; e.g. pla33810-Cat3 CS2SA* =5.81!x!10^7 vs. MATLS 5.80!x!10^7, S5 5.80!x!10^7;		
Competition	N	Code	Yes
Budget / Runs	600s / 10	Hardware	Java (CS2SA), C++ (MATLS)

Abstract. El Yafrani and Ahiod (2018) introduced **CS2SA*** and **CS2SA-R**, two refined versions of the single-solution CS2SA algorithm that incorporate **offline instance-based parameter tuning** via the `irace` automatic configuration framework. Rather than using fixed SA cooling schedules, the algorithms use **spline interpolation** to derive instance-size-specific parameter profiles, adaptively setting the number of SA trials and the cooling rate based on the number of cities and items. Evaluated on the full benchmark with 60 representative instances across all three categories and KP types, CS2SA* and CS2SA-R achieve highly competitive performance, particularly in **Categories B and C** where higher knapsack capacities create a rich item-selection landscape that benefits from the stochastic exploration enabled by SA. This work established the single-solution SA paradigm as a strong baseline for subsequent TTP research.

Contribution	CS2SA* and CS2SA-R, CoSolver-based single-solution heuristics combining steepest-ascent 2-opt hill climbing for the tour and simulated annealing with bit-flip for packing, with several complexity-reduction techniques (Delaunay neighbourhood, objective-value recovery, 2-opt early break, insertion-without-elimination); CS2SA* is <code>irace</code> -tuned with a size-adaptive trials formula, CS2SA-R is a random-restart variant.
Authors' claims	Improving the packing plan has more impact on the objective than improving the tour; CS2SA*, CS2SA-R and S5 surpass MATLS on most instances, especially mid-size and very large; CS2SA* is best for large instances and higher-capacity categories while S5 is best on category 1; CS2SA-R is generally better on small instances; a 2-opt neighbourhood beats random selection for the TSP component.
Evidence base	The categorised 60-instance benchmark: 20 TSP base graphs (76-33810 cities) at 3 fixed category points (Cat1 F1/T3/C1, Cat2 F5/T2/C5, Cat3 F10/T1/C10). 10 runs, 600 s. Baselines MATLS (C++) and S5 (Java) re-run under the same budget. CS2SA in Java, public source code. <code>irace</code> tuning of alpha, T0 and the trials count on 8 size groups. Statistics: only pairwise Vargha-Delaney A-test (Table 8, success rates); no omnibus test. The 'packing matters more than tour' claim rests on a single-instance experiment (Table 1).

Stated limitations Authors state the simple 2-opt still lacks exploration and a more sophisticated TSP heuristic would be preferred, that the heuristics are extremely sensitive to the initial tour, that CS2SA-R lacks proper tuning, that CS2SA* still has drawbacks at low knapsack capacity, and that the strength relies on strong domain assumptions; future work is other multi-component problems and EA hybridisation.

Strengths. ✓ Two well-engineered CoSolver variants with several concrete, clearly explained complexity-reduction techniques • ✓ CS2SA* is a genuine state-of-the-art single-objective method (later a standard RDI baseline) • ✓ Systematic irace tuning with a size-adaptive trials formula • ✓ Public source code; 10 runs with relative standard deviation; MATLS and S5 re-run under an identical 600 s budget • ✓ Ablations: 2-opt vs random selection and an impact-of-component-solvers study • ✓ Honest about the weak TSP-side exploration and the low-capacity weakness

Weaknesses. ✗ No systematic KP-type/capacity sweep beyond the three fixed category points in the main comparison • ✗ Statistical analysis is only a pairwise A-test with a success-rate summary; no omnibus (Friedman) test and no multiple-comparison correction • ✗ Only 2 baselines (MATLS, S5); no CoSolver2B, MA2B, MMAS or the contemporaneous CoCo despite citing them • ✗ The 'packing matters more than the tour' claim is supported by a single-instance experiment plus an argument, not established across instances • ✗ CS2SA-R is essentially CS2SA* without tuning; its inclusion mainly shows tuning helps • ✗ CS2SA*'s advantage is confined to higher-capacity categories; S5 still wins category 1 • ✗ Best mean values only; no best-known or optimal reference, so absolute quality is not situated

[28] El Yafrani et al. (2018) — A hyperheuristic approach based on low-level heuristics for the travelling thief problem

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	GPHS*	Family (L1/L2)	Hyper-heuristic / GP-based

Benchmark	—	Size	Investigates the use of hyper-heuristics for solving the TTP, a multi-component optimization problem combining the TSP and KP. Proposes GPHS*, a Genetic Programming-based hyper-heuristic that performs automatic online heuristic selection to dynamically identify effective combinations of existing heuristics for both problem components. Unlike traditional approaches that separately optimize routing and packing, GPHS* adaptively selects heuristics during the search process to better coordinate the interaction between the TSP and KP components. Experimental evaluations on benchmark TTP instances demonstrate that the proposed approach is competitive with state-of-the-art algorithms, particularly on small and medium-sized instances. Partially following the literature categorization A, B, C of instances. Source code available at https://bitbucket.org/yafrani/gphs-online/src/master/ & Cities: 51; 52; 76; 100; 280; 439; 783 Items: 1x; 5x; 10x KP-Types: bsc; uco; usw
Metric	approximation ratio (avg 10 runs), Vargha-Delaney A-test.	Stat. test	Friedman + Dunn-Sidak ($\alpha=0.05$), Shapiro-Wilk, A-test
Baselines	MA2B, MATLS, S5.		
Key result	GPHS* beats all three for $100 < n < 783$ cities; competitive with MATLS/S5 below 100 cities (MA2B still best on small); S5 best above 783; Friedman+Dunn-Sidak: GPHS* significantly better than S5 on eil51/eil76/kroA100/rat783, better than MA2B on a280.		
Competition	N	Code	Yes
Budget / Runs	10min / 10	Hardware	—
Abstract. El Yafrani et al. (2018) proposed GPHS*, a Genetic Programming-based hyper-heuristic that performs automatic online heuristic selection for the TTP, evolving selection strategies that dynamically pick which low-level heuristic to apply at each step of the search. Unlike offline hyper-heuristics that pre-evolve a fixed strategy, GPHS* adapts its heuristic selection in real time based on the current solution state, learning which operators are beneficial in different regions of the search landscape. Tested on instances from 51 to 783 cities with all KP types, GPHS* achieves performance competitive with hand-crafted state-of-the-art algorithms, particularly on small and medium instances, while requiring no problem-specific tuning. The publicly available source code enables full reproducibility and facilitates comparison with future hyper-heuristic approaches for the TTP.			
Contribution	GPHS / GPHS*, a genetic-programming hyper-heuristic for online low-level-heuristic selection in TTP: a GP tree of connectors and acceptance (if) nodes over 7 heuristic terminals (MATLS bit-flip, CS2SA simulated annealing, Delaunay 2-opt, and three disruptive moves), executed as a sequence on the current solution; offline (GPHS) and online per-instance (GPHS*) variants.		

Authors' claims	GPHS* beats GPHS and a standard one-point-crossover GA on all 7 instance groups; it is statistically better than S5 on eil51/eil76/kroA100, indistinguishable from S5 on berlin52/a280/pr439, and worse than S5 on rat783; MA2B still outperforms all approaches under 100 cities; evolved models tend to start with a disruptive operator, use search heuristics in the middle and always end with local search.
Evidence base	189 instances from 7 representative TSP base graphs (max 783 cities) x 3 KP types x 3 item factors x 3 capacities. Baselines MA2B, MATLS, S5 re-run under an identical 10-min budget on the same machine. 10 runs. Statistics: Vargha-Delaney A-test per instance (Tables 4-10), Friedman + Dunn-Sidak per group (Table 11), Shapiro-Wilk. GPHS* offline-tuned by irace. Approximation ratio against a self-referential best-found value; aggregate comparison as degree-6 polynomial trend lines. Public source for GPHS, GPHS* and the GA baseline.
Stated limitations	Authors call it 'a preliminary attempt', note disruptive operators 'need further improvement', that GPHS* degrades on rat783 because the 10-min limit gives less compute per instance, that 'a fairer comparison would be against S5', and that global optima are unknown 'even for the tiniest instances'; future work is self-tuning of heuristic parameters and testing scalability on larger instances.

Strengths. ✓ Strong statistical protocol: A-test per instance on all 189 instances, Friedman + Dunn-Sidak per group, Shapiro-Wilk, 10 runs • ✓ Compares against 3 genuine state-of-the-art methods, all re-run under an identical 10-min budget on the same machine • ✓ irace offline tuning, anti-bloat parsimony pressure, and a GPHS vs GPHS* vs GA ablation that isolates the benefit • ✓ Public source for GPHS, GPHS* and the GA baseline • ✓ Honest: GPHS* only matches or beats S5 on small/mid instances and is significantly worse on rat783; MA2B still dominates under 100 cities • ✓ Useful descriptive analysis of evolved model structure

Weaknesses. ✗ All 189 instances ≤ 783 cities; no medium or large instances; scalability explicitly untested • ✗ GPHS* is beaten by MA2B on small instances and by S5 on the largest tested (rat783); the net gain is a modest edge over S5 in a narrow range • ✗ Online per-instance GP training consumes much of the 10-min budget on evolution rather than solving, starving the search on larger instances • ✗ Approximation ratio uses a self-referential best-found reference (no optima known even for tiny instances) • ✗ Aggregate comparison shown as degree-6 polynomial trend lines • ✗ The 7 heuristic terminals are all borrowed from MATLS/CS2SA/2-opt; the hyper-heuristic recombines rather than generates operators • ✗ The GA baseline is a deliberately basic one-point-crossover GA

[29] Nieto-Fuentes et al. (2018) — *A Guided Local Search Approach for the Travelling Thief Problem*

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	?	Family (L1/L2)	Problem def./benchmark/analysis / ?
Benchmark	—	Size	Cities: 52 to 4461; Items: 10x

Metric	avg fitness, best-found, mean objective.	Stat. test	Shapiro-Wilk + Levene + ANOVA/Welch/Kruskal-Wallis (=5%)
Baselines	RES (hill-climber + restart), S5, CS2SA, MA2B; best-known.		
Key result	RES_GLS finds 3 new best-known (benefits concentrated on smaller instances, e.g. fnl4461_unc =6,744,903); RES_GLS vs. RES roughly even; MA2B best on small (a280_unc =429,099), S5 best on large; CS2SA did not improve.		
Competition	N	Code	Yes
Budget / Runs	12h, restart every 5min (600s limit removed in the 3rd competition) / 30	Hardware	—

Abstract. Nieto-Fuentes et al. (2018) applied **Guided Local Search (GLS)** to the tour-optimization component of the TTP, using penalty terms on tour edges that are repeatedly included in locally optimal solutions but do not improve the TTP objective. By adding penalties to features associated with poor TTP performance, GLS systematically diversifies the tour search beyond the narrow region explored by standard 2-opt hill-climbers. Evaluated on instances from 52 to 4,461 cities with 10 items per city, the method generates new best-known solutions on several benchmark instances, with the greatest improvements on small and medium-scale problems. A critical observation is that several state-of-the-art methods are **over-specialized to a narrow subset of near-optimal TSP tours**, and that GLS's ability to escape this region by penalizing repeated structure is the main driver of its superiority, despite its higher computational cost and limited scalability to very large instances.

Contribution	RES_GLS, which applies guided local search to the tour-optimization stage of a CoSolver-style framework (penalising unfavourable edges in a modified fitness), alternating 2-opt with bit-flip and adding S5-style multi-start restarts; presented as the first trajectory metaheuristic to focus on the tour.
Authors' claims	RES_GLS gives competitive average fitness versus the state of the art and finds new best-known solutions on 3 instances; the dependency on the initial tour remains large so restarting is mandatory; benefits are mainly on small/medium instances; on the largest instances too few restarts are triggered and it underperforms; the standard 600-second budget is too restrictive (12 h is used).
Evidence base	12 instances only (6 TSP base graphs at max capacity, item factor 10, 2 KP types). 12-hour budget, 5-min restart criterion, 30 repetitions. Statistics: Shapiro-Wilk, Levene, then ANOVA/Welch or Kruskal-Wallis, alpha=5 percent. Baselines CS2SA, MA2B, S5 re-run from source under the same extended stopping criterion. Public source code and an updated best-known archive. RES_GLS wins about 5/12 instances; S5 wins all the large ones.
Stated limitations	Authors explicitly state the large computational requirements drastically reduce the number of restarts on large instances, so only a few regions are explored and state-of-the-art results could not be obtained there; future work is complexity reduction, population-based integration and parallelisation.

Strengths. ✓ Rigorous statistical protocol: 30 runs, Shapiro-Wilk + Levene + ANOVA/Welch/Kruskal-Wallis with a clear superiority definition • ✓ Fair comparison: competitors' code re-run under the same extended stopping criterion, not quoted • ✓ Novel and well-motivated: guided local search is the first tour-stage diversification for TTP, with an explicit RES vs RES_GLS ablation • ✓ Reports new best-known solutions and publishes code plus a best-known archive • ✓ Honestly documents the restart-starvation failure mode on large instances • ✓ Makes the useful point that the 600-second competition budget is too tight

Weaknesses. ✗ Only 12 instances (6 graphs, single capacity/item-factor); not the categorised 60-subset • ✗ Requires a 12-hour budget; under a realistic budget the restart mechanism starves and the method loses to S5 on every large instance • ✗ The guided-local-search benefit is confined to small/medium instances • ✗ The ablation is ambiguous: RES_GLS wins 4 and RES wins 5 of the head-to-head instances, so GLS does not clearly dominate even its own ablation • ✗ Penalty weight and restart-time parameters set without a tuning study • ✗ New best-known solutions obtained with 12-hour budgets are not comparable to standard-budget results

[30] Przybylek et al. (2018) — *Decomposition Algorithms for a Multi-hard Problem*

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	CoSolver-H / MCTS / ACO	Family (L1/L2)	Population-based metaheuristic / Decomposition (MCTS/ACO)
Benchmark	—	Size	Cities: 3 to 76; Items: 10 to 146
Metric	profit, quality $Q\% = (P - P^{\#}) / (P^* - P^{\#})$ (100% = optimal, 0% = random), Gain = $P - P^{\#}$ for large.	Stat. test	95% confidence intervals
Key result	CoSolver-H and MCTS never produce bad solutions; CoSolver-H best (known-graph avg Gain 648 > MCTS 324 > ACO 161); MCTS beats ACO on most; original CoSolver/CoSolver-E can be misled (negative Q on greedy-proof instances); CoSolver-H greatly improves scalability.		
Competition	N	Code	Yes
Budget / Runs	— / 16 (ACO/MCTS, 95% CI)	Hardware	—

Abstract. Przybyłek et al. (2018) introduced the concept of **multi-hardness** to formalize the difficulty arising from the interaction of multiple NP-hard subproblems in a single integrated optimization problem. The paper argues that existing TTP-specific methods lack generalizability because they exploit problem structure without modeling the underlying source of difficulty. To address this, several **decomposition-based algorithms** are proposed for an abstract multi-hard problem and compared against heuristic benchmarks on self-generated instances ranging from 3 to 76 cities. The study provides a theoretical framework for understanding why independently strong subproblem solvers often fail in an integrated setting and establishes formal measures of multi-hardness that could guide the design of more robust decomposition strategies for future multi-component problems.

Contribution	A conceptual framework of 'multi-hard' problems with TTP as the model case: a formal decomposition into TSKP + KRP with gap-preserving reductions (no PTAS for TSKP unless P=NP; an FPTAS for KRP), decomposition algorithms (original CoSolver, CoSolver-E with exact KRP, CoSolver-H fully heuristic), two non-decomposition heuristics (Monte-Carlo tree search and ant colony optimization for TTP), an exact branch-and-bound reference solver, and a public benchmark generator with controllable coupling.
Authors' claims	CoSolver-H and MCTS never produce bad solutions and CoSolver-H beats the original CoSolver on average (heuristic components are 'more stable'); the original CoSolver and CoSolver-E can be misled to worse-than-random solutions on greedy-proof instances; MCTS beats ACO in most cases; decomposition/negotiation becomes less effective as component coupling rises, with MCTS relatively better than CoSolver-H on highly coupled instances.
Evidence base	Own generated instances, 3-76 cities and 10-146 items for the absolute-quality measure $Q = (P - P\#)/(P^* - P\#)$ with P^* the exact optimum and $P\#$ the average of 1000 random solutions; for larger TSPLIB-derived graphs only 'Gain over the random average' is reported. Stochastic methods run 16 times with 95 percent confidence intervals and error bars. No omnibus statistical test. Not the polyakovskiy2014 9720-suite. Journal marked (c)2015, so no comparison with S5, MATLS, CoSolver2B, MMAS or CS2SA.
Stated limitations	Authors state further work is needed to design algorithms that better exploit problem structure, that very high coupling makes negotiation less effective and 'better methods may still be discovered', and that the long-term goal is a multi-hard methodology validated in an industrial environment.

Strengths. ✓ Rare conceptual contribution: formalises multi-hardness and gap-preserving TSKP/KRP decomposition with two proven complexity results • ✓ Introduces and publicly releases a benchmark generator with controllable coupling / sequentiality • ✓ Compares six methods against true optima on small instances (a genuine absolute-quality measure) • ✓ 16 runs for stochastic methods with 95 percent confidence intervals and error bars • ✓ Well-supported finding that decomposition/negotiation degrades as component coupling rises, quantified across several coupling levels • ✓ Honest that CoSolver can be misled to worse-than-random on greedy-proof instances

Weaknesses. ✗ Absolute-quality results only on tiny instances (3-76 cities); large-instance results use only 'gain over a random average', a very weak reference • ✗ No omnibus statistical test; comparisons rest on averages and confidence intervals read off plots • ✗ Uses its own non-standard instances, not the 9720-suite, so results are not comparable to the mainstream literature • ✗ MCTS and ACO are basic and untuned; the head-to-head wins are on the authors' own implementations • ✗ By its 2018 publication date it does not compare against S5, MATLS, CoSolver2B, MMAS or CS2SA • ✗ The exact branch-and-bound solver is not characterised (size limits, runtimes)

[31] Wagner et al. (2018) — A case study of algorithm selection for the traveling thief problem

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	SATzilla-style selection	Family (L1/L2)	Learning-based / Algorithm selection
Benchmark	—	Size	Cities: 51 to 85,900; Items: 1x; 3x; 5x; 10x KP-Types: bsc; uco; usw
Metric	normalized objective score (1.0 = optimal).	Stat. test	—
Key result	best selectors close 90% of the single-best oracle gap; S5 has highest standalone perf & Shapley value but only 2nd marginal contribution, while CS2SA has lowest standalone but highest marginal contribution; selection frequency S5 0.38, M4B 0.16, M3/M3B 0.12, CS2SA 0.09.		
Competition	N	Code	Unknown
Budget / Runs	— / —	Hardware	—

Abstract. [Wagner et al. \(2018\)](#) presented the first **large-scale algorithm selection and portfolio construction study** for the TTP, assembling a comprehensive dataset of the performance of 21 TTP algorithms evaluated on all 9,720 benchmark instances. The study defines 55 **instance features** describing geometric, statistical, and combinatorial properties of TTP instances, and uses these features as input to train per-instance algorithm selectors that predict which algorithm will perform best on an unseen instance. A portfolio constructed from a subset of complementary algorithms is shown to **significantly outperform the single best standalone algorithm** in terms of average solution quality across the benchmark. Analysis of portfolio composition reveals that no single algorithm dominates across all instance types, and that algorithm complementarity—particularly between methods specialized for Category A and those effective in Categories B and C—is the primary driver of portfolio performance.

Contribution	The first algorithm-selection / portfolio study for TTP: a public benchmark of 21 TTP algorithms run on all 9720 instances on identical hardware and budget, 55 instance features, the first TTP algorithm portfolios (SATzilla'09/'11, ISAC, 3S selectors in ASlib format), and a complementarity analysis via Spearman clustering, marginal contribution and Shapley values.
Authors' claims	The single best algorithm is S5 (scaled performance 0.959) and a perfect per-instance oracle would score 1.0; SATzilla'11-style selection reaches 0.993, closing about 90 percent of the gap; only 5 definitional features (capacity, renting ratio, item count, KP type, city count) matter and give near-oracle selection at essentially zero feature-computation cost; there is no single best algorithm; CS2SA has the worst average performance but the highest marginal (Shapley) contribution.

Evidence base All 9720 instances x 21 algorithms, 10-min budget, Xeon E5430, Java 1.8. Performance = per-instance objective linearly rescaled to [0,1] (best of the 21 = 1), with timeouts/crashes scored -1. FlexFolio selectors, 10-fold cross-validation, ASlib scenario 'TTP-2016' released publicly. No statistical test on the selector comparison. Feature importance via Gini importance across pairwise random forests. MATLS unsolved on 204 instances, the M-series on 700-1342 each, CS2SA crashes on 6284.

Stated limitations Authors explicitly state the instance set contains many small instances, biasing the comparison toward algorithms good on small instances; that the 10-min budget is 'rather artificial'; that the [0,1] rescaling gives 'only a first indication'; and that most of the 55 features are too expensive (hours) for large instances. Future work: TTP-specific features, a representative instance subset, and variable-budget multi-algorithm selection.

Strengths. ✓ The first comprehensive uniform benchmark: 21 algorithms x all 9720 instances on identical hardware and budget, released publicly (ASlib TTP-2016) • ✓ Rigorous algorithm-selection methodology (multiple selectors, 10-fold CV, ASlib reproducibility) • ✓ The key finding (5 header features give near-oracle selection with no feature computation) is clean and actionable • ✓ Sophisticated complementarity analysis (Spearman clustering, marginal contribution, Shapley values) revealing CS2SA's hidden value and S5's portfolio role • ✓ Honest about the small-instance bias and the artificiality of the 10-min budget • ✓ Confirms empirically that there is no single best TTP algorithm

Weaknesses. ✗ The [0,1] per-instance rescaling makes 'performance' pool-relative; absolute quality vs true optima is not assessed and the numbers depend entirely on which 21 algorithms are in the pool • ✗ Scoring timeouts/crashes as -1 heavily penalises MATLS, the M-series and especially CS2SA (6284 crashes), distorting the rankings; the authors note CS2SA's issues are implementation bugs • ✗ The 21-algorithm pool stops at 2016, predating CoCo, CS2SA* and all later state of the art; the headline 0.993 is relative to an outdated pool • ✗ No statistical testing of the selector comparison or the feature-subset results • ✗ The acknowledged small-instance bias means the aggregate rankings over-weight instances few researchers target • ✗ 47 of the 55 features are shown to be prohibitively expensive for large instances, undercutting most of the feature-engineering contribution

[32] Ali et al. (2019) — A novel differential evolution mapping technique for generic combinatorial optimization problems

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	BMV-DE	Family (L1/L2)	Population-based metaheuristic / Differential evolution
Benchmark	—	Size	Cities: 51; 52; 70; 76; 99; 100; 101; 105; 107; 150; 198; 280; 783 Items: 1x KP-Types: bsc
Metric	average error AE% (best/mean/worst), STD, time (s).	Stat. test	Wilcoxon signed-rank - BMV significantly better on all 12 TSPs (p<0.002), statistically tied (~) on KPs and TTPs

Key result on TTPs BMV best avg AE% =10.50% (vs. TP 18.59, Set-based 15.31, LRV 17.45); on TSP BMV improves DE best/mean/worst by 17.5/27.7/32% (lowest avg error 359.80); on KP not significantly different.

Competition N **Code** No

Budget / Runs — / 30 (5000 max FEs) **Hardware** —

Abstract. Ali et al. (2019) proposed a new **mapping mechanism for Differential Evolution (DE)** that enables the continuous-domain DE framework to solve combinatorial optimization problems with permutation and binary decision variables. The mapping converts real-valued DE trial vectors into valid TSP tours and binary knapsack solutions by ranking and thresholding operations, while a best-solution-guided repair strategy drives the population toward high-quality regions of the discrete space. Compared against eight competing DE mapping techniques on the TSP, KP, and TTP—using instances with 51 to 783 cities and bounded strongly correlated items—the proposed method achieves the lowest average optimality gap on the TSP and competitive performance on TTP, demonstrating that the quality of the continuous-to-discrete mapping function is a critical factor in DE-based approaches to combinatorial optimization.

Contribution BMV (Best-Matched Value), a discretisation/mapping method for differential evolution on permutation and binary combinatorial problems: rank the continuous mutant to a permutation, then swap some genes toward the current best solution to give the population a ‘direction’; for binary problems threshold at the vector average. Applied to TSP, 0/1 KP and TTP.

Authors’ claims BMV significantly outperforms 8 other mapping methods on TSP (AE about 53 percent lower than their average, 92-99 percent faster, Wilcoxon ‘+’ on all 12 TSPs) and ‘achieves promising results’ on KP and TTP; the paper’s own Wilcoxon results show BMV vs the other mappings is ‘not significantly different’ on all TTP comparisons.

Evidence base 13 TTP instances, all bounded-strongly-correlated, single capacity/item-factor, 50-1000 cities. 30 runs, 5,000 fitness evaluations, MATLAB, identical DE settings for all 9 mappings. Wilcoxon signed-rank test (Table 10). No local search / no problem-specific operators (deliberately). No comparison with any real TTP solver. TTP objective values are large and negative.

Stated limitations Authors state BMV may need more computational time, that its population diversity may drop over generations because of matching genes to the best solution, and that a well-designed local search and a new DE-mutation operator for combinatorial problems are needed as future work.

Strengths. ✓ Systematic controlled comparison: 9 mapping methods under identical DE settings, parameters, budget and instances • ✓ 30 runs plus a Wilcoxon signed-rank test with a clear +/=/- decision scheme • ✓ Parametric analysis of BMV’s two parameters with confidence intervals • ✓ Honest that for TTP (and KP) BMV is not significantly better than the alternatives, only ‘promising’

Weaknesses. ✗ For TTP the paper’s own Wilcoxon results show no significant difference between BMV and any of the 8 other mappings; the ‘outperforms’ claim holds only for TSP • ✗ TTP evaluation is 13 instances of a single KP type, single capacity/item-factor; not the categorised subset • ✗ No comparison with any real TTP solver; DE-BMV without local search gives large negative objectives and is not a competitive TTP method • ✗ Only 5,000 fitness evaluations; no wall-clock budget; not comparable to the 10-min-budget TTP literature • ✗ TTP interdependence is completely ignored (permutation + threshold-binary, no coordination) • ✗ The contribution is to DE methodology; TTP is one of three testbeds and the weakest result

[33] [Namazi et al. \(2019b\)](#) — A Cooperative Coordination Solver for Travelling Thief Problems

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	CoCo (PGCH/SGCH/LGCH/MBFS)	Family (L1/L2)	Single-solution metaheuristic / Coordinated local search (CoCo)
Benchmark	—	Size	Cities: 76; 100; 130; 159; 280; 574; 724; 1000; 1304; 1577; 2103; 3038; 4461; 7397; 11849; 13509; 14051; 15112; 18512; 33810 Items: 1x; 5x; 10x KP-Types: bsc; uco; usw
Metric	avg RDI per category;	Stat. test	ANOVA + paired t-test (95% CI), significance starred
Competition	N	Code	Unknown
Budget / Runs	10min / 10	Hardware	Intel Xeon X5650 2.66GHz

Abstract. [Namazi et al. \(2019b\)](#) introduced the **Cooperative Coordination (CoCo) solver**, a fundamentally new paradigm for TTP optimization based on **explicit coordination of 2-opt tour moves with packing plan updates**. The core observation is that standard 2-opt operators frequently reject beneficial tour reversals because the fixed packing plan drops in value when the reversed segment is evaluated, hiding the true potential of the new tour topology. To address this, the **Profit Guided Coordination Heuristic (PGCH)** synchronizes each 2-opt reversal with item reassignments guided by **prefix-minimum and postfix-maximum profitability trends** along the tour, replacing unprofitable items in the reversed segment with more profitable uncollected items from its boundary. A complementary **Boundary Bit-Flip search** further refines packing by focusing only on “marginal” items near the profit-neutrality boundary. Evaluated on the full 60-instance benchmark with the **Relative Deviation Index (RDI)** as the comparison metric, CoCo significantly outperforms MATLS, S5, and CS2SA*, establishing coordination-aware local search as the leading paradigm for the TTP.

Contribution	The CoCo (Cooperative Coordination) solver: PGCH (a profit-guided expanded 2-opt that, when reversing a tour segment, coordinately adjusts the collection plan using prefix-minimum / postfix-maximum profitability-ratio anchors) plus Boundary Bit-Flip (restricting bit-flips to the lowest-profit collected and highest-profit uncollected items to free time budget for more tour evaluations); an extended version of the SoCS 2019 PGCH paper.
Authors' claims	PGCH is considerably better than plain 2-opt on all three categories; Boundary Bit-Flip helps on Categories B and C but hurts on Category A (item scarcity); CoCo outperforms MATLS, S5 and CS2SA* statistically (ANOVA + t-test, 95 percent CI) in the vast majority of instances and per category, with mean RDI A/B/C = 96.5/95.8/93.6 vs S5 83.3/72.4/73.1, MATLS 48.2/66.1/65.8, CS2SA* 32.3/37.4/38.5.

Evidence base	The categorised 60-instance benchmark (20 per A/B/C, 76-33810 cities). 10 runs, 10-min timeout, Xeon X5650, 2 GB memory limit. $RDI = (G_{\text{mean}} - G_{\text{min}}) / (G_{\text{max}} - G_{\text{min}}) \times 100$, a HIGHER-is-better pool-relative measure (0 = worst run, 100 = best run across all runs of all solvers); raw gain values in an appendix. ANOVA + pairwise t-tests with significance marking. Baselines MATLS, S5, CS2SA* re-run from source under identical conditions. A 4-variant ablation isolates PGCH vs Boundary Bit-Flip.
Stated limitations	Authors note Boundary Bit-Flip degrades on Category A because item scarcity over-restricts the search space, and that the 0.01 percent restart margin is empirically set.

Strengths. ✓ Genuinely novel, well-motivated mechanism: PGCH tightly couples tour segment reversal with a principled collection-plan update, directly attacking the weak-coordination problem • ✓ Clear worked example and figures showing why uncoordinated 2-opt moves are rejected • ✓ Rigorous evaluation: full categorised 60-instance benchmark, 10 runs, 10-min budget, ANOVA + t-tests, raw gain values in an appendix • ✓ Baselines (MATLS, S5, CS2SA*) re-run from source under identical conditions • ✓ 4-variant ablation isolates the contribution of PGCH and Boundary Bit-Flip • ✓ Establishes CoCo as the new state of the art on the categorised benchmark and the standard strong baseline for later work • ✓ Honest that Boundary Bit-Flip hurts on Category A

Weaknesses. ✗ Published as an arXiv preprint; the peer-reviewed SoCS 2019 short version covers only PGCH, not this extended CoCo • ✗ RDI here uses the higher-is-better, pool-relative convention (0 = worst run, 100 = best run); the numbers are not absolute-optimality measures and depend on which solvers are in the pool • ✗ Only 3 baselines (MATLS, S5, CS2SA*); no MA2B, MMAS, CoSolver2B or RES_GLS • ✗ Large RDI gaps over CS2SA* on Category A sometimes correspond to small differences in the underlying gain values relative to the min-max spread • ✗ The restart-margin parameter is set empirically with no sensitivity study • ✗ No omnibus test (Friedman/Nemenyi) of the RDI ranking; only pairwise t-tests on gain values

[34] Namazi et al. (2019a) — A Profit Guided Coordination Heuristic for Travelling Thief Problems

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	PGCH	Family (L1/L2)	Single-solution metaheuristic / Coordinated heuristic (CoCo)
Benchmark	—	Size	Cities: 76; 100; 130; 159; 280; 574; 724; 1000; 1304; 1577; 2103; 3038; 4461; 7397; 11849; 13509; 14051; 15112; 18512; 33810 Items: 1x; 5x; 10x KP-Types: bsc; uco; usw
Metric	—	Stat. test	—

Key result PGCH consistently 2-OPT, e.g. Cat~A a280 5.53% vs. 1.81%, Cat~B pla33810 7.86% vs. 2.20%, Cat~C pla33810 3.83% vs. 0.89%; PGCH accepts longer reversal segments;

Competition N **Code** Unknown

Budget / Runs 10min / 10 **Hardware** —

Abstract. [Namazi et al. \(2019a\)](#) proposed an **extended 2-opt operator** for the TTP that integrates item-reassignment decisions directly into the segment-reversal procedure, predating and motivating the full CoCo framework of [Namazi et al. \(2019b\)](#). When a tour segment is reversed, items collected in the early cities of the reversed segment—which, after reversal, are now closer to the end of the tour and thus generate less profit—are released, and more profitable uncollected items in the later cities of the original segment are added to the plan. This joint reversal-and-reassignment move is evaluated against the full TTP objective, allowing the solver to accept reversals that improve the integrated tour-packing quality even when the tour alone becomes locally longer. Evaluated on the full benchmark with Category A/B/C classification, the integrated heuristic outperforms several state-of-the-art TTP solvers, providing the empirical foundation for the subsequent CoCo development.

Contribution The Profit Guided Coordination Heuristic (PGCH): an expanded 2-opt that, after reversing a tour segment, adjusts the picking plan using prefix-minimum profitability-ratio anchors (unpick low-ratio early items, pick high-ratio later items); embedded in a coordinated TTP solver with CLK init, best-of-PackIterative/Insertion packing init, PGCH over the Delaunay neighbourhood, random single bit-flip, and S5-style restarts. (SoCS 2019, the peer-reviewed short version of the CoCo work.)

Authors' claims The coordinated solver outperforms MATLS, S5 and CS2SA* in a vast majority of instances in all three categories, with mean RDI A/B/C = 92.5/92.0/92.1 vs S5 79.2/66.7/66.6, MATLS 50.3/70.4/71.3, CS2SA* 33.5/39.5/42.0; PGCH obtains a higher objective than plain 2-opt on all tested instances.

Evidence base The categorised 60-instance benchmark (20 per A/B/C, 76-33810 cities). 10 runs, 10-min timeout. $RDI = (G_{\text{mean}} - G_{\text{min}}) / (G_{\text{max}} - G_{\text{min}}) \times 100$ (higher is better, pool-relative). Baselines MATLS, S5, CS2SA* re-run from source under identical conditions (fresh CLK per run). A PGCH-vs-2-opt ablation on 9 instances (3 per category). No statistical test in this short version.

Stated limitations None stated explicitly (5-page paper); the 0.01 percent restart margin is empirically set.

Strengths. ✓ Peer-reviewed (SoCS 2019); introduces the core PGCH mechanism cleanly with a strong motivating analysis of the profitability trend • ✓ Full categorised 60-instance benchmark, 10 runs, 10-min budget • ✓ Baselines (MATLS, S5, CS2SA*) re-run from source under identical conditions • ✓ PGCH-vs-2-opt ablation directly isolates the mechanism's contribution • ✓ Establishes the coordination-heuristic paradigm that becomes state of the art

Weaknesses. ✗ No statistical significance testing in this version (added only in the extended arXiv); RDI comparisons are pool-relative point estimates • ✗ RDI uses the higher-is-better, pool-relative convention; numbers depend on which solvers are in the pool and are not absolute-optimality measures • ✗ Only 3 baselines (MATLS, S5, CS2SA*); no MA2B, MMAS or CoSolver2B • ✗ The packing side is a plain random single bit-flip; under-developed (improved only in the later CoCo extension) • ✗ The PGCH-vs-2-opt ablation uses only 9 instances • ✗ 5-page paper; no analysis of failure cases or parameter sensitivity

[35] [Neumann et al. \(2019\)](#) — A Fully Polynomial Time Approximation Scheme for Packing While Traveling

Variant	PWT	Objective / #obj	Single-objective / 1
Route	Fixed route	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	FPTAS (PWT)	Family (L1/L2)	Approximation scheme / FPTAS
Benchmark	—	Size	Cities: 101; Items: 100 to 1000
Metric	approximation rate AR%=100 OBJ/ OPT, runtime (s);	Stat. test	—
Baselines	exact MIP (eMIP), branch-infer-and-bound (BIB), approximate MIP (aMIP) polyakovskiy2015packing;		
Key result	FPTAS AR =100% for 0.0001,0.01; DP far faster than eMIP/BIB (e.g. uco-sw_10 1000 items: DP 15s vs. eMIP 35min); relaxing 0.0001 0.75 cuts FPTAS time 50-95% with AR still 100%;		
Competition	N	Code	No
Budget / Runs	— / 1 (deterministic)	Hardware	4GB / 3.06GHz Intel Dual Core (exp1); 128GB / 8x2.8GHz AMD 6-core cluster (exp2)

Abstract. [Neumann et al. \(2019\)](#) developed the first **exact dynamic programming algorithm** for the PWT problem and an accompanying **Fully Polynomial-Time Approximation Scheme (FPTAS)** that maximizes the net profit above the baseline travel cost of an empty tour. The DP algorithm exploits the ordered structure of the fixed route to solve the problem optimally, but its running time is pseudopolynomial in the item count. The FPTAS scales this to a polynomial-time approximation with a provable $(1 - \epsilon)$ guarantee by rounding item profits and applying the DP to the rounded instance. Tested on instances with 101 cities and item counts ranging from 100 to 1,000, the FPTAS consistently outperforms the MIP-based approaches of [Polyakovskiy and Neumann \(2015\)](#), combining theoretical guarantees with practical efficiency and establishing the FPTAS as the method of choice for exact-quality PWT solutions within tight time limits.

Contribution For the Packing While Travelling Problem (fixed route, one item per city, capacity constraint): an exact pseudo-polynomial $O(nW)$ dynamic programming algorithm with approximate domination, an FPTAS for the shifted objective $b(S) - b(\text{empty})$ giving $(1-\epsilon)\text{OPT}$ in time polynomial in n and $1/\epsilon$, and a strengthened inapproximability result (deciding non-negative objective is NP-complete, NP-hard even with unbounded capacity), so no finite-ratio approximation for the unshifted objective unless $P=NP$.

Authors' claims	The DP solves most exactly-solvable instances much faster than the prior MIP/branch-infer-and-bound (e.g. 15 s vs 35 min on a 1000-item instance), superior on 24 of 27; the FPTAS keeps a 100 percent ratio for small eps and only approaches the approximate-MIP accuracy at eps=0.25; on hard large-weight-range instances, relaxing eps to 0.75 speeds the FPTAS 50-95 percent with no visible accuracy loss; the choice of approximation guarantee significantly affects runtime; the FPTAS holds for any increasing-convex travel-time-per-distance law.
Evidence base	TTP benchmark, fixed route. Experiment 1: a 101-city route, 27 instances (3 KP types x 3 item counts x 3 capacities), same machine as polyakovskiy2017 for comparability, baselines eMIP/BIB/aMIP quoted. Experiment 2: new hard instances with profit/weight ranges up to [1,1e7], on a 128 GB cluster, no MIP comparison. Metrics: OPT, runtime, approximation ratio. Deterministic.
Stated limitations	Authors frame the increasing-convex assumption as a strength (generality), note the DP is pseudo-polynomial (runtime blows up on large-weight-range instances), and state that understanding what makes instances hard for MIP vs FPTAS should guide a future mixed algorithm.

Strengths. ✓ The first FPTAS for the shifted PWT objective, with a matching inapproximability result showing the shift is necessary • ✓ Clean, correct proofs; the approximate-domination-without-explicit-rounding technique is elegant and reusable • ✓ The DP is simple, exact, and empirically much faster than the prior MIP/BIB methods on most instances • ✓ Careful experimental design: same machine and instances as polyakovskiy2017 for a fair exact-methods comparison, plus a harder synthetic set • ✓ Honest that the FPTAS is often slower than the approximate MIP on easy instances and pays off only on specific hard cases • ✓ Generality: the FPTAS extends to any increasing-convex travel-time-per-distance law

Weaknesses. ✗ Addresses only the packing sub-problem on a fixed route, not the full TTP • ✗ The FPTAS guarantee is for the shifted objective; for the actual PWT objective no finite-ratio approximation exists, so the guarantee's practical meaning depends on the empty-vehicle reference being acceptable • ✗ The DP is pseudo-polynomial ($O(nW)$); scalability is capacity-bound and the paper's own data show runtime blow-up on large-weight-range instances • ✗ Empirical evaluation is on ≤ 1000 -item instances plus synthetic hard instances; nothing at the $1e5$ - $1e6$ -item scale the approximate MIP of polyakovskiy2017 handled • ✗ FPTAS approximation ratios are 100 percent almost everywhere, so the eps-vs-quality trade-off it is designed to expose is barely visible in the data • ✗ Experiment 2 has no MIP comparison at all

[36] Zouari et al. (2019) — A new hybrid ant colony algorithms for the traveling thief problem

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	?	Family (L1/L2)	Problem def./benchmark/analysis / ?
Benchmark	—	Size	Cities: 280; Items: 1x
Metric	found-solution value and execution time (s) (convergence curves).	Stat. test	none

Key result OMMACS (dynamic) outperforms both CMMACS and DMMACS in quality and speed; the parallel DMMACS is slower than the serial version on these small instances (no parallel benefit).

Competition N **Code** No

Budget / Runs — / — **Hardware** —

Abstract. Zouari et al. (2019) investigated three **Hybrid Ant Colony Optimization (ACO)** strategies for the TTP, differing in how pheromone information and solution construction are organized. The **centralized static** variant maintains a single global pheromone matrix updated by the best ant; the **dynamic** variant adapts the pheromone update rule based on search progress; and the **distributed** variant partitions the colony into subgroups that independently construct tours and exchange the best found solutions at synchronization points. Applied to a pool of ten instances with 280 cities and 1 item per city, the hybrid ACO strategies demonstrate competitive performance, with the **dynamic** variant (OMMACS) achieving the best balance of solution quality and execution time. The study highlights the potential of swarm intelligence paradigms for multi-component problems and motivates further investigation of large-scale ACO frameworks for the TTP.

Contribution Three hybrid ant-colony algorithms for TTP built on a multi-level Max-Min ant colony system: CMMACS (static, centralised: 2-opt for the tour + MMACS for packing), OMMACS (online/streaming variant), DMMACS (distributed variant, serial and parallel).

Authors' claims OMMACS improves the process of obtaining a better and faster solution and outperforms both CMMACS and DMMACS; the parallel DMMACS is slower than serial on small instances; OMMACS's speed may come from lower per-step complexity.

Evidence base 10 bounded-strongly-correlated a280 instances (280 cities, 279 items) from polyakovskiy2014. Results shown only as small unlabelled curves (found solutions and execution time). No comparison with any external TTP solver. No statistical test, no run count. 2-page extended abstract.

Stated limitations Authors state that verifying the algorithms on large instances, to measure the benefit of the multithread architecture, is future work.

Strengths. ✓ Explores three under-studied angles at once: online/streaming, distributed and parallel resolution • ✓ ACO-based, building on the authors' prior multi-level MMACS work

Weaknesses. ✗ 2-page extended abstract; the algorithms are only sketched, not specified in reproducible detail • ✗ Evaluated on a single instance type (10 a280 bounded-strongly-correlated instances); no size or KP-type variation; not the categorised subset • ✗ No comparison with any established TTP solver; only the three proposed variants against each other • ✗ No statistical test, no run count; results shown only as small unlabelled curves • ✗ The distributed/parallel contribution is not demonstrated to help (parallel is slower than serial on the tested instances) • ✗ The online (OMMACS) framing is not connected to any dynamic-TTP formulation and 'input as a stream' is undefined • ✗ No absolute quality context (best-known gaps)

[37] Ali et al. (2020) — *Differential Evolution Algorithm for Multiple Inter-dependent Components Traveling Thief Problem*

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static

Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	DB-DE	Family (L1/L2)	Population-based metaheuristic / Differential evolution
Benchmark	—	Size	Cities: 51; 52; 70; 76; 99; 100; 101; 105; 107; 150; 198; 280; 783 Items: 1x KP-Types: bsc
Metric	average objective (30 runs), Friedman rank.	Stat. test	Friedman rank test
Baselines	21 algorithms from wagner2018case		
Key result	DB-DE best on 4/13 (eil51 =4019.4, st70, eil101, d198), better-than-mean on 4 more; Friedman: ranked 2nd in objective, 1st in time; avg 41.79s vs. 600s for the 21 algorithms (saves 93.03%).		
Competition	N	Code	No
Budget / Runs	— / 30	Hardware	MATLAB R2017b, 16GB, i7

Abstract. Ali et al. (2020) proposed a **modified Differential Evolution** algorithm tailored to the mixed-variable structure of the TTP, where the tour is a permutation and the packing plan is a binary vector. The method introduces dedicated **mapping and repairing mechanisms** to convert continuous DE trial vectors into valid combinatorial solutions, an **adapted mutation operator** that exploits the best solution's structure to guide perturbation, and problem-specific local search procedures for both subcomponents. A novel initialization strategy generates a diverse initial population in which the packing plan is optimally adjusted to the corresponding tour using a greedy KP heuristic. Evaluated on 13 strongly correlated instances with 51 to 783 cities (1x, bsc) and compared against 21 algorithms, the method achieves competitive solution quality and computational efficiency, demonstrating that continuous-domain evolutionary algorithms can be successfully adapted to multi-component combinatorial problems with appropriate domain-specific engineering.

Contribution	DB-DE (Discrete-Binary differential evolution) for TTP: dual representation, BMV mapping, an ensemble of three adaptively applied mutations, k-means repair of 10 percent of the tour population, 3-opt on the best tour, and a TTP-aware KP re-derivation that places items by carry-distance-adjusted profit each generation, so only the tour is evolved and the packing is re-derived.
Authors' claims	DB-DE significantly improves over its own ablations (+94.5 percent over DE+BMV, +9-16 percent over DE+BMV+2/3-Opt); versus 21 state-of-the-art algorithms (values quoted from Wagner et al.) it gets the best average on 4 of 13 instances and averages 6.5 percent above the pool minimum / 6.3 percent below the maximum; it runs in about 42 s vs 600 s ('93 percent less time') and ranks 2nd on objective and 1st on time by the Friedman rank test.
Evidence base	13 TTP instances, all bounded-strongly-correlated, single capacity/item-factor, <= 783 cities. MATLAB, i7, 30 runs, 10,000 fitness evaluations. Taguchi/Minitab parameter tuning. Baselines: 3 self-comparison DE variants plus 21 algorithms from Wagner et al. quoted as min/mean/max (not re-run). Friedman rank test on objective and time. No per-instance, per-algorithm, or statistical comparison against the 21.
Stated limitations	Authors state each component will be investigated further and that multi-objective TTP is future work; no dedicated limitations section.

Strengths. ✓ A genuinely engineered TTP method (ensemble mutation + k-means repair + local search + TTP-aware KP re-derivation), not a generic DE • ✓ Only evolves the tour and re-derives the packing each generation, cutting time • ✓ Taguchi parameter tuning; Friedman rank test on both objective and time; 30 runs • ✓ Honest ablation (DE+BMV to +2Opt to +3Opt to +repair) clearly isolates where quality comes from • ✓ Very fast (about 42 s vs 600 s), a real practical advantage if quality is competitive

Weaknesses. ✗ All 13 instances are a single KP type, single capacity/item-factor, ≤ 783 cities; not the categorised 60-subset • ✗ Comparison with the '21 state-of-the-art algorithms' is against quoted min/mean/max, not re-runs; DB-DE's mean vs the pool's min/mean/max is a weak comparison with no per-instance data or statistical test • ✗ DB-DE gets the best average on only 4 of 13 instances and averages 6.3 percent below the pool maximum • ✗ 10,000 fitness evaluations is a small budget; the 42 s vs 600 s comparison is not like-for-like (different stopping criteria and hardware) • ✗ The 21-algorithm pool from Wagner et al. is a mix of simple heuristics; '2nd by Friedman rank' among them is weak evidence against CoCo/CS2SA*/MATLS • ✗ Heavy reuse of the authors' own prior components (BMV, k-means repair, ensemble mutation); incremental • ✗ MATLAB implementation

[38] Ali and Mohamedkhair (2020) — *Hyper-Heuristic Approaches for the Travelling Thief Problem*

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	SR-IO/KAB-IO/RD-MSA/RPD-MSA	Family (L1/L2)	Hyper-heuristic / Selection hyper-heuristic
Benchmark	—	Size	Applies four single-point selection hyper-heuristic approaches to the TTP, where heuristics are repeatedly selected from a predefined set of low-level operators to iteratively improve solutions. Experimental results show that the proposed hyper-heuristic methods achieve performance comparable to state-of-the-art approaches reported in the literature. Only proposed approaches are evaluated and compared between them. No comparison with literature methods is provided.& Cities: 51; 76; 289 Items: 1x KP-Types: bsc; uco; usw
Metric	average objective (profit).	Stat. test	Mann-Whitney U (95%)
Baselines	S5, M3, M4, M4B, MATLS.		

Key result proposed HHs best on all eil51/eil76 (e.g. eil51_corr SR-IO 4438.35; eil76_uncorr KAB-IO 5658.46); SOTA best on a280 (a280_corr S5 18398.02, a280_uncorr M3 19312.18); SR-IO dominates the other proposed methods on a280.

Competition N

Code No

Budget / Runs 10 min/instance / 5

Hardware Intel-4510 CPU 2GHz, C++11

Abstract. Ali and Mohamedkhair (2020) applied four **single-point selection hyper-heuristics** to the TTP, each combining a heuristic-selection mechanism with a move-acceptance criterion: Simple Random with Improve Only (SR-IO), K-Arm-Bandit with Improve Only (KAB-IO), Random Descent with Modified Simulated Annealing (RD-MSA), and Random Permutation Descent with Modified SA (RPD-MSA). At each iteration one low-level operator is chosen from a predefined pool of TSP, KP and combined TTP operators (built by pairing each TSP move with each KP move) and applied to the current solution. The study compares the four hyper-heuristic variants against each other and against five state-of-the-art algorithms on small/medium TTP instances (51, 76 and 280 cities, 1 item per city, all KP types) and demonstrates that they achieve performance comparable to the literature, winning on eil51/eil76 while the state-of-the-art methods retain the edge on a280. The results suggest that even simple hyper-heuristic selection strategies, when equipped with a well-designed set of domain-specific low-level operators, can navigate the TTP search space effectively without requiring algorithm-specific parameter tuning or problem-structure exploitation.

Contribution Four single-point selection hyper-heuristics for TTP (SR-IO, KAB-IO, RD-MSA, RPD-MSA: four selection methods x two move-acceptance methods) over 23 categorised TSP/KP/TTP operators, with an LKH init tour, greedy KP init, and penalised infeasible solutions allowed as a diversification device.

Authors' claims The proposed selection hyper-heuristics give comparable results to the state of the art; SR-IO dominates the other proposed variants on a280 instances (Mann-Whitney U, 95 percent CI); RPD-MSA outperforms KAB-IO and RD-MSA on eil76 bounded-strongly-correlated; the proposed methods achieve best results on eil51 and eil76 while S5/M4B/M3 are best on a280.

Evidence base Only 9 instances from polyakovskiy2014 (eil51, eil76, a280; 3 KP types; 1 item/city). Baselines S5, MATLS, M3, M4, M4B (5 SOTA). 5 runs, 10-min budget, 2 GHz machine, C++11. Statistics: Mann-Whitney U at 95 percent CI, but only for the pairwise comparison among the 4 proposed variants; the SOTA comparison (Tables III-V) reports only bolded average objective with no variance and no statistical test.

Stated limitations Authors state future work is applying the method to very large instances, tuning the selection and move-acceptance parameters, and studying the effect of the initial-solution method.

Strengths. ✓ Systematic hyper-heuristic design space (4 selection x 2 acceptance methods, 23 categorised operators) • ✓ Mann-Whitney U test with 95 percent CI for the pairwise comparison of the 4 proposed variants • ✓ Compares against 5 state-of-the-art methods, including per-instance average objective • ✓ Allowing penalised infeasible solutions as a diversification device is a defensible, clearly explained choice • ✓ Honest: the proposed methods win only the small/medium instances; S5/M4B/M3 win a280

Weaknesses. ✗ Only 9 instances (3 city sizes ≤ 280 , 1 item/city); not the categorised subset; no medium/large • ✗ Only 5 runs per instance; the SOTA comparison has no variance and no statistical test (only bolded averages) • ✗ Whether the S5/MATLS/M-series values were re-run under the same setup or quoted is not clearly stated • ✗ KAB and MSA parameters set to fixed values without tuning (acknowledged) • ✗ 6-page paper at a regional venue; thin • ✗ No absolute quality context; ‘comparable to state-of-the-art’ is on 9 tiny instances where the proposed methods only match on the smallest

[39] Maity and Das (2020) — *Efficient hybrid local search heuristics for solving the travelling thief problem*

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	hybrid local search	Family (L1/L2)	Single-solution metaheuristic / Hybrid local search
Benchmark	—	Size	Cities: 76; 100; 130; 159; 280; 574; 724; 1000; 1304; 1577; 2103; 3038; 4461; 7397; 11849; 13509; 14051; 15112; 18512; 33810 Items: 1x KP-Types: bsc; uco; usw
Metric	mean objective, Friedman rank;	Stat. test	Friedman ($F = 12nk(k+1) r^{2-3n(k+1)}$)
Baselines	SH, RLS, (1+1)EA, DH, CS2SA, CS2SA*, CS2SA-R, MATLS;		
Key result	best on Cat1 (bsc, small cap); MATLS/CS2SA better on Cat2/Cat3;		
Competition	N	Code	No
Budget / Runs	600s / 10	Hardware	Intel Core i5-3337U 1.80GHz

Abstract. Maity and Das (2020) focused on the design of an effective **scoring-based item-selection heuristic** for the TTP, operating on fixed near-optimal tours generated by the **Chained Lin-Kernighan Heuristic (CLKH)**. The proposed scoring function combines an item’s profit with a penalty term that reflects the cumulative travel cost incurred by carrying its weight over the remaining tour, enabling more informed selection decisions than simpler profit-to-weight ratios. A **bit-flip local search** then iteratively refines the binary packing plan by toggling item-collection decisions to escape local optima. Evaluated exclusively on 1x-density instances spanning 76 to 33,810 cities across all three KP types, the method is benchmarked using **Friedman’s non-parametric ranking test**, which confirms statistically significant superiority over competing approaches on most benchmark configurations, providing rigorous statistical validation for TTP algorithm comparison.

Contribution	A hybrid local-search TTP1 heuristic: a CLKH tour with a new item-scoring function (profit/weight ratio raised to an exponent beta, weighted by time-from-start over time-to-end) for greedy packing, followed by a randomised bit-flip (add/remove a random item index, keep if improved); beta = 7.4 chosen empirically over 200 tours.
Authors' claims	The method surpasses the state of the art on most instances, especially Category 1 (bounded-strongly-correlated, 1 item/city, small capacity), because its greedy KP sorting suits that KP type; for Categories 2 and 3, MATLS and CS2SA do better on many instances; the method is competitive with CS2SA* and CS2SA-R on the bsc category; the KP component matters more than the TSP component.
Evidence base	28 base problems x 3 categories = 84 instances. Baselines SH, RLS, (1+1) EA, CS2SA, MATLS, DH re-run on the same families, 600 s, 10 runs, mean objective; plus a separate CS2SA*/CS2SA-R comparison on 22 bsc instances. Friedman test (statistic, chi-square, mean ranks) per category. MATLAB, i5-3337U 1.80 GHz. The method has the best Friedman rank on Category 1 (1.71), is 2nd on Category 2 (behind MATLS), and 3rd on Category 3 (behind CS2SA and MATLS).
Stated limitations	Authors state they will improve run-time and space exploration with a more sophisticated search, and note MATLS and CS2SA do better on Categories 2/3 'due to strong assumptions about the domain'.

Strengths. ✓ Proper statistical testing: Friedman test with reported statistic, chi-square and mean ranks, per category; 10 runs; 600 s standard budget • ✓ Baselines (SH, RLS, EA, CS2SA, MATLS, DH) re-run on the same families, not quoted • ✓ Separate comparison against the strong CS2SA*/CS2SA-R on the bsc category • ✓ Simple low-complexity method; the new scoring function is a sensible refinement of the PACK score • ✓ Honest that the method only wins Category 1 and that MATLS/CS2SA win Categories 2 and 3

Weaknesses. ✗ The method only optimises the packing plan on a single fixed CLKH tour; the tour is never optimised, a fundamental ceiling versus CoCo/S5 • ✗ Wins only on Category 1, the category where a greedy KP sort is naturally strong; 2nd-to-4th on Categories 2 and 3 • ✗ 84 instances at 3 fixed category points; not the full categorised 60-subset • ✗ No comparison with CoCo/PGCH (2019), the actual state of the art by the 2020 publication date • ✗ beta = 7.4 chosen empirically with no reported sensitivity analysis or per-category tuning • ✗ The randomised bit-flip does a single pass of m random indices per call; its effect versus a full deterministic bit-flip is not analysed • ✗ MATLAB on a 1.80 GHz i5; a language/implementation confound even though baselines were re-run

[40] Namazi et al. (2020) — *Surrogate Assisted Optimisation for Travelling Thief Problems*

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	CoCo-SM (SVR)	Family (L1/L2)	Learning-based / Surrogate-assisted

Benchmark	—	Size	Cities: 76; 100; 130; 159; 280; 574; 724; 1000; 1304; 1577; 2103; 3038; 4461; 7397 Items: 1x; 5x; 10x KP-Types: bsc; uco; usw
Metric	% of initial tours filtered out, # missed best solutions (out of 10 runs).	Stat. test	none
Key result	30% of initial tours filtered (avg A/B/C =37/30.5/28%) at a cost of 1 missed best per 10 runs (avg 0.75/1/0.78), saving 30% solver time; surrogate filtering far better than random (e.g. hardest Cat-C instance: 10,000s for CoCo to process 1000 tours vs. 15s to train+use the surrogate).		
Competition	N	Code	No
Budget / Runs	— / 10	Hardware	—

Abstract. [Namazi et al. \(2020\)](#) introduced an **adaptive surrogate-assisted filtering framework** for TTP optimization, motivated by the observation that iterative restart-based TTP solvers waste significant computation exploring tours that, regardless of how well the KP component is subsequently solved, cannot lead to high-quality integrated solutions. A **customized Support Vector Regression (SVR)** model is trained online to learn the correlation between structural properties of initial TSP tours—such as tour length and geometric regularity—and the quality of TTP solutions eventually produced from those tours. The surrogate model is then used to **filter out non-promising tour initializations** before expensive TTP optimization is performed, concentrating the computational budget on the most promising starting regions of the search space. Tested across all three KP categories on instances up to 7,397 cities, the surrogate-based approach significantly reduces the number of non-promising evaluations with only a minor impact on solution quality, offering a principled framework for improving solver efficiency in iterative multi-component optimization.

Contribution	CoCo-SM: adds an adaptive kernel-SVR surrogate (with a novel tour-similarity RBF kernel based on city-position differences) to the CoCo solver to predict the final objective an initial CLK tour will lead to, and filter non-promising initial tours before the full optimization; adaptive retraining triggered by a normalised-error measure.
Authors' claims	The first surrogate-assisted approach for TTP; on average about 30 percent of initial tours are filtered out at the cost of missing about one best solution out of 10 runs; the surrogate overhead is negligible (about 15 s vs about 10,000 s to process 1,000 tours on the hardest instance); surrogate filtering is considerably better than random filtering; more filtering means a higher chance of missing the best.
Evidence base	32 instances at 574-7397 cities, 3 categories. Both CoCo and CoCo-SM run 10 times; CoCo-SM reuses the same 1,000 initial tours. Metric: percentage of tours filtered + number of missed best solutions (out of 10), for beta in 0,1,2,3. Only baseline is random filtering. No statistical test. Hyper-parameters 'empirically selected'.
Stated limitations	None stated explicitly (5-page paper); the filtering/quality trade-off is acknowledged (more filtering means more missed best solutions).

Strengths. ✓ First surrogate-assisted approach for TTP; the tour-similarity kernel is a sensible novel design for comparing tours • ✓ Adaptive online retraining with a principled normalised-error measure and a probabilistic (not hard-threshold) filtering rule • ✓ Honest cost/benefit framing: quantifies both the time saved and the quality risk • ✓ Uses the real CoCo solver and identical initial tours for a controlled comparison of the filtering effect

Weaknesses. ✗ Does NOT report whether CoCo-SM's final solution quality (objective/RDI) is better, equal or worse than plain CoCo; only filtering accuracy and missed-best counts • ✗ Only 32 instances at 574-7397 cities; no small instances and none larger; not the standard categorised 60 • ✗ Only baseline is random filtering; no comparison with other tour-quality predictors or with simply doing fewer restarts • ✗ Several hyper-parameters 'empirically selected' with no sensitivity study beyond beta in 0,1,2,3 • ✗ No statistical testing • ✗ The surrogate's predictive accuracy is not directly reported, only its downstream filtering outcome

[41] Ali et al. (2021) — A novel approach for solving travelling thief problem using enhanced simulated annealing

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	ESA	Family (L1/L2)	Single-solution metaheuristic / Simulated annealing
Benchmark	—	Size	Cities: 76; 100; 130; 159; 280; 574; 724; 1000; 1304; 1577; 2103; 3038; 4461; 7397; 11849; 13509; 14051; 15112; 18512; 33810 Items: 1x; 5x; 10x KP-Types: bsc; uco; usw
Metric	mean objective (std, time T).	Stat. test	none (+/- comparison)
Baselines	S5, CS2SA/CS2SA-R.		
Key result	ESA reports very large gains on small/medium (e.g. eil76_bsc =18,835 vs. S5 3742/CS2SA 3842; eil76_un =155,765 vs. 88,136; rl1304_bsc =1,705,291 vs. 80,066) and is much faster (108/159/345s vs. 600s); wins (+) on bsc across all 20 bases (Table 3), mixed on uncorrelated, mostly + on usw; gains so large vs. S5/CS2SA that an objective-scaling/setup discrepancy is likely (interpret cautiously).		
Competition	N	Code	Unknown
Budget / Runs	— / multiple (std reported)	Hardware	—

Abstract. Ali et al. (2021) proposed a **three-stage heuristic** for the TTP combining a random picking initialization, a tour generated by the **Lin-Kernighan (LK) heuristic**, and a modified Simulated Annealing phase that removes cities from the tour whose inclusion negatively affects the net profit. The SA phase performs a stochastic search over a neighborhood of city-exclusion moves, evaluating each removal against the TTP objective and accepting improvements with probability proportional to a cooling schedule. Evaluated on the full range of benchmark instances (76 to 33,810 cities, all three item densities and KP types), the method reports competitive performance, particularly on small and medium instances. However, the reported comparisons must be interpreted cautiously, as parameter inconsistencies in the experimental setup make direct comparison with literature results unreliable for some instance configurations.

Contribution	ESA (enhanced simulated annealing), a rearranged-steps TTP heuristic: start from a random picking plan, build an LKH tour, ELIMINATE from the tour all cities from which no item is picked, then apply modified simulated annealing to the packing and 2-opt to the tour, iterating while improving.
Authors' claims	ESA outperforms S5 and CS2SA on small and medium instances and gives competitive results on larger ones, at much lower runtime; on eil76 uncorrelated ESA reports about 155,765 profit vs about 88,100 for S5/CS2SA.
Evidence base	About 75 instances (28 base graphs, 51-33810 cities). Baselines S5 and a hybrid SA/hill-climb method (labelled 'CS2A/CS2SA R'), with no clear re-run protocol. 10 runs, mean/std/time. No statistical test. Table 2 shows ESA standard deviation of 0 on almost every instance despite a stochastic SA from a random start. ESA objective values are inconsistent in scale and direction versus the baselines on the same named instances (from about one third to about twenty times the baseline).
Stated limitations	Authors state the city-elimination step can make a downstream city unreachable so that 'we may get the infinity value as a profit' (the paper's conclusion ends on this sentence), and that the city-elimination process and large-instance capability need further work.

Strengths. ✓ Open access (PeerJ CS) with code and data in supplemental files; standard TTP instances used • ✓ The skip-cities-with-no-picked-items idea is intuitive for reducing travel cost • ✓ Reasonably broad instance table across the full size range • ✓ Reports runtime (ESA is fast)

Weaknesses. ✗ The method appears to solve a RELAXED problem (visit only cities with picked items), not standard TTP1 (which mandates visiting all cities), so the reported profit is not comparable to S5/CS2SA and the large 'gains' are an artefact of drastically reduced travel cost • ✗ Reported ESA objective values are inconsistent in scale and direction versus S5/CS2SA on the same named instances, a strong indicator of an objective-computation or instance-parameterisation error • ✗ Table 2 shows ESA standard deviation of 0 on almost every instance despite a stochastic SA starting from a random plan, which is implausible • ✗ The authors themselves acknowledge the city-elimination step can make the tour infeasible ('infinity value as a profit'); the method is not sound as stated • ✗ Only two baselines, with no clear re-run protocol, no statistical test, and an ambiguous baseline identity • ✗ Starting from a random picking plan is a weak, unjustified design choice; the methodology is not reproducible from the text

[42] Bossek and Wagner (2021) — *Generating Instances with Performance Differences for More Than Just Two Algorithms*

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	evolved instances	Family (L1/L2)	Problem def./benchmark/analysis / Instance generation

Benchmark	—	Size	Proposes fitness functions for evolving problem instances that generate large performance differences among multiple algorithms simultaneously, extending previous approaches that focused only on pairwise algorithm comparisons. Applies the proposed strategies to the multi-component TTP using three incomplete TTP solvers as a proof of concept. The evolved instances are intended to reveal algorithm strengths and weaknesses and support automatic per-instance algorithm selection and configuration. Experimental results demonstrate the potential of the proposed approach, while also showing that its effectiveness strongly depends on the degree of performance complementarity among the considered algorithms. No explicit comparison to the literature is provided.& Cities: 200 Items: 1x; 3x; 5x; 10x KP-Types:
Metric	—	Stat. test	—
Key result	success strongly portfolio-dependent - C2 dominates (EA succeeds on 85% pairwise, 97.5% for C2>S4/C2>S2), instances where S2/S4 win are scarce; fitness functions can be fooled by bi-modal solver performance.		
Competition	Y	Code	Yes
Budget / Runs	— / —	Hardware	R 4.0.0, HPC; TTP heuristics in Java
Abstract. Bossek and Wagner (2021) addressed the automatic generation of discriminating TTP instances using an evolutionary instance generator that maximizes the performance difference among multiple TTP algorithms simultaneously. Unlike prior instance generation approaches that focus on pairwise algorithm differentiation, the proposed fitness functions target configurations in which three or more incomplete TTP solvers exhibit the largest mutual performance spread, supporting the development of per-instance algorithm selectors. The evolved instances reveal complementary algorithm strengths that are not apparent in standard benchmark instances and provide a richer foundation for algorithm portfolio construction. Applied to instances with 200 cities and item densities of 1x, 3x, 5x, and 10x, the study demonstrates that the effectiveness of instance generation strongly depends on the degree of performance complementarity among the candidate algorithms being differentiated.			
Contribution	Fitness functions for evolutionary instance generation that produce TTP instances discriminating MORE THAN TWO algorithms simultaneously with a target performance ranking: a no-order function (sum of products of consecutive performance gaps) and a lexicographic explicit-ranking function; a (1+1)-EA proof-of-concept for N=3 faulkner2015 heuristics (S2, S4, C2).		
Authors' claims	The strategies are promising but their success strongly relies on the algorithms' performance complementarity; the explicit-ranking function succeeds in about 85 percent of jobs overall but only 10-20 percent when a non-dominant heuristic must beat the dominant one; the fitness functions can be fooled by bi-modal single-solver performance distributions even with 30 re-evaluation runs.		

Evidence base	720 instances for the no-order function, 240 for the ranking-sensitive functions (24 (n, item-count, ranking) combinations x 10). n=200 nodes; k=5 heuristic runs per fitness evaluation, 30 re-evaluations; 48-hour EA runs on a cluster; public GitHub; 55 features + PCA. Analysis via success-rate bar charts and boxplots; no statistical test on outcomes; no comparison with prior TTP instance generators.
Stated limitations	Authors state (1) it is easy to evolve instances reliably solved by the dominating algorithm but hard/impossible for the reverse, so the portfolio must be complementary, and (2) the fitness functions can be fooled by statistical artefacts; future work is task-specific mutation operators and alternative summary statistics.

Strengths. ✓ Tackles a real, previously open methodological problem: evolving instances that discriminate $N \geq 3$ algorithms with a target ranking • ✓ Three principled fitness formulations, one with a clear lexicographic structure • ✓ Honest, well-analysed negative results with a mechanistic explanation of the median-fitness-fooling artefact • ✓ Substantial compute, public code and data, PCA feature analysis of evolved instances

Weaknesses. ✗ Proof-of-concept only: $N=3$ heuristics, all from faulkner2015 (not state of the art); no modern solvers or larger portfolios • ✗ The core outcome is that the method largely fails when the desired ranking contradicts the portfolio's dominance structure, exactly the interesting case • ✗ The generated instances are not released as a named benchmark or shown useful for downstream algorithm selection • ✗ No statistical testing of the success-rate results; k=5 runs per fitness evaluation is small • ✗ The KP component is treated 'as another TSP' for feature extraction, a crude approximation • ✗ GECCO Companion (workshop-tier) paper; explicitly preliminary

[43] Misir (2021) — Benchmark Set Reduction for Cheap Empirical Algorithmic Studies

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	benchmark analysis	Family (L1/L2)	Problem def./benchmark/analysis / Instance analysis
Benchmark	—	Size	Cities: 51 to 85,900; Items: x1; x3; x5; x10 KP-Types: bsc; uco; usw
Metric	Spearman rank-order correlation between full-set vs. reduced-set algorithm ranking (Shapley-value based).	Stat. test	Spearman
Key result	most subsets >0.9 ; Subset-k5-1 (62 instances, $<1\%$ of the benchmark) $=0.974$; Subset-k5-20 (1240 instances) $=0.991$; k=5 best rank-preservation (k=6 anomalously low 0.80, ignored); 7 identifiable clusters (e.g. Cluster~1 14,000-city 1-3 item instances, Cluster~2 1800-city 5-10 item).		
Competition	N	Code	Unknown
Budget / Runs	— / —	Hardware	—

Abstract. [Mısırlı \(2021\)](#) introduced a **benchmark set reduction strategy** that automatically identifies a compact representative subset of TTP instances from the full 9,720-instance benchmark, reducing the computational burden of repeated algorithm evaluation during development and testing. The method derives **latent instance representations** from a performance matrix of 21 TTP algorithms evaluated on all instances, capturing the relative difficulty and structural characteristics of each instance as perceived by a diverse algorithm portfolio. Clustering in the latent space then selects a minimal subset of instances that preserves the full diversity of performance patterns. The study demonstrates that evaluating algorithms on this reduced set produces rankings and relative performance estimates that closely match those obtained on the full benchmark, providing practitioners with a principled, data-driven approach to reducing the cost of TTP algorithm benchmarking without sacrificing representativeness.

Contribution	A benchmark-set reduction method: extract latent features from the 21-algorithm x 9720-instance performance matrix via SVD, cluster instances by k-means on the latent features, and select the instances closest to each cluster centroid, giving a small representative subset; applied to the wagner2018case TTP data.
Authors' claims	A subset covering under 1 percent of the 9720 instances preserves algorithm-selection performance and the Shapley-based algorithm ranking (Spearman rho 0.974 for a 62-instance subset, 0.991 for a 1240-instance subset); only a few (mostly KP/definitional) features carry signal, echoing wagner2018case; k=6 subsets give surprisingly low rho and are ignored.
Evidence base	The wagner2018case performance dataset only. Metrics: FlexFolio algorithm-selection performance on subsets and Spearman rho of Shapley-value ranks (subset vs full), plus Shapley/marginal-contribution plots. No new algorithm runs; no statistical test beyond Spearman rho; no comparison with simpler subsetting (stratified random sampling, the categorised 60).
Stated limitations	Authors state the approach is problem-independent and will be extended to other ASlib problems, with future work on matrix completion for incomplete data and on a fair (not just representative) benchmark set and a diversity-and-fairness instance generator.

Strengths. ✓ Directly addresses a real, acknowledged problem: the 9720-instance suite is too expensive and ad-hoc subsetting is biased • ✓ Principled: uses the instance-algorithm performance matrix itself (not just instance features) to define representativeness • ✓ Validates preservation of two distinct structures (selection performance and Shapley-based ranking) • ✓ Reproducible (uses the public wagner2018case dataset and ASlib) • ✓ Reinforces the wagner2018case finding that only a few features carry signal

Weaknesses. ✗ Representativeness is defined and validated only relative to the same 2016-era 21-algorithm pool used to build the subset; no test of whether the subset generalises to modern methods (CoCo, CS2SA*) • ✗ No new algorithm experiments; the paper is entirely an analysis of an existing performance matrix • ✗ The k=6 subsets give 'surprisingly low' rho and are discarded without explanation, a red flag about the clustering's stability • ✗ No statistical test beyond Spearman rho; no comparison with simpler subsetting or with the categorised 60 • ✗ The recommended subset is not published as a named, adopted benchmark; the community kept using the categorised 60 • ✗ Circular: the subset is chosen to preserve FlexFolio performance and then evaluated by FlexFolio performance

[44] [Naumov and Pawluś \(2021\)](#) — *Identifying the Optimal Packing and Routing to Improve Last-Mile Delivery Using Cargo Bicycles*

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained

Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	cargo-bike CVRP	Family (L1/L2)	Problem def./benchmark/analysis / Application
Benchmark	—	Size	Cities: 10 Items: KP-Types:
Metric	total route time (s), total route distance (m).	Stat. test	none
Key result	all three find near-identical best (Bin-Pack-3D =1525s/7914m; MTSP =1526s/7913m; CVRP =1525s/7914m); CVRP reaches the optimum in 2s while MTSP does not, so CVRP is preferred for dynamic real-life use.		
Competition	N	Code	Unknown
Budget / Runs	MTSP/CVRP limited to 2s / —	Hardware	—

Abstract. Naumov and Pawluś (2021) applied TTP-inspired models to the **real-world optimization of cargo bicycle logistics** in urban areas subject to motorized traffic restrictions. The paper integrates concepts from the KP (selecting which goods to carry), the TSP (finding efficient delivery routes), and the TTP (modeling the speed reduction due to cargo weight) into a unified optimization framework for electric cargo bike routing. Optimization algorithms implemented in Python are evaluated on synthetically generated datasets with 10 delivery points, providing practical evidence that TTP-derived models have direct applicability to last-mile urban logistics. While no comparison to the TTP benchmark literature is provided, the work contributes to the growing body of evidence that the TTP's nonlinear speed-weight coupling captures a relevant operational characteristic of load-dependent vehicle routing in real-world settings.

Contribution	Applies the load-dependent-speed idea to last-mile cargo-bicycle delivery: formulates a combined 3D-bin-packing + routing problem for a fleet of cargo bikes (vehicle starts full and unloads at deliveries, the opposite of standard TTP) and compares three Python/OR-Tools approaches: a deterministic exhaustive Bin-Pack-3D benchmark, MTSP-First, and CVRP-First (both Cheapest-Arc + guided local search).
Authors' claims	All three approaches find near-identical route parameters (total time 1525/1526/1525 s) but different node sequences; CVRP-First reaches the optimum in 2 s and is best suited for real-life dynamic settings; MTSP-First does not reach the optimum; Bin-Pack-3D is only a non-scalable benchmark.
Evidence base	A single synthetic instance: 4 bicycles, about 7-10 consignments, capacity 100 kg, semi-random coordinates and cargo dimensions/weights. Bin-Pack-3D enumerated about 828,000 feasible packings in about 864 s; MTSP/CVRP given a 2 s solver limit. Metrics: total route time and distance, plus histograms of feasible-solution populations. No statistical test. No standard TTP benchmark or comparison with any TTP solver. Data on GitHub.
Stated limitations	Authors explicitly state Bin-Pack-3D does not scale, and that the models are a theoretical framework to be further developed and validated on real-world data; future work is LIFO 3D packing and a constraint-programming formulation of the whole problem.

Strengths. ✓ Genuine application-motivated adaptation of the load-dependent-speed idea to cargo-bike last-mile delivery, with 3D-loading feasibility

- ✓ Three approaches compared, including a deterministic exhaustive benchmark to establish the true optimum on the test instance
- ✓ Open data and Python/OR-Tools implementation; grounded in an EU Horizon 2020 project
- ✓ Honest that it is a theoretical framework not yet validated on real data

Weaknesses. ✗ Evaluated on a single tiny synthetic instance; no instance set, scaling study or sensitivity analysis; the efficiency claims rest on one data point

- ✗ Not connected to the standard TTP benchmark or literature; no comparison with any TTP solver
- ✗ The problem is really a load-dependent CVRP with 3D loading, not TTP1; the profit-maximising thief objective is absent (all consignments must be delivered)
- ✗ No statistical testing; results are single-run route parameters
- ✗ The three approaches give essentially identical objective values, so the comparison does not discriminate them on quality; 'CVRP-First is best' rests on solver speed on one instance
- ✗ Published in an energy journal, with limited methodological depth on the optimization side

[45] Zhang et al. (2021) — Solving the Traveling Thief Problem Based on Item Selection Weight and Reverse-Order Allocation

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	RWS	Family (L1/L2)	Constructive/iterative heuristic / Scoring heuristic
Benchmark	—	Size	Cities: 76; 100; 130; 159; 280; 574; 724; 1000; 1304; 1577; 2103; 3038; 4461; 7397; 11849; 13509; 14051; 15112; 18512; 33810 Items: 1x; 5x; 10x KP-Types: bsc; uco; usw
Metric	objective value, RDI;	Stat. test	—
Baselines	S5, MATLS, CS2SA*;		
Key result	RWS strongest on Category~C (high capacity, uco); in the later cross-comparison of nguyen2023simulated RWS ranks best on Cat~C (avg rank 1.85) and second on Cat~B (2.3);		
Competition	N	Code	Yes
Budget / Runs	600s / 10	Hardware	—

Abstract. Zhang et al. (2021) proposed **RWS (Reverse-order Weighted Selection)**, an item-selection algorithm for the TTP that introduces two key innovations over prior scoring heuristics. First, a novel **EW scoring formula** applies an exponent of 1.5 to the item weight in the profit-to-weight scoring, amplifying the negative contribution of heavy items and reducing the tendency to include items whose weight penalty dominates their profit gain at the current stage of the tour. Second, **reverse-order allocation** selects items starting from the last city and working backward toward the first, ensuring that heavy items are loaded near the tour's end where they are carried over the shortest distances and thus incur the least travel cost penalty. Evaluated with the standard Category A/B/C classification on the full benchmark, RWS proves particularly effective in **Category C**, where high capacity and uncorrelated item types allow aggressive weight-aware selection to substantially improve total profit.

Contribution	RWS, a TTP1 method for packing-plan construction on a repeatedly resampled LKH tour: an item-scoring function that raises the weight to an exponent and divides by distance-to-tour-end, plus reverse-order allocation (select good items in cities near the end of the tour first, so they are carried a short distance), wrapped in an S5-like restart loop.
Authors' claims	RWS is competitive on many instances of various sizes and types; the item weight has a greater effect on profit than value or location due to the cumulative effect; RWS beats MATLS, S5 and CS2SA* on Category B (best avg rank 2.25) and Category C (best avg rank 1.7) but is worst on Category A; solver-1 (exponent 1.5) beats solver-2 (exponent 1.0) especially on large instances.
Evidence base	The categorised 60-instance benchmark: 20 TSP base graphs (76-33810 cities) at the standard 3 category points (A/B/C); a 4th point (Category D) is defined but not reported in the result tables. 10 runs, 600 s, i5-8500. Friedman test + Nemenyi post-hoc per category, relative standard deviation reported. Baselines MATLS, S5, CS2SA* run under identical conditions. Exponent 1.5 and per-category beta set by informal 'experimental study over dozens of times'. Public source code (via a 2023 correction).
Stated limitations	Authors explicitly state that the premature consideration of the weight's cumulative effect and the reverse item selection reduce the search space, making the algorithm 'not prominent in small-scale examples' (Category A); future work is bi-objective TTP, more than two components, and a list of metaphor metaheuristics.

Strengths. ✓ Proper statistics: Friedman test + Nemenyi post-hoc per category, 10 runs, relative standard deviation, 600 s standard budget • ✓ Baselines (MATLS, S5, CS2SA*) run under identical conditions on the same machine • ✓ Clear, well-motivated mechanism: reverse-order allocation exploits the cumulative-weight effect • ✓ Exponent ablation (solver-1 vs solver-2) with box plots across categories • ✓ Public source code; honest that RWS is weak on Category A and explains why

Weaknesses. ✗ Defines a fourth Category D (uncorrelated, high capacity, nine items/city) but reports no results for it in the main tables • ✗ Only 3 baselines (MATLS, S5, CS2SA*); no CoCo/PGCH (2019), the state of the art by the 2021 publication date, and no MA2B or MMAS • ✗ RWS wins only Categories B and C; on Category A it is beaten by both S5 and MATLS • ✗ The exponent and per-category beta are set by informal trial, with no systematic tuning or sensitivity table • ✗ The 'weight matters more than value/location' claim is motivational and not isolated from the reverse-order-allocation change • ✗ Mean + relative standard deviation only; no best-known or optimal reference • ✗ The future-work list of metaphor metaheuristics signals a weak methodological direction

[46] Nikfarjam et al. (2022a) — Co-evolutionary diversity optimisation for the traveling thief problem

Variant	TTP1	Objective / #obj	Single-objective / 1
----------------	------	-------------------------	----------------------

Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	Co-EA (QD+EDO)	Family (L1/L2)	Population-based metaheuristic / Co-evolutionary diversity
Benchmark	—	Size	Cities: 51; 152; 280 Items: 1x; 3x; 5x KP-Types: bsc; uco; usw
Metric	entropy $H(P_2)$, best TTP quality $z(p^*)$.	Stat. test	Kruskal-Wallis + Bonferroni and Mann-Whitney U (5%)
Baselines	QD-based EA nikfarjam2022evolutionary and standard EDO.		
Key result	Co-EA beats standard EDO in entropy on 14/18 (2 ties) and beats QD in quality $z(p^*)$ on 12/18 (e.g. avg best 19,943.8 vs. 19,861.8); Gamma_2 is the best self-adaptation; Co-EA converges faster with higher entropy and lower std than EDO.		
Competition	N	Code	Unknown
Budget / Runs	10^6 m fitness evals, =10% / 10	Hardware	—

Abstract. Nikfarjam et al. (2022a) proposed a **Co-evolutionary Algorithm (Co-EA)** for the TTP that simultaneously explores both **behavioral diversity**—measured in the space of TSP and KP objective scores—and **structural diversity**—measured by solution encoding distances. The framework bridges Quality Diversity methods, which excel at filling behavioral niches, and EDO methods, which maximize structural differences, by co-evolving two populations: one targeting behavioral space coverage and one targeting structural dispersion. Tested on instances with 51, 152, and 280 cities, Co-EA achieves substantially higher diversity scores than the baseline EDO methods reported in the literature, including those of Nikfarjam et al. (2022b). The results confirm that explicitly co-evolving complementary diversity objectives produces a richer and more informative solution set, with potential applications in multi-scenario decision support for real-world TTP-like problems.

Contribution	A co-evolutionary algorithm (Co-EA) that simultaneously maintains a quality-diversity population (MAP-Elites over tour length and item profit) and an evolutionary-diversity-optimisation population (max structural entropy subject to a quality slack); bi-level offspring (EAX + inner (1+1)-EA) with self-adaptation of the inner budget. Key advantage: does not require the optimum to be known a priori.
Authors' claims	Co-EA achieves significantly higher structural diversity than the baseline EDO algorithm on 14/18 instances and competitive solution quality (better mean best-TTP-score on 12/18); it also beats the QD-only baseline on quality on many instances; self-adaptation of the inner budget beats a fixed budget on all 18 instances; Co-EA converges faster and to higher entropy with lower variance than standard EDO.
Evidence base	The same 18 TTP instances as nikfarjam2022evolutionary (3 TSP base graphs, ≤ 280 cities). 10 runs, $10^6 \times m$ fitness evaluations, 10 percent quality slack. Statistics: Kruskal-Wallis + Bonferroni, Mann-Whitney U. Baselines are exclusively the authors' own two prior papers. Self-adaptation parameters 'based on preliminary experiments'. Peer-reviewed (PPSN 2022).

Stated limitations Future work: more complicated MAP-Elites survival selection; framed as a transition from benchmark problems to real-world problems where imperfect modelling is common.

Strengths. ✓ Peer-reviewed (PPSN 2022) • ✓ Removes the key limitation of prior EDO work: the optimum no longer needs to be known a priori • ✓ Unifies the QD and EDO diversity paradigms in one co-evolutionary loop • ✓ Self-adaptation of the inner budget with a principled mechanism, ablated on all 18 instances • ✓ Proper non-parametric statistics, 10 runs, large evaluation budget

Weaknesses. ✗ Same narrow instance set as its predecessor: 18 instances, 3 TSP base graphs, ≤ 280 cities; not the categorised subset • ✗ Baselines are exclusively the authors' own two prior papers; the whole QD-for-TTP line is one lab and self-referential • ✗ The contribution is diversity of solution sets, not solving TTP better; 'competitive quality' is against the authors' own QD baseline, not the state of the art • ✗ The diversity measure still adds two unnormalised entropies • ✗ Self-adaptation parameters 'based on preliminary experiments' without a reported sensitivity study • ✗ 10 percent quality slack fixed; the practical value of a diverse set of solutions all 10 percent worse than optimal is not established • ✗ 72-hour worst-case CPU budget per instance is impractical

[47] Nikfarjam et al. (2022b) — *Evolutionary diversity optimisation for the traveling thief problem*

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	EDO (entropy)	Family (L1/L2)	Population-based metaheuristic / Evolutionary diversity optimisation
Benchmark	—	Size	Cities: 51; 152; 280 Items: 1x; 3x; 5x KP-Types: bsc; uco; usw
Metric	diversity $H/H_e/H_i$; $=0.1$, $=50$.	Stat. test	Mann-Whitney U and Kruskal-Wallis + Bonferroni (5%)
Key result	DP yields higher edge diversity H_e , (1+1) EA higher item diversity H_i , H best overall; EDO outperforms the QD-based EA nikfarjam2022coevolutionary in diversity on most instances.		
Competition	N	Code	Unknown
Budget / Runs	10,000 iterations / 10	Hardware	—

Abstract. Nikfarjam et al. (2022b) investigated the TTP through the lens of **Evolutionary Diversity Optimization (EDO)**, shifting the research objective from finding a single high-quality solution to generating a **diverse set of high-quality solutions** that cover different structural regions of the solution space. The paper proposes a bi-level evolutionary algorithm designed to maximize a diversity measure—quantifying structural differences among solutions—subject to a quality constraint, and compares it against a **Quality Diversity (QD)** framework. Tested on instances with 51, 152, and 280 cities at item densities of 1x, 3x, and 5x across all KP types, the proposed two-stage EDO algorithm achieves significantly higher structural diversity than the QD-based approach on most benchmark configurations. This work establishes that the TTP is a valuable testbed for diversity-oriented evolutionary research, and that the entropy-driven EDO framework provides a superior diversity-quality trade-off compared to Quality Diversity for multi-component combinatorial problems.

Contribution	First application of evolutionary diversity optimisation (EDO) to a multi-component problem: a structural-diversity measure for TTP solutions (edge entropy + item entropy) and a bi-level EA that generates tours via EAX, computes packings (DP or (1+1)-EA), and uses EDO survival selection to keep a set of solutions that are all within a quality slack of the optimum but maximally diverse.
Authors' claims	Focusing on overall diversity brings greater item diversity than focusing only on item diversity (an artefact of the tour-first bi-level order); (1+1)-EA as KP operator gives more item diversity than DP (which gives identical packings for identical tours); using edge entropy as fitness is best for edge/total diversity with less computation; the EDO-EA gives significantly higher structural diversity than the QD-based EA on most instances and more robust populations.
Evidence base	18 instances from 3 TSP base graphs (eil51, pr152, a280; max 280 cities), 3 KP types, mixed item counts. Quality slack 10 percent, mu=50, 10 runs, 10,000 iterations. Statistics: Mann-Whitney U, Kruskal-Wallis + Bonferroni. Assumes the optimal/best-known objective is known a priori. Baselines are the authors' own QD method. arXiv preprint.
Stated limitations	Authors state the approach assumes the optimal/near-optimal objective is known a priori (in line with the EDO literature), and propose multi-objective-optimisation indicators for the edge/item diversity trade-off as future work.

Strengths. ✓ First to bring evolutionary diversity optimisation to a multi-component problem; a new research direction for TTP • ✓ Principled diversity measure grounded in the EDO/TSP literature • ✓ Careful ablations (DP vs (1+1)-EA, and three fitness variants) with proper non-parametric tests and 10 runs • ✓ Direct comparison with the QD-based alternative on the same instances • ✓ A robustness experiment operationalises why diversity matters (alternatives available when an edge/item becomes unavailable)

Weaknesses. ✗ Only 18 instances, 3 TSP base graphs, max 280 cities; not the categorised subset • ✗ Assumes the optimal or best-known objective is known a priori for every instance, which does not hold for real TTP instances • ✗ The contribution is diversity of solution sets, not solution quality; it does not advance the state of the art in solving TTP • ✗ arXiv preprint; overlap with the group's other diversity papers risks double-counting • ✗ The diversity measure adds two unnormalised entropies on different scales • ✗ The 'overall diversity beats item diversity for item diversity' result is an artefact of the tour-first algorithm design, not a general insight • ✗ Fixed small budget (10,000 iterations); convergence not reached on some instances

[48] Namazi et al. (2023) — Solving Travelling Thief Problems using Coordination Based Methods

Variant	TTP1	Objective / #obj	Single-objective / 1
----------------	------	-------------------------	----------------------

Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	CoCoP / CoCoL	Family (L1/L2)	Single-solution metaheuristic / Coordinated local search (CoCo)
Benchmark	—	Size	Cities: 76; 100; 130; 159; 280; 574; 724; 1000; 1304; 1577; 2103; 3038; 4461; 7397; 11849; 13509; 14051; 15112; 18512; 33810 Items: 1x; 5x; 10x KP-Types: bsc; uco; usw
Metric	RDI per category (Table~2) + new best objective values (Table~5);	Stat. test	Wilcoxon signed-rank (p=0.00001 CoCoP vs. S5) + 95% CI
Baselines	MATLS, S5, CS2SA*, and MEA2P wu2018evolutionary-style memetic;		
Key result	CoCoP/CoCoL best on almost all instances in all 3 categories; e.g. Cat~A eil76 CoCoP/CoCoL = 100.0 vs. S5 = 100.0, CS2SA* = 15.4; new best-known objective values on most mid/large instances (e.g. pla33810-CatC 5.89!x!10^7);		
Competition	N	Code	Yes
Budget / Runs	10min (also 1h robustness test) / 10	Hardware	—

Abstract. [Namazi et al. \(2023\)](#) presented a **comprehensive extension of the CoCo framework** that systematically progresses from hand-crafted to machine-learned coordination heuristics. Building on the PGCH of [Namazi et al. \(2019b\)](#), the paper introduces the **Search Guided Coordination Heuristic (SGCH)**, which refines item reassignment using a look-ahead profit evaluation, and the **Learning Guided Coordination Heuristic (LGCH)**, which trains an **online neural network** to predict whether each item should be collected based on normalized profitability trends and city positions, replacing the static trend-based rules of PGCH with a non-linear classifier that adapts to each instance. The KP search is formalized as a **Marginal Bit-Flip Search (MBFS)** governed by a **Coordinated Item Selection Heuristic (CISH)** that targets items at the profitability boundary. Evaluated on the full 60-instance benchmark across all categories and KP types, the full CoCo solver significantly outperforms all prior state-of-the-art methods, demonstrating that machine learning can automate and improve upon the coordination heuristics that were previously designed by domain experts.

Contribution A comprehensive journal treatment of coordination-based TTP solving: a formal framework (TSP/KP components, cyclic-tour quality NP-hard, item-profitability-ratio trend analysis), three coordination heuristics inside the 2-opt tour search (SGCH search-guided, PGCH profit-guided, LGCH learning-guided / neural classifier), and a marginal bit-flip search for the KP component; the final solvers are CoCoP (PGCH + marginal bit-flip) and CoCoL (LGCH + marginal bit-flip). Public source code.

Authors' claims	A simple local-search coordination approach (SGCH) does not work in TTP (too slow, too few restarts); PGCH and LGCH vastly outperform no-coordination on RDI, especially on Categories B and C; CoCoP and CoCoL significantly outperform MATLS, S5 and CS2SA* (Wilcoxon signed-rank, p about $1e-5$), with CoCoP the best; marginal bit-flip beats simulated-annealing search because it converges faster and restarts more; CoCoP obtains new best objective values on most large instances and beats MEA2P on large instances while running orders of magnitude faster.
Evidence base	The categorised 60-instance benchmark (20 per A/B/C), 10 runs, 10-min timeout plus a 1-hour robustness check. RDI is pool-relative (higher is better, 0 = worst run, 100 = best run across the 5 compared solvers). Wilcoxon signed-rank test on RDI; extensive appendix with per-instance median/mean/stddev. Baselines MATLS, S5 (self-reconstructed), CS2SA* (objectives corrected, different tour per run) and MEA2P (separate 2500-restart protocol). Public source code (github.com/majid75/CoCo). Open access.
Stated limitations	Authors state SGCH is too slow (motivating PGCH/LGCH), that Category A is hard for LGCH because a classification error on the only item cannot be compensated, and that LGCH's learned pattern is 'arguably not very high quality' so its prediction does not help once better solutions are found; they also document that CS2SA* and several other methods use a faulty rounded-distance objective that makes them incomparable.

Strengths. ✓ Rigorous comprehensive journal treatment: formal framework with proofs, an NOCH/SGCH/PGCH/LGCH ablation, a KP-search ablation, 10 runs, Wilcoxon tests, a 1-hour robustness check, and an extensive per-instance appendix • ✓ Full categorised 60-instance benchmark with baselines re-run from source under identical conditions • ✓ Openly documents and corrects a systemic rounded-distance objective error in CS2SA*/el2016/el2018/maity2020/zhang2021 and explains how it distorted their conclusions, a genuine service to the field • ✓ Establishes CoCoP as the state of the art on the categorised benchmark and reports many new best-known objective values • ✓ Public source code; open access; deep diagnostic analysis of why coordination and restarts help

Weaknesses. ✗ RDI is pool-relative (higher is better, 0 = worst run); the reported values depend entirely on which 5 solvers are in the pool and are not absolute-optimality measures • ✗ The main comparison pool is only MATLS, S5 and CS2SA*; MEA2P is compared under a different protocol and MA2B, MMAS, CoSolver2B, RES_GLS and SAVI are not compared • ✗ CS2SA* is kept in the normalisation pool even though the authors argue it is incomparable in its original form • ✗ LGCH, the machine-learning coordination heuristic and a headline contribution, is consistently beaten by PGCH and adds little • ✗ Statistical testing is Wilcoxon signed-rank on aggregate RDI; no per-category omnibus test or critical-difference analysis • ✗ New best-known values are relative to the Wuijts and Thierens 2019 / Wagner et al. 2018 reference values, not an independent optimum • ✗ 58 pages of heavy notation; the practical takeaway (PGCH + marginal bit-flip + restarts) is buried • ✗ The 1-hour-vs-10-min check shows 'similar' performance, i.e. the method has largely converged by 10 min and stops improving, a ceiling not discussed as such

[49] [Nguyen et al. \(2023\)](#) — *Simulated Annealing with Dynamic Programming-based Vertex Insertion for Efficiently Solving the Traveling Thief Problem*

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting

Algorithm	SAVI	Family (L1/L2)	Single-solution metaheuristic / Simulated annealing (DP-hybrid)
Benchmark	—	Size	Cities: 76; 100; 130; 159; 280; 574; 724; 1000; 1304; 1577; 2103; 3038; 4461; 7397; 11849; 13509; 14051; 15112; 18512; 33810 Items: 1x; 5x; 10x KP-Types: bsc; uco, usw
Metric	mean objective, RSD (%), average rank (1-4, lower better);	Stat. test	Wilcoxon signed-rank
Baselines	S5, RWS, CS2SA*;		
Competition	N	Code	Yes
Budget / Runs	600s / 10	Hardware	AMD Ryzen~7 5800U 1.9GHz, Java

Abstract. [Nguyen et al. \(2023\)](#) introduced **SAVI**, combining **Simulated Annealing** for tour search with an efficient **Dynamic Programming-based vertex insertion** procedure for tour improvement. The DP insertion evaluates the profit gain of reinserting each city into all possible positions in the current tour in $O(n)$ time by reformulating the evaluation as a recursive computation over the tour structure, a significant improvement over the $O(n^2)$ complexity of naive insertion enumeration. The SA component controls the acceptance of tour modifications, allowing the algorithm to escape local optima in the tour space while the DP-insertion procedure continuously seeks high-quality reinsertion moves within the current neighborhood. Evaluated on the full benchmark (76 to 33,810 cities, all item densities and KP types), SAVI produces competitive results against state-of-the-art algorithms, with the greatest gains on medium and large instances where tour quality has the strongest impact on the final packing plan.

Contribution	SAVI, which extends CS2SA* by replacing the Insertion operator with InsertionDP: a dynamic-programming vertex-insertion that computes the objective for every insertion position in $O(1)$ via a recursive update, reducing the full insertion search from $O(n^3)$ to $O(n^2)$, inside a CoSolver framework (Delaunay 2-opt, SA with vertex insertion, SA with bit-flip, bit-flip boosting).
Authors' claims	SAVI outperforms S5, CS2SA* and RWS on medium and large instances, especially Categories B and C; SAVI is weakest on Category A (few items, so altering the tour order does not help); SAVI converges well within 2-3 minutes on large instances.
Evidence base	The categorised 60-instance benchmark: 20 TSP base graphs x 3 categories (A F1/T3/C1, B F5/T2/C5, C F10/T1/C10). 10 runs, 600 s, AMD Ryzen 7. Wilcoxon signed-rank test between the best-mean algorithm and each other, plus average ranking per category. Baselines S5, CS2SA*, RWS re-run in Java under identical conditions. Public source code. Per the average rankings, SAVI is best only on Category B (1.75); it is the WORST method on Category A (3.0) and 2nd on Category C (behind RWS).
Stated limitations	Authors explicitly state SAVI is weakest on Category A because SA with vertex insertion does not help when there are few items and can reduce performance; future work is enhancing the KP component, co-evolutionary / multi-objective extensions and linkage learning.

Strengths. ✓ Concrete algorithmic contribution: InsertionDP reduces the vertex-insertion evaluation from $O(n^3)$ to $O(n^2)$ via a clean recursive objective update • ✓ Full categorised-style evaluation (20 base graphs x 3 categories), 10 runs, 600 s standard budget • ✓ Wilcoxon signed-rank test per instance with +/-/= marking and average ranking per category • ✓ Baselines (S5, CS2SA*, RWS) re-run in the same language under identical conditions • ✓ Public source code; honest that SAVI loses on Category A and only leads Category B

Weaknesses. ✗ SAVI wins only Category B; it is the worst method on Category A and 2nd on Category C, so ‘highly competitive on medium and large instances’ oversells a category-B-only lead • ✗ The trials parameter uses an ad-hoc formula adapted from CS2SA* with no tuning study • ✗ Only 3 baselines; no CoCo/CoCoP (2023), the state of the art by the publication date, and no MATLS or MA2B • ✗ SA parameters taken from CS2SA*, not re-tuned for SAVI • ✗ The Wilcoxon test compares only the best-mean algorithm vs each other per instance; no omnibus test or correction • ✗ Mean + relative standard deviation only; no best-known or optimal reference • ✗ Incremental: SAVI is CS2SA* with the Insertion operator swapped for InsertionDP

[50] Rodríguez et al. (2023) — A Sequence-Based Hyper-Heuristic for Traveling Thieves

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	Sequence-based HH	Family (L1/L2)	Hyper-heuristic / Sequence-based
Benchmark	—	Size	Cities: 5 Items: varying between 1 to 10 per city KP-Types:
Metric	achieved profit and normalized profit score ([0,1]).	Stat. test	none (distributional/violin analysis)
Baselines	1000 randomly generated operator sequences (length 64) and instance-specialized HHs vs. general HHs; no comparison with literature TTP solvers.		
Key result	trained HHs lie above the 85th percentile of random sequences (random median ~100, trained median ~300 profit on instance~1); general HH (90%) reaches normalized score >0.7 on unseen correlated instances and beats specialized HHs on 16/20 instances; on uncorrelated Set~3 a 100% threshold gives normalized profit >0.69 on all instances.		
Competition	N	Code	On request
Budget / Runs	— / 10 per HH (20 SA-trained sequences in preliminary stage)	Hardware	—

Abstract. [Rodríguez et al. \(2023\)](#) proposed a **sequence-based selection hyper-heuristic** for the TTP in which an entire solution is constructed by executing a learned sequence of elementary operators, each operator either advancing the tour by selecting the next city to visit or extending the packing plan by selecting the next item to collect. The hyper-heuristic searches for operator sequences of fixed length that perform well across multiple TTP instances, comparing **instance-specific sequences** tuned for individual instances against **general-purpose sequences** optimized across the full instance set. The central finding is that general-purpose sequences consistently outperform instance-specific ones, suggesting that the most effective operator orderings capture structural properties common to all TTP instances rather than exploiting idiosyncratic instance features. Applied to small synthetic instances with 5 cities, the study establishes sequence-based hyper-heuristics as a promising direction for developing robust, training-free TTP solvers.

Contribution	A sequence-based selection hyper-heuristic for TTP: the hyper-heuristic is an array of low-level operators (one greedy movement heuristic; three item-selection heuristics), executed once through, trained by simulated annealing over sequence-perturbation operators, with a 'democratic removal' step that trims operators disregarded on most training instances; plus a 5-city TTP instance generator.
Authors' claims	The trained sequence-based hyper-heuristic outperforms randomly generated sequences (median profit about 300 vs about 100, and more consistent); trained general sequences surpass per-instance specialised sequences; simulated annealing is a valid training method; the work is 'a stepping stone toward a more robust solver'.
Evidence base	Self-generated instances only: 60 instances across 3 sets, ALL with 5 cities. The only baseline is 1000 randomly generated operator sequences. No standard benchmark, no comparison with any TTP solver (not even a greedy PackIterative or S1), no statistical test (only violin plots), no RDI or best-known reference. Fixed RNG seed; MDPI open access with the generator described.
Stated limitations	Authors call it a 'base model that can be improved upon', note it 'may hinder performance if problems are too different', and frame it as a 'stepping stone'.

Strengths. ✓ Novel hyper-heuristic representation: a plain sequence of operators with no problem-feature vector, simple and interpretable • ✓ The 'democratic removal' mechanism to trim useless operators from the learned sequence is a sensible idea • ✓ Studies generalisation explicitly (train on one set, test on same-kind and different-kind sets) • ✓ Reproducible (fixed RNG seed, described generator, open access) • ✓ Honest that it is a preliminary base model

Weaknesses. ✗ ALL experiments are on self-generated 5-CITY instances; this is a toy, not the TTP as studied by the field, and no standard benchmark instance is used anywhere • ✗ The only baseline is random operator sequences; no comparison with any TTP heuristic, so 'outperforms random sequences' is near-vacuous • ✗ No statistical test; results are violin plots of raw profit • ✗ The single greedy movement heuristic fixes the tour by a greedy rule; the hyper-heuristic only reorders item-selection operators and cannot improve the routing • ✗ No RDI, best-known reference or absolute quality context; objective values are frequently negative even for the trained model • ✗ SA training parameters chosen informally • ✗ A proof-of-concept on toy instances with no connection to the TTP state of the art

[51] Ni and Spector (2024) — *Leveraging Symbolic Regression for Heuristic Design in the Traveling Thief Problem*

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained

Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	Symbolic regression init	Family (L1/L2)	Learning-based / Symbolic regression
Benchmark	—	Size	Cities: 280; 575; 1323; 3038 Items: 1x; 5x; 10x KP-Types: bsc; uco; usw
Metric	objective value and number of objective evaluations (rank frequencies 1st/2nd/3rd).	Stat. test	none (rank-frequency analysis)
Baselines	Insertion, packIterative, DH, SH.		
Key result	6T ranks 1st in objective on ~80% of the 450 instances (packIterative mostly 2nd, Insertion mostly 3rd) and uses fewer objective evaluations than packIterative, though it is slower than the cheap Insertion heuristic.		
Competition	N	Code	Yes
Budget / Runs	— / 30 per stochastic heuristic (1 for deterministic Insertion/packIterative/DH/SH)	Hardware	Amherst College HPC (NSF grant)

Abstract. [Ni and Spector \(2024\)](#) employed **symbolic regression** to automatically discover interpretable mathematical expressions that predict features of near-optimal packing plans from the structural properties of TTP instances. The discovered expressions are then used to design **efficient population initialization schemes** for Genetic Algorithms, seeding the population with near-optimal packing plans that closely match the structure of the best-known solutions. Unlike random or heuristic initialization, the symbolic regression-derived scheme provides individuals that are already in promising regions of the binary packing space, dramatically accelerating convergence. Evaluated on instances with 280 to 3,038 cities at all three item densities and KP types, the approach validates significantly faster convergence and better final solutions compared to prior initialization methods, demonstrating that learning from near-optimal solution structure via symbolic regression is a powerful and interpretable approach to knowledge-guided population initialization.

Contribution	Uses symbolic regression twice: first to characterise near-optimal packing plans in a 2D feature space of standardised item profitability ratio and standardised distance-to-tour-end (finding a smooth nonlinear boundary separates packed from unpacked items) and to evolve which polynomial feature combinations matter (feature sets 2T-6T); then to predict the optimal boundary-line coefficients directly from TTP instance features (they depend almost entirely on the capacity factor). This yields a family of fast, interpretable packing-INITIALISATION heuristics refined by a bidirectional step search and a small (1+1) EA on the coefficients.
Authors' claims	High-quality packing plans have a smooth curved IPR/rDist boundary that hand-designed heuristics (0-1 tunable parameters) cannot match; on 450 unseen test instances the 6T heuristic has the highest frequency of best objective and generally needs fewer objective-value computations than PackIterative, though more than Insertion; the 4T heuristic needs the fewest evaluations.

Evidence base	Training on 4 base graphs (a280, rat575, rl1323, pcb3038; 360 TTP instances). Testing on 5 different base graphs (eil51 to pla85900; 450 instances), 30 runs per heuristic, shared starting tour. Baselines PackIterative and Insertion (deterministic, run once). Results reported ONLY as ranking-frequency bar charts (1st/2nd/3rd counts) with NO objective values, NO gap to best-known, NO statistical test, NO effect sizes. Near-optimal training packings from a (1+1) EA at 1e6 generations. Two anomalous evolved solutions were manually replaced with linear fits. arXiv / EuroGP-style preprint.
Stated limitations	Authors state the best algorithms will not use a single initialisation heuristic (they recommend also always running Insertion), that separate coefficient-prediction functions are needed per KP type, and that a single cross-instance function and other weight/profit distributions are future work.

Strengths. ✓ Genuinely novel and interpretable: uses symbolic regression to discover the geometric structure of good packing plans, then again to make the resulting metaheuristic fast enough to be an initialisation heuristic • ✓ The critique that hand-designed scoring heuristics are limited by having 0-1 tunable parameters (a fixed shape) is sharp and well-argued • ✓ Careful train/test separation: 4 base graphs for training, 5 different base graphs (incl. the smallest and largest) for the 450 test instances • ✓ 30 runs per heuristic, shared starting tour, hardware/language-independent cost proxy (evaluation count) • ✓ Honest: recommends still running Insertion alongside and positions this as an initialisation heuristic, not a full solver

Weaknesses. ✗ Results reported ONLY as ranking-frequency bar charts; no objective values, no gap to best-known/optimal, no statistical test, no effect sizes, so the magnitude of any improvement is unknown • ✗ Evaluates only the packing-initialisation step in isolation, not its effect on a full solver's final objective • ✗ Baselines are packing-init heuristics only (PackIterative, Insertion); no comparison with any complete method or with simply running a longer (1+1) EA • ✗ The 6T heuristic itself contains a (1+1) EA plus a bidirectional step search; it is not a simple deterministic heuristic • ✗ Separate coefficient-prediction functions must be evolved per KP type; no generalisation to unseen weight/profit distributions (acknowledged) • ✗ Two anomalous symbolic-regression solutions were manually replaced with linear fits, an ad-hoc patch • ✗ arXiv preprint; the claim that the heuristics 'work better with larger instances' is a hypothesis weakly supported by a discreteness argument

[52] Nikfarjam et al. (2024) — *On the Use of Quality Diversity Algorithms for the Travelling Thief Problem*

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	BMBEA (QD, journal)	Family (L1/L2)	Population-based metaheuristic / Quality diversity
Benchmark	—	Size	Cities: 51; 152; 280; 575; 1000; 2152; 4461 Items: 1x; 3x; 5x KP-Types: bsc; uco; usw

Metric	TTP objective score (average and best) and quality-diversity coverage of the behavioural space.	Stat. test	pairwise statistical-significance test (5%)
Baselines	(+1)EA and EnBEA (entropy-based structural-diversity EA).		
Key result	with the DP packing operator BMBEA gives the highest mean on 11/18 instances and statistically outperforms (+1)EA and EnBEA on most instances at 10^5 iterations; it improves the best-known TTP value on 2 large instances; MAP-Elitism boosts both diversity and solution quality.		
Competition	N	Code	No
Budget / Runs	10^4 and 10^5 iterations (also up to 72h CPU) / multiple independent runs (statistical)	Hardware	n/r

Abstract. Nikfarjam et al. (2024) explored the use of **MAP-Elites Quality Diversity** optimization for the TTP, constructing a behavioral map in which cells are indexed by the TSP and KP objective scores of solutions, and each cell retains the highest-quality TTP solution whose subproblem scores fall within the cell's boundaries. This behavioral discretization allows the algorithm to **systematically explore the interaction between routing and packing objectives**, revealing the distribution of high-quality TTP solutions across the joint TSP-KP objective space. The MAP-Elites framework employs well-known TSP and KP search operators as variation sources and is evaluated on instances with 51 to 4,461 cities at three item densities across all KP types. The approach improves several best-known solution values on standard benchmark instances and provides novel structural insights into how TSP and KP quality trade off in optimal TTP solutions.

Contribution	The first quality-diversity (MAP-Elites) study for a combinatorial optimization problem: BMBEA, a bi-level MAP-Elites-based EA over the tour-length / item-profit behavioural space (EAX or 2-OPT for tours; DP or (1+1)-EA for packing; MAP-Elitism survival), plus a relaxed-map method that sets the grid bounds from the initial population to remove two hard-to-tune parameters, and comparisons with a ($\mu+1$)-EA and an entropy-based EA. This entry covers a single research study reported in two publications: the GECCO 2022 conference paper (nikfarjam2022use) and its extended, corrected ACM TELO 2024 journal version (nikfarjam2024use, the primary reference), which the authors state already contained the main contributions and merely corrects an implementation error and adds the relaxed-map method and larger instances.		
Authors' claims	QD/MAP-Elites is applicable to TTP and shows that the best TTP solution lies at a behavioural descriptor of roughly $(1.005 \times \text{optimal tour length}, 0.9\text{-}0.95 \times \text{optimal packing profit})$, i.e. both sub-problems slightly sub-optimal; EAX dominates 2-OPT and (1+1)-EA competes with DP while being far faster on large instances; BMBEA improves the best-known TTP solution on a few benchmark instances; MAP-Elitism beats a ($\mu+1$)-EA and an entropy-based EA on TTP score; the relaxed-map method matches the tuned method while removing two parameters.		
Evidence base	34 TTP instances (eil51/pr152/a280 small, plus rat575/u2152/fnl4461/dsj1000 up to 4461 cities), one per sub-group. 10 runs, 10^4 or 10^5 iterations or 72 h. Kruskal-Wallis + Bonferroni at 5 percent. Best-known values from chagas2022weighted and wuijts2019 (both relative to the 2016-era Wagner pool). Different termination criterion from the standard 600 s. The linked repository shares instances and the best solutions found, but no source code.		
Stated limitations	Authors state the grid-bound parameters need careful per-instance tuning in the prefixed method (motivating the relaxed method), that DP does not scale on larger instances, and that they corrected incorrect results from the conference version due to an implementation error; future work is CVT-MAP-Elites and relaxing the quality constraint.		

Strengths. ✓ Peer-reviewed journal (ACM TELO); the first quality-diversity / MAP-Elites study for any combinatorial optimization problem • ✓ Openly corrects an implementation error from the GECCO 2022 conference version • ✓ Thorough experimental design: operator ablation, survival-selection ablation, two iteration budgets, and the relaxed-map contribution • ✓ Proper non-parametric statistics (Kruskal-Wallis + Bonferroni), 10 runs • ✓ Reports improved best-known TTP solutions on a few benchmark instances (a concrete quality result) • ✓ Instances and best solutions shared in a public repository; the relaxed-map method removes two hard-to-tune parameters • ✓ Insightful behavioural-space finding: the best TTP solution needs both sub-problems slightly sub-optimal

Weaknesses. ✗ 34 instances but only one per sub-group; max 4461 cities; not the largest benchmark instances and not the categorised 60-subset • ✗ Best-known baselines come from chagas2022weighted / wuijts2019, which compare to the 2016-era 21-algorithm Wagner pool, not to CoCo+/CS2SA*/SAVI-era results • ✗ Different termination criterion ($10^4/10^5$ iterations or 72 h) than the standard 600 s; not like-for-like with the mainstream literature (acknowledged) • ✗ The primary contribution is behavioural-space characterisation and diverse-set generation, not solving TTP better • ✗ Heavily self-referential: builds on and compares against the same group's EDO / Co-EA / EnBEA papers • ✗ 72-hour worst-case CPU budget per instance is enormous and impractical • ✗ DP as KP operator does not scale (about 19,000 s per run on a 130-item instance), mitigated only by switching to (1+1)-EA

Remark. *Extended and corrected journal version of the GECCO 2022 conference paper; both publications represent the same underlying research study and are therefore merged to avoid double counting.*

[53] Wu et al. (2024) — ReinforceNS: Reinforcement Learning-based Multi-start Neighborhood Search for Solving the Traveling Thief Problem

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	ReinforceNS	Family (L1/L2)	Learning-based / Reinforcement learning
Benchmark	—	Size	Cities: 76; 100; 130; 159; 280; 574; 724; 1000; 1304; 1577; 2103; 3038; 4461; 7397; 11849; 13509; 14051; 15112; 18512; 33810 Items: 1x KP-Types: bsc; uco, usw
Metric	best/avg objective, RDI, 95% CI;	Stat. test	95% CI
Baselines	MATLS, S5, CoCo namazi2023solving, GECCO-2023 competition best;		
Key result	12 new best-known on 18 competition instances; beats CoCo on 38/60 instances; best-count (out of 20/cat.) ReinforceNS = 14/13/11 (A/B/C) vs. CoCo = 6/13/9, S5 = 0/0/0, MATLS = 0/0/0;		

Competition	Y	Code	No
Budget / Runs	600s / 10	Hardware	Intel Xeon Platinum 8255C 2.50GHz, 40GB

Abstract. Wu et al. (2024) proposed **ReinforceNS**, the first **reinforcement learning-based multi-start neighborhood search** algorithm for the TTP. A preprocessing phase identifies and eliminates dominated items and city-edge combinations using problem-specific bounds, reducing the effective search space before optimization begins. High-quality initial solutions are generated by combining a high-quality TSP tour with an iterated greedy packing heuristic. The core of the algorithm is a **reinforcement learning agent** that dynamically selects which neighborhood structure to apply at each iteration, learning from the rewards of accepted improvements which operators are most productive at different stages of the search. A post-optimization phase applies targeted local search to further refine the best solution. Evaluated on 60 representative benchmark instances across Categories A, B, and C (1x item density, all KP types), ReinforceNS achieves 12 new best-known results on 18 instances from a recent TTP competition, demonstrating the effectiveness of learning-guided adaptive operator selection for high-performance TTP optimization.

Contribution	ReinforceNS, the first reinforcement-learning-based method for TTP: a multi-start iterated local search where a tabular Q-learning policy (initialised from LKH alpha-nearness) guides the choice of 2-opt exchange partner via epsilon-greedy selection, with LKH-style neighbourhood reduction, an iterated greedy packing initialisation, and a post-optimisation phase (item flipping + route refining).
Authors' claims	ReinforceNS outperforms MATLS, S5 and CoCo on solution quality within the same 600 s limit and is the most stable (lowest variance); it beats the GECCO 2023 TTP competition best average on 12 of 18 instances (12 new results); it beats CoCo on 38 of 60 instances; an ablation shows the RL mechanism helps on 12/12/15 of the three 20-instance category sets.
Evidence base	60 categorised benchmark instances (one instance file per base graph per category, '-1' suffix) plus 18 GECCO-2023 competition instances ('-r'). C++ -O3, Xeon Platinum 8255C, 600 s, 10 runs. RDI is pool-relative (higher is better, 0 = worst, 100 = best across MATLS/S5/CoCo/ReinforceNS). Baselines re-run on the same platform. 95 percent confidence intervals; RL vs no-RL ablation on all 60. No omnibus statistical test (Friedman/Nemenyi).
Stated limitations	Authors note ReinforceNS does relatively better as knapsack capacity and item count increase, and suggest investigating other machine-learning / deep-learning techniques for TTP as future work; no dedicated limitations section.

Strengths. ✓ First RL-based TTP method; fills a real gap (the learning-for-routing wave had not reached TTP) • ✓ Q-learning is used sensibly (guide 2-opt partner selection), warm-started from LKH alpha-nearness • ✓ Full 60-instance categorised set plus 18 GECCO-2023 competition instances, 10 runs, 600 s, C++ • ✓ Baselines (MATLS, S5, CoCo) re-run on the same platform for fairness (explicitly) • ✓ Clean RL vs no-RL ablation on all 60 instances showing the RL mechanism adds value • ✓ Reports 12 new best results vs the GECCO 2023 competition; 95 percent confidence intervals; claims lowest variance

Weaknesses. ✗ RDI is pool-relative (higher is better); values depend on the 4-algorithm pool and are not absolute-optimality measures • ✗ Only 3 external baselines; CS2SA*, SAVI, RWS and MA2B not compared, and it is unclear whether 'CoCo' here is CoCo or the stronger CoCoP • ✗ The comparison uses only one instance file per category per base graph, not the full categorised grid • ✗ No omnibus statistical test; only 95 percent confidence intervals and best-counts • ✗ Many hyper-parameters set without a tuning study • ✗ The 'RL' is tabular Q-learning over city-pair edges (no deep learning, no cross-instance generalisation); closer to an adaptive local search than the learned-heuristic paradigm the framing suggests • ✗ The post-optimisation phase is essentially bit-flip + Insertion from prior work; the ablation isolates only RL vs no-RL, not the other components

[54] Xiang et al. (2024b) — Five-Element Cycle Optimization Algorithm Based on an Integrated Mutation Operator for the Traveling Thief Problem

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	FECOIMO	Family (L1/L2)	Hyper-heuristic / Five-element cycle
Benchmark	—	Size	Cities: 10; 20; 50; 100 Items: KP-Types:
Metric	objective (benefit G) - mean, max and std.	Stat. test	Friedman + post-hoc multiple comparison (5%)
Baselines	ESA, IGWO (Improved Grey Wolf), IWOA (Improved Whale), GA, PGCH.		
Key result	FECOIMO Friedman-rank 1 on all three statistics; significantly beats GA/IGWO/IWOA/PGCH (e.g. GA vs FECOIMO adj. $p=6.35 \times 10^{-5}$) but ties ESA ($p>0.05$); global Friedman p-values 2.89×10^{-38} , 7.17×10^{-37} , 5.37×10^{-35} .		
Competition	N	Code	No
Budget / Runs	per-instance k_max tuned to convergence (100 for n=10 up to 6000 for n=100) / 30	Hardware	n/r

Abstract. Xiang et al. (2024b) proposed **FECOIMO**, a **Five-Element Cycle Optimization** algorithm enhanced with multiple mutation strategies for the TTP. The five-element cycle structure organizes the evolutionary search around five distinct solution archetypes—representing exploitation, exploration, diversification, intensification, and boundary search—and applies different mutation operators to each archetype, maintaining a structured balance between global search and local refinement. Evaluated on a self-generated 39-instance set (cities $n \in \{10, 20, 50, 100\}$, items per city m varying, single item factor, capacity ratios $\tau \in \{25, 50, 75\}$), FECOIMO is compared against ESA, IGWO, IWOA, GA, and PGCH using Friedman non-parametric ranking tests with post-hoc multiple comparisons. FECOIMO ranks first in mean, maximum and standard deviation of the objective across all instances, with its advantage growing as instance scale increases; the post-hoc tests show it significantly outperforms GA, IGWO, IWOA and PGCH, while the difference against the mature ESA is *not* statistically significant—FECOIMO matches ESA while better balancing exploration and exploitation. This establishes it as a competitive structured multi-operator approach, particularly for larger TTP instances, despite a slightly higher execution time per iteration.

Contribution	FECOIMO (Five-Element Cycle Optimization with an Integrated Mutation Operator) for single-objective TTP: a population metaheuristic based on the Chinese Wu Xing five-elements theory (generating/restricting forces among metal/wood/water/fire/earth), combined with an integrated mutation operator that fuses three perturbation moves (flip, insert, move) with usage probabilities derived from measured per-operator effect, plus a heuristic operator for the packing plan.
Authors' claims	FECOIMO outperforms GA, ESA, IWOA, IGWO and PGCH across all instances, particularly large-scale ones; Friedman tests give p-values around $1e-35$ to $1e-38$; the multiple-comparison tests, however, show FECOIMO is NOT significantly different from ESA on mean, maximum or standard deviation; FECOIMO has near-zero variance and a longer runtime with comparable complexity.
Evidence base	39 self-generated instances from the Bonyadi 2013 generator (not the polyakovskiy suite), n in 10,20,50,100 (max 100 cities), 3 capacity-tightness levels. 30 runs, MATLAB, Friedman test + multiple-comparison post-hoc (Tables 6-11). Per-instance maximum-iteration count individually tuned to the convergence point. Baselines: GA, ESA (ali2021novel), IWOA, IGWO, PGCH (namazi2019profit re-implemented). Most reported objective values are large and negative.
Stated limitations	Authors acknowledge FECOIMO has a longer execution time (framed as 'comparable complexity') and outline future research in the concluding remarks.

Strengths. ✓ Proper statistical rigour: Friedman test + multiple-comparison post-hoc tests, 30 runs, mean/max/SD reported • ✓ Data-driven design: the integrated-mutation probabilities are set from a measured per-operator effect with a full 39-instance breakdown • ✓ The update-probability parameter is tuned via a Friedman-ranked sweep • ✓ FECOIMO has near-zero variance (very stable) • ✓ Honest that FECOIMO does not significantly beat ESA and is slower • ✓ Open access

Weaknesses. ✗ FECHOIMO does NOT significantly outperform ESA on mean, maximum or standard deviation (the paper's own post-hoc tests, all $p > 0.05$); the abstract's 'outperforms all algorithms across all instances' contradicts its own statistics • ✗ The ESA baseline (ali2021novel) is itself methodologically unsound (relaxed problem, inconsistent objective scales), so 'matches ESA' is a comparison against a broken reference • ✗ The PGCH baseline performs far worse here (rank about 4.5) than in namazi2019profit's own paper where it is state of the art, strongly indicating a faulty re-implementation • ✗ Only 39 self-generated instances, max 100 cities, from the bonyadi2013 generator; not the standard suite or categorised 60; no medium/large instances • ✗ No comparison with any genuine TTP state of the art (S5, MATLS, CS2SA*, CoCo/CoCoP, SAVI, RWS, ReinforceNS) • ✗ Metaphor-heavy framing with operators that reduce to standard flip/insert/move perturbations plus a heuristic packing step • ✗ Per-instance maximum-iteration tuning is an undisclosed form of per-instance calibration; almost all objective values are large negative (a regime where no method does well); MATLAB, slower than competitors

[55] Pathirage Don et al. (2025a) — *Evolutionary Multitasking for the Scenario-based Travelling Thief Problem*

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	Multitasking EA	Family (L1/L2)	Population-based metaheuristic / Evolutionary multitasking
Benchmark	—	Size	Represents the TTP as a set of multiple scenarios in which each scenario shares a similar TSP component while differing in the KP component. Experimental scenarios are generated using both fixed weights and weights drawn uniformly at random. An evolutionary multitasking optimization approach is employed to solve multiple scenarios in parallel. Experiments conducted on a variety of TTP instances compare the performance of basic and advanced multitasking approaches against classical methods. The results indicate that multitasking provides a competitive advantage over classical methods, particularly under limited computational time budgets. No explicit benchmark results are reported.& Cities:51; 152; 280; 575; 1000 Items: 1x KP-Types: bsc; uco
Metric	mean objective value std (error plots).	Stat. test	none (mean std error plots)
Baselines	classical (1+1)EA, S1, S5.		

Key result MT-(1+1)EA gives slight advantage on fixed bsc scenarios; MT-S1 beats S1 in most (all uniform) cases; MT-S5 fails to improve over S5; S5/MT-S5 reach the best solutions overall; multitasking advantage concentrates under tight time budgets and larger instances.

Competition N **Code** No

Budget / Runs 1 minute max runtime per algorithm / 30 **Hardware** Phoenix HPC, University of Adelaide

Abstract. Pathirage Don et al. (2025a) formulated the TTP as an **evolutionary multitasking optimization** problem by decomposing it into multiple scenarios that share the same TSP component but differ in the KP component—generated with either fixed weights or weights drawn uniformly at random. Solving all scenarios simultaneously allows the evolutionary algorithm to exploit inter-scenario similarities by transferring genetic material between related KP configurations, improving convergence speed by reusing partial solutions that are effective across multiple weight settings. Evaluated on instances from 51 to 1,000 cities (1x item density, uncorrelated and bounded strongly correlated types), the multitasking approach demonstrates a competitive advantage over classical single-task methods, particularly under limited computational time budgets, suggesting that modeling TTP parameter uncertainty as a set of related optimization tasks is an effective strategy for producing robust solutions across diverse operating conditions.

Contribution First study of evolutionary multitasking / multi-factorial optimisation for TTP: represents an instance as $k=5$ ‘scenarios’ (same TSP, different perturbed KP weight profiles) solved in parallel with a shared population and knowledge transfer; multitasking variants of (1+1) EA and of the S1/S5 heuristics, with a fixed shared CLK tour and only packing plans evolved and transferred.

Authors’ claims Multitasking brings a competitive advantage over classical methods on small time budgets; MT-(1+1) EA slightly beats (1+1) EA on bounded-strongly-correlated and larger instances; MT-S1 beats S1 in most cases (all cases for uniformly-distributed scenarios); MT-S5 fails to improve on S5; S5 (and MT-S5) still exceed all algorithms tested; multitasking helps more when scenarios are similar.

Evidence base 10 TTP benchmark instances, 51-1000 cities, 1 item/city, 2 weight profiles; $k=5$ scenarios per instance; 30 runs, 1-min budget; error plots (mean +/- std) comparing each multitasking algorithm to its classical baseline. Transfer frequency 100. No statistical test (only overlapping error bars). No comparison with any state-of-the-art TTP solver.

Stated limitations Authors explicitly state the advantage is ‘conditional’ (small budget, similar scenarios, weak base algorithm), that MT-S5 does not help, that S5 itself remains best, and that fixed weight profiles are harder to transfer between; framed as opening a research area.

Strengths. ✓ First application of evolutionary multitasking / transfer optimisation to TTP; opens a clear new direction • ✓ Careful, honest framing: the advantage is explicitly conditional and bounded • ✓ Reports negative results plainly (MT-S5 does not help; multitasking does not beat S5 itself) • ✓ Clean ablation design: two multitasking (1+1)-EA variants plus S1/S5 multitasking, each vs its own classical baseline • ✓ 30 runs, error bars, transfer-count tracking, same seed for reproducibility; peer-reviewed (GECCO 2025)

Weaknesses. ✗ Fixes and shares a single CLK tour across all scenarios; only packing plans are evolved and transferred, so this is multitasking on the KP sub-problem under a fixed route, not the full TTP • ✗ No statistical significance testing at all; conclusions rest on visually comparing overlapping error bars • ✗ Only 10 instances, 1 item per city, 2 weight profiles; not the categorised subset • ✗ No comparison with any state-of-the-art TTP solver; even within the study S5 remains best • ✗ The 'scenario-based TTP' is a synthetic construction; its link to a real application is asserted, not demonstrated • ✗ Transfer frequency and weight-perturbation parameters set without a sensitivity study • ✗ 1-minute budget; results not comparable to the mainstream TTP literature; the payoff is small even where it works

[56] Zhang et al. (2025) — A two-stage algorithm based on greedy ant colony optimization for travelling thief problem

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	ICA + GACO	Family (L1/L2)	Population-based metaheuristic / Swarm (ACO)
Benchmark	—	Size	Cities:51; 76; 101 Items: 1x KP-Types: bsc; uco; usw
Metric	objective (profit) and average ranking.	Stat. test	none (average-ranking comparison)
Baselines	benchmark heuristics S1/S2/C1/C2, SAVI nguyen2023simulated, CS2SA* el2018efficiently, and LK-GISS / GA-GISS ablations.		
Key result	GISS beats S1/S2/C1/C2 and competes with SAVI (avg rank: LK-GISS 2.39 vs SAVI 2.48 in Table~5; LK-GISS 2.2 vs CS2SA* 2.7, SAVI 2.8 in Table~6); GACO-GISS beats LK-GISS overall (avg rank 1.43 vs 1.57) despite longer tours, confirming TTP-aware routing helps; performance degrades on large-scale instances.		
Competition	N	Code	On request
Budget / Runs	— / n/r	Hardware	n/r

Abstract. Zhang et al. (2025) proposed a two-phase framework combining an **Item Combination Algorithm (ICA)** and a **Guided ACO (GACO)** for joint tour and item optimization in the TTP. The ICA precomputes high-quality item combinations by analyzing profit and spatial distribution patterns across the cities, constructing a library of promising packing sub-plans that can be efficiently assembled into complete packing plans during the search. The GACO algorithm then builds TTP-aware tours guided by pheromone information derived from the quality of TTP solutions—rather than TSP tour lengths—producing routes that are more compatible with the precomputed item combinations. An additional **Global Item Selection Strategy (GISS)** based on a Genetic Algorithm performs global search over the packing space without relying on hand-crafted scoring rules. Evaluated on small instances with 51, 76, and 101 cities (1x item density, all KP types), GACO produces more effective routes than the standard LK heuristic, and GISS outperforms several state-of-the-art item-selection methods.

Contribution	A two-stage TTP1 algorithm: Stage 1 GACO (greedy ACO for routing that USES item information) - an item-classification step pre-computes a beneficial item set by value-density and spatial clustering, and biases the ant route construction (via a per-item selection matrix) to place cities holding those items near the tour end, then 2-opt polish; Stage 2 GISS - a plain genetic algorithm for item selection replacing hand-designed scoring rules.
Authors' claims	GISS beats the S1/S2/C1/C2 scoring rules and SAVI on the same LK route (best average ranking); GACO-GISS beats LK-GISS (average ranking 1.43 vs 1.57) even though GACO routes are 12-30 percent longer than LK routes; optimising one sub-problem in isolation gets stuck in local optima; performance deteriorates on large instances.
Evidence base	Standard TTP dataset. Table 5: 24 instances from eil51/eil76/eil101 (≤ 101 cities). Table 6: 10 instances ≤ 280 cities. 10 runs per instance, average reported. 'Average ranking' reported with NO Friedman statistic, NO p-value, NO omnibus test. Parameter-sensitivity table. Public source code. No time budget (iteration-based). Baselines S1/S2/C1/C2, CS2SA*, SAVI.
Stated limitations	Authors explicitly state satisfying performance is confined to small/medium instances and deteriorates on large instances; future work is efficiency for large/complex TTPs and real-life instances.

Strengths. ✓ Genuinely novel first-stage idea: use item value-density and spatial clustering to bias ANT route construction so valuable items end up near the tour end (coordination at construction time, not just local search) • ✓ GISS (a plain GA) as a 'no complex rules' alternative to hand-designed scoring functions is an honest framing • ✓ Parameter-sensitivity table across 5 instances and 7 parameters; public source code • ✓ Honest that performance deteriorates on large instances and confined claims to small/medium • ✓ Table 7's finding (a 12-30 percent longer GACO route can still yield a better final objective) supports the non-separability thesis

Weaknesses. ✗ All experiments ≤ 280 cities (mostly ≤ 101); no medium-large or large instances; not the categorised subset • ✗ 'Average ranking' reported with no Friedman statistic, p-value or post-hoc test; the tiny ranking differences (1.43 vs 1.57) are untested for significance • ✗ No time budget; ACO + GA vs scoring-rule baselines is not a like-for-like effort comparison • ✗ SAVI and CS2SA* are compared only on the item-selection stage using a fixed LK route, which handicaps those whole-solution methods and is not a fair test of them • ✗ Baselines top out at CS2SA*/SAVI (2018); no CoCo/CoCoP or ReinforceNS • ✗ The item-classification step has many components set 'according to experience'; reproducibility is hard despite the code • ✗ Many table entries are large negative objectives (uncorrelated small-capacity instances), a regime where no method does well

[57] Pathirage Don et al. (2026) — Greedy Approaches for Packing While Travelling with Deterministic and Stochastic Constraints

Variant	PWT	Objective / #obj	Single-objective / 1
Route	Fixed route	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	Greedy PACK + HH	Family (L1/L2)	Constructive/iterative heuristic / Greedy + hyper-heuristic
Benchmark	—	Size	Cities: 51; 152; 280; 575; 1000 Items: 1x KP-Types: bsc; unc
Metric	mean objective std.	Stat. test	Kruskal-Wallis ($p < 0.05$)
Baselines	original reward r1 (and cross-comparison among r2-r7, HH_1-HH_6).		
Key result	deterministic - r3 best one-shot, r5 (benefit-to-weight ratio) best iterative; HH_4 best overall and significantly beats r5 on 7/10 instances. Stochastic - r7 best one-shot, HH_6 best overall and significantly beats r7 on 5/10 instances; tighter =0.999 lowers achievable objective.		
Competition	N	Code	No
Budget / Runs	1000 iterations / 30 PWT instances/setting	Hardware	Phoenix HPC, Adelaide

Abstract. Pathirage Don et al. (2026) extended the PWT literature by proposing and analyzing **greedy algorithms for both deterministic and stochastic variants** of the Packing While Traveling problem. For the deterministic PWT, new **reward functions** are introduced that more accurately model the trade-off between item profit and the additional travel cost induced by carrying weight over the remaining route, yielding better greedy selection decisions than prior scoring rules. These reward functions are further incorporated into a **hyper-heuristic framework** that selects among multiple greedy strategies at each decision step based on local solution state, providing additional flexibility. For the stochastic PWT, item weights are modeled as random variables subject to a **chance-constrained formulation** that guarantees the knapsack capacity is not exceeded with a specified probability, and the reward functions are adapted to account for weight uncertainty under this framework. Evaluated on PWT instances derived from the standard TTP benchmark (51 to 1,000 cities, single item per city, uncorrelated and bounded-strongly-correlated KP types), the proposed methods demonstrate competitive performance on both deterministic and stochastic configurations, advancing the state of the art for risk-aware PWT optimization.

Contribution New reward (goodness) functions for the PACK greedy packing heuristic on PWT (fixed tour): a benefit-minus-travel-cost reward r2, its benefit-to-weight ratio r3, iterative versions r4/r5 that recompute scores after each pick using the accumulated weight, a hyper-heuristic framework that treats the reward functions as low-level heuristics, and an extension to the chance-constrained PWT (stochastic weights) with reward functions r6/r7 using Hoeffding/Chebyshev increased-expected-weights and stochastic hyper-heuristics.

Authors' claims	Deterministic: r3 (one-time benefit-to-weight ratio) beats the standard reward on all instances and r5 (iterative benefit-to-weight ratio) gives the best mean on most; raw-benefit rewards (r2, r4) underperform, so the ratio form is key; the hyper-heuristic HH4 significantly beats r5 on 7 of 10 instances. Stochastic: r7 beats the standard reward and r6, and the hyper-heuristic HH6 significantly beats r7 on 5 of 10 instances; a tighter chance constraint makes high optimisation levels harder.
Evidence base	300 PWT instances = 10 TTP base instances (51-1000 cities, 1 item/city, bsc + unc) x 30 random CLK tours. Mean/std over the 30 tours; Kruskal-Wallis test with +/-/* significance marking. Stochastic: delta=20, alpha in 0.9, 0.999. 1000-iteration stopping. The standard reward baseline is PACK with the exponent fixed to 1 (weaker than PackIterative). No comparison with any exact/approximate PWT method (polyakovskiy2017 MIP, neumann2019 FPTAS/DP), so no optimality gap. arXiv preprint.
Stated limitations	None stated explicitly; the conclusion notes the hyper-heuristics give improvement 'at an additional cost'.

Strengths. ✓ Systematic study of reward-function design for the PACK greedy heuristic with a clear finding (the benefit-to-weight RATIO, especially iteratively updated, is what helps; raw benefit does not) • ✓ Covers both deterministic PWT and chance-constrained PWT in one framework, a rare bridge to the CCTTP line • ✓ 300 instances (10 bases x 30 random tours), mean/std, Kruskal-Wallis tests with +/-/* marking • ✓ The iterative reward functions correctly account for a picked item changing downstream items' travel cost • ✓ Hyper-heuristic significance results are honestly partial (7/10, 5/10)

Weaknesses. ✗ Fixed-tour PWT only; the tour is deliberately not optimised, so the results say nothing about full TTP • ✗ The baseline is PACK with the exponent fixed to 1, weaker than faulkner's PackIterative; 'beats the standard reward' is partly beating a deliberately un-tuned baseline • ✗ No comparison with any exact/approximate PWT method, so no optimality gap and no absolute quality context, despite such methods existing for exactly this problem • ✗ Only 10 base instances, 1 item per city, bsc + unc only • ✗ The improvements are modest and often not significant (HH4 beats r5 on only 7/10; HH6 beats r7 on only 5/10) • ✗ Many overlapping reward-function and hyper-heuristic variants; the practical recommendation is buried • ✗ The stochastic extension is incremental over the authors' own CCTTP paper; the exponent is fixed to 1 throughout • ✗ No runtime/cost quantification for the 'additional cost' of the hyper-heuristics

[58] Eube et al. (2026b) — Approximation Algorithms for the Traveling Thief Problem

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	increasing (general; linear is a special case) / time-renting
Algorithm	(9+e,9+e)-approx Pareto set; (2e+e) Weighted-TSP	Family (L1/L2)	Approximation schemes / Polynomial-time bi-criteria approximation
Benchmark	none (theoretical)	Size	N/A (theoretical; general finite metric space)

Metric	approximation ratios (proved)	Stat. test	none (theoretical)
Baselines	n/a (theoretical)		
Key result	polynomial-time $(9+\epsilon, 9+\epsilon)$ -approximate Pareto set on a finite metric space; $(2e+\epsilon)$ -approximation for the Weighted TSP (fixed item set).		
Competition	N	Code	No
Budget / Runs	— / 1 (deterministic)	Hardware	—

Abstract. Eube et al. (2026b) gave the **first approximation algorithms for any variant of the TTP**, complementing the many heuristics and non-polynomial exact methods with the first solutions carrying provable performance guarantees. The travel speed is modelled as a general increasing function of the accumulated weight, of which the usual linear speed-decrease law is a special case. Because the TTP has two competing objectives (maximise collected profit, minimise travel time), the authors target an (α_1, α_2) -**approximate Pareto set**: a set of solutions such that every feasible solution is dominated by one collecting at least a $1/\alpha_1$ fraction of its profit while using at most α_2 times its travel time. The main result is a polynomial-time algorithm that computes a $(9 + \epsilon, 9 + \epsilon)$ -**approximate Pareto set** on a finite metric space. For the restricted setting in which the set of collected items is fixed in advance, so that only a load-dependent minimum-time tour must be found (the **Weighted TSP**), a $(2e + \epsilon)$ -approximation is given. The work is purely theoretical, with no implementation or computational experiments, and the constant factor of 9 is acknowledged to be large.

Contribution	The first approximation algorithms for any variant of the TTP. The travel speed is a general increasing function of the accumulated weight (linear speed-decrease is a special case). Because the TTP has two competing objectives (maximise profit, minimise travel time), the target is an (α_1, α_2) -approximate Pareto set. Main result: a polynomial-time algorithm computing a $(9+\epsilon, 9+\epsilon)$ -approximate Pareto set on a finite metric space. For the sub-setting where the collected items are fixed in advance (the Weighted TSP: a load-dependent minimum-time tour), a $(2e+\epsilon)$ -approximation.
Authors' claims	A $(9+\epsilon, 9+\epsilon)$ -approximate Pareto set can be computed in polynomial time; a $(2e+\epsilon)$ -approximation exists for the Weighted TSP; these are the first approximation guarantees for the TTP, complementing prior inapproximability results.
Evidence base	Purely theoretical. Published at ESA 2026 (LIPIcs vol. 388). Proofs of the approximation ratios; no implementation, no computational experiments, no benchmark.
Stated limitations	The constant factor of 9 is acknowledged to be large; the work is theoretical with no implementation.

Strengths. ✓ The first approximation algorithm for any TTP variant, filling a genuine gap between heuristics and non-polynomial exact methods

- ✓ Handles a general increasing speed function, not only the linear special case
- ✓ Correctly frames the guarantee as a bi-criteria (α_1, α_2) -approximate Pareto set, matching the two competing objectives
- ✓ A separate, cleaner $(2e+\epsilon)$ -approximation for the fixed-item-set Weighted TSP
- ✓ Rigorous; connects to and complements the existing TTP inapproximability literature

Weaknesses. ✗ Large constant factor $(9+\epsilon, 9+\epsilon)$; of limited practical value as an approximation

- ✗ No implementation, no experiments, no comparison with heuristics on any benchmark instance
- ✗ The result is for general finite metric spaces; no sharper ratio for the Euclidean instances the benchmark uses

[59] Eube et al. (2026a) — *Effective Traveling for Metric Instances of the Traveling Thief Problem*

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	W-TSP DP / (8+e)-approx	Family (L1/L2)	Exact methods / DP + approximation (tour subproblem)
Benchmark	—	Size	Cities: 76 to 33810 (path metric variant) Items: 1x; 5x; 10x KP-Types: bsc; uco; usw
Metric	route-cost improvement (%) of the DP tour over the S5 tour under the same packing plan.	Stat. test	none
Baselines	S5 (Chained Lin-Kernighan + PackIterative).		
Key result	over 25 instances, average improvement 13.58%, best 69.71%, worst 0.00% (DP is optimal for the path metric, so never worse than S5). A n-means clustering approximation cuts DP runtime by ~99.28% for only 1.18% average cost increase.		
Competition	N	Code	Yes
Budget / Runs	10 min/instance / n/r (deterministic DP)	Hardware	MacBook Pro 2022, Apple M2, 24GB RAM

Abstract. Eube et al. (2026a) investigated the **tour-optimization subproblem of the TTP under a fixed packing plan** from a theoretical computational complexity perspective. The paper formulates this subproblem as a **weighted variant of the TSP** in which the travel cost between consecutive cities depends on the cumulative weight of all items collected from the tour's start up to the current city, making the edge costs asymmetric and order-dependent. Two distance metrics are analyzed: for the **path metric**, an $O(n^2)$ dynamic programming algorithm is derived; for the **star metric**, the problem is shown to be NP-hard, and an $(8 + \varepsilon)$ -approximation algorithm is constructed using a combination of minimum spanning trees and tree path decompositions. The study establishes for the first time that even the tour-optimization component of the TTP is NP-hard under natural distance metrics, providing a rigorous theoretical lower bound on the computational complexity of any exact TTP solver.

Contribution A theoretical study of the TOUR-optimisation component of TTP under a fixed packing plan, formulated as the Weighted TSP (travel cost per unit distance is a monotone function of the cumulative collected weight): proves W-TSP is NP-hard even on a star metric, gives an $(8+\varepsilon)$ -approximation on star graphs, an $O(n^2)$ exact dynamic program on a path metric with a general cost function, and an $O(\log n)$ -approximation on a general metric with the linear speed-decrease cost; plus a $\text{sqrt}(n)$ -cluster grouping heuristic to accelerate the DP.

Authors' claims	W-TSP is NP-hard on a star metric (proven); (8+eps)-approx on star graphs and $O(\log n)$ -approx on general metric with linear cost (proven); the $O(n^2)$ path-metric DP is exact; integrating the DP into TTP (re-optimising the tour under S5's fixed packing) improves route cost by 13.58 percent on average (best 69.71 percent, worst 0 percent since the DP is optimal for the induced W-TSP); the grouping heuristic cuts DP runtime by about 99 percent for about 1 percent cost increase; good tours defer collecting weight until the return path.
Evidence base	TTP benchmark instances from TSPLIB with y-coordinates set to 0 to make them path metrics (n up to 574, 1 item/city for the larger ones). S5 (10-min, MacBook Pro M2) provides the reference tour+packing; the DP re-optimises the tour under S5's packing. 25 instances; summary statistics only. A synthetic 3x5x5 experiment for runtime scaling. Deterministic. Published at PPSN 2026 (pp. 282-297).
Stated limitations	The DP is for path metrics only (experiments force $y=0$); the star (8+eps)-approx and the general-metric $O(\log n)$ -approx are theoretical, not implemented; node-weighted-TSP techniques do not extend to the general cost function; the DP is $O(n^2)$ per start and needs the grouping heuristic to scale.

Strengths. ✓ Rigorous theoretical contribution filling a real gap: the tour sub-problem of TTP had almost no complexity / approximation theory • ✓ Multiple solid, proven results (NP-hardness on star, (8+eps)-approx on star, $O(n^2)$ exact DP on path, $O(\log n)$ -approx on general metric with linear cost) • ✓ The path-metric structural lemma (leftmost-or-rightmost) is elegant and yields a practical $O(n^2)$ exact algorithm • ✓ Experiments show the DP genuinely improves S5's tours (avg 13.58 percent, up to 69.71 percent) under S5's own packing and never worsens them • ✓ The grouping heuristic scales the DP to thousands of nodes for about 1 percent cost loss • ✓ Provides an exact yardstick for the tour component so other papers' tour optimisation can be judged against optimality

Weaknesses. ✗ The exact DP applies only to path (line) metrics; the experiments artificially flatten TSPLIB instances to 1D, so the 13.58 percent improvement is on degenerate, non-standard instances, not the real 2D-Euclidean TTP benchmark • ✗ The star (8+eps) and general-metric $O(\log n)$ approximations are purely theoretical, not implemented, with large constants • ✗ No comparison with any other tour-optimisation approach (2-opt, LK, PGCH tour moves); the only baseline is S5's tour • ✗ The experiments re-optimize the tour under a fixed S5 packing and do not close the loop (re-pack after re-touring), so the end-to-end TTP objective improvement is not reported • ✗ Only 25 instances, ≤ 574 cities, all degenerate path metrics • ✗ The grouping heuristic re-introduces heuristic approximation, undercutting the 'exact' framing for large instances

[60] Kapancioglu et al. (2026) — Formulations and an exact algorithm for the Traveling Thief Problem

Variant	TTP1	Objective / #obj	Single-objective / 1
Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	MILP (SCF/MCF/2-index) + exact algorithm	Family (L1/L2)	Exact methods / MILP formulations + exact algorithm

Benchmark	generated symmetric/asymmetric instances + TTP benchmark	Size	Generated instances: up to 20 nodes; TTP benchmark: new optima for 15 instances (2 with >50 nodes) KP-Types: uncorrelated; similar-weights; strongly-correlated
Metric	LP-relaxation gap (%), MILP solve time (s), #instances solved to optimality	Stat. test	none
Baselines	the three-index vs two-index formulations against each other; prior exact methods (B&B, CP)		
Key result	new optimal values for 15 benchmark instances (incl. two with >50 nodes); the exact algorithm performs well on similar-item-weight instances; three-index formulations are memory-bound while two-index formulations solve larger instances.		
Competition	N	Code	No
Budget / Runs	3600 s (1 h) time limit / 1 (deterministic)	Hardware	—

Abstract. [Kapancioglu et al. \(2026\)](#) developed **mixed-integer linear programming formulations and an exact algorithm** for the TTP, addressing the near-total absence of formulation-based baselines on the standard benchmark. Three families of models are proposed and compared: two three-index formulations based respectively on single-commodity and multi-commodity flow, and a more compact two-index formulation with cut constraints that trades a weaker LP relaxation for far lower memory use, allowing it to be built for instances on which the three-index models run out of memory. The paper also revisits a branch-and-bound search and a constraint-programming model. On generated symmetric and asymmetric instances of up to 20 nodes the formulations are compared through LP-relaxation gaps and solve times, and the dedicated exact algorithm, which is targeted at instances whose items have similar weights, is shown to close several previously open instances: **new optimal values are reported for 15 benchmark instances, two of which have more than 50 nodes**. The approach remains confined to small instances, and the authors note that stronger formulations or decomposition are needed to reach benchmark-scale instances.

Contribution	Mixed-integer linear programming formulations and an exact algorithm for the TTP: two three-index formulations (single-commodity and multi-commodity flow) and a compact two-index formulation with cut constraints (weaker LP relaxation, much lower memory), plus a revisited branch-and-bound search and constraint-programming model; and a dedicated exact algorithm targeted at instances whose items have similar weights.
Authors' claims	The formulations are compared through LP-relaxation gaps and solve times on generated symmetric/asymmetric instances up to 20 nodes; the two-index formulation can be built where the three-index models run out of memory; the exact algorithm performs well on similar-item-weight instances and closes several open instances, with new optimal values reported for 15 benchmark instances, two of which have more than 50 nodes.
Evidence base	62 generated instances (up to 20 nodes) plus a subset of the TTP benchmark. Metrics: LP-relaxation gap (%), MILP solve time (s), number of instances solved to optimality, within a 1-hour time limit. Deterministic. Published in the European Journal of Operational Research.
Stated limitations	Authors state the three-index formulations are hindered by memory, that computational times grow sharply with instance size, and that stronger formulations or decomposition are needed to efficiently solve the instances in the TTP literature.

Strengths. ✓ Provides a formulation-based and exact baseline for a benchmark dominated by heuristics • ✓ Three principled formulation families with a clear LP-gap vs memory-usage trade-off analysis • ✓ Reports new optimal values for 15 benchmark instances, including two with more than 50 nodes (a genuine advance on exact TTP solving) • ✓ Peer-reviewed in a top OR journal; also revisits branch-and-bound and constraint programming

Weaknesses. ✗ Exact solving still scales only to about 50-node instances; the benchmark's hundreds-to-thousands-of-city instances remain far out of reach • ✗ The three-index formulations are memory-bound; the two-index ones have a weak LP relaxation • ✗ Generated instances (up to 20 nodes) are non-standard and were used only for the formulation comparison • ✗ The dedicated exact algorithm works well only on similar-item-weight instances • ✗ No comparison with the earlier exact work (wu2017exact) on a common instance set

[61] **Murjani and Sobhanan (2026)** — *Drive, Pack, Fly: The Travelling Thief Problem with Drone*

Variant	TTP-D	Objective / #obj	Single-objective / 1
Route	Full tour (vehicle) + drone sorties	Timing	Flight synchronisation (drone launch/rendezvous timing)
Agents	Vehicle + drone (two cooperating agents)	Capacity	single (shared)
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear (load-dependent vehicle speed) / time-renting
Algorithm	MILP + metaheuristics + attention DRL + hybrid	Family (L1/L2)	Learning-based methods / Deep reinforcement learning (attention) + metaheuristic + MILP
Benchmark	two benchmark families (TTP-derived + drone-routing)	Size	Two benchmark families (a TTP-derived set and a drone-routing set); MILP: small instances only
Metric	collected profit net of rental cost; makespan; runtime	Stat. test	none
Baselines	MILP optimum (small instances); metaheuristic baseline (large instances)		
Key result	MILP solves small instances to optimality; the learner-initialised hybrid recovers most of the metaheuristic quality at a fraction of the budget, but the largest instances still need the full metaheuristic; sensitivity analysis: the rental ratio is the primary profitability driver, fleet parameters matter only at the margin.		
Competition	N	Code	No
Budget / Runs	— / —	Hardware	—

Abstract. Murjani and Sobhanan (2026) introduced the **Travelling Thief Problem with Drone (TTP-D)**, extending the load-dependent-speed model with a second, cooperating agent. A ground vehicle whose speed decreases as its load grows is paired with an onboard drone that retrieves outlying, high-value items; because travel time is load-dependent, every item the vehicle collects shifts the arrival times that govern the drone's launch and rendezvous points, tightly coupling routing, packing and **flight synchronisation**. The objective is to maximise collected profit net of a time-based rental cost by jointly optimising all three. The authors provide a mixed-integer linear program that solves small instances to optimality, metaheuristics and an **attention-based deep-reinforcement-learning policy** for larger instances, and a learner-initialised hybrid in which the reinforcement-learning policy constructs a starting solution that a short annealing run then refines; on two benchmark families this hybrid recovers most of the metaheuristic baseline's quality at a fraction of its budget, though the largest instances still require the full metaheuristic. A sensitivity analysis identifies the rental ratio as the primary driver of profitability, with fleet parameters mattering only at the margin.

Contribution	Introduces the Travelling Thief Problem with Drone (TTP-D): a load-slowed ground vehicle paired with an onboard drone that retrieves outlying high-value items, where each item the vehicle collects shifts the arrival times that govern the drone's launch and rendezvous, tightly coupling routing, packing and flight synchronisation. Objective: maximise collected profit net of a time-based rental cost. Provides a mixed-integer linear program (small instances to optimality), metaheuristics, an attention-based deep-reinforcement-learning policy, and a learner-initialised hybrid (DRL construction + short annealing refinement).
Authors' claims	The MILP solves small instances to optimality; on two benchmark families the learner-initialised hybrid recovers most of the metaheuristic baseline's quality at a fraction of its computational budget, though the largest instances still require the full metaheuristic; a sensitivity analysis identifies the rental ratio as the primary driver of profitability, with fleet parameters mattering only at the margin.
Evidence base	Two benchmark families (a TTP-derived set and a drone-routing set). MILP (Gurobi) for small instances; metaheuristic baseline plus DRL and hybrid for larger instances; comparison of solution quality vs computational budget; a rental-ratio / fleet-parameter sensitivity analysis. arXiv preprint.
Stated limitations	Authors state the MILP is intractable beyond small instances and that the learner-initialised hybrid still needs the full metaheuristic on the largest instances.

Strengths. ✓ Introduces a genuinely new, well-motivated variant (drone-assisted collection with load-dependent speed and flight synchronisation) with real application scenarios • ✓ Full solver stack: MILP for optimality certificates on small instances, metaheuristics, an attention-based DRL policy, and a learner-initialised hybrid • ✓ The hybrid's practical value is honestly bounded (recovers most quality at low budget, but not on the largest instances) • ✓ A sensitivity analysis that isolates the rental ratio as the key economic driver

Weaknesses. ✗ arXiv preprint; not yet peer-reviewed • ✗ MILP scales only to small instances; the largest instances still need the full metaheuristic • ✗ Two benchmark families are introduced but their sizes and construction are not standardised against any prior variant • ✗ As a new variant with a single paper, there is no external baseline and no independent validation • ✗ The load-dependent-speed + drone-synchronisation coupling makes cross-comparison with plain TTP1 results impossible

[62] Tran and Luong (2026) — Integrating Ant Colony Optimization and Simulated Annealing with Restart for the Traveling Thief Problem

Variant	TTP1	Objective / #obj	Single-objective / 1
----------------	------	-------------------------	----------------------

Route	Full tour (all cities)	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	ACO-RSA	Family (L1/L2)	Population-based metaheuristic / Ant colony optimization + simulated annealing with restart
Benchmark	TTP benchmark (polyakovskiy2014) + categorized A/B/C	Size	Cities: 51; 52; 76; 101; 318; 724; 1577 Items: 1x; 5x; 10x KP-Types: bsc; uco; usw
Metric	mean objective, average rank, per-instance gap (%)	Stat. test	per-instance significance markers (+/-/=)
Baselines	S5, CS2SA*, SAVI, GACO-GISS		
Key result	ACO-RSA competitive and often best on small/medium instances (eil51/76/101, berlin52); outperformed by S5 and SAVI on the larger instances (lin318, u724, fl1577).		
Competition	N	Code	No
Budget / Runs	— / —	Hardware	—

Abstract. [Tran and Luong \(2026\)](#) proposed **ACO-RSA**, a hybrid metaheuristic in which a MAX-MIN Ant System with item-aware tour construction builds the initial route and **simulated annealing with restart** performs a global search over both the tour and the packing plan, the restart mechanism providing the diversification needed to escape local optima. The method is compared against S5, CS2SA*, SAVI and the two-stage GACO-GISS on eil51, eil76, eil101, berlin52, lin318, u724 and fl1577 instances and on the categorised A/B/C benchmark. ACO-RSA is competitive and frequently the best method on the small and medium instances, but it is clearly outperformed by S5 and SAVI on the larger instances (lin318 and above), where the ant-construction and annealing budget do not scale. The paper is a short conference contribution and does not release source code.

Contribution	ACO-RSA, a hybrid metaheuristic: a MAX-MIN Ant System with item-aware tour construction builds the initial route, and simulated annealing with restart performs a global search over both the tour and the packing plan, the restart mechanism providing diversification to escape local optima.
Authors' claims	ACO-RSA achieves competitive performance and is frequently the best method on small and medium instances (eil51/76/101, berlin52); it is outperformed by S5 and SAVI on the larger instances (lin318 and above).
Evidence base	TTP benchmark instances (eil51/76/101, berlin52, lin318, u724, fl1577) and the categorised A/B/C set; baselines S5, CS2SA*, SAVI and GACO-GISS; per-instance mean objective, average rank and gap (%), with per-instance +/-/= significance markers. ICIIT 2026 (ACM). No source code released.
Stated limitations	The paper acknowledges ACO-RSA does not scale to the larger instances, where the ant-construction and annealing budget are insufficient.

Strengths. ✓ Sensible hybrid design: item-aware ACO construction for the tour, simulated annealing with restart for global search over tour and packing • ✓ Compares against four genuine recent baselines (S5, CS2SA*, SAVI, GACO-GISS), not weak straw baselines • ✓ Per-instance results with significance markers on both the standard graphs and the categorised A/B/C set • ✓ Honest that ACO-RSA loses on lin318 and above

Weaknesses. ✗ Does not scale: outperformed by S5 and SAVI on all the larger instances (lin318, u724, fl1577) • ✗ Short conference paper at a regional venue; no source code • ✗ The two components (MMAS, SA-with-restart) are both standard; the novelty is the combination • ✗ No comparison with CoCo/CoCoP or ReinforceNS, the strongest TTP1 methods; runs count / hardware not clearly reported • ✗ Only eil/berlin/lin318/u724/fl1577 bases; not the full categorised 60-subset

Table 3 — Literature Review of TTP2 (16 papers)

[1] Blank et al. (2017) — Solving the Bi-objective Traveling Thief Problem with Multi-objective Evolutionary Algorithms			
Variant	TTP2	Objective / #obj	Bi-objective (time vs profit) / 2
Route	Full tour	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Time-decaying profits (drop rate)	Speed / Cost	linear / value-drop-rate
Algorithm	NSGA-II (BI-TTP)	Family (L1/L2)	Population-based metaheuristic / NSGA-II
Benchmark	—	Size	Cities: 10; 20; 50; 100 Items: 1; 5; 10 per city
Metric	hypervolume of normalized median Pareto front (median via Empirical Attainment Function);	Stat. test	none
Baselines	Greedy, ISA, ISA-LOCAL, NSGA-II (PACK-OPTIMAL factory + 2-opt/OPT tour factory + EDGE crossover);		
Key result	NSGA-II best on all 18 instances - e.g. multi-cluster-05 HV 0.884 (vs ISA-LOCAL 0.868, ISA 0.867, Greedy 0.813); multi-0010-05-m 0.881; multi-0020-05-m 0.853; avg only 3.4 distinct tours per Pareto front;		
Competition	Y	Code	No
Budget / Runs	100,000 function evaluations/run / 30	Hardware	not reported
Abstract. Blank et al. (2017) provide an in-depth analysis of the Bi-Objective Traveling Thief Problem (BITTP), focusing on understanding the objective space properties of the problem before proposing solution methods. The authors characterize the Pareto front shape and identify non-convex and disconnected regions caused by the TSP–KP interaction, which complicates the application of classical scalarization techniques. Two deterministic algorithms are proposed: a greedy algorithm that constructs a TSP tour (via nearest-neighbour or solver-based tours) and picks items in decreasing value-per-weight order, and an extension that replaces greedy packing with an exact KP solver combined with local Pareto front improvement. Population-based NSGA-II variants with evolutionary factory and recombination operators are also evaluated on instances up to 100 cities and 1000 items. The study establishes a first systematic baseline for BITTP, demonstrating that greedy deterministic methods can already produce competitive partial Pareto approximations and that exploiting the KP substructure within the greedy framework yields significant gains.			
Contribution	One of the first dedicated studies of the bi-objective TTP (min travel time, max profit; value drop omitted): an objective-space characterisation (symmetry of the tour under an empty knapsack, horizontal lines as embedded TSPs at fixed profit, a right-shift when items are picked), two deterministic reference algorithms (Greedy = LKH tour + greedy b/w picking; ISA = LKH tour + KNP solver at evenly spaced capacities, with an optional local-optimisation variant), and an NSGA-II with a factory/operator study.		

Authors' claims	NSGA-II (with a KNP-solver packing factory, a TSP-solver tour factory, edge-recombination crossover) has the best hypervolume on all 18 instances; a KNP solver as packing factory beats greedy picking; ISA-LOCAL is not significantly better than ISA; edge crossover brings little improvement because only two tours seed the population; on average only 3.39 distinct tours appear per Pareto front.
Evidence base	18 self-generated instances (random-generator and clustered, 10-100 cities, 1/5/10 items/city, 3 capacity categories). 30 runs, max 100,000 function evaluations, median Pareto front via the empirical attainment function. Metric: hypervolume of the normalised median Pareto front. No statistical test; no external BI-TTP baseline (this is one of the first). vmin=0.1, vmax=1.
Stated limitations	Authors state tour diversity in the Pareto front is low and 'might be improvable with other evolutionary operators', that operators considering the interwovenness (not recombining tour and packing independently) 'might be useful', and that performance on larger/other instances should be studied further.

Strengths. ✓ Pioneering: a genuinely useful objective-space characterisation (symmetry, horizontal TSP lines, right-shift) that later BI-TTP work builds on • ✓ Provides two deterministic reference algorithms explicitly as baselines 'that must be outperformed' • ✓ Systematic factory/operator ablation for NSGA-II (3 packing factories x 4 tour factories) • ✓ 30 runs, fixed 100,000-evaluation budget, median Pareto front via the empirical attainment function • ✓ Honest about the low tour-diversity problem and the ineffectiveness of edge crossover here • ✓ Exhaustive Pareto front for a 4-city example as ground-truth illustration

Weaknesses. ✗ 18 self-generated instances only (≤ 100 cities); not a standard benchmark (predates the BI-TTP competition suite); results not comparable to later BI-TTP work • ✗ Only baselines are the authors' own Greedy/ISA; no attempt to adapt single-objective TTP solvers • ✗ No statistical significance testing; 'clearly best on all' asserted from hypervolume point values • ✗ Neglects the item value drop that bonyadi2013's TTP2 specifies; this is really a time-vs-total-profit problem • ✗ NSGA-II's advantage over ISA is modest on the larger/high-capacity instances while using far more evaluations • ✗ The OPT tour/packing factories rely on external LKH/KNP solvers, so the starting population has essentially one tour, which the paper itself identifies as crippling the evolutionary tour search

[2] El Yafrani et al. (2017) — Multi-objectiveness in the single-objective traveling thief problem

Variant	TTP2	Objective / #obj	Bi-objective (time vs profit) / 2
Route	Full tour	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Time-decaying profits (drop rate)	Speed / Cost	linear / value-drop-rate
Algorithm	EMOA-TTP	Family (L1/L2)	Population-based metaheuristic / Multi-objective EA
Benchmark	—	Size	Cities: 52; 100 Items: 255; 495
Metric	single-objective TTP score (knee region of bi-objective Pareto front), EAF attainment surfaces;	Stat. test	none

Baselines	MA2B, MMAS, S5 (best single-objective TTP solvers); algorithm EMOA-TTP = NSGA-II (jMetal) with Null Crossover + 2 local-search mutations;		
Key result	best-known TTP solutions lie in the knee region of the bi-objective front; EMOA-TTP competitive with 3 leading single-objective solvers (e.g. kroA100 example: EMOA-TTP 24820, MA2B 24612, S5 24794, MMAS 24993);		
Competition	Y	Code	No
Budget / Runs	10min/run / 50 and 200 (EAF analysis)	Hardware	Phoenix HPC, University of Adelaide

Abstract. El Yafrani et al. (2017) investigate the hypothesis that TTP1—traditionally solved as a scalar single-objective problem—is inherently a bi-objective problem, and that treating it as TTP2 (with both travel time and profit as explicit objectives) can produce solutions competitive with the best-known TTP1 algorithms. The authors implement **NSGA-II** using the jMetal framework, with a representation encoding both tour and item selection. Without applying any weighted-sum scalarization, the algorithm evolves an approximate **Pareto front** over travel time and profit. The empirical results demonstrate that the best-known TTP1 solutions (produced by leading single-objective algorithms) reside on the Pareto front found by NSGA-II, confirming the inherent multi-objective nature of TTP1. This finding is significant because it implies that a multi-objective framework, which does not commit to any specific weight combination, naturally discovers solutions that single-objective approaches require manual tuning to reach. The study serves as foundational evidence for the TTP2 research direction, motivating multi-objective treatments of multi-component combinatorial problems.

Contribution	A 2-page position paper arguing that single-objective TTP1 is a scalarisation of a bi-objective (time, profit) problem and that a multi-objective approach (EMOA-TTP: NSGA-II in jMetal, with two disruptive operators and two local searches as mutations, Null Crossover, 10-min stopping) can produce TTP1-quality solutions implicitly (the best TTP-score solution sits in the knee of the Pareto front).
Authors' claims	The best-known single-objective TTP solutions lie in the Pareto set produced by EMOA-TTP; EMOA-TTP can compete with S5, MA2B and MMAS and find better single-objective solutions implicitly; on kroA100_n495_bsc_01, EMOA-TTP 24820 vs S5 24794, MA2B 24612, MMAS 24993.
Evidence base	The empirical content is two figures on two instances (kroA100_n495_bsc_01 for the Pareto-front-vs-scalar comparison; berlin52_n255_usw_01 for an attainment-surface-vs-number-of-runs analysis at 50 vs 200 runs). 10-min budget. No results table, no instance set, no statistical test.
Stated limitations	It is a 2-page companion paper explicitly framed as preliminary; the interpretation of bi-objective results needs attainment functions and the attainment surface shifts with the number of runs.

Strengths. ✓ Sharp conceptual point: TTP1's objective is a scalarisation of (time, profit), so a multi-objective solver covers a range of trade-offs and contains the single-objective solution in its knee • ✓ Uses the standard 10-min TTP budget rather than a generation count • ✓ Notes the methodological issue that BI-TTP result interpretation needs attainment functions

Weaknesses. ✗ 2-page companion paper; the empirical evidence is two figures on two instances, with no results table, instance set or statistical test • ✗ The 'competes with S5, MA2B, MMAS' claim rests on a single instance where EMOA-TTP is not even best (MMAS wins) • ✗ EMOA-TTP uses Null Crossover (cloning), so it is really a multi-start parallel local search dressed as an EMOA • ✗ No hypervolume or IGD; only visual Pareto-front / attainment-surface inspection • ✗ 'Profit' here is total item value, not decayed profit • ✗ A motivation piece, not a solver contribution; superseded by later BI-TTP work

[3] [Wu et al. \(2018\)](#) — *Evolutionary computation plus dynamic programming for the bi-objective travelling thief problem*

Variant	TTP2	Objective / #obj	Bi-objective (time vs profit) / 2
Route	Full tour	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Time-decaying profits (drop rate)	Speed / Cost	linear / value-drop-rate
Algorithm	IBEA + DP	Family (L1/L2)	Population-based metaheuristic / Multi-objective EA (DP-hybrid)
Benchmark	—	Size	Cities: 51; 76; 101 Items: 50; 75; 100
Metric	hypervolume (LHV, LSC indicators) + total reward (single-objective TTP score);	Stat. test	Welch's t-test (log-scaled p-values)
Baselines	MA2B el2016population (single-objective SOTA); 7 parent-selection rules (AS-BST, AS-EXT, FPS, RBS-EXP, RBS-HAR, RBS-IQ, TS) x 2 indicators = 16 settings, Welch's t-test vs UAR baseline;		
Key result	IBEA beats MA2B on the majority of cases (e.g. eil101_n100 Uncorrelated FPS-LHV mean 3620.844 vs MA2B 3339.600; eil76_n75 Uncorrelated 5445.624 vs 5275.067); FPS recommended; best on uncorrelated (easiest), worst on bounded-strongly-correlated;		
Competition	N	Code	No
Budget / Runs	20,000 generations, population 50 / 30	Hardware	supercomputer 5568 Intel Xeon 2.30GHz cores, 12TB, 170,000 CPU-hours total

Abstract. [Wu et al. \(2018\)](#) propose a **hybrid indicator-based bi-objective evolutionary algorithm (IBEA)** for TTP2 in which an exact **dynamic programming (DP)** algorithm is embedded as a surrogate packing optimizer. Given a TSP tour evolved by the EA, the DP efficiently computes the exact Pareto front over all feasible packing plans for the Packing While Travelling (PWT) sub-problem, with objectives of minimizing accumulated knapsack weight and maximizing total profit. The resulting DP Pareto data informs **novel indicator-based parent selection** and customized crossover operators, enabling the EA to preferentially evolve tours that yield favourable DP front shapes. This tight integration of exact DP within the EA loop avoids the suboptimality of treating tour and packing decisions independently. On benchmark instances with up to 101 cities and 100 items, the hybrid outperforms state-of-the-art single-objective TTP1 solvers as well as standard multi-objective algorithms, suggesting that inner-loop exact solvers are a powerful design choice when sub-problem structure permits exact computation.

Contribution	An indicator-based hybrid EA for a (reward, weight) bi-objective TTP: an EA evolves tours, each tour is fed to the exact $O(mC)$ dynamic program for PWT which returns the whole Pareto front of packing plans for that fixed tour, and the global front is assembled from segments of many tours' DP fronts. Two novel indicators computed on DP fronts (Loss of Surface Contribution, Loss of Hypervolume) select which tours to mutate/cross; 8 parent-selection rules are studied.
Authors' claims	The by-product (max-reward point of the bi-objective front) beats the state-of-the-art single-objective result (MA2B) in most cases; among selection rules RBS-HAR is best on Loss of Hypervolume, AS-BST on Loss of Surface Contribution, and FPS is recommended as consistently good; the method does best on uncorrelated instances and worst on bounded-strongly-correlated ones.
Evidence base	Only 9 instances: eil51/eil76/eil101 (≤ 101 cities) $\times$ 3 KP types. 30 runs, 20,000 generations, population 50. Baseline MA2B (single-objective) only, compared on total reward. Welch's t-test of each selection rule vs the UAR baseline. About 170,000 CPU hours on a 5568-core supercomputer.
Stated limitations	Authors note the experiments are computationally expensive, that unknown global optima and varying means make cross-instance comparison hard, that the crossover probability is un-optimised, and that hardness conclusions are 'conjecture'.

Strengths. ✓ Genuinely novel: the first TTP approach to use the exact PWT DP as a subroutine inside an EA, exploiting that the DP returns a whole packing Pareto front per tour • ✓ Principled novel indicators (LSC, LHV) that measure a tour's marginal contribution to the global front • ✓ Systematic 16-combination study (2 indicators $\times$ 8 selection rules) with Welch's t-test vs a UAR baseline, 30 runs • ✓ Connects to the exact-methods line (neumann2019 DP), which almost no other TTP heuristic work does • ✓ Honest about the enormous compute cost and the difficulty of cross-instance comparison

Weaknesses. ✗ Only 9 instances, all ≤ 101 cities; the exact DP subroutine is $O(mC)$ per tour, so the whole approach is confined to tiny instances and cannot scale • ✗ About 170,000 CPU hours on a supercomputer for 9 tiny instances; wildly impractical • ✗ The (reward, weight) formulation differs from the (time, profit) BI-TTP of blank2017/yafrani2017 and the competition suite, so results are not comparable to the mainstream BI-TTP line • ✗ The only external baseline is MA2B (single-objective), compared only on the max-reward point; no bi-objective baseline • ✗ Generation-based budget (20,000 gen), not wall-clock; no effort comparison with MA2B • ✗ 'Beats MA2B in most cases' is on ≤ 101 -city instances where exact methods can find true optima anyway

[4] Laszczyk and Myszkowski (2019) — A Specialized Evolutionary Approach to the bi-objective Travelling Thief Problem

Variant	TTP2	Objective / #obj	Bi-objective (time vs profit) / 2
Route	Full tour	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Time-decaying profits (drop rate)	Speed / Cost	linear / value-drop-rate
Algorithm	NTGA (specialized)	Family (L1/L2)	Population-based metaheuristic / EA (NTGA)
Benchmark	—	Size	Cities: 51 Items: 50; 150; 250; 500

Metric	5 quality measures - ED (Euclidean distance to front), HV (hypervolume), PFS (Pareto front size), RNI (ratio of non-dominated individuals), S (spacing);	Stat. test	none
Baselines	NSGA-II and NTGA, each in generic vs specialized (TTP-aware operator) configuration;		
Key result	specialized NTGA best overall - avg HV 0.8419 (NTGA-spec) vs 0.8029 (NSGA-II-spec), 0.7303 (NTGA-gen), 0.7414 (NSGA-II-gen); avg PFS 45.6 vs 22.3; specialization gain > paradigm choice; NTGA-spec increasingly dominates NSGA-II-spec on larger instances;		
Competition	Y	Code	No
Budget / Runs	not reported / multiple (EAF-averaged fronts)	Hardware	not reported

Abstract. Laszczyk and Myszkowski (2019) conduct a systematic comparison of two multi-objective evolutionary frameworks—**NSGA-II** and the **Non-Dominated Tournament Genetic Algorithm (NTGA)**—for bi-objective TTP, examining the impact of problem-specific operator design. Both algorithms are tested in a **generic configuration** (standard order-based encoding and operators without TTP awareness) and a **specialized configuration** (TTP-specific crossover and mutation operators tailored to the TSP and KP sub-components). In the generic configuration, both algorithms achieve similar performance; in the specialized setting, NTGA significantly outperforms NSGA-II due to its modified selective pressure and diversity maintenance via non-dominated tournament selection. Evaluation uses multiple quality indicators including hypervolume (HV), convergence, uniformity, and spread, measured on instances with 51 cities and up to 500 items. The study concludes that operator specialization has a greater impact than algorithmic paradigm choice in combinatorial bi-objective problems, providing important design guidance for practitioners.

Contribution	A factorial comparison of NSGA-II and NTGA (an NSGA-II variant with tournament selection, no parent-child competition, and clone prevention instead of crowding distance) on the bi-objective TTP (min time, max profit, no value drop), each in a generic-representation and a TTP-specialised-representation configuration, to isolate the effect of operator specialisation.
Authors' claims	With generic representation NSGA-II and NTGA are statistically equivalent on all quality measures; with specialised representation, specialisation significantly improves hypervolume (all 12 instances) and NTGA-specialised further improves over NSGA-II-specialised (about 40 percent better Euclidean distance on average, better hypervolume and Pareto-front size on all 12); Wilcoxon signed-rank confirms all differences significant.
Evidence base	12 instances, ALL from eil51 (51 cities) with 50/150/250/500 items x 3 KP types. Per-configuration Taguchi orthogonal-array parameter tuning. 50 runs, 2000 generations. Quality measures ED, HV, PFS, RNI, S. Wilcoxon signed-rank test. Empirical attainment function for averaged fronts. No comparison with any external BI-TTP method.
Stated limitations	Authors give a nuanced interpretation (NTGA's hypervolume gains come from Nadir distance not spread; ED can worsen with a better-distributed front) and note the clone-prevention comparison cost.

Strengths. ✓ Clean factorial design (2 algorithms x 2 representations) with per-config Taguchi parameter tuning • ✓ Proper statistics: 50 runs, Wilcoxon signed-rank test, multiple quality measures, attainment-function visualisations • ✓ Careful, honest interpretation of the quality measures • ✓ Isolates the contribution of operator specialisation vs the algorithm choice (specialisation matters more) • ✓ Introduces NTGA to the BI-TTP context, where it later becomes competitive

Weaknesses. ✗ All 12 instances are eil51 (51 cities); no city-size variation; not the BI-TTP competition suite; results do not transfer • ✗ No comparison with any external BI-TTP method (blank2017, wu2018, yafrani2017); the comparison is entirely internal • ✗ Generation-based budget (2000 gen), not wall-clock; no effort comparison • ✗ The bi-objective formulation omits the value drop • ✗ Random initialisation (no LKH/greedy seeding), so absolute solution quality is likely far from good; the comparison is between two weak configurations on tiny instances • ✗ NTGA's improvements are statistically significant but modest, and Euclidean distance on 2 instances goes the wrong way

[5] Kumari and Srivastava (2020) — Variable Neighbourhood Search for Bi-Objective Travelling Thief Problem

Variant	TTP2	Objective / #obj	Bi-objective (time vs profit) / 2
Route	Full tour	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Time-decaying profits (drop rate)	Speed / Cost	linear / value-drop-rate
Algorithm	GVNS	Family (L1/L2)	Single-solution metaheuristic / Variable neighbourhood search
Benchmark	—	Size	Cities: 51 Items: 50; 150; 250; 500
Metric	normalized hypervolume (HV);	Stat. test	none
Baselines	3 VNS variants (MVNS, MVNS_Imp, GVNS) vs NTGA; neighbourhoods City_Swap/OneItemRemove/AllItemRemove + Greedy_Heuristic1/2 construction;		
Key result	GVNS best of the three variants on all 12 instances and beats NTGA on 8 of 12 (avg HV 0.8360 vs NTGA 0.8281 where it wins; 0.8540 vs 0.86825 where NTGA wins) - e.g. eil51_n250_uncorr-similar-weight GVNS 0.858414 vs NTGA 0.8471;		
Competition	N	Code	No
Budget / Runs	population size ps=50 (tuned), nbd_size=20 / 10	Hardware	Intel Core i3 laptop 2.00GHz, 4GB RAM, C++

Abstract. Kumari and Srivastava (2020) apply **Variable Neighbourhood Search (VNS)** to the bi-objective TTP, positioning it as a lightweight single-solution alternative to population-based evolutionary methods. VNS systematically changes the neighbourhood structure to escape local optima, applying a sequence of neighbourhood operators for both the TSP component (e.g., 2-opt, node insertion) and the KP component (bit-flip, swap). An acceptance criterion based on Pareto dominance determines whether a new solution replaces the current one after each neighbourhood search. The algorithm is evaluated on benchmark instances with 51 cities and between 50 and 500 items. Results show that VNS achieves performance comparable to state-of-the-art NSGA-II-based approaches while requiring fewer function evaluations. The study demonstrates that systematic neighbourhood change can effectively navigate the multi-objective TTP landscape without maintaining a population, offering a computationally efficient strategy for problems where evaluation is expensive.

Contribution	Three variable-neighbourhood-search variants for the bi-objective TTP (time vs profit): MVNS (multi-objective VNS with a population), MVNS_Imp (MVNS plus an improvement operator), and GVNS (general VNS adaptation with sequential neighbourhoods and shaking); with two problem-specific greedy initialisers, an improvement operator (no-item cities to the tour start, heavy cities to the end) and three neighbourhoods.
Authors' claims	GVNS outperforms MVNS and MVNS_Imp on all 12 instances and outperforms NTGA on 8 of the 12 (average hypervolume 0.836 vs 0.828 on those 8; on the 4 where GVNS loses, 0.854 vs 0.868).
Evidence base	12 instances, ALL eil51 (51 cities) with 50/150/250/500 items x 3 KP types (the same set as laszczyk2019). C++, i3 2.00 GHz. Population-size tuning on 3 representative instances (10 runs). Comparison: 3 VNS variants + NTGA (values quoted from laszczyk2019). Average/SD of hypervolume; number of runs for the main comparison not clearly stated. No statistical test. Could not compare with blank2017 for lack of normalisation bounds.
Stated limitations	Authors explicitly state they could not compare with blank2017 due to unavailable normalisation bounds; future work is more improved operators.

Strengths. ✓ First VNS approach for BI-TTP, with three principled variants and a clean ablation • ✓ Two problem-specific greedy initialisers and an improvement operator that exploit TTP structure • ✓ Population-size tuning with reported avg/SD • ✓ Compares against NTGA, the then-best specialised BI-TTP metaheuristic • ✓ Honest that it could not normalise against blank2017's results

Weaknesses. ✗ All 12 instances are eil51 (51 cities); no city-size variation; not the GECCO-2019 competition suite • ✗ NTGA values quoted from laszczyk2019, not re-run; different hardware/implementation • ✗ No statistical significance testing; GVNS actually loses on 4 of 12 and by a larger average margin where it loses than where it wins, so the headline is a wash • ✗ MVNS (the pure variant) performs very poorly; the contribution is really the improvement operator, not VNS per se • ✗ Number of runs for the main 12-instance comparison not clearly stated • ✗ Small conference (ICRITO); 5 pages; thin experimental design

[6] Santana and Shakya (2020) — Dynamic programming operators for the bi-objective Traveling Thief Problem

Variant	TTP2	Objective / #obj	Bi-objective (time vs profit) / 2
Route	Full tour	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Time-decaying profits (drop rate)	Speed / Cost	linear / value-drop-rate
Algorithm	MO-DP	Family (L1/L2)	Population-based metaheuristic / DP-based multi-objective
Benchmark	—	Size	Cities: 280; 4461; 33,810 Items: 1; 5; 10 per city
Metric	hypervolume (HV); fixed Pareto-front size per group (a280 100, fnl4461 50, pla33810 20);	Stat. test	points-based ranking

Baselines	HPI, Jomar (NDS-BRKGa), NTGA (GECCO-2019 top-3); MO-DP with DP-operator variants (DP20, DP_30_PI, Rot_PP_DP10_PI, DP10_PI);		
Key result	best DP variant DP_30_PI wins 7 of 9 instances among DP-based; Rot_PP_DP10_PI best on largest pla33810-n169045 (0.7954) and n338090 (0.8671); joint ranking points HPI 26 (1st), Jomar 13 (2nd), DP_30_PI 5 (3rd, above NTGA 3) - DP operators competitive with Jomar/NTGA, beaten only by HPI;		
Competition	Y	Code	No
Budget / Runs	max 1500 evaluations/run, max gen g=1000, DP window w=150 / 5	Hardware	not reported (Python DP kernel)

Abstract. [Santana and Shakya \(2020\)](#) propose integrating **dynamic programming (DP) operators** directly into population-based multi-objective TTP solvers, advancing the idea that DP—previously used only in single-objective TTP frameworks—can effectively guide bi-objective search. Several DP operator variants are introduced, each exploiting different structural characterizations of TTP instances (e.g., tour-order dominance, partial DP, truncated trade-off fronts) to improve packing plans within the evolutionary loop. A key contribution is a **fast simultaneous multi-solution evaluation strategy** that evaluates multiple individuals efficiently via shared DP computations, reducing per-generation cost. The resulting **Hybrid Evolutionary-DP Search** framework is tested on very large instances (up to 33,810 cities and 338,100 items) from the Polyakovskiy benchmark set, where standard evolutionary operators alone would be insufficient. Results show that DP-based operators consistently improve hypervolume of Pareto set approximations, establishing DP operators as scalable and effective components for large-scale bi-objective TTP.

Contribution	Dynamic-programming-based local-search operators for the bi-objective TTP (time vs profit): a hybrid MOEA (MO-DP) whose non-dominated archive doubles as population, where a DP operator modifies a solution's packing plan and multiple partial solutions (removing items from a suffix of cities) are evaluated at once; DP operators are parameterised by window size, value metric (profit or distance-to-tour-end), weight proportion and intensity; windowed DP scales to large instances.
Authors' claims	Scoring items by distance-to-tour-end beats scoring by profit inside the DP; more DP applications improve quality; the best config (distance metric, original weight, 10 repetitions) is the absolute winner on all 9 instances though sometimes by a small margin; MO-DP with the best operators is competitive with the top-3 GECCO-2019 BI-TTP competition entries but HPI still leads.
Evidence base	The 9-instance GECCO-2019 BI-TTP competition benchmark (a280, fnl4461, pla33810 x 3 KP configs, 280-33810 cities). Hypervolume metric, fixed-size Pareto-front output. Operator study: 2 classes x 8 configs, 1000 generations, window size 150. SOTA comparison: 5 executions, max 1500 evaluations, vs HPI/Jomar/NTGA competition submissions (which had unlimited time/evaluations). No statistical test on the SOTA comparison. Python DP kernel. No crossover.
Stated limitations	Authors state their goal is not a new state-of-the-art algorithm; the DP is exact only for ≤ 280 items and infeasible for larger (hence windowing); crossover is left for future work.

Strengths. ✓ Uses the standard GECCO-2019 competition benchmark and compares directly with the top-3 entries • ✓ Scales DP to large instances (33810 cities) via windowing, unlike wu2018's whole-solution DP • ✓ The 'evaluate multiple partial solutions from one solution' trick is an efficient, clean idea for bi-objective TTP • ✓ Systematic operator-parameter study (2 classes x 8 configs, all 9 instances) • ✓ Useful finding: distance-to-tour-end item scoring beats profit scoring inside the DP • ✓ Honest that it is an operator study, not a state-of-the-art claim

Weaknesses. ✗ Only 9 instances (the GECCO-2019 set); a280/fnl4461/pla33810 only • ✗ Only 5 executions and 1500 evaluations for the SOTA comparison; no statistical test; hypervolume point estimates • ✗ The SOTA comparison is against competition submissions that had unlimited time/evaluations while MO-DP had 1500 evaluations, an acknowledged confound • ✗ No crossover; MO-DP is a DP-augmented local search, not a full EA • ✗ The best DP config's hypervolume advantage is small on several instances, and the gain over profit-based DP is modest on uncorrelated instances • ✗ HPI remains first; MO-DP does not win the competition benchmark • ✗ Windowed DP loses the DP's optimality guarantee, so 'DP operator' oversells a windowed heuristic for large instances

[7] [Chagas et al. \(2021\)](#) — *A Non-Dominated Sorting Based Customized Random-Key Genetic Algorithm for the Bi-Objective Traveling Thief Problem*

Variant	TTP2	Objective / #obj	Bi-objective (time vs profit) / 2
Route	Full tour	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Time-decaying profits (drop rate)	Speed / Cost	linear / value-drop-rate
Algorithm	NDS-BRKGA	Family (L1/L2)	Population-based metaheuristic / BRKGA (multi-objective)
Benchmark	—	Size	Cities: 280; 4461; 33,810 Items: 1; 5; 10 per city
Metric	hypervolume (HV, normalized to [0,1] vs. min-time/max-profit reference);	Stat. test	none reported (competition HV ranking)
Baselines	5 other teams (EMO-2019), 12 other teams (GECCO-2019);		
Key result	best HV on 7 of 9 instances at EMO-2019 1st place; 2nd place at GECCO-2019 (won a280_n2790, 2nd on five others; top-5 on all pla33810); on pla33810_n169045 and n338090 the lead over the 2nd-best exceeded 0.65 HV;		
Competition	Y	Code	Yes
Budget / Runs	10min/run (competition); subproblem solvers TSP-t=300s (LKH 2.0.9, avg gap <0.23%), KP =5x10 ⁴ (GH+DP) / 10-30 independent	Hardware	Intel Xeon 2.30GHz cluster, single-processor (sequential), C/C++ (BRKGA framework)

Abstract. Chagas et al. (2021) propose **NDS-BRKGA** (Non-Dominated Sorting Biased Random-Key Genetic Algorithm) as a highly competitive solver for the BI-TTP (minimize travel time, maximize profit). The algorithm uses a **biased random-key encoding** to represent combined tour-packing solutions as continuous vectors, decoded into permutations and binary item selections via a deterministic procedure. The initial population is seeded with high-quality solutions derived from near-optimal TSP (Lin-Kernighan) and KP (greedy fractional) subproblem solvers, providing a domain-aware warm start. A **custom repair operator** corrects infeasible packing plans before fitness evaluation, ensuring the evolutionary operators freely explore the solution space. Non-dominated sorting and crowding distance from NSGA-II govern elite selection. A comprehensive parameter sensitivity study identifies robust settings. NDS-BRKGA **won 1st place at the EMO-2019** and **2nd place at the GECCO-2019** BI-TTP competitions, establishing it as a leading baseline algorithm for subsequent TTP2 research.

Contribution	NDS-BRKGA, a non-dominated-sorting biased random-key genetic algorithm for the bi-objective TTP (min time, max profit; the EMO/GECCO-2019 competition formulation): BRKGA random-key encoding + NSGA-II survival, with domain-knowledge customisations (biased initial population from LKH tours + a greedy-heuristic-plus-DP knapsack solver, a feasibility repair that removes last-collected items, elite/non-elite split, periodic 2-opt + bit-flip local search). Won 1st at EMO-2019 and 2nd at GECCO-2019.
Authors' claims	NDS-BRKGA won EMO-2019 (best hypervolume on 7/9 instances) and placed 2nd at GECCO-2019; the final version outperforms all other competition approaches on 3 instances and is 2nd on 3 more (HPI wins the pla33810 instances); domain-knowledge initialisation gives a significant improvement on larger instances; the single-objective by-product beats the best-known TTP1 score only on the two smallest instances.
Evidence base	9 medium/large instances (a280, fnl4461, pla33810 × 3 KP configs, 280-33810 cities). C/C++, LKH 2.0.9.3. Hypervolume metric (normalised). Parameter study: 3072 configurations × 10 runs × 5 h = about 1,382,400 CPU hours (~158 CPU years). 5 h convergence plots at 10-min resolution. Competition results quoted from the official EMO/GECCO-2019 rankings. Public source code and all non-dominated solutions.
Stated limitations	Authors note hypervolume maximisation is not equivalent to an optimal Pareto approximation, that the parameter study cost 158 CPU years, that local search gives no major improvement after 5 h, and that the single-objective score is not an explicit BI-TTP goal.

Strengths. ✓ The definitive BI-TTP competition winner (1st EMO-2019, 2nd GECCO-2019), documented with the actual competition rankings • ✓ Genuine, well-motivated domain-knowledge customisations, each ablated (LKH+GH+DP biased init, feasibility repair, elite/non-elite, periodic local search) • ✓ Enormous, rigorous parameter study (3072 configs, 10 runs, 5 h) with clear takeaways • ✓ 5 h convergence plots at 10-min resolution showing fast convergence • ✓ Public source code and all non-dominated solutions • ✓ Honest: hypervolume is not optimality; local search's marginal benefit after 5 h is small; the single-objective score is not the goal • ✓ Journal-length treatment that transparently extends two competition versions

Weaknesses. ✗ Only 9 instances (the competition suite); a280/fnl4461/pla33810 only; no small instances or other city sizes • ✗ The 158-CPU-year parameter study is not reproducible by most researchers and raises the question of how much of the win is tuning vs method • ✗ Hypervolume point estimates only; no statistical test in the main competition comparison or the final-version table • ✗ The competitions had no compute regulations, so the cross-team hypervolume comparison is partly a compute/engineering win • ✗ HPI beats NDS-BRKGA on the largest (pla33810) instances, by a large margin at GECCO-2019; the method does not scale to the largest instances as well as HPI • ✗ The (time, profit) competition formulation is not bonyadi2013's value-drop TTP2 • ✗ Local search is throttled to 10 percent of elites every omega cycles with $\min(100, n^2)$ moves for compute reasons; its full potential is untested

[8] [Myszkowski and Laszczyk \(2021\)](#) — Diversity based selection for many-objective evolutionary optimisation problems with constraints

Variant	TTP2	Objective / #obj	Bi-objective (time vs profit) / 2
Route	Full tour	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Time-decaying profits (drop rate)	Speed / Cost	linear / value-drop-rate
Algorithm	NTGA2	Family (L1/L2)	Population-based metaheuristic / EA (NTGA)
Benchmark	—	Size	Cities: 51 Items: 500
Metric	7 quality measures - ED, HV, PFS, RNI, S, IGD, Purity (P);	Stat. test	Wilcoxon signed-rank, all differences significant
Baselines	NSGA-II, -DEA, NTGA (bi-objective TTP); U-NSGA-III, -DEA, NTGA (many-objective);		
Key result	on TTP, NTGA2 best on 6 of 7 measures - HV avg 0.8813 (vs NTGA 0.8751, NSGA-II 0.8720, -DEA 0.8621), PFS 880.57 (171% larger than NTGA), Purity 0.7007, IGD 0.0006 (70.91% better than NTGA); -DEA wins only ED (0.2915, 29% better);		
Competition	Y	Code	No
Budget / Runs	max 50,000 births (evaluations); tuned via modified Taguchi / 50 (for PF visualization)	Hardware	not reported

Abstract. [Myszkowski and Laszczyk \(2021\)](#) introduce **NTGA2** (Non-Dominated Tournament Genetic Algorithm 2), a many-objective evolutionary algorithm with a novel **gap selection operator** designed to guide the population toward the least-explored regions of the current Pareto front approximation. Unlike distance-based crowding metrics that aggregate objective dimensions, the gap operator evaluates each objective independently to avoid distortion from dimension scaling differences—a relevant issue in combinatorial problems with objectives of very different magnitudes. NTGA2 is applied to both bi-objective TTP and a 5-objective multi-skill resource-constrained project scheduling problem (MS-RCPSP), demonstrating its generality. On the bi-objective TTP benchmark, NTGA2 consistently outperforms NTGA, h-DEA, and U-NSGA-III in spread and diversity metrics while maintaining competitive convergence (hypervolume). The study also achieved **3rd place at the GECCO-2019 BI-TTP competition**, and demonstrates that dimension-sensitive diversity promotion is more effective than generic crowding in combinatorial multi-objective optimization.

Contribution NTGA2, a diversity-based selection method extending NTGA with a ‘gap selection’ operator that uses the archive to find gaps in the Pareto-front approximation and directs the search to fill them, processing each objective dimension separately to avoid the combinatorial curse-of-dimensionality distance bias; applied to bi-objective TTP (time vs renting cost, no value drop) and to many-objective MS-RCPSP. NTGA2 took 3rd place in the GECCO-2019 BI-TTP competition.

Authors' claims	For bi-objective TTP, NTGA2 outperforms NSGA-II, theta-DEA and NTGA on almost all quality measures: best hypervolume (+0.72 percent over NTGA, statistically significant), best Pareto-front size (+171 percent), best inverted generational distance (+70.91 percent over NTGA), best purity, spacing and ratio of non-dominated individuals; the exception is Euclidean distance, on which theta-DEA is about 29 percent better; the gap operator only helps once the population has converged.
Evidence base	12 TTP instances, ALL from eil51 (51 cities) with 50/150/250/500 items x 3 KP types. 50 runs, 50,000 births (fixed budget). Baselines NSGA-II, theta-DEA, NTGA re-run in the same framework, all forced to use the archive; per-method Taguchi + Gray Relational Analysis tuning. Statistics: Wilcoxon signed-rank test. 7 quality measures. Inverted generational distance and purity use a union-of-methods reference (no true Pareto front). Published in Information Sciences.
Stated limitations	Authors note the gap operator may waste computation on a middle-of-front gap that constraints make unfillable, suggest turning it on/off dynamically, note that making tournament size contingent on dimension domain size is difficult, and that the bi-objective quality measures (inverted generational distance, purity) 'might not always be useful' and a better many-objective measure is needed.

Strengths. ✓ Published in a top journal (Information Sciences); NTGA2 is a genuinely competitive BI-TTP method (3rd at GECCO-2019) • ✓ The gap-selection operator is novel and well-motivated: use the archive to find Pareto-front gaps and search there, per-dimension to avoid the curse-of-dimensionality distance bias • ✓ Rigorous statistics: 50 runs, Wilcoxon signed-rank test, 7 quality measures, per-method Taguchi + Gray Relational Analysis tuning • ✓ Fair comparison: all baselines re-run in the same framework and forced to use the archive • ✓ Honest about the gap operator's limitations and the theta-DEA exception (better convergence) • ✓ Also demonstrates many-objective scalability (3/4/5-objective MS-RCPSP)

Weaknesses. ✗ All 12 TTP instances are eil51 (51 cities); no city-size variation; not the GECCO-2019 competition suite; results do not transfer to the instance sizes the competition uses • ✗ No comparison with the actual BI-TTP competition winners (NDS-BRKGA, HPI) on the competition instances; only NSGA-II, theta-DEA and NTGA on eil51 • ✗ The BI-TTP formulation is (time, renting cost) = TTP1's objective split, not bonyadi2013's value-drop TTP2 • ✗ NTGA2's hypervolume advantage over NTGA is small (+0.72 percent); the large gains are in spread/gap-filling metrics, while theta-DEA beats it on convergence • ✗ Inverted generational distance and purity use a union-of-methods reference, so the '70.91 percent better' figure is relative to that pool • ✗ The GECCO-2019 3rd place is mentioned but not analysed (no per-instance competition hypervolume comparison) • ✗ TTP is one of two testbeds; it gets the smaller experimental section (12 eil51 instances)

[9] Yang et al. (2021) — An MOEA:D-ACO algorithm with finite pheromone weights for Bi-objective TTP

Variant	TTP2	Objective / #obj	Bi-objective (time vs profit) / 2
Route	Full tour	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Time-decaying profits (drop rate)	Speed / Cost	linear / value-drop-rate
Algorithm	MOACO-PW	Family (L1/L2)	Population-based metaheuristic / Swarm (multi-objective ACO)

Benchmark	—	Size	Cities: 280; 4461; 33,810 Items: 1; 5; 10 per city
Metric	normalized hypervolume (HV), reference point (30000, 30000);	Stat. test	none
Baselines	Greedy, ISA, ISA-Local, NSGA-II, MOEA/D-ACO;		
Key result	MOACO-PW best HV on most datasets - a280-n279 0.874 (vs NSGA-II 0.825, MOEA/D-ACO 0.851), a280-n790 0.792, fnl4461-n446 0.798, pla33810-n809 0.628; MOEA/D-ACO better on a280-n395 (0.872) and fnl4461-n230 (0.823); NSGA-II best on pla33810-n338 (0.858);		
Competition	N	Code	No
Budget / Runs	1000 iterations (~100,000 evaluations), $G_{\infty}=3000$; params: ants $N=24$, groups $K=3$, neighbours $T_0=10$, $\alpha=0.95$, $\beta=1$, $\gamma=2$ / 30	Hardware	not reported

Abstract. Yang et al. (2021) propose **MOACO-PW** (Multi-Objective Ant Colony Optimization with Finite Pheromone Weights) for bi-objective TTP, combining the MOEA/D decomposition framework with ACO-based tour construction. A key contribution is **distance-related pheromone weight initialization** that incorporates geographic distance information to guide ants toward shorter tours. A new **weighted Tchebycheff aggregation function** blends the weighted-sum and Tchebycheff aggregation approaches via a tunable proportion parameter, exploiting both methods' strengths in approximating non-convex Pareto fronts. **Dynamic adjustment of ant neighbor numbers** across iterations prevents premature convergence, and an **adaptive mutation operator** perturbs solutions to escape local optima. Evaluated on large bi-objective TTP benchmark instances (up to 33,810 cities and 338,100 items), MOACO-PW outperforms GA, ISA, ISA-Local, and standard NSGA-II in hypervolume and spread indicators. The work extends pheromone-weight-aware decomposition as an effective paradigm for ACO in multi-objective multi-component optimization.

Contribution	MOACO-PW, a MOEA/D-ACO (decomposition + ant colony) algorithm with 'finite pheromone weights' for the bi-objective TTP (time vs profit, no value drop): distance-weighted pheromone initialisation, a weighted-Tchebycheff aggregation blending weighted-sum and Tchebycheff, adaptive weight-vector adjustment toward under-searched regions, and an adaptive ant-neighbour count.
Authors' claims	MOACO-PW has competitive performance versus GA, ISA, ISA-Local, NSGA-II and the base MOEA/D-ACO; higher average hypervolume in most datasets; better convergence speed and a more uniform Pareto front; MOACO-PW beats all baselines on 6 of the 9 large instances.
Evidence base	A custom dataset (10/20/50/100 cities, 100,000 iterations) and the '9 Polyakovskiy' instances (1000 iterations). 30 runs, averaged. Normalised hypervolume with a fixed reference point (30000, 30000). Baselines GA/ISA/ISA-Local/NSGA-II from blank2017 (quoted), MOEA/D-ACO re-run. No statistical test.
Stated limitations	Authors note the dataset-specification problem, and that the number of iterations, the number of averages and multi-dimensional targets need further study.

Strengths. ✓ Brings MOEA/D-ACO (decomposition + ant colony) to BI-TTP, a different algorithmic family from the NSGA-II mainstream • ✓ Four sensible mechanistic enhancements, each motivated • ✓ 30 runs, compares against 5 baselines including the base MOEA/D-ACO (an ablation of the enhancements) • ✓ Convergence-speed analysis (hypervolume vs iterations)

Weaknesses. ✗ The instance table has garbled/incorrect names and city counts (a280 listed with 280/395/790 cities; fnl4461 as 'fnl446'; pla33810 as 'pla338'), so it is unclear which instances were actually solved • ✗ The hypervolume reference point is a fixed (30000, 30000) regardless of instance scale, which makes the hypervolume values meaningless for the large instances (objectives in the millions) • ✗ The large-instance comparison uses 1000 iterations vs 100,000 for the custom set; on 4461-/33810-city instances that is far too few, and the comparison against blank2017's quoted values is not like-for-like • ✗ No statistical significance testing; averaged hypervolume point values only • ✗ MOACO-PW loses to MOEA/D-ACO on 2/9 and to NSGA-II on 1/9 large instances; 'better in most cases' rests on a flawed evaluation • ✗ Baselines quoted from blank2017; unexplained '-' entries where methods 'did not obtain a result' • ✗ Poor English throughout; a minor proceedings venue; not built upon by the mainstream BI-TTP line

[10] Chagas and Wagner (2022b) — *A weighted-sum method for solving the bi-objective traveling thief problem*

Variant	TTP2	Objective / #obj	Bi-objective (time vs profit) / 2
Route	Full tour	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Time-decaying profits (drop rate)	Speed / Cost	linear / value-drop-rate
Algorithm	WSM	Family (L1/L2)	Population-based metaheuristic / Weighted-sum BRKGA
Benchmark	—	Size	Cities: up to 85,900 Items: 1; 3; 5; 10 per city
Metric	hypervolume (HV, normalized to [0,1]); % HV variation vs. NDSBRKGA; RPD on single-objective TTP score;	Stat. test	Wilcoxon signed-rank, 5%
Baselines	NDSBRKGA chagas2021non, HPI, NTGA, shisunzhang, ALLAOUI, SSteam (EMO/GECCO-2019 entries), and the 21 single-objective algorithms surveyed in wagner2018case;		
Key result	significantly better than NDSBRKGA on 789/960 instances (82.2%), tied 27 (2.8%), worse 144 (15%) by Wilcoxon; best HV on 6 of 9 competition instances (e.g. fnl4461_01_bsc HV 0.934685 vs NDSBRKGA 0.933942; pla33810_01_bsc 0.930580 vs HPI 0.927214); 379 new best-known single-objective TTP scores beating all 21 prior algorithms;		
Competition	N	Code	Yes
Budget / Runs	10min/run (tuning/main); 5h/run (competition comparison) / 30 independent	Hardware	Intel Xeon X5650 2.67GHz, Java 8, CentOS 7.4

Abstract. Chagas and Wagner (2022b) propose a **Weighted-Sum Method (WSM)** for the BI-TTP that decomposes the bi-objective problem into a parametric family of single-objective subproblems by assigning convex weights to the travel time and profit objectives. Each weighted subproblem is solved by a **two-stage heuristic**: first, a high-quality TSP tour is constructed using the **Chained Lin-Kernighan (CLK)** algorithm; second, a **randomized variant of the PACKITERATIVE heuristic** generates diverse packing plans by randomly varying item processing order, enabling multiple Pareto-equivalent solutions per weight setting. Local search operators borrowed from single-objective TTP are applied for further improvement. The method is tuned on 96 instance groups and evaluated on competition benchmarks, outperforming 6 of 9 competition entries. Crucially, by solving weighted single-objective subproblems, the method discovers **379 new best-known solutions** for standard TTP1 instances, demonstrating that bi-objective decomposition can simultaneously advance single-objective TTP research.

Contribution	WSM, a weighted-sum decomposition method for the bi-objective TTP (time vs profit, no value drop): scalarise into many single-objective TTP problems and solve each with a two-stage heuristic (Chained-LK tour + a randomised version of a deterministic packing heuristic), incorporating single-objective TTP local-search operators; 6 parameters tuned by irace on 96 instance groups (up to 85,900 cities).
Authors' claims	WSM outperforms the competition participants on 6 of 9 competition instances (HPI and NDS-BRKGA win the 3 smallest) and finds new best solutions to 379 single-objective TTP instances; versus NDS-BRKGA (re-tuned identically, 30 runs, 10 min), a Wilcoxon signed-rank test on hypervolume finds no significant difference on 2.8 percent of instances, WSM significantly better on 82.2 percent and worse on 15 percent; a single averaged parameter configuration performs essentially as well as per-instance tuning on 100 unseen instances.
Evidence base	96 instance groups (eil51 to pla85900). irace tuning. WSM vs NDS-BRKGA: 30 runs, 10 min each, normalised hypervolume, Wilcoxon signed-rank test at 5 percent across about 960 instances. Competition comparison: 5 h runtime, 9 instances, hypervolume, vs the official EMO/GECCO-2019 rankings. Public source code and all irace logs.
Stated limitations	Authors note weighted-sum methods may not generate a dispersed Pareto front even with consistent weight changes (argued not to hurt here), that the packing heuristic struggles on uncorrelated-item instances, that WSM was not developed for a single-objective purpose, and that the HPI implementation was unavailable for the tuning-based comparison.

Strengths. ✓ Rigorous, journal-quality (COR): 96-group irace tuning with a full parameter characterisation, a generalisation test on 100 unseen instances, and a 5 h competition comparison • ✓ Proper statistics: Wilcoxon signed-rank test on 30-run hypervolume distributions across about 960 instances, one of the most statistically thorough BI-TTP comparisons • ✓ Genuinely competitive: beats the competition winners on 6/9 competition instances and NDS-BRKGA on the large instances; new best-known on 379 single-objective TTP instances • ✓ The randomised-packing-heuristic idea (randomise a deterministic heuristic for diversity across scalarisations) is clean and effective • ✓ Fair NDS-BRKGA comparison: re-tuned with the same irace procedure, same 10-min budget, 30 runs • ✓ Public code and all irace logs; scales to 85,900 cities • ✓ Honest about the dispersed-front limitation of weighted-sum and the packing-heuristic weakness on uncorrelated instances

Weaknesses. ✗ WSM loses to HPI and NDS-BRKGA on the 3 smallest competition instances, by the authors' own account • ✗ The competition comparison uses 5 h runtime and quotes competition hypervolume values from competitions with no compute regulations, so the cross-team comparison is not compute-controlled • ✗ The BITTP formulation omits the value drop; WSM's scalarisation is exactly the single-objective TTP objective, so WSM is arguably 'solving TTP1 many times' rather than genuinely multi-objective • ✗ Weighted-sum cannot find non-convex Pareto regions by construction (argued not to matter empirically here, but a structural limitation) • ✗ The 379 new best-known single-objective solutions are relative to literature reference values, not independent optima, and the authors caution WSM is not a single-objective method • ✗ On uncorrelated-item instances WSM is beaten by NDS-BRKGA; the packing weakness there is acknowledged but not fixed • ✗ Per-instance hypervolume tables are not in the body; the main evidence is comparison heatmaps

[11] **Erdoğan (2022)** — *Comparing ALNS and NSGA-II on Bi-Objective Travelling Thief Problem*

Variant	TTP2	Objective / #obj	Bi-objective (time vs profit) / 2
Route	Full tour	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Time-decaying profits (drop rate)	Speed / Cost	linear / value-drop-rate
Algorithm	ALNS vs NSGA-II	Family (L1/L2)	Single-solution metaheuristic / ALNS (comparison)
Benchmark	—	Size	Cities: 52; 280 Items: 1; 5; 10 per city
Metric	hypervolume (HV) and C-Metric (coverage/dominance ratio);	Stat. test	none
Baselines	ALNS vs NSGA-II, each hybridized with 4 TSP + 4 KP local searches (CLKH, 2-opt, swap, insert / KP-random, KP-by-score, bit-flip, swap);		
Key result	NSGA-II beats ALNS on all 6 - e.g. berlin52_n51_bsc HV NSGA-II 0.99 vs ALNS 0.96; a280_n279_bsc 0.88 vs 0.73; C-Metric(NSGA-II,ALNS) 25-86% while C-Metric(ALNS,NSGA-II) ~0%; new reference Pareto fronts produced;		
Competition	N	Code	Unknown
Budget / Runs	600s/run (SA T_0=100, cooling 0.99) / 30	Hardware	not reported

Abstract. Erdoğdu (2022) conduct a systematic comparison between two contrasting metaheuristic paradigms for the BI-TTP: **Adaptive Large Neighbourhood Search (ALNS)** (a single-solution intensification approach) and **NSGA-II** (a population-based diversification approach), both hybridized with TSP and KP local search. ALNS uses a destroy-and-repair mechanism with adaptively weighted operators (CLKH for TSP, PACKITERATIVE for KP) to explore the search space from a single solution, accepting improvements based on dominance. NSGA-II maintains a Pareto population with standard evolutionary operators augmented by the same local search routines. Both methods are initialized using high-quality TSP tours and packing plans. Experiments on six Polyakovskiy benchmark instances (52 and 280 cities, 1/5/10 items per city) yield new reference Pareto fronts for these instances. The study concludes that no single paradigm uniformly dominates across all instances, and that hybridization with problem-specific local search is the decisive factor for performance in both frameworks.

Contribution	Hybridises and compares two metaheuristic types on the bi-objective TTP (time vs profit, no value drop): adaptive large neighbourhood search (single-solution, simulated-annealing-driven, adaptive local-search selection) versus NSGA-II (population), both with Chained-LK tour init + greedy KP init + 4 TSP local searches + 4 KP local searches; publishes new Pareto fronts.
Authors' claims	NSGA-II outperforms ALNS on all 6 instances: higher hypervolume, and by the C-metric NSGA-II dominates most of ALNS's front while ALNS dominates about 0 percent of NSGA-II's front.
Evidence base	Only 6 instances: berlin52 (52 cities) and a280 (280 cities), each x bsc_01, usw_05, uc_10. 30 runs, 600 s. Metrics: hypervolume (nadir from the combined front) and C-metric. No statistical test; detailed results on a website. No comparison with any established BI-TTP method; only ALNS vs NSGA-II, both authored here.
Stated limitations	None stated explicitly; 'space limitation' for detailed results.

Strengths. ✓ Clean single-solution-vs-population comparison on BITTP with matched components (same init, same 8 local searches) • ✓ Uses two complementary multi-objective metrics (hypervolume and C-metric) • ✓ 30 runs, 600 s (standard TTP protocol) • ✓ Publishes the Pareto fronts

Weaknesses. ✗ Only 6 instances (2 city sizes ≤ 280 , 1 config per KP type); not the GECCO-2019 suite; not systematic • ✗ No comparison with any established BI-TTP method; only the two hybrids against each other • ✗ No statistical test; hypervolume reported to 2 decimals with no variance • ✗ The hypervolume normalisation uses a combined-front nadir, so values are pool-relative to just these two methods • ✗ 'ALNS loses to NSGA-II' is about this specific ALNS hybrid and the paper does not analyse why • ✗ On a280_n2790 the hypervolume gap is tiny (0.52 vs 0.54) and ALNS dominates a few NSGA-II points, so the 'NSGA-II wins all' headline is marginal there • ✗ Regional OM conference; 10 pages; thin; value drop omitted

[12] Garbaruk et al. (2022) — Comparative Study by Using a Greedy Approach and Advanced Bio-Inspired Strategies in the Context of the Traveling Thief Problem

Variant	TTP2	Objective / #obj	Bi-objective (time vs profit) / 2
Route	Full tour	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static

Value model	Time-decaying profits (drop rate)	Speed / Cost	linear / value-drop-rate
Algorithm	greedy vs NSGA-II	Family (L1/L2)	Problem def./benchmark/analysis / Comparison/analysis
Benchmark	—	Size	Cities: 280; 4461; 33,810 Items: 1; 5; 10 per city
Metric	Pareto front shape (negative profit vs travel time, visual comparison);	Stat. test	none (visual)
Baselines	Greedy (Nearest-Neighbour tour + value/weight greedy packing) vs NSGA-II (population sizes 20-500, mutation rates 1-50%);		
Key result	Greedy surprisingly outperforms NSGA-II on all population-size/mutation-rate settings (NSGA-II Pareto front consistently dominated, shorter spread); recommended NSGA-II config: population ~100, mutation rate 30%;		
Competition	N	Code	No
Budget / Runs	10,000 generations / not reported	Hardware	not reported

Abstract. Garbaruk et al. (2022) present a comparative analysis of two **generic, instance-independent algorithms** for the TTP: a **greedy construction algorithm** and a **bio-inspired NSGA-II-based strategy**. The motivation is to identify an algorithm that performs robustly across diverse TTP instances without requiring instance-specific parameter tuning. The greedy algorithm constructs a tour and sequentially selects items to maximize profit subject to capacity constraints. NSGA-II jointly evolves tour and packing decisions using standard operators without TTP specialization. Both are evaluated on a broad range of Polyakovskiy benchmark instances (280 to 33,810 cities, 1/5/10 items per city). Results show that while NSGA-II generally produces more diverse Pareto approximations, the greedy algorithm is competitive on smaller instances and provides rapid initial solutions. The study provides practical guidance for practitioners who need baseline results quickly and cannot afford algorithm specialization, while also identifying instance characteristics (size, item density, correlation type) that favour each approach.

Contribution	Compares a simple greedy algorithm (nearest-neighbour tour + greedy benefit/weight item picking, generating a Pareto front along the way) versus a from-scratch NSGA-II (random init, simple order-based recombination, swap/bit-flip mutation, capacity repair) for the bi-objective TTP (time vs profit, no value drop), using the GECCO-2019 competition instances.
Authors' claims	Surprisingly, the greedy algorithm performed far better than NSGA-II across all population sizes (20-500) and mutation rates (1-50 percent); the EA's advantage (multiple paths) is ineffective because the paths are generated randomly and the instances are too large; population about 100 is recommended; future work is seeding NSGA-II with the greedy solution.
Evidence base	Detailed results shown for ONE instance (a280, 279 items, capacity 25936). Two figures of Pareto fronts by population size / mutation rate vs greedy, 10,000 generations. No table, no hypervolume values, no statistical test, no other instances shown despite claiming to use the 9-instance GECCO-2019 set. Nearest-neighbour heuristic for the tour (not LKH).
Stated limitations	Authors state NSGA-II's random path generation is the problem and the instances are too large; future work is an optimised initial population (from greedy) and a second-best-path perturbation for diversity.

Strengths. ✓ An honest negative result: a plain greedy algorithm beats a from-scratch NSGA-II on large BI-TTP instances • ✓ Correctly diagnoses why (random tour init means the EA cannot find fast tours) • ✓ Uses the GECCO-2019 competition instances (at least nominally) • ✓ Points to the fix (seed with the greedy solution), which is exactly what the winning NDS-BRKGGA does

Weaknesses. ✗ The empirical content is two figures on a single instance (a280_n279); the claim to use 9 test instances is not backed by any per-instance results, table or hypervolume • ✗ No hypervolume, C-metric or statistical test; the 'greedy far better' conclusion rests on visual inspection of two plots • ✗ NSGA-II here is a straw baseline: random tour initialisation, nearest-neighbour tour for the greedy, simple recombination, no local search; not representative of any competent BI-TTP EA • ✗ The 'surprising' finding is already known: blank2017 and chagas2021 explicitly seed their EAs with LKH+greedy for exactly this reason • ✗ The greedy itself uses nearest-neighbour for the tour, so even the winner is far from good absolute quality • ✗ Short conference paper; no comparison with any real BI-TTP method; value drop omitted

[13] Kumari and Srivastava (2023) — NSGAII for Travelling Thief Problem with Dropping Rate

Variant	TTP2	Objective / #obj	Bi-objective (time vs profit) / 2
Route	Full tour	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Time-decaying profits (drop rate)	Speed / Cost	linear / value-drop-rate
Algorithm	NSGA-II (drop rate)	Family (L1/L2)	Population-based metaheuristic / NSGA-II
Benchmark	—	Size	Cities: 51 Items: 50; 150; 250; 500
Metric	hypervolume (HV) with computed ideal/nadir vectors; non-dominated set size NDS ;	Stat. test	none
Baselines	none (TTP2 with dropping rate previously unsolved); NSGA-II with 3 construction heuristics + TTP2-aware crossover/mutation/repair;		
Key result	HV ranges 0.196 (100_100_1_25) to 0.730 (50_50_3_75); NDS up to 42; HV and NDS both grow with iterations; small instances converge by 100 iterations, large ones still improving;		
Competition	N	Code	No
Budget / Runs	100 iterations (max) / not reported	Hardware	not reported

Abstract. Kumari and Srivastava (2023) present the **first dedicated algorithmic study of TTP2** with the dropping-rate feature originally defined in Bonyadi et al. (2013)—a formulation that had been largely overlooked as previous bi-objective TTP work had focused on BI-TTP (same as TTP1 formulated as bi-objective without dropping rate). The authors design **NSGA-II with three construction heuristics** for generating initial populations that incorporate TTP2-aware solution quality (accounting for both time and the value decay caused by travel time). Problem-specific **crossover and mutation operators** are tailored to the combined tour-packing encoding: operators preserve partial TSP paths while simultaneously adjusting item selections to account for the time-dependent profit. Experiments are conducted on 20 benchmark instances and evaluated using hypervolume and spread metrics. Since no prior algorithm specifically addresses TTP2, no competitive baseline comparison is possible, and the work establishes the first systematic experimental results for this variant, providing a reference for future TTP2 algorithm development.

Contribution	The first metaheuristic for the actual value-drop TTP2 (min time, max time-decayed profit): an NSGA-II with three construction heuristics (CLKH tour + profit/weight-ratio greedy; CLKH + profit/(weight × min-distance-carried); nearest-distance tour + the second KP heuristic), problem-specific crossover (swap tours, segment-swap packing), a perturbation mutation that moves item pickup to a later city to reduce carry time and decay, and a repair operator.
Authors' claims	Construction heuristic Cons_3 gives the best mean hypervolume (0.38980) and the most best-hypervolume values (9/19); population size restricted to floor($m/2$); the algorithm's hypervolume converges over 100 iterations; no competitive testing is possible because no previous work for TTP2 exists.
Evidence base	20 self-generated/adapted TTP2 instances, all small (n in 10,20,50,100); one instance has hypervolume 0 (no feasible items). 100 iterations, population size floor($m/2$). Normalised hypervolume (nadir + ideal from NSGA-II runs). Number of runs not clearly stated; single hypervolume values in the tables. No statistical test. No baseline (first TTP2 work). The construction-heuristic comparison is the only experiment.
Stated limitations	Authors explicitly state competitive testing is not possible because no previous work for TTP2 exists, that the optimal Pareto front is unknown, and that hypervolume is not free from arbitrary scaling (addressed via normalisation).

Strengths. ✓ Genuinely fills a gap: the first metaheuristic for the actual value-drop TTP2 as bonyadi2013 defined it (most 'BI-TTP' work drops the value decay) • ✓ Three thoughtfully differentiated construction heuristics with a clear comparison • ✓ Problem-specific crossover/mutation/repair operators designed for the TTP2 structure (e.g. delaying item pickup to reduce decay) • ✓ Uses normalised hypervolume to address the scaling issue • ✓ Honest that no competitive comparison is possible

Weaknesses. ✗ 20 self-generated/adapted instances, all small ($n \leq 100$); no standard TTP2 benchmark exists and this paper does not establish one (instance-generation details are thin) • ✗ No baseline at all (unavoidable for the first work), so 'the algorithm works' is shown only by hypervolume converging, not by beating anything • ✗ Number of independent runs not clearly stated; tables appear to report single hypervolume values; no variance, no statistical test • ✗ Only the 3 construction heuristics are actually compared; the NSGA-II itself is textbook with a tiny 100-iteration budget • ✗ Population size and iteration count are ad-hoc, tied to instance size with no tuning • ✗ The dropping-rate parameters are instance parameters but their effect on solution structure or difficulty is not analysed • ✗ Short conference paper; a minimal proof-of-concept for a significant 'first' claim

[14] Santiyuda et al. (2024) — Solving biobjective traveling thief problems with multiobjective reinforcement learning

Variant	TTP2	Objective / #obj	Bi-objective (time vs profit) / 2
Route	Full tour	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Time-decaying profits (drop rate)	Speed / Cost	linear / value-drop-rate
Algorithm	MORL (Pointer/Attention)	Family (L1/L2)	Learning-based / Multi-objective RL
Benchmark	—	Size	Cities: 76; 280; 4461 Items: 1; 5; 10 per city
Metric	hypervolume (HV);	Stat. test	Friedman + Iman-Davenport, Hochberg post-hoc, =0.05
Baselines	NDS-BRKGA chagas2021non (pop N=500, =10, 10min/instance); 6 proposed = PN,AMxDRL-MOA, PHN, MBPS;		
Key result	DRL-MOA methods achieve higher median HV than NDS-BRKGA on most graphs; beat NDS-BRKGA at M _{city} =1 and on high-item-density; NDS-BRKGA stronger as capacity C grows; inference ≥1 order of magnitude faster (no per-instance re-solve);		
Competition	Y	Code	Yes
Budget / Runs	training N _s =100 agents, 10,000 epochs (DRL-MOA) / 100 instances/eval	Hardware	NVIDIA GTX 2080 Ti, Intel i9-7900X (20 cores), PyTorch

Abstract. [Santiyuda et al. \(2024\)](#) propose the first **end-to-end Multi-Objective Reinforcement Learning (MORL)** framework for the bi-objective TTP, targeting the joint city tour and item selection problem without decomposing into independent subproblems. Two encoder architectures are evaluated: **Pointer Network (PN)** and **Attention Mechanism (AM)**, each combined with three MORL training strategies: **DRLMOA** (deep reinforcement learning multi-objective algorithm, optimizing the Pareto front via scalarized rewards), **PHN** (multi-sample Pareto hypernetwork, generating solutions conditioned on preference vectors), and **MBPS** (manifold-based policy search, exploring the Pareto manifold via smooth preference interpolation). Since PN and AM generate single sequences (city tours), a **novel encoding/decoding scheme** is introduced to simultaneously produce city visit order and binary item selection without modifying the base architectures. All models are trained exclusively on small Eil76-based random instances and tested on larger diverse instances. PN-DRLMOA and AM-DRLMOA achieve competitive Pareto set quality, especially on high-item-density instances, while **outperforming NDS-BRKGA by over an order of magnitude in solution generation speed**, highlighting MORL’s potential for real-time or large-scale TTP2 optimization.

Contribution The first end-to-end multi-objective reinforcement learning (MORL) approach for the bi-objective TTP (time vs profit, no value drop): two neural architectures (pointer network, attention model) x three MORL methods (deep RL multi-objective, manifold-based policy search, Pareto hypernetwork), with a solution encoding/decoding scheme that lets single-sequence architectures produce the two sequences (tour + item selection) of TTP; trained only on small Eil76-based instances.

Authors' claims	Competitive with NDS-BRKGA especially on instances with many items; the DRL-MOA methods generalise to different/larger graphs; MORL methods significantly outperform NDS-BRKGA on inference running time (seconds/minutes vs 600 s) and manifold-based policy search trains in under 15 percent of DRL-MOA's training time; honest negatives: on few-item instances MORL solutions are all dominated by NDS-BRKGA, MORL cannot produce low-travel-time solutions, and there are overlapping/dominated solutions.
Evidence base	Trained on small Eil76-subgraph instances; tested on 10 graphs up to 1379 cities, M_city in 1,3,5,10, capacity in 1..10, 3 profit-weight correlations. Baseline NDS-BRKGA only. Hypervolume indicator. Statistics: Friedman with Iman-Davenport extension + Hochberg post-hoc (NDS-BRKGA as control), alpha=0.05, on every instance group (Appendices A/B), 100 runs. Public code and full evaluations repository.
Stated limitations	Authors explicitly state the biggest drawback is long training time, that generalisation needs better problem-instance design, that MORL cannot generate low-travel-time solutions and needs better route optimisation, and that overlapping/dominated solutions occur.

Strengths. ✓ Genuine first: end-to-end MORL for TTP, a real new algorithmic paradigm for the variant • ✓ Thorough experimental design: 6 architecture x method combinations, 10 test graphs (incl. sizes and correlations not in training), 3 instance parameters swept • ✓ Proper statistics: Friedman + Iman-Davenport + Hochberg post-hoc with NDS-BRKGA as control, on every instance group, 100 runs • ✓ Honest, detailed negative results (dominated on few-item instances, cannot produce fast-tour solutions, overlapping solutions) • ✓ The key practical finding (MORL is orders of magnitude faster at inference; manifold-based policy search trains in a fraction of DRL-MOA time) is well-supported • ✓ Demonstrates generalisation to graphs and item counts not in training • ✓ Public code and full evaluations repository

Weaknesses. ✗ Only one baseline (NDS-BRKGA); no comparison with NTGA2, WSM, MO-DP or any other BI-TTP method • ✗ MORL is dominated by NDS-BRKGA on few-item instances (a large part of the benchmark) with far lower hypervolume there; 'competitive' holds only for many-item instances • ✗ MORL cannot generate low-travel-time solutions at all; the route-optimisation side is fundamentally weak (acknowledged as future work) • ✗ The BI-TTP formulation omits the value drop • ✗ Trained only on Eil76-derived small instances; the training-instance design is acknowledged as inadequate • ✗ Long training time is the dominant cost; the inference-speed advantage is offset by a large one-time training cost • ✗ Hypervolume is the only quality indicator; many overlapping/dominated solutions shrink the effective front size

[15] Xiang et al. (2024a) — Multi-Objective Five-Element Cycle Optimization Algorithm Based on Multi-Strategy Fusion for the Bi-Objective Traveling Thief Problem.

Variant	TTP2	Objective / #obj	Bi-objective (time vs profit) / 2
Route	Full tour	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Time-decaying profits (drop rate)	Speed / Cost	linear / value-drop-rate
Algorithm	MOFECO-MS	Family (L1/L2)	Hyper-heuristic / Five-element cycle (MO)
Benchmark	—	Size	Cities: 280; 4461; 33,810 Items: 1; 5; 10 per city

Metric	hypervolume (HV), spacing (SP), pure diversity (PD); reporting Best/Worst/Median/Average/StdDev;	Stat. test	descriptive (Best/Worst/Median/Avg/StdDev)
Baselines	8 algorithms - NSGA-II, MOPSO, MOAO, MOHBA, MOWOA, MOGWO, MOFECO, LSMOFECO;		
Key result	MOFECO-MS best HV (e.g. eil51_n50 avg 2.30×10^9 vs LSMOFECO 1.54×10^9 , MOFECO 7.34×10^8 , NSGA-II 8.64×10^8), and best SP and PD on most instances;		
Competition	N	Code	No
Budget / Runs	$c_ = N \times 1000 = 100,000$ evaluations, $N=100$; tuned $L=5$, $q=20$, $l=3$, $p_c=0.9$, $p_m=0.9$, SM4 update / 15	Hardware	MATLAB R2018a, Intel Xeon E5645 2.4GHz, 32GB RAM, Windows 10

Abstract. Xiang et al. (2024a) propose **MOFECO-MS** (Multi-Objective Five-Element Cycle Optimization with Multi-Strategy Fusion), a novel nature-inspired algorithm for the BITTP drawing from the **Five-Element Cycle Model** (the five elements: metal, wood, water, fire, earth and their generative/destructive relationships). Each individual in the population is assigned an element type, governing its behaviour through a unique **element update mechanism** that determines how solutions evolve based on inter-element relationships. The population is partitioned into elemental groups; **intra-group crossover** promotes local exploitation of similar solutions, while **inter-group crossover** introduces global diversity by combining information across different elemental groups. A dedicated **mutation operator** is applied to elite individuals to enable fine-grained local search. The multi-strategy fusion mechanism dynamically coordinates these components based on population diversity and fitness landscape. Evaluated against eight state-of-the-art algorithms on nine BITTP competition instances, MOFECO-MS achieves best **hypervolume (HV)**, **spread (SP)**, and **pure diversity (PD)** indicators, demonstrating that culturally inspired update mechanisms can effectively balance convergence and diversity in multi-objective combinatorial optimization.

Contribution	MOFECO-MS, a multi-objective Five-element Cycle Optimization algorithm with multi-strategy fusion for the bi-objective TTP (time vs profit, GECCO-2023 dataset, no value drop): based on the Chinese Five-element Cycle Model, with population grouping, within-group and between-group crossover, mutation local search on the best individuals, and 4 element-update search-method variants.
Authors' claims	MOFECO-MS consistently outperforms 8 state-of-the-art multi-objective algorithms on 9 BITTP instances, especially in hypervolume and Spread, and is competitive (not always best) on Pure Diversity; it is robust across scales.
Evidence base	9 BITTP instances (eil51/ch150/a280, max 280 cities; GECCO-2023 dataset). 15 runs. 100,000 objective evaluations. Baselines: NSGA-II plus 5 metaphor swarm algorithms (MOPSO, MOWOA, MOGWO, MOAO, MOHBA) plus MOFECO (base) plus LSMOFECO (the authors' prior), with the swarm baselines using their default (non-TTP) parameters. Metrics: hypervolume, Spread, Pure Diversity, reported as Best/Worst/Median/Average/StdDev over 15 runs. NO statistical test. Extensive parameter-tuning study. Open access; public dataset.
Stated limitations	Future work: adaptive parameter adjustment, dynamic update strategies by evolutionary stage, other evolutionary operators, co-evolution.

Strengths. ✓ Extensive parameter-tuning study (crossover/mutation probabilities, cycle parameters, 4 search-method variants) with reported hypervolume statistics • ✓ 9 instances across 3 city sizes; 15 runs; consistent 100,000-evaluation budget • ✓ Open access; public GECCO-2023 dataset; an ablation of the 4 element-update variants • ✓ MOFECO-MS clearly beats the generic MO baselines and its own predecessors on hypervolume/Spread

Weaknesses. ✗ NOT ONE real BI-TTP method is used as a baseline; all 8 baselines are generic MO metaheuristics (NSGA-II plus 5 metaphor swarm algorithms plus 2 prior FECO variants), none designed for TTP, and the swarm baselines use default non-TTP parameters • ✗ ‘Outperforms 8 state-of-the-art algorithms’ does not mean it competes with NDS-BRKGA, WSM, NTGA2 or the GECCO competition winners • ✗ No statistical significance testing at all; 27 tables of Best/Worst/Median/Average/StdDev with no p-values • ✗ Only 15 runs; max 280 cities; not the full GECCO competition suite • ✗ Metaphor-heavy framing; the ‘novel selection and update strategy’ reduces to population grouping + crossover + mutation local search • ✗ Beating widely criticised metaphor swarm algorithms (MOWOA/MOGWO/MOAO/MOHBA) is not evidence of quality • ✗ Raw hypervolume reported without a stated reference point / normalisation, so cross-instance and cross-paper comparison is impossible • ✗ 35 pages, mostly tuning and per-instance tables; same group and pattern as xiang2024five

[16] Viet et al. (2026) — *Advanced Quantum Annealing for the Bi-Objective Traveling Thief Problem An varepsilon-Constraint-Based Approach*

Variant	TTP2	Objective / #obj	Bi-objective (time vs profit) / 2
Route	Full tour	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Time-decaying profits (drop rate)	Speed / Cost	linear / value-drop-rate
Algorithm	QA + eps-constraint (QUBO)	Family (L1/L2)	Quantum / hybrid quantum / Hybrid quantum-classical
Benchmark	—	Size	Cities: 280; 4461; 33,810 Items: 1; 5; 10 per city
Metric	hypervolume (HV, min-max normalized, reference [1,1]) and running time;	Stat. test	none
Baselines	NSGA-II, U-NSGA-III, MOEA/D (Pymoo, pop 500, crossover 0.8, mutation 0.1; 10,000 iters a280 / 4,000 iters ch150); also weighted-sum and no-LEA ablations;		
Key result	QA -constraint HV consistently >0.7; with LEA up to 35% higher quality than weighted-sum (a280_n279_bsc +LEA HV 0.89 vs weighted-sum 0.64); up to 8.1x faster than NSGA-II (a280_n279_bsc 4161s vs 33,771s); LEA overhead only 0.5% of runtime; S=10 segments + equal initialization optimal;		
Competition	N	Code	No
Budget / Runs	S=10 -segments (one QA call + one LEA each) / 3 independent	Hardware	Kaggle Intel Xeon 4-core 2.20GHz/30GB, D-Wave Advantage2 QPU (Zephyr, 4400 qubits) via Leap hybrid CQM solver, Python 3.10/Dimod 0.12/Pymoo 0.6

Abstract. Viet et al. (2026) present an **advanced Quantum Annealing (QA) approach** for the BI-TTP, addressing the scalability limitations that classical heuristics face due to the complex routing-packing interdependence and NP-hard combined structure. The core idea is to apply the **ε -constraint method** to convert the bi-objective problem into a parametric family of single-objective problems by restricting the profit objective through adjustable ε -levels computed within analytically derived upper and lower bounds. Each ε -level subproblem involves a sum of **fractional terms** (arising from the variable speed–weight interaction), which is reformulated with auxiliary binary variables into an equivalent non-fractional form, then encoded as a **Quadratic Unconstrained Binary Optimization (QUBO) model**. The QUBO model is submitted directly to the **D-Wave quantum annealer**, exploiting quantum tunnelling to escape local minima. Solutions from the quantum annealer are post-processed by a tailored classical heuristic for refinement. By selecting ε -levels broadly, the method captures diverse regions of the Pareto front, producing well-spread approximations. Experimental results on standard BI-TTP benchmarks demonstrate that the approach outperforms classical baselines in **time efficiency**, positioning hybrid quantum-classical computing as a scalable alternative for large-scale multi-objective combinatorial optimization.

Contribution	The first quantum-computing approach for TTP (single- or bi-objective): a hybrid quantum-annealing + epsilon-constraint method for the bi-objective TTP (time vs profit, no value drop) that turns the problem into single-objective subproblems (minimise time s.t. profit $\geq$ epsilon), reformulates the sum-of-fractional-quadratic-terms objective into an equivalent QUBO via an auxiliary-variable transformation (with equivalence proofs), solves each QUBO on D-Wave’s hybrid CQM solver, and refines with a ‘Later and Enough Accumulation’ heuristic that delays item pickups.
Authors’ claims	First QA for TTP; outperforms MOEA/D, NSGA-II and U-NSGA-III in time efficiency (up to 9x faster on bounded-strongly-correlated instances, 8.1x faster than NSGA-II on a280_n279_bsc), with the runtime advantage growing with instance size; the QA method’s hypervolume consistently exceeds 0.7; NSGA-II and U-NSGA-III fail to cover the lower profit end on bsc instances; epsilon-constraint beats weighted-sum by up to 35 percent hypervolume; the LEA heuristic drops travel time about 40 percent for the lowest-travel-time solution at 0.5 percent overhead.
Evidence base	Synthetic dataset from polyakovskiy2014, 3 KP weight distributions. Only 2 base graphs (ch150_n149, a280_n279), 6 instances total. Baselines MOEA/D, NSGA-II, U-NSGA-III (classical MOEAs), re-run. D-Wave hybrid CQM solver (up to 500,000 binary variables). Hypervolume (min-max normalised, ref point (1,1)). 3 independent runs with std; hypervolume-vs-iteration plots and a runtime table. No Friedman/Wilcoxon test on hypervolume. Ablations: epsilon-constraint vs weighted-sum, with/without LEA, number of segments, epsilon-initialisation.
Stated limitations	Authors state current quantum hardware limits scale and that advances in qubit count and QPU reliability are needed; value drop is explicitly omitted; U-NSGA-III ‘gradually converges toward comparable hypervolume levels’, so given enough time the classical methods catch up on quality.

Strengths. ✓ Genuine first: the first quantum-annealing approach for TTP (single or bi-objective); opens a new paradigm • ✓ Rigorous formulation work: proves the fractional-quadratic to QUBO reformulation is equivalence-preserving; the auxiliary-variable transformation is a real technical contribution • ✓ epsilon-constraint (vs weighted-sum) is the right choice for non-convex Pareto fronts, and the paper demonstrates weighted-sum’s failure (35 percent hypervolume gap) • ✓ The LEA refinement heuristic is simple, well-motivated, cheap (0.5 percent overhead) and clearly helps (about 40 percent travel-time reduction) • ✓ Ablations of epsilon-constraint vs weighted-sum, with/without LEA, number of segments and epsilon-initialisation • ✓ Peer-reviewed (IEEE TQE), 3 runs with std; honest that the runtime advantage is the main claim and that classical methods converge toward comparable hypervolume given more time

Weaknesses. ✗ Only 2 base graphs (ch150, a280), ≤ 280 cities, 6 instances; not the GECCO-2019 competition suite; nowhere near the scale the 'scalability' claims would need • ✗ The headline 'outperforms' is really 'faster'; the paper concedes U-NSGA-III converges toward comparable hypervolume, so the quality advantage is a single-run-vs-limited-iterations artefact • ✗ Runs on D-Wave's HYBRID CQM solver where most compute is classical (QPU access time is a small fraction), so it is not really 'quantum solving' the problem • ✗ No statistical test on hypervolume (Friedman/Wilcoxon); the comparison is hypervolume-vs-iteration plots plus a 3-run runtime table • ✗ Only classical MOEA baselines; no comparison with the actual BI-TTP state of the art (NDS-BRKGA, WSM, HPI) despite citing them • ✗ The QUBO has up to 500,000 binary variables for a ≤ 280 -city instance; the encoding does not scale and current QPUs cannot embed it directly (hence the hybrid solver) • ✗ Value drop omitted; not yet built upon

Table 4 — Literature Review of MTTP and DTTP (4 papers)

[1] Chand and Wagner (2016) — Fast heuristics for the multiple traveling thieves problem			
Variant	MTTP	Objective / #obj	Single-objective / 1
Route	Full tour	Timing	Unconstrained
Agents	Multiple cooperative thieves (shared capacity)	Capacity	shared-multi-agent
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	MTTP fast heuristic	Family (L1/L2)	Constructive/iterative heuristic / Greedy (MTTP)
Benchmark	—	Size	Cities: 195 to 85,900 Items: 3; 10 per city
Metric	average objective score, average rank over independent runs;	Stat. test	average-rank analysis
Baselines	initialization schemes Init1-4 (Init4 best); local search S1-S5 (S4/S5 best); complex algorithms C1-C3; restart C1 with 1 thief best single-thief TTP algorithm of faulkner2015approximate;		
Key result	2 thieves is the “sweet spot”; MTTP approaches outperform the best single-thief TTP algorithm by 50-100% regularly; C1 (restart) best on average rank, C3 best on largest instances; negative scores for 5 thieves on some instances (not-travelling becomes optimal);		
Competition	N	Code	Unknown
Budget / Runs	10min/instance / 30	Hardware	Intel Xeon E5430 2.66GHz, Java 1.8
Abstract. Chand and Wagner (2016) introduce the Multiple Traveling Thieves Problem (MTTP), a novel variant of the TTP designed to address a key realism limitation of the original formulation: the use of a single thief visiting all cities. In MTTP, multiple thieves are deployed simultaneously, each constructing an individual tour and packing plan, with the collective goal of maximizing total group profit. Unlike the standard TTP, not all cities need to be visited and multiple thieves may visit the same city, making the problem structurally different from vehicle routing problems (VRP) where all demand locations must be served. The interdependence between different thieves’ tours and packing plans means that individually optimal subproblem solutions per thief do not necessarily yield the best collective result, preserving the TTP’s characteristic multi-component interaction at a group level. The authors propose several fast construction heuristics including simple greedy strategies and a dedicated Cross Thief Mutation (CTM) operator that transfers items and city assignments across thieves to exploit inter-thief complementarities. The formulation is designed to be backward-compatible with standard single-thief TTP instances, enabling direct comparison of algorithms. Experiments on benchmark instances up to 85,900 cities demonstrate the heuristics’ scalability and establish the MTTP as a practically motivated and extensible TTP variant.			

Contribution	Introduces the Multiple Traveling Thieves Problem (MTTP), a relaxed generalisation of TTP with t thieves who all start/end at city 1, SHARE a single knapsack capacity, collectively maximise profit, and need not all visit every city; reduces to TTP at $t=1$ and reuses the existing 9720-instance suite. Proposes fast heuristics: 4 initialisations, two new operators (DropCity, CrossThiefMutation), and simple (S1-S5) and complex (C1-C3) heuristic combinations.
Authors' claims	Init4 (chunked CLK tour + PackIterative per thief) is best and about 2 thieves is a 'sweet spot' on average (more thieves raise total travel cost because of the shared start/end); the problem-specific operators (DropCity, CrossThiefMutation) give massive improvements over the faulkner2015 operators; the restart-only C1 has the best average rank while C3 is best for large instances; C1 with one thief equals the best faulkner2015 TTP algorithm and the MTTP approaches 'always outperform' it, regularly by 50-100 percent.
Evidence base	The 72-instance faulkner subset, reused unmodified (6 city sizes 195-85,900, 2 item factors, 3 KP types, 2 capacities). 10-min budget, 30 repetitions. Results reported almost entirely as AVERAGE RANKS (figures) plus a decision tree and one colour-coded objective-score figure for 2 instances. No statistical test. Public code and results. Negative objectives are common.
Stated limitations	Authors note C3 is sometimes beaten by C1 so the TSP part of MTTP 'remains of very high importance', that the 5-thief setup sometimes cannot transition back to 4 thieves (negative scores), and conjecture that a thief could be dropped entirely for efficiency but the operators cannot do this.

Strengths. ✓ Introduces MTTP with a clean, well-motivated relaxation (shared capacity, selective visiting, collective profit) and a real-world motivation (transport stolen goods by multiple vehicles) • ✓ Designed so existing TTP instances and the $t=1$ reduction work • ✓ Systematic 3-study design (initialisations, simple operators, complex algorithms) with 30 repetitions, 10-min budget, decision-tree analysis • ✓ Two genuinely new operators (DropCity, CrossThiefMutation); CrossThiefMutation doubles as a crossover for single-thief TTP • ✓ Public code and results; honest that about 2 thieves is the sweet spot and restart still beats the complex operators

Weaknesses. ✗ Results reported almost entirely as average ranks; no objective-value tables except one colour-coded figure for 2 instances; hard to reuse or verify • ✗ No statistical significance testing • ✗ The '50-100 percent better than the best TTP algorithm' claim compares a multi-thief solution (more agents, effectively more usable capacity) against a single-thief one, so it is not a like-for-like algorithm comparison • ✗ Only the 72-instance subset; MTTP is only ever compared internally (Init and S/C variants) • ✗ No dedicated MTTP benchmark instances; the number of thieves t is a free parameter with no principled choice • ✗ Negative objectives are pervasive, indicating many configurations produce unprofitable solutions • ✗ MTTP has had essentially no follow-up: this remains the only MTTP paper years on

[2] [Herring et al. \(2020a\)](#) — *Dynamic multi-objective optimization of the travelling thief problem*

Variant	TTP2, DTTP	Objective / #obj	Bi-objective (time vs profit) / 2
Route	Full tour	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Time-decaying profits (drop rate)	Speed / Cost	linear / value-drop-rate
Algorithm	DMOTTP EA	Family (L1/L2)	Population-based metaheuristic / EA (dynamic multi-objective)

Benchmark	—	Size	Cities: 52; 280; 783; 2319 Items: 1; 5; 10 per city
Metric	hypervolume (HV) and spread, ranked at end of each dynamic interval (5 intervals), median over repeats;	Stat. test	median end-of-interval rank aggregation
Baselines	9 static initialization strategies (ss/sr/sg/rs/rr/rg/gg, where s=solver, g=greedy, r=random for TSPxKP); 8 dynamic response methods (pS, pR, pG, mS, mR, mG, mC, mN); 3 dynamics Loc/Ava/Val;		
Key result	solver-based pS best HV on a280A/B/C and u2319A; pG/mG best balance (high HV + high spread, capturing both knee and high-profit regions); random pR/mR worst HV but high spread; composite mC mid-range and scales best to large TSP components;		
Competition	N	Code	Unknown
Budget / Runs	1000 generations, population 60 / 30 repeats x 10 dynamic-change patterns per instance	Hardware	not reported

Abstract. [Herring et al. \(2020a\)](#) introduce the **Dynamic Multi-Objective Travelling Thief Problem (DMOTTP)**, extending the static bi-objective TTP with three types of environmental dynamics: changes to **city locations**, changes to **item availability**, and changes to **item values**. Each dynamic type is formalized independently, with different implications for the Pareto optimal front after each change event. The study evaluates the role of population **initialization strategies** for re-seeding the EA after changes: solver-based initialization (using TSP and KP near-optima), greedy initialization, random initialization, and a composite strategy combining multiple approaches. Experiments are conducted on benchmark instances from 52 to 2319 cities with 1 to 10 items per city. The key finding is that solver-based and greedy initialization mechanisms recover faster from changes (higher hypervolume at early generations post-change) compared to random restarts, while the composite strategy achieves the best overall performance. This work establishes a new and practically motivated research direction for dynamic multi-objective combinatorial optimization.

Contribution	Defines the dynamic bi-objective TTP (min tour time, max time-decayed profit; unlike sachdeva2020, it keeps bonyadi2013's value decay) with three realistically motivated dynamic types (city locations, item-city availability map, item values), and a systematic study of 8 population re-seeding / response mechanisms (solver, greedy, random, passive, and mixed combinations) inside an NSGA-II-style EA, asking which strategy responds best to which dynamic type. (arXiv extended version; cross-listed as TTP2 and DTTP.)
Authors' claims	Randomisation-based re-seeding is not competitive with any other method; solver-based methods achieve the highest hypervolume ranks under location dynamics but poor spread (they converge to the minimum-tour region); greedy methods do well on small-TSP instances; the passive method is competitive and increasingly so as the TSP component grows; a combined method gives the best all-round ranks; no single method is best for every dynamic type/instance, and the nature (not just size) of the TSP problem controls whether solver or greedy methods win.
Evidence base	12 base problems (berlin52, a280, rat783, u2319 x A/B/C KP types; max 2319 cities). Location dynamics move only $d_N=2$ cities per change. 30 repeats x 10 dynamic instances. Fixed generation budget (1000 generations, change every 200). Metrics: end-of-interval hypervolume and Zitzler's spread, aggregated as median RANKS (polar plots). No statistical test; no absolute hypervolume values; no per-instance tables (true Pareto front unknown).

Stated limitations Authors state the change-magnitude parameters should be investigated further, that dynamics are limited to one type at a time (combined is future work), that change frequency/synchronicity is future work, and suggest island-model / cooperative co-evolution to isolate solution-generation methods.

Strengths. ✓ Defines the dynamic BI-TTP with three distinct, realistically motivated dynamic types, and keeps the value decay of bonyadi2013's TTP2 (unlike sachdeva2020) • ✓ Systematic 8-response-mechanism x 3-dynamic-type x 12-instance study, 30 repeats • ✓ Nuanced, well-argued findings (randomisation is unhelpful; solver-vs-greedy depends on TSP structure not just size) • ✓ Connects DTTP to the dynamic-multi-objective-optimisation literature • ✓ Thorough related-work coverage (extended arXiv version)

Weaknesses. ✗ The entire analysis is median-RANK polar plots plus prose; no statistical test, no absolute hypervolume/spread values, no per-instance tables • ✗ Ranks are pool-relative over the 8 response mechanisms and say nothing about absolute solution quality • ✗ No new DTTP algorithm; a study of population re-seeding strategies using NSGA-II with existing initialisers • ✗ Only 12 base problems (max 2319 cities); the location dynamic moves only 2 cities regardless of instance size • ✗ Only one dynamic type at a time; the realistic combined case is entirely future work • ✗ Generation-based budget, not wall-clock; conclusions hedged throughout ('in some instances', 'mixed results') • ✗ arXiv preprint; overlaps heavily with herring2020responsive (same authors, problem and period)

[3] **Herring et al. (2020b)** — *Responsive Multi-population Models for the Dynamic Travelling Thief Problem*

Variant	DTTP	Objective / #obj	Single- and bi-objective / 1 or 2
Route	Full tour	Timing	Epoch-based disruption
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Dynamic (item/city toggling)
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	DTTP responsive EA	Family (L1/L2)	Population-based metaheuristic / EA (dynamic)
Benchmark	—	Size	Cities: 52; 100; 101; 144; 280; 318 Items: 1; 5; 10 per city
Metric	2D hypervolume (nadir reference point), % of generations with significantly higher mean HV;	Stat. test	unpaired one-tailed t-tests, Bonferroni-corrected, =0.05
Baselines	3 population models - Si (single-population), Mo (multi-pop, objective-focusing, isolated), Mi (multi-pop with migration); each resolving vs non-resolving (-nr) the TSP optimum on re-seed;		
Key result	ordering Si < Mo < Mi (migration best) - migration significantly better on 63% of generations (A-type), 95% (B and C); resolving the problem components before re-seeding beats non-resolving on most problems (e.g. berlin52A 61.4% of generations);		
Competition	N	Code	No

Budget / Runs	global population 360, crossover 0.9, mutation 2/N_items & 2 (bit-flip/swap), 1000 generations, =10%, d_N=2 / 20 repeats x 10 change patterns	Hardware	not reported
----------------------	---	-----------------	--------------

Abstract. [Herring et al. \(2020b\)](#) investigate the benefit of **multi-population evolutionary architectures** for the Dynamic TTP, addressing the challenge of simultaneously maintaining solution diversity and exploiting objective-specific structure under time-varying conditions. Three population models are compared: (1) a **single-population EA** (baseline), (2) a **multi-population model with objective focusing** where isolated sub-populations each evolve toward distinct objective directions without information exchange, and (3) a **multi-population model with migration** where solutions are periodically shared between sub-populations with different objective focuses. A key contribution is the design of **responsive re-seeding**: when a dynamic event occurs, each sub-population is re-seeded with problem information relevant to its objective focus and to the current (post-change) problem state, ensuring that exploited information remains current. Migration between differently-directed sub-populations counterbalances the risk of premature convergence in isolated sub-populations by introducing cross-objective diversity. A **per-interval performance analysis method** is developed to capture improvements throughout the varying dynamic epochs, enabling fine-grained comparison beyond end-of-run metrics. Results demonstrate significant hypervolume improvements for the migration model compared to both the single-population baseline and the isolated multi-population model, confirming that information sharing between sub-populations is a decisive factor. The study concludes that even simple multi-population topologies with straightforward migration patterns can substantially improve performance on dynamic multi-objective combinatorial optimization, motivating further exploration of island model and co-evolutionary strategies for DTTP.

Contribution	A multi-population / island-model approach for the dynamic bi-objective TTP under location dynamics: three topologies (single population; multi-population with objective biasing; the latter plus migration and a mixing population), with responsive seeding that replaces offspring generation with solver-based re-initialisation (Concorde / dynamic programming, or a fast Inver-Over repair) after a change; asks whether updating the exploited component optima matters and which topology responds best. (The peer-reviewed IEEE SSCI companion of herring2020dynamic .)
Authors' claims	Re-solving the component optima after a change gives significantly higher hypervolume than keeping the initial optima (clear for the multi-population topologies); multi-population beats single population, with the ordering single < objective-biased < objective-biased-plus-migration in most problems; migration plus a mixing population helps even with equated function evaluations; for berlin52B/C, disconnected populations beat migration (attributed to sub-optimal migration parameters).
Evidence base	18 base problems (berlin52, kroA100, eil101, pr144, a280, lin318 x A/B/C KP; max 318 cities). Location dynamics move d_N=2 cities per change from a fixed schedule; change every 100 generations. 20 repeats x 10 dynamic patterns. Global population 360 (evaluation ratio 1:1.6:1.45 across topologies). Metric: 2D hypervolume with a nadir point. Statistics: pairwise one-tailed t-tests (alpha=0.05, Bonferroni), reported as the percentage of generations with a significantly higher mean hypervolume. Peer-reviewed (IEEE SSCI 2020).
Stated limitations	Authors note sub-optimal migration frequency/schedule may explain the berlin52B/C anomaly and that tuning the migration mechanisms is reserved for future work.

Strengths. ✓ Peer-reviewed (IEEE SSCI); a focused study of island models for the dynamic BI-TTP • ✓ Novel angle: island models with objective-biased sub-populations plus migration and a mixing population, applied to a dynamic combinatorial multi-objective problem • ✓ Rigorous generation-level statistics: pairwise one-tailed t-tests with Bonferroni correction, reported as the fraction of generations with significant improvement, to avoid the information loss of averaging over dynamic intervals • ✓ Equates function evaluations across topologies for fairness; reproducible (fixed dynamic schedules and parameters) • ✓ Answers clear, useful questions: re-solving the component optima after a change matters, and multi-population helps • ✓ Honest about the berlin52B/C anomaly and its likely cause

Weaknesses. ✗ Only 18 base problems (max 318 cities); location dynamics only; $d_N=2$ cities move regardless of instance size, a tiny perturbation • ✗ Metrics are pool-relative: the fraction of generations where one topology significantly beats another, not absolute quality or a true Pareto front • ✗ No new DTTP solver in the usual sense; a study of population topologies and re-seeding for an NSGA-II-style EA using external solvers • ✗ The hypervolume values themselves are never reported, so the magnitude of the improvement is unknown • ✗ Only location dynamics; availability and value dynamics are excluded 'for brevity' • ✗ Migration parameters chosen heuristically; the berlin52B/C anomaly shows they are not well-tuned • ✗ 8-page paper; overlaps heavily with herring2020dynamic; DTTP has had no follow-up beyond this trio

[4] Sachdeva et al. (2020) — *The Dynamic Travelling Thief Problem: Benchmarks and Performance of Evolutionary Algorithms*

Variant	DTTP	Objective / #obj	Single- and bi-objective / 1 or 2
Route	Full tour	Timing	Epoch-based disruption
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Dynamic (item/city toggling)
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	DTTP EA	Family (L1/L2)	Population-based metaheuristic / EA (dynamic)
Benchmark	—	Size	Cities: 280; 4461; 33,810 Items: 1; 5; 10 per city
Metric	normalized objective score over 10 epochs (AUC = area under curve, and END = final value);	Stat. test	650 pairwise significance tests
Baselines	7 algorithms - item-toggling: Bitflip, REA_m, PackIterative, PackIterative+Bitflip; city-toggling: Insertion, CLK, CLK+Insertion;		
Key result	toggling items restart-from-scratch preferred (PackIterative+Bitflip best; Bitflip recovers only at $d=1\%$, fails at $d=3\%$); toggling cities recovery preferred (Insertion best, since restart resets the TSP tour); disruptive events significantly reshape algorithm rankings;		
Competition	N	Code	Unknown
Budget / Runs	10 epochs/scenario / multiple (650 pairwise statistical tests)	Hardware	not reported

Abstract. Sachdeva et al. (2020) define and benchmark the **Dynamic Travelling Thief Problem (DynTTP)**, the first systematic study of dynamic disruptions in the standard single-objective TTP. Two independent disruption mechanisms are introduced: **toggling item availability** (a fraction $d\%$ of items switch between available and unavailable) and **toggling city availability** (a fraction $d\%$ of cities are inserted or removed from the tour). These disruptions are studied in isolation to ensure clean analysis of their respective effects. A controlled benchmark framework is established with 72 experimental scenarios ($9 \text{ instances} \times 4 \text{ disruption magnitudes} \times 2 \text{ disruption types}$) by varying instance size, disruption magnitude $d \in \{1\%, 3\%, 10\%, 30\%\}$, and epoch length. Seven algorithms—spanning diverse strategies including restart-from-scratch, recovery-from-previous-solution, and hybrid approaches—are evaluated across all scenarios. The central finding is nuanced: *restart* strategies outperform *recovery* strategies for large disruptions, while *recovery* is preferable for small disruptions; the transition depends on instance size and algorithm. This confirms that no single strategy is universally superior and that dynamic algorithm selection is necessary in practice. The work provides a reproducible benchmark framework and the first empirical evidence on evolutionary algorithm behaviour under dynamic changes to multi-component combinatorial optimization problems.

Contribution	Introduces the Dynamic Traveling Thief Problem (DTTP), single-objective TTP with run-time changes: d percent of items OR d percent of cities toggle availability each epoch, studied independently (tour fixed while toggling items; packing not re-optimised while toggling cities), with design rules that keep solutions valid; a portfolio study of 7 algorithm combinations asking whether to RESTART from scratch or RECOVER the previous solution after a disruption.
Authors' claims	For toggling items, starting from scratch is preferred (PackIterative+BitFlip best), except that at $d=1$ percent BitFlip can beat it on convergence speed; for toggling cities, recovering the tour (Insertion) is preferred over restarting; disruptive events have a major impact on performance and significantly change the relative rankings of algorithms.
Evidence base	9 instances (the GECCO-2019 subset: a280/fnl4461/pla33810 $\times$ 3 KP configs). 3 disruption types $\times$ 4 magnitudes (d in 1,3,10,30 percent) $\times$ period $z=m$. 10 epochs, 30 runs. Metrics END (final quality per epoch) and AUC (convergence speed + quality), min-max normalised across the 7 algorithms. Statistics: about 650 pairwise one-sided Mann-Whitney U tests. Dedicated RNG so all algorithms see the same disruptions. Full version = arXiv 2004.12045. Public cross-validated C++ code.
Stated limitations	Authors note this CCIS version is condensed (the full arXiv has the variants review, heuristic details, and a single-event study), and that the two dynamic modes are studied only independently.

Strengths. ✓ Introduces the DTTP with careful, well-specified design rules for maintaining solution validity under item/city toggling • ✓ Reuses the standard GECCO-2019 subset with a dedicated RNG so disruption sequences are comparable across algorithms • ✓ Systematic: 3 disruption types $\times$ 4 magnitudes $\times$ 9 instances $\times$ 10 epochs $\times$ 30 runs; two complementary metrics • ✓ Proper statistics: about 650 pairwise Mann-Whitney U tests • ✓ Clear, actionable findings: restart for item toggling, recover for city toggling, and disruption magnitude flips the ranking • ✓ Public cross-validated C++ code and results

Weaknesses. ✗ Only 9 instances (the GECCO-2019 subset); a280/fnl4461/pla33810 only • ✗ END/AUC are min-max normalised across the 7 algorithms, so they are pool-relative, not absolute quality • ✗ No new DTTP algorithm; a benchmark + restart-vs-recover portfolio study using existing operators • ✗ The two dynamic modes are studied only independently; the realistic combined case is left open • ✗ Results are largely conveyed via heatmaps; the ~650 tests are summarised in prose, not tabulated per instance • ✗ The condensed CCIS version omits the heuristic details and the single-event study, so it is not fully self-contained • ✗ $z=m$ evaluations per epoch is an arbitrary period; sensitivity to z is not studied

Table 5 — Literature Review: Thief Orienteering Problem (ThOP) (8 papers)

[1] Santos and Chagas (2018) — <i>The Thief Orienteering Problem: Formulation and Heuristic Approaches</i>			
Variant	ThOP	Objective / #obj	Single-objective / 1
Route	Selective routing (subset of cities)	Timing	Global time budget
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear/constant / time-budget
Algorithm	ILS / BRKGA (ThOP)	Family (L1/L2)	Population-based metaheuristic / BRKGA + ILS
Benchmark	—	Size	Cities: 51; 107; 280; 1000 Items: 1; 3; 5; 10 per city
Metric	convergence (% of best-known profit, avg / best over runs);	Stat. test	none
Baselines	ILS vs BRKGA;		
Key result	BRKGA stable 90% convergence across all groups; ILS degrades as items/city increase (97% <60%); BRKGA better on large/dense instances, ILS better only on smallest a280/pr107 with 1 item (e.g. a280-01-bsc ILS 95.0 vs BRKGA 92.4; a280-10-usw BRKGA 91.3 vs ILS 54.7);		
Competition	N	Code	Unknown
Budget / Runs	m/10 seconds/run; BRKGA P=100, P_e=25, P_m=30, =0.6, L=50 / 10	Hardware	Intel Xeon E5-2660 v2 2.20GHzx40, 384GB RAM, CentOS 6.8, C/C++

Abstract. Santos and Chagas (2018) introduce the **Thief Orienteering Problem (ThOP)**, a multi-component combinatorial optimization problem combining the **Orienteering Problem (OP)** and the **0–1 Knapsack Problem (KP)**. In ThOP, a thief carries a capacitated knapsack and must travel from a fixed start city to a fixed end city within a time limit T , visiting a *subset* of cities (unlike TTP, where all cities must be visited) and collecting items to maximize total profit. The thief’s travel speed is inversely proportional to knapsack weight: $v = v_{max} - w \cdot (v_{max} - v_{min}) / W$, coupling route choice and item selection. A **Mixed Integer Non-Linear Programming (MINLP)** formulation is provided. Two heuristics are proposed: an **Iterated Local Search (ILS)** with 2-opt and node-swap neighborhood operators acting on the route, and a **Biased Random-Key Genetic Algorithm (BRKGA)** encoding solutions as random-key vectors with a biased crossover that biases offspring toward elite parents. A benchmark of 432 instances is derived from TSPLIB, varying city count (51 to 1000) and items per city (1, 3, 5, 10). BRKGA outperforms ILS on the majority of large instances, highlighting the value of population-based diversification for the selective routing structure of ThOP.

Contribution	Introduces the Thief Orienteering Problem (ThOP): Orienteering Problem (selective routing, fixed start/end, hard time limit T) + Knapsack (capacity W, load-dependent speed); not all points visited. Provides a mixed-integer nonlinear programming formulation, an instance generator built from TTP instances plus a 432-instance benchmark, and two heuristics (iterated local search, biased random-key GA) with a solution encoding as a sequence of items.
Authors' claims	BRKGA outperforms ILS on a large majority of instances, especially larger ones (BRKGA convergence about 91-93 percent on dsj1000 vs ILS 59-79 percent); ILS is competitive only on small instances and single-item cases; BRKGA finds routes with fewer edge crossings.
Evidence base	432 instances: 4 TSP bases (eil51, pr107, a280, dsj1000) x 4 item factors x 3 KP types x 3 capacity classes x 3 time classes (T = 50/75/100 percent of the authors' GVNS TTP-competition route time). Stopping = $\text{ceil}(m/10)$ seconds. 10 runs, avg + best. Metric $\text{chi} = \text{alg_avg} / \max(\text{ILS_best}, \text{BRKGA_best}) \times 100$ percent, a convergence-to-best-known measure. Public benchmark and generator. No statistical test.
Stated limitations	First heuristics for a new problem; suggests further investigation.

Strengths. ✓ Introduces a genuinely distinct multi-component variant (selective routing + hard time budget) with a clean MINLP formulation, worked example and real-world motivation (reverse logistics with load-dependent speed) • ✓ Delivers a public 432-instance benchmark AND an instance generator derived from TTP instances, which the whole subsequent ThOP literature adopts • ✓ Two heuristics from different families, 10 runs, a systematic instance grid • ✓ Time-budget-adaptive T (50/75/100 percent of a reference route) is a sensible difficulty control • ✓ Public results and supplementary material

Weaknesses. ✗ The convergence metric chi is self-referential (best-known = best of the two proposed heuristics), so it measures convergence to the better of ILS/BRKGA, not to a true optimum • ✗ Only 2 heuristics, both authored here; no external or adapted-TTP/OP baseline • ✗ No statistical significance testing • ✗ The time limit T is set from the authors' own unpublished GVNS TTP solution, so instance difficulty depends on that reference • ✗ BRKGA parameters tuned 'empirically following recommended settings', not systematically; the $\text{ceil}(m/10)$ -second stopping criterion is non-standard • ✗ 4 TSP base graphs only (max 1000 cities) • ✗ ILS is quite weak on larger/multi-item instances, so 'BRKGA outperforms ILS' is partly about ILS being under-developed

[2] Chagas and Wagner (2020) — *Ants can orienteer a thief in their robbery*

Variant	ThOP	Objective / #obj	Single-objective / 1
Route	Selective routing (subset of cities)	Timing	Global time budget
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear/constant / time-budget
Algorithm	ACO (ThOP)	Family (L1/L2)	Population-based metaheuristic / Swarm (ACO)
Benchmark	—	Size	Cities: 51; 107; 280; 1000 Items: 1; 3; 5; 10 per city

Metric	approximation ratio (% of best-known upper bound), avg over runs;	Stat. test	Wilcoxon signed-rank, 5%
Baselines	Santos & Chagas (2018) ILS/BRKGA; two variants ACOTThOP* (Irace-tuned) and ACOTThOP (general config: ants=196, =1.24, =5.46, =0.51, ptries=1, no local search);		
Key result	ACOTThOP* better on 409/432 (95%) (worse 2, tie 21), ACOTThOP 401/432 (93%); new best solutions on 410/402 instances; average 320%/313% improvement over prior best (e.g. pr107_10_usw_10_03 profit 474,464 vs 133,925);		
Competition	N	Code	Yes
Budget / Runs	m/10 s/run, Irace maxExperiments=5000 / 10	Hardware	Intel Xeon E5-2660 2.20GHz, CentOS 7, ACOTSP 1.0.3 (C)

Abstract. Chagas and Wagner (2020) present the first **Ant Colony Optimization (ACO)**-based approach for ThOP, structured as a **two-phase algorithm**. In phase one, ACO constructs a time-feasible route by guiding ants with pheromone trails accumulated on city-to-city transitions and a heuristic component based on profitability and distance. In phase two, a novel **randomized packing heuristic** generates diverse item packing plans for each constructed route, iterating over items in decreasing profit-to-weight ratio and selecting them probabilistically to avoid deterministic greedy traps. **Automated algorithm configuration** using irace is applied to optimize the ACO hyperparameters, identifying configurations separately tuned for instance groups. The resulting ACO algorithm outperforms ILS, BRKGA, and GA on over **90% of the 432 benchmark instances**, with an average improvement exceeding **300%**, establishing ACO as the dominant approach for ThOP and demonstrating the effectiveness of decoupling route construction from packing.

Contribution	A two-phase swarm-intelligence approach for the ThOP: ant colony optimization (ACOTSP framework) for route construction plus a new randomized packing heuristic (ptries attempts); automated configuration (irace) on 48 instance groups for insight; two versions (per-group tuned ACOTThOP* and a single general-config ACOTThOP).
Authors' claims	ACOTThOP*/ACOTThOP outperform santos2018 on 419/410 of 432 instances (average quality) and by about 320/313 percent on average; Wilcoxon signed-rank (10 runs, 5 percent) confirms ACOTThOP* significantly better on 409 (95 percent), worse on 2; the improvement is largest on larger/multi-item instances; tuned configs mostly prefer NO local search and ptries=1.
Evidence base	432-instance benchmark, ceil(m/10)-second budget, 10 runs. irace per-group (maxExperiments 5000). Metric: approximation ratio = avg / self-referential best-known ('upper bound' = best found across methods). Wilcoxon signed-rank test on all 432 instances. Public code and full irace logs. Peer-reviewed (EvoCOP 2020, published in Operations Research Letters).
Stated limitations	Best-known used as an upper bound is a proxy for the true optimum.

Strengths. ✓ Rigorous methodology: irace per-group tuning (48 groups) with insight plots, a general config derived and tested separately, 10 runs, Wilcoxon signed-rank test on all 432 instances • ✓ Large, well-supported improvement over santos2018 (95 percent significantly better, about 320 percent average) - becomes the ThOP baseline • ✓ The randomized packing heuristic (ptries) is a clean, reusable idea; the finding that tuned configs prefer no local search and ptries=1 is a genuine insight • ✓ Public code and full irace logs (reproducible); peer-reviewed

Weaknesses. ✗ The baseline (santos2018 ILS/BRKGA) is weak, especially on large/multi-item instances; a 320 percent improvement over a method that was itself far from good, with no external/exact reference • ✗ Approximation ratio uses a self-referential best-known, so it is pool-relative, not a true optimality gap • ✗ Only the 432-instance benchmark, 4 TSP bases (max 1000 cities) • ✗ ACOThOP* needs per-group irace tuning (48 groups); the practical single-config version is slightly worse • ✗ The '300+ percent average improvement' is driven by large-instance groups where santos2018 nearly fails; on eil51 the improvement is modest • ✗ No comparison with an adapted TTP method or a MIP; the ThOP MINLP is never solved even for tiny instances

[3] Faêda and Santos (2020) — A Genetic Algorithm for the Thief Orienteering Problem

Variant	ThOP	Objective / #obj	Single-objective / 1
Route	Selective routing (subset of cities)	Timing	Global time budget
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear/constant / time-budget
Algorithm	thopGA	Family (L1/L2)	Population-based metaheuristic / GA
Benchmark	—	Size	Cities: 51; 107; 280; 1000 Items: 1; 3; 5; 10 per city
Metric	convergence (% of best-known); count of best/average wins;	Stat. test	Friedman (p<0.05) + Wilcoxon (thopGA vs BRKGA p<0.05), Bonferroni
Baselines	ILS, BRKGA santos2018thief;		
Key result	thopGA best on 304/432 instances (>70%); superior in all instance groups (e.g. best-solution wins: eil51 thopGA 64 vs BRKGA 26 vs ILS 12; dsj1000 thopGA 92 vs BRKGA 15 vs ILS 1); thopGA stable 85% convergence while BRKGA drops 93% 44% as items increase;		
Competition	N	Code	Unknown
Budget / Runs	m/10 seconds/run; it_max=50, p_k=100, p_c=0.90, p_m=0.85 / 10	Hardware	not reported (notes BRKGA ran on a faster machine)

Abstract. Faêda and Santos (2020) propose a **Genetic Algorithm (GA)** specifically designed for ThOP, representing solutions as ordered sequences of items (from which the thief's route is uniquely determined by item locations). The GA applies **tournament selection** to favor high-profit individuals, a **Partially Mapped Crossover (PMX)** variant adapted for item sequences to preserve relative item ordering, and **swap and insert mutation** to explore neighborhood structures. A **greedy feasibility repair** mechanism is applied after recombination to remove items that cause time or capacity constraint violations, prioritizing removal of items with the worst profit-to-weight ratio. The algorithm is tested on all 432 benchmark instances, and the GA outperforms both ILS and BRKGA in the majority of cases. The study demonstrates that encoding item selection order—rather than city tour sequence—is a natural and effective representation for ThOP, where the route emerges implicitly from item choices.

Contribution	A genetic algorithm (thopGA) for the ThOP: a standard GA with local search, high mutation rate, and an ordered-item-list encoding; compared against the ILS and BRKGA of santos2018.
Authors' claims	thopGA is superior in the majority of cases with stable convergence (about 85 percent) across most groups while BRKGA degrades from 93 percent to about 44 percent as the number of checkpoints/items grows; thopGA finds the best solution on 304 of 432 instances (over 70 percent); BRKGA is better only for single-item and lowest-capacity groups; the Friedman test confirms significant differences.
Evidence base	The same 432-instance benchmark, $\text{ceil}(m/10)$ -second budget, 10 runs, the same self-referential chi metric (best-known = best of the 3 methods). Baselines ILS and BRKGA are QUOTED from santos2018 and were run on DIFFERENT hardware (BRKGA on a faster machine), a confound the authors themselves acknowledge and use to explain thopGA's losses on tight-budget instances. Friedman test applied (noting the samples are not independent).
Stated limitations	Authors acknowledge the hardware difference, note the algorithms still depend on a good weight-value relationship, and that tighter time/capacity limits are harder.

Strengths. ✓ Uses the established 432-instance benchmark, the same chi metric and the same time budget as santos2018 • ✓ Friedman test applied, correctly noting the samples are not independent • ✓ Systematic breakdown of chi by all 4 instance factors; reports best-result counts per method with a ties column • ✓ thopGA is genuinely more stable than BRKGA as instance size grows • ✓ Honest about the hardware confound

Weaknesses. ✗ Baselines ILS and BRKGA are quoted from santos2018 and run on different (and, for BRKGA, faster) hardware than thopGA; with the $\text{ceil}(m/10)$ -second budget this is a serious confound acknowledged by the authors • ✗ The chi metric is self-referential (best-known = best of the 3 methods) • ✗ Only 3 methods, all from the same two-lab ThOP line; no external or exact baseline • ✗ thopGA is a textbook GA; the novelty is minimal • ✗ No absolute-quality context (no optimal or MIP reference) • ✗ Same 432-instance benchmark, 4 TSP bases (max 1000 cities); 8-page conference paper

[4] Chagas and Wagner (2022a) — *Efficiently solving the thief orienteering problem with a max–min ant colony optimization approach*

Variant	ThOP	Objective / #obj	Single-objective / 1
Route	Selective routing (subset of cities)	Timing	Global time budget
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear/constant / time-budget
Algorithm	ACO++	Family (L1/L2)	Population-based metaheuristic / Swarm (ACO++)
Benchmark	—	Size	Cities: 51; 107; 280; 1000 Items: 1; 3; 5; 10 per city
Metric	approximation ratio; pairwise better-or-equal %; OP ratio to exact optimum (branch-and-cut);	Stat. test	Wilcoxon signed-rank, 5%

Baselines	ILS, BRKGA, GA faeda2020genetic, ACO chagas2020ants;		
Key result	ACO++ better-or-equal vs ILS 99.54%, vs BRKGA 96.06%, vs GA 99.54%, vs ACO 98.15%; statistically better than ACO on 409/432 (95%) (worse 11, tie 12); on OP, ACO++ avg ratio-to-optimal 0.980 (vs ACO 0.949, BRKGA 0.863, ILS 0.705); finds OP optimum on all dsj1000 (1.000);		
Competition	N	Code	Yes
Budget / Runs	Irace-tuned (ants 50-200, 2-opt local search) / 30	Hardware	not reported (MMAS over ACOTSP, C)

Abstract. Chagas and Wagner (2022a) refine the ACO framework of Chagas and Wagner (2020) within the **MAX-MIN Ant System (MMAS)** paradigm, producing an enhanced algorithm called **ACO++**. Key improvements include: (1) a **new pheromone initialization scheme** using instance-specific information to tighten initial pheromone bounds and accelerate convergence; (2) a **refined candidate list mechanism** that restricts ant moves to a small set of high-quality neighboring cities, substantially reducing per-iteration computational cost; (3) configurable **2-opt local search** applied to routes found by ants. A thorough re-tuning of all competing algorithms (ILS, BRKGA, GA, ACO) via automated configuration ensures fair comparison. ACO++ achieves statistically significant improvements over all prior algorithms on almost all 432 instances, setting a new state-of-the-art for ThOP heuristics and providing a rigorous benchmarking protocol for future work.

Contribution	An improved ACO for the ThOP (ACO++): MAX-MIN Ant System plus the randomized packing heuristic plus local searches, with extensive per-group irace tuning; also re-tunes the competitor algorithms (ILS, BRKGA, GA) with the same irace procedure, and evaluates all methods on the classical Orienteering Problem sub-problem against branch-and-cut optima.		
Authors' claims	ACO++ outperforms ILS, BRKGA, GA and the base ACO on at least 96 percent of the 432 instances; Wilcoxon signed-rank shows ACO++ significantly better than the base ACO on 409 of 432; ACO++ finds more efficient (shorter, fewer-crossing) routes, hence better knapsack usage; on the classical OP, ACO++ averages 0.980 of the branch-and-cut optimum (vs ACO 0.949, BRKGA 0.863, ILS 0.705).		
Evidence base	432-instance benchmark plus 48 OP instances (single-item, unbounded capacity, constant speed). $\text{ceil}(m/10)$ -second budget, irace per-group, 10 runs (ThOP) / 30 runs (OP), Wilcoxon signed-rank test. For the OP sub-problem, comparison is against TRUE OPTIMA from Fischetti et al.'s branch-and-cut. Baselines ILS/BRKGA/GA RE-TUNED with irace. Public code and irace logs.		
Stated limitations	Future work is a multiple-thieves ThOP variant and a bi-objective version (max profit, min distance).		

Strengths. ✓ Journal-length, rigorous: re-tunes ALL competitor algorithms with the same irace procedure, correcting the hardware/tuning confounds of faeda2020 • ✓ MAX-MIN Ant System is a well-motivated upgrade with a clean ACO-vs-ACO++ ablation (Wilcoxon, 409/432 significant) • ✓ Anchors quality to TRUE OPTIMA for the OP sub-problem (ACO++ averages 0.980 of optimum), a real optimality-gap result rare in the ThOP literature • ✓ Structural analysis showing ACO++'s advantage comes from more efficient routes, with route visualisations • ✓ 10 + 30 runs, Wilcoxon signed-rank test throughout, public code and irace logs • ✓ Establishes ACO++ as the ThOP state of the art (until SAAS-HC)

Weaknesses. ✗ For the full ThOP the quality anchor is still the self-referential best-known; only the OP special case is compared to true optima • ✗ Same 432-instance benchmark, 4 TSP bases (max 1000 cities) • ✗ ACO++ requires per-group irace tuning (48 groups); the practical single-config performance is not the headline • ✗ The improvement over the base ACO, while significant on 95 percent of instances, is modest in magnitude on many groups • ✗ The OP-optimum comparison is easiest on the instances where the OP is trivial (dsj1000); the harder OP instances still show a gap • ✗ Same two-author line that introduced the problem and the prior ACO; the ThOP MINLP is still never solved even for tiny instances

[5] Bloch-Hansen et al. (2023) — A Polynomial-Time Approximation Scheme for Thief Orienteering on Directed Acyclic Graphs

Variant	ThOP	Objective / #obj	Single-objective / 1
Route	Selective routing (subset of cities)	Timing	Global time budget
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear/constant / time-budget
Algorithm	PTAS (DAG ThOP)	Family (L1/L2)	Approximation scheme / PTAS
Benchmark	—	Size	—
Metric	—	Stat. test	—
Key result	(i) ThOP on general (undirected) graphs has no constant-factor approximation unless $P=NP$ (reduction from longest path); (ii) PTAS for ThOP on DAGs via DP with weight/profit rounding + exact enumeration of the constant-many highest-profit items and exact (unrounded) weights for travel-time; (iii) extensions to outerplanar/series-parallel graphs; (iv) FPTAS for constant-speed ($V_- = V_+$) special cases		
Competition	N	Code	No
Budget / Runs	— / —	Hardware	—

Abstract. Bloch-Hansen et al. (2023) provide the first **complexity and approximability results** for ThOP. A reduction from the longest path problem establishes that ThOP on general undirected graphs admits **no constant-factor approximation algorithm** unless $P = NP$. Despite this hardness, a **Polynomial-Time Approximation Scheme (PTAS)** is designed for ThOP on **Directed Acyclic Graphs (DAGs)**, based on a dynamic programming formulation with careful rounding of item weights and profits. Two core challenges are overcome: (1) rounding small high-profit item weights can degrade solution quality substantially—solved by explicitly enumerating a constant number of items with largest profit; (2) rounding item weights causes compounded errors in travel time—solved by using exact (unrounded) weights when computing travel time during DP transitions. The PTAS is further extended to outerplanar and series-parallel graphs, and **FPTAS** variants are derived for special constant-speed cases. This is the first theoretical approximation result for ThOP.

Contribution	The first approximation-theory results for ThOP: proves there is no constant-factor approximation for ThOP unless $P=NP$ (reduction from longest path), even for bounded-degree graphs with unit edges and one unit item per vertex; and gives a polynomial-time approximation scheme (PTAS) for ThOP when the input graph is a directed acyclic graph, via exact dynamic programming with careful rounding (round profits, enumerate a constant number of highest-profit items, use actual item weights for travel-time computation).
Authors' claims	ThOP has no constant-factor approximation and no $2^{O(\log^{1-\epsilon} n)}$ -approximation under standard complexity assumptions, even for trivial instance structure; there is a PTAS for ThOP on DAGs, and it adapts to a restricted undirected case.
Evidence base	Pure theory (IWOCA 2023, LNCS, 12 pages). Proofs of the inapproximability theorem and corollary, an exact DP algorithm with correctness proof, the PTAS construction and its analysis. No implementation, no experiments.
Stated limitations	The PTAS is for DAGs; an adaptation is given only for a restricted undirected case.

Strengths. ✓ The first hardness result and the first approximation scheme for ThOP • ✓ Clean handling of the rounding challenge (actual weights for travel time, rounded profits, enumerate the top items) • ✓ The inapproximability result is strong: it holds even for bounded-degree graphs with unit edges and unit items • ✓ Rigorous proofs; connects ThOP to the longest-path and knapsack literature

Weaknesses. ✗ The PTAS is only for directed acyclic graphs; the standard TTP-derived ThOP benchmark instances are on general/complete graphs, so this has no direct bearing on the santos2018 benchmark • ✗ No implementation or experiments • ✗ It is a PTAS, not an FPTAS; the dynamic-programming table, while polynomial, has a large exponent • ✗ The undirected adaptation is explicitly 'restricted'

[6] Huynh et al. (2023) — Self-Adaptive Ant System with Hierarchical Clustering for the Thief Orienteering Problem

Variant	ThOP	Objective / #obj	Single-objective / 1
Route	Selective routing (subset of cities)	Timing	Global time budget
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear/constant / time-budget
Algorithm	SAAS-HC	Family (L1/L2)	Population-based metaheuristic / Swarm hybrid (ACO+SA)
Benchmark	—	Size	Cities: 51; 107; 280; 1000 Items: 1; 3; 5; 10 per city
Metric	approximation ratio; pairwise better-or-equal %; mean error (best/worst/avg vs best-known);	Stat. test	Wilcoxon signed-rank, 5%
Baselines	ILS, BRKGA, ACO, ACO++ chagas2022efficiently;		
Key result	SAAS-HC better-or-equal vs ILS 99.77%, vs BRKGA 97.92%, vs ACO 98.61%, vs ACO++ 78.24%; vs ACO++ by Wilcoxon: better on 176, worse 170, tie 86 (roughly even, but) SAAS-HC finds best-known on 330 instances vs ACO++'s 180, using a single parameter set (vs ACO++'s 48 group-tuned sets); mean error best 0.14%, avg 1.25%;		

Competition	N	Code	No
Budget / Runs	0.1ms/run; CMA-ES self-tuning, 45,000 experiments on 21 instances / 30	Hardware	Intel Core i7-8750H 2.20GHz

Abstract. Huynh et al. (2023) propose **SAAS-HC** (Self-Adaptive Ant System with Hierarchical Clustering), an improved variant of ACO++ that eliminates the need for expensive per-instance parameter tuning. Two adaptive parameter control mechanisms are introduced: **self-adaptation** using evolution strategies adjusts pheromone-related parameters (evaporation rate and pheromone bounds) automatically during runtime based on solution quality feedback; **adaptation** using profit diversity statistics from the current ant population adjusts other algorithm parameters. Two efficiency techniques reduce computational overhead: **hierarchical clustering** groups geographically proximate cities into clusters, dramatically reducing the number of city transitions ants must evaluate; **lazy evaporation** defers pheromone trail updates to only modified entries, cutting per-iteration update time. A single parameter configuration is used across all 432 benchmark instances. SAAS-HC achieves superior results compared to ACO++ and all previous ThOP algorithms, demonstrating that dynamic self-adaptation with structural efficiency improvements can surpass the performance of extensively tuned configurations.

Contribution	SAAS-HC (Self-Adaptive Ant System with Hierarchical Clustering) for the ThOP, an improvement over ACO++ that removes the per-group tuning burden: CMA-ES self-adaptation of 6 ACO parameters during the run, profit-diversity-driven adaptation of ant count and evaporation rate, hierarchical-clustering cluster trees for $O(\log n)$ next-city selection, and lazy pheromone evaporation; one parameter configuration for all 432 instances.
Authors' claims	SAAS-HC produces superior results versus ILS, BRKGA, ACO and ACO++ using a single parameter set; better-or-equal versus ACO++ on 78.24 percent of instances; SAAS-HC finds the best-known on 330 instances vs ACO++'s 180; mean error 0.14 percent (best), 1.25 percent (average). The Wilcoxon signed-rank test shows SAAS-HC is statistically WORSE than ACO++ on 170 instances, no different on 86, and BETTER on 176.
Evidence base	432-instance benchmark, $\text{ceil}(m/10)$ -second budget, 30 runs. ALL methods (ILS, BRKGA, ACO, ACO++, SAAS-HC) re-run on the SAME machine (i7-8750H) - a genuine like-for-like comparison. GA excluded (source unavailable). SAAS-HC tuned with CMA-ES (45,000 experiments) into one config for all 432 instances (vs ACO++'s 240,000 experiments / 48 configs). Wilcoxon signed-rank test. Approximation ratio vs self-referential best-known. Public code. Parameter-sensitivity study showing ACO++ is sensitive to its parameter sets.
Stated limitations	Future work is time windows, multiple depots and multiple objectives; the paper acknowledges SAAS-HC vs ACO++ is close (statistically worse on 170, better on 176).

Strengths. ✓ The strongest experimental protocol in the ThOP line: all methods re-run on the SAME machine, 30 runs, Wilcoxon signed-rank test, correcting all the hardware confounds of the earlier ThOP papers • ✓ Removes the per-group tuning burden of ACO++: one CMA-ES-tuned config for all 432 instances, with a parameter-sensitivity study justifying why this matters • ✓ Multiple genuine technical contributions (CMA-ES self-adaptation, profit-diversity adaptation, hierarchical-clustering cluster trees, lazy evaporation) • ✓ Public source code; peer-reviewed • ✓ Honest: SAAS-HC vs ACO++ is close (better on 176, worse on 170, no difference on 86); the real advantage is best-known count (330 vs 180) and no tuning burden • ✓ Establishes SAAS-HC as the current ThOP state of the art

Weaknesses. ✗ Against ACO++, SAAS-HC is statistically WORSE on 170 of 432 instances and better on only 176 - essentially a tie on strict pairwise significance; the selling point is ‘no per-group tuning’ and more best-knowns, not clearly higher quality • ✗ The quality anchor is still the self-referential best-known (approximation ratio); no true optima for the full ThOP • ✗ Same 432-instance benchmark, 4 TSP bases (max 1000 cities); no new or larger instances • ✗ The method is complex (CMA-ES + hierarchical clustering + lazy evaporation + 2 adaptation mechanisms + MMAS); no ablation isolating each component’s contribution • ✗ SAAS-HC still needs a one-time CMA-ES tuning (45,000 experiments); it reduces but does not eliminate tuning • ✗ GA excluded from the comparison; 8-page regional-venue paper; the abstract’s ‘superior results’ oversells what the Wilcoxon numbers show (a near-tie with ACO++)

[7] Bloch-Hansen and Solis-Oba (2024) — *The Thief Orienteering Problem on Series-Parallel Graphs*

Variant	ThOP	Objective / #obj	Single-objective / 1
Route	Selective routing (subset of cities)	Timing	Global time budget
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear/constant / time-budget
Algorithm	series-parallel theory	Family (L1/L2)	Approximation scheme / Approximation theory
Benchmark	—	Size	—
Metric	—	Stat. test	—
Competition	N	Code	No
Budget / Runs	— / —	Hardware	—

Abstract. Bloch-Hansen and Solis-Oba (2024) extend the DAG-based PTAS of Bloch-Hansen et al. (2023) to ThOP on **series-parallel graphs**, a graph class with applications in production scheduling, VLSI layout, electrical circuit analysis, and system reliability design. The core contribution is a **polynomial-time transformation algorithm** that converts any ThOP instance defined on a series-parallel graph into an equivalent ThOP instance on a DAG, after which the existing DAG PTAS can be applied directly. The main difficulty is preserving all simple s -to- t paths during the transformation without introducing cycles or an exponential blowup in graph size. This is resolved by creating carefully constructed copies of vertices and directed edges exploiting the inductive definition of series-parallel graphs. The authors prove correctness of the transformation (path equivalence) and show the resulting DAG has polynomial size in the input. Motivating applications to **production optimization** (manufacturing stage scheduling) and **series-parallel system reliability optimization** are discussed, grounding the theoretical contribution in practical problem domains.

Contribution A polynomial-time algorithm that transforms a ThOP instance on a SERIES-PARALLEL graph into an equivalent ThOP instance on a directed acyclic graph (preserving all simple start-to-end paths, without cycles or exponential blow-up), thereby extending the DAG PTAS of bloch2023polynomial to series-parallel graphs. Motivated by manufacturing process plans and series-parallel reliability systems.

Authors' claims	The transformation preserves all simple paths and runs in polynomial time by exploiting the recursive structure of series-parallel graphs, yielding a PTAS for ThOP on series-parallel graphs.
Evidence base	Pure theory (Springer journal, 33 pages). The transformation algorithm with a detailed inductive case analysis (series and parallel compositions, terminal-vertex handling). No implementation, no experiments.
Stated limitations	Still restricted to series-parallel graphs (via the DAG reduction).
Strengths.	✓ Extends the ThOP PTAS to a new, application-motivated graph class • ✓ The transformation is non-trivial: preserving all simple paths without exponential blow-up or cycles requires exploiting the series-parallel structure • ✓ Journal-length rigorous treatment
Weaknesses.	✗ Still not general graphs; series-parallel graphs are a narrow class and the TTP-derived ThOP benchmark is not series-parallel • ✗ No implementation or experiments • ✗ 33 pages of intricate case analysis for a mostly technical reduction • ✗ The practical motivation (manufacturing, reliability) is asserted, not demonstrated with any instance

[8] Bloch-Hansen et al. (2025) — *Algorithms for the thief orienteering problem on directed acyclic graphs*

Variant	ThOP	Objective / #obj	Single-objective / 1
Route	Selective routing (subset of cities)	Timing	Global time budget
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear/constant / time-budget
Algorithm	FPTAS (ThOP)	Family (L1/L2)	Approximation scheme / FPTAS
Benchmark	—	Size	—
Metric	—	Stat. test	—
Key result	(i) no polynomial-time constant-factor approximation on general graphs unless P=NP; (ii) bicriteria PTAS for ThOP on DAGs - profit (1-)OPT using travel time $T(1+)$; (iii) FPTAS for undirected graphs when $V_- = V_+$, integer edge lengths, $T=L+K$ (constant K); (iv) FPTAS for clique graphs when $V_- = V_+$ (generalizes multi-group knapsack)		
Competition	N	Code	No
Budget / Runs	— / —	Hardware	—

Abstract. Bloch-Hansen et al. (2025) present a comprehensive journal-version study of approximation algorithms for ThOP on DAGs, extending and unifying earlier conference contributions. The paper first strengthens the inapproximability result: ThOP on general graphs admits no polynomial-time constant-factor approximation unless $P = NP$. The main result is a **PTAS for a relaxed version of ThOP on DAGs**: the algorithm finds a solution whose profit is within a factor of $(1 - \epsilon)$ of optimal while using travel time at most $T(1 + \epsilon)$, for any constant $\epsilon > 0$. The DP-based algorithm rounds item weights and profits while enumerating the highest-profit items exactly to control rounding error, using unrounded weights for travel time computation. The paper additionally derives: (1) an **FPTAS for ThOP on undirected graphs** when $V_{\min} = V_{\max}$ (speed is independent of load), edge lengths are integer, and $T = L + K$ for a constant K ; (2) an **FPTAS for ThOP on clique graphs** when $V_{\min} = V_{\max}$, a generalization of the multi-group knapsack. These results complete the complexity and approximability landscape for ThOP on DAGs and related graph classes, bridging theoretical foundations with the growing body of heuristic results.

Contribution	A consolidated theory reference for ThOP: the inapproximability result (no constant-factor approximation unless $P=NP$); a PTAS for a RELAXED ThOP on DAGs that produces solutions using time at most $T(1+\epsilon)$; an FPTAS for ThOP on arbitrary undirected graphs where travel time depends only on edge lengths and T equals a shortest-path length plus a constant; and an FPTAS (an $O(n \cdot I \cdot I ^2)$ exact algorithm) for ThOP on a clique with unit edges and load-independent speed, via a clique profit table with dominance pruning.
Authors' claims	ThOP has no constant-factor approximation unless $P=NP$; there is a resource-augmentation PTAS on DAGs (time budget relaxed to $T(1+\epsilon)$); there are FPTAS for the two restricted load-independent cases (undirected with a near-shortest-path budget, and cliques).
Evidence base	Pure theory (Theoretical Computer Science, 2025, open access). Proofs of the hardness result, the relaxed-PTAS construction and analysis, and the two FPTAS with their profit-table algorithms and dominance arguments. No implementation, no experiments.
Stated limitations	The DAG PTAS is for a relaxed time constraint; the FPTAS results require load-independent travel time.

Strengths. ✓ The definitive theory reference for ThOP: collects the hardness result, a resource-augmentation PTAS on DAGs, and two FPTAS for restricted cases • ✓ Published in a top theory venue (Theoretical Computer Science); open access • ✓ The clique FPTAS is a clean exact $O(n \cdot |I| \cdot |I|^2)$ algorithm • ✓ Rigorous proofs throughout

Weaknesses. ✗ The PTAS on DAGs is now for a RELAXED problem (time budget $T(1+\epsilon)$, i.e. the hard time constraint is violated by an ϵ factor); strict-time ThOP on DAGs is not solved by a PTAS here • ✗ The FPTAS results require LOAD-INDEPENDENT travel time ($v_{\min}=v_{\max}$), which removes the defining feature of ThOP; they are FPTAS for degenerate special cases, not the real ThOP • ✗ No implementation or experiments • ✗ The practical ThOP benchmark (general graphs, load-dependent speed) has no approximation guarantee from this work

Table 6 — Literature Review: TTP with Time Windows (TTPTW) (1 paper)

[1] Angmalisang and Neumann (2026) — <i>The Traveling Thief Problem with Time Windows: Benchmarks and Heuristics</i>			
Variant	TTPTW	Objective / #obj	Single-objective / 1
Route	Full tour	Timing	Per-city time windows + mandatory wait
Agents	Single thief	Capacity	single
Uncertainty	Deterministic (none)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	DSEA	Family (L1/L2)	Population-based metaheuristic / EA (DSEA)
Benchmark	—	Size	Cities: 51 to 33810 Items: 1; 3; 5 per city
Metric	objective value (mean OB, std OB) and feasible rate (FR %);	Stat. test	Kruskal-Wallis + Dunn-Bonferroni post-hoc
Baselines	S4, S5, C5 (with new tour initialization), LKH-3, VSR-LKH-3; DSEA variants DSEA_1/DSEA_2/DSEA_3 (no-repair / Repack / repair+optimizer);		
Key result	DSEA_1 best (avg rank 1.42 vs DSEA_3 2.00, DSEA_2 2.33); window-aware tour initialization lets S4/S5/C5 reach feasible solutions (vs near-zero FR without it); DSEA dominates S4/S5/C5/LKH-3/VSR-LKH-3 in feasibility and objective on almost all instances;		
Competition	N	Code	Unknown
Budget / Runs	1,000,000 function evaluations/run; DSEA =0.5, =1000, =10, =5, M=10000, h=10; PackIterative c=5, =2.5, q=20 / 30	Hardware	Ubuntu, Intel Core i7-1255U 1.70GHz (10 cores), 16GB RAM, Python

Abstract. Angmalisang and Neumann (2026) introduce the **Traveling Thief Problem with Time Windows (TTPTW)**, the first systematic study of the classical TTP augmented with **time window constraints**. In TTPTW, each city i is associated with a time window $[e_i, l_i]$: if the thief arrives before window e_i he must wait until it opens (increasing knapsack rental time and hence cost), and if he arrives after l_i the city is infeasible. This fundamentally alters the TTP objective, since total travel time now includes mandatory waiting periods, and the feasible region is drastically narrower than in the unconstrained TTP. The paper makes three major contributions. First, new **benchmark instances** are generated from standard TSPLIB-based TTP instances (51 to 1000 cities) by superimposing time windows at multiple tightness levels (controlled by a window-width parameter l), enabling controlled evaluation of time-window sensitivity. Second, a novel **tour initialization algorithm** that orders cities by window midpoint is proposed and shown to substantially reduce infeasibility rates; this algorithm is retrofitted into existing TTP heuristics S4, S5, and C5 (CLK + PackIterative) as well as into the TSPTW solvers **LKH-3** and **VSR-LKH-3** (which adds reinforcement-learning-guided (Q-learning, Sarsa, Monte Carlo) variable strategy selection to LKH). Third, the **Dual Search Evolutionary Algorithm (DSEA)** is proposed, simultaneously maintaining a tour-search thread and a packing-plan-search thread with modified operators and two packing plan repair strategies: a *weight-based repair* that removes heavy items and a *profit-based repair* that removes low-profit items when constraints are violated. Experiments across the full benchmark show that DSEA outperforms all adapted baselines, establishing that co-evolutionary, window-aware joint optimization is required for TTPTW.

Contribution	Introduces the TTP with Time Windows (TTPTW): TTP1 plus per-city time windows with mandatory waiting if the thief arrives early, feasible only if arrival is within the window, with the objective penalising total elapsed time including waiting; a constraint-objective separation (Deb 2000). Provides a new benchmark (20 TTP instances plus time windows at several tightness levels, and two instance types: Type A with a known basic-TTP optimum, Type B realistic) and DSEA, a Dual Search Evolutionary Algorithm with a feasibility-aware tour initialisation, two search operators (perturbed 2-opt, random insertion) and packing-plan repair schemes; three DSEA variants.
Authors' claims	DSEA1 (no packing-plan repair) is the best DSEA variant (average rank 1.42 vs 2.33 and 2.00), because the repair wastes function evaluations and DSEA1 explores more tours; the proposed tour initialisation significantly helps S4/S5/C5 find feasible solutions; DSEA1 outperforms adapted S4/S5/C5, LKH-3 and VSR-LKH-3 on TTPTW (better or equal on almost all instances), mainly on feasibility rate; the algorithms do not become increasingly challenged as time windows shrink or city count grows.
Evidence base	20 TTP instances (6 TSP bases, max 1000 cities), each with several time-window tightness levels and Type A/B. Python (DSEA), Java (S4/S5/C5), C (LKH-3, VSR-LKH-3). 30 runs, 1,000,000 function evaluations stopping (not CPU time, for the cross-language comparison). Kruskal-Wallis significance test + Dunn-Bonferroni post-hoc. Baselines re-adapted for TTPTW and re-run. Type A instances have a known basic-TTP optimum (windows built from the optimal TSP route). Published at PPSN 2026 (pp. 600-615).
Stated limitations	DSEA3 (integrated repair) is good only on small bounded-strongly-correlated instances; some instances at the tightest window level have no feasible solution even with tour init; future work is the new problem and benchmark itself.

Strengths. ✓ Introduces TTPTW with a careful formal treatment: mandatory waiting, constraint-objective separation, a new tunable-tightness benchmark, and two instance types • ✓ Type A instances have a KNOWN basic-TTP optimum, giving a genuine reference point, rare for a new variant • ✓ Function-evaluation-based stopping rather than CPU time, the right choice for a cross-language comparison (Python vs Java vs C) • ✓ Proper statistics: 30 runs, Kruskal-Wallis + Dunn-Bonferroni post-hoc, per-instance significance markings • ✓ Compares against adapted S4/S5/C5 AND two strong TSPTW solvers (LKH-3, VSR-LKH-3), all re-adapted and re-run • ✓ Clean ablation: 3 DSEA variants, 2 search operators, with/without tour initialisation; honest, nuanced findings

Weaknesses. ✗ The headline result is that DSEA1 (the variant WITHOUT the packing-plan repair, one of the paper's own contributions) is the best, so two of the three proposed repair mechanisms are shown to hurt • ✗ 20 TTP instances (6 TSP bases, max 1000 cities); mostly bounded-strongly-correlated, with uncorrelated/similar-weights only at a few points • ✗ DSEA1's advantage is largely about feasibility rate (finding any feasible solution), not objective quality; on objective the margins are often small • ✗ Only Type A instances have a known reference optimum, and even there the comparison reported is DSEA vs baselines, not DSEA vs the known optimum • ✗ Single lab (Adelaide); the only TTPTW paper • ✗ Python implementation for DSEA; even with FE-based stopping, the per-FE cost differs and practical runtime is not comparable • ✗ Many DSEA parameters from preliminary experiments, not systematic tuning; no exact/MIP method for TTPTW even on tiny instances

Table 7 — Literature Review: Chance Constrained TTP (CCTTP) (2 papers)

[1] Pathirage Don et al. (2024) — <i>The Chance Constrained Travelling Thief Problem: Problem Formulations and Algorithms</i>			
Variant	CCTTP	Objective / #obj	Single-objective / 1
Route	Full tour	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Stochastic weights (chance constraint) (weights)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	S1/S5/C1/C5 + surrogate	Family (L1/L2)	Constructive/iterative heuristic / Chance-constrained adaptation of S1/S5/C1/C5
Benchmark	—	Size	Cities: 51 to 1000 Items: 1 per city
Metric	mean objective score over 30 runs (+ std).	Stat. test	Kruskal-Wallis + Dunn with Bonferroni correction
Baselines	four algorithmic arrangements S1/S5/C1/C5 (CLK tour + Pack_SF/Pack_S), each tested under both the surrogate-based (Chebyshev/Hoeffding bound) and sampling-based (1000 weight samples per item) models; uncertainty levels 0.9,0.99,0.999, 20,50 fixed and [20,50] variable.		
Key result	S5 statistically significantly better than S1 (more stable, std 0 on small bsc/unc); C5 statistically similar to C1 on the surrogate model; objective degrades as tightens and grows; sampling-based model less stable (higher std) than surrogate.		
Competition	N	Code	No
Budget / Runs	20 minutes per run / 30 independent	Hardware	Java JDK 1.8.0, Phoenix HPC, University of Adelaide

Abstract. Pathirage Don et al. (2024) introduce the **Chance Constrained Travelling Thief Problem (CCTTP)**, the first stochastic extension of the TTP in which item **weights are uncertain random variables** rather than fixed deterministic values. This reflects real-world settings where item weights may vary due to measurement imprecision, environmental conditions, or perishability. The classical feasibility constraint—that total knapsack weight $\sum w_i y_i \leq W$ —is replaced by a **chance constraint**: $\Pr[\sum \tilde{w}_i y_i \leq W] \geq \alpha$, where $\tilde{w}_i$ are random variables and $\alpha \geq 0.8$ is a prescribed confidence level. Two constraint evaluation methods are developed: (1) a **surrogate-based approach** exploiting the Central Limit Theorem to compute the probability analytically under the assumption that summed stochastic weights are approximately normally distributed, enabling fast closed-form evaluation without sampling; (2) a **sampling-based approach** that estimates the probability empirically by drawing multiple realizations of item weights through Monte Carlo simulation, trading computational cost for accuracy. Both methods are incorporated as feasibility oracles into high-performing deterministic TTP algorithms including S5, C5, and evolutionary algorithm variants. Experiments on standard TTP benchmark instances (51 to 1000 cities) with stochastic weights at varying uncertainty levels compare the two methods, finding that the surrogate approach provides a practical efficiency–accuracy balance while the sampling approach achieves higher fidelity at greater cost. This work establishes CCTTP as a principled and practically motivated stochastic TTP variant, providing the first unified framework addressing both the multi-component *interdependence* and *uncertainty* aspects of combinatorial packing-and-routing problems.

Contribution	Introduces the Chance-Constrained TTP (CCTTP): TTP1 with stochastic item weights (uniform random variables) and a chance constraint $\Pr[\text{total weight} \leq B] \geq \alpha$ ($\alpha \geq 0.9$). Two models: a surrogate-based model that replaces the stochastic constraint with a deterministic surrogate via the one-sided Chebyshev inequality ($\alpha=0.9$) or the Hoeffding bound (α in 0.99, 0.999); and a sampling-based model using $K=1000$ Monte-Carlo weight samples with an empirical-feasibility constraint. Adapts the deterministic S1/S5/C1/C5 heuristics to both models via a modified packing routine that checks the surrogate/sampling constraint.
Authors' claims	S5 is statistically significantly better than S1 (Kruskal-Wallis + Dunn-Bonferroni), with lower (often near-zero) standard deviation, especially on smaller instances; C5 is similarly better than C1; S5 and C5 have statistically similar performance, so focused packing-plan optimisation (S5) is more effective in computational complexity than optimising tour and packing together (C5); higher uncertainty and tighter confidence lower the mean objective; the sampling-based model gives a less stable outcome than the surrogate model.
Evidence base	10 TTP instances (5 city sizes 51-1000, 1 item/city, 2 KP types uncorrelated + bounded-strongly-correlated). Surrogate: delta in 20, 50, alpha in 0.9, 0.99, 0.999. Sampling: $K=1000$ samples. 20-min budget, 30 runs. Kruskal-Wallis significance test + Dunn-Bonferroni post-hoc, tables and error plots. Java. Baselines S1/S5/C1/C5 all adapted here; no external CCTTP baseline and no comparison with a deterministic-TTP solution or a MIP. Peer-reviewed (GECCO 2024).
Stated limitations	Authors note the sampling-based model is less stable and does not converge in the time frame, that the chance constraint is the only modification to the surrogate model versus deterministic TTP, and frame future work as understanding uncertainty impact and robustness.

Strengths. ✓ Introduces CCTTP with two principled, well-grounded models (surrogate via Chebyshev/Hoeffding tail bounds; sampling via Monte Carlo), connecting to the chance-constrained knapsack / submodular-function literature • ✓ Adapts the established S1/S5/C1/C5 heuristics cleanly (only the packing routine's constraint check changes) • ✓ Proper statistics: 30 runs, Kruskal-Wallis + Dunn-Bonferroni post-hoc, per-instance significance markings, error plots • ✓ Systematic sweep of the uncertainty level and confidence level • ✓ Uses the standard TTP benchmark instances; 20-min budget; Java • ✓ Honest that the sampling model is less stable and that the surrogate model differs from deterministic TTP only in the constraint

Weaknesses. ✗ Only 10 instances (5 city sizes, 1 item/city, 2 KP types); a very limited benchmark for a new variant • ✗ No comparison with a deterministic-TTP solution's feasibility under uncertainty, or with a MIP / exact chance-constrained solve on tiny instances, so solution quality is not situated against any optimum • ✗ The method is essentially 'run S1/S5/C1/C5 with a different constraint check'; the algorithmic novelty is modest • ✗ The uniform-weight assumption is a strong, unexamined modelling choice • ✗ S5 'wins' mainly by having near-zero standard deviation (it converges to the same restart solution), so the result is partly about stability, not peak quality • ✗ K=1000 is a fixed choice; sensitivity to K is not studied; only weights are stochastic (profits deterministic), a narrow uncertainty model

[2] Pathirage Don et al. (2025b) — *Weighted-Scenario Optimisation for the Chance Constrained Travelling Thief Problem*

Variant	CCTTP	Objective / #obj	Single-objective / 1
Route	Full tour	Timing	Unconstrained
Agents	Single thief	Capacity	single
Uncertainty	Stochastic weights (chance constraint) (weights)	Dynamics	Static
Value model	Fixed profits	Speed / Cost	linear / time-renting
Algorithm	Weighted-scenario CCTTP	Family (L1/L2)	Single-solution metaheuristic / Chance-constrained scenario heuristic
Benchmark	—	Size	Cities: 51 to 1000 Items: 1 per city
Metric	mean expected objective score over 30 runs (+ std).	Stat. test	Kruskal-Wallis + Dunn with Bonferroni correction
Baselines	three scenario-adapted algorithms (1+1)EA_ws, S5_ws, C5_ws; thresholds 0.8,0.9.		
Key result	S5_ws/C5_ws heuristics statistically beat (1+1)EA_ws on small/medium instances; (1+1)EA_ws slightly better on bsc1000/unc1000 (more iterations fit in budget); C5_ws equal-or-narrowly-better than S5_ws; Set~C > Set~A > Set~B on expected objective.		
Competition	N	Code	No
Budget / Runs	10 minutes per run / 30 independent	Hardware	Java JDK 1.8.0, Phoenix HPC, University of Adelaide

Abstract. Pathirage Don et al. (2025b) propose a **weighted-scenario optimisation** framework as a tractable approach to solving the CCTTP of Pathirage Don et al. (2024). Rather than using parametric distributional assumptions or expensive Monte Carlo sampling, the stochastic item weights are represented by a **finite set of k discrete weighted scenarios** $\{(\mathbf{w}^{(1)}, p_1), \dots, (\mathbf{w}^{(k)}, p_k)\}$, where each scenario j provides a complete weight profile for all items and p_j is its probability of occurrence. The optimization objective becomes maximizing the **expected fitness**: $\sum_{j=1}^k p_j \cdot z(x, y; \mathbf{w}^{(j)})$, where z is the standard TTP objective evaluated under scenario j 's weights, subject to the chance constraint that a sufficient proportion of scenarios satisfy the knapsack capacity. Well-known TTP evolutionary and heuristic algorithms (S5, C5, co-evolutionary EA) are **adapted to the scenario-based model**: fitness evaluation is replaced by the weighted average across scenarios, and constraint satisfaction is assessed as the weighted proportion of feasible scenarios. A suite of CCTTP instances is derived from standard TTP benchmarks (51 to 1000 cities, one item per city) using $k = 5$ distinct scenarios per instance and three **scenario sets** that differ in how the probability of occurrence P_T is assigned across scenarios: Set A (near-uniform probabilities), Set B (higher-weight scenarios more probable), and Set C (lower-weight scenarios more probable). The analysis finds that scenario-adapted heuristics substantially outperform direct application of deterministic TTP solvers, that Set B yields the lowest expected objective (higher chance of constraint violation) while Set C gives the best, and that the $(1 + 1)$ EA only overtakes the heuristics on the largest instances. The framework provides an efficient and extensible methodology for stochastic TTP, bridging the gap between exact stochastic programming and practical heuristic search.

Contribution	A weighted-scenario model for the CCTTP: instead of surrogate bounds or many Monte-Carlo samples, use a limited number k of weighted scenarios, each a slightly perturbed weight profile with an occurrence probability; feasibility is the probability-weighted fraction of scenarios in which the capacity holds, and the objective is the expected TTP score over feasible scenarios. Uses $k=5$ scenarios and three scenario sets (A uniform, B heavier weights more likely, C lighter weights more likely). Adapts three algorithms: $(1+1)$ EA, S5 and C5.
Authors' claims	For smaller instances the two heuristic algorithms (S5, C5) beat the $(1+1)$ EA across all scenario sets; for larger instances the $(1+1)$ EA is slightly better (more iterations fit the budget); S5 performs better than the $(1+1)$ EA overall and C5 is equal or narrowly better than S5; Scenario Set B (heavier weights more likely) gives the lowest expected objectives and Set C the best; a tighter confidence level lowers objectives and raises variance.
Evidence base	The SAME 10 TTP instances as pathirage2024chance (5 bsc + 5 unc, 51-1000 cities, 1 item/city). $k=5$ scenarios, $\delta=20$, 3 scenario sets, α in 0.8, 0.9. 10-min budget, 30 runs. Kruskal-Wallis + Dunn-Bonferroni post-hoc; per-instance mean/std/significance tables. Baselines $(1+1)$ EA, S5, C5 all adapted here; NO comparison with the authors' own prior surrogate or sampling models. IEEE CEC 2025 / arXiv.
Stated limitations	None stated explicitly; the conclusion frames the weighted-scenario model as a relaxed strategy for stochastic multi-component problems with chance constraints.

Strengths. ✓ A third, distinct model for CCTTP (weighted scenarios) that is computationally lighter than $K=1000$ sampling and less conservative than surrogate bounds • ✓ The scenario-set idea (three occurrence-probability distributions) is a clean way to study how the uncertainty structure affects the solution • ✓ Proper statistics: 30 runs, Kruskal-Wallis + Dunn-Bonferroni, per-instance significance markings, two confidence levels • ✓ Uses the standard TTP instances; 10-min budget • ✓ Sensible, well-explained findings (Set B tighter $\rightarrow$ lower objective; heuristics win small, EA wins large due to iteration count)

Weaknesses. ✗ No comparison with the authors' own prior CCTTP models (surrogate, sampling) from pathirage2024chance, despite identical instances, authors and lab; so it is impossible to tell whether the weighted-scenario model produces better/worse solutions or feasibility • ✗ Only 10 instances (5 bsc + 5 unc, 1 item/city); the same limited benchmark as pathirage2024chance • ✗ $k=5$ scenarios is a fixed choice; no study of how k affects feasibility accuracy, and no validation that the 5-scenario feasibility estimate corresponds to the true chance constraint being met • ✗ The three algorithms are just deterministic S5/C5/(1+1) EA with a scenario-based feasibility check; minimal algorithmic novelty • ✗ The scenario occurrence probabilities are hand-assigned, not derived from a real distribution • ✗ No absolute-quality anchor (no optimum, no MIP, no comparison to the deterministic-TTP solution's scenario feasibility) • ✗ S5 'wins' partly by low variance (converges to the same restart solution), the same pattern as pathirage2024chance

References

- Alharbi, S. T. (2018a). The design and development of a modified artificial bee colony approach for the traveling thief problem. *International Journal of Applied Evolutionary Computation (IJAE)*, 9(3):32–47.
- Alharbi, S. T. (2018b). A hybrid genetic algorithm with tabu search for optimization of the traveling thief problem. *International Journal of Advanced Computer Science and Applications*, 9(11):276–287.
- Ali, F. and Mohamedkhair, M. (2020). Hyper-heuristic approaches for the travelling thief problem. In *2020 International Conference on Computer, Control, Electrical, and Electronics Engineering (ICCCEE)*, pages 1–6. IEEE.
- Ali, H., Rafique, M. Z., Sarfraz, M. S., Malik, M. S. A., Alqahtani, M. A., and Alqurni, J. S. (2021). A novel approach for solving travelling thief problem using enhanced simulated annealing. *PeerJ Computer Science*, 7:e377.
- Ali, I. M., Essam, D., and Kasmarik, K. (2019). A novel differential evolution mapping technique for generic combinatorial optimization problems. *Applied Soft Computing*, 80:297–309.
- Ali, I. M., Essam, D., and Kasmarik, K. (2020). Differential evolution algorithm for multiple inter-dependent components traveling thief problem. In *2020 IEEE Congress on Evolutionary Computation (CEC)*, pages 1–8. IEEE.
- Angmalisang, H. Y. and Neumann, F. (2026). The traveling thief problem with time windows: benchmarks and heuristics. In *International Conference on Parallel Problem Solving from Nature*, pages 600–615. Springer.
- Araujo, R. P., Rios, E., Coelho, I. M., Marzulo, L. A., and Castro, M. C. (2018). A novel list-constrained randomized vnd approach in gpu for the traveling thief problem. *Electronic Notes in Discrete Mathematics*, 66:183–190.
- Beham, A., Fechter, J., Kommenda, M., Wagner, S., Winkler, S. M., and Affenzeller, M. (2015). Optimization strategies for integrated knapsack and traveling salesman problems. In *International Conference on Computer Aided Systems Theory*, pages 359–366. Springer.
- Blank, J., Deb, K., and Mostaghim, S. (2017). Solving the bi-objective traveling thief problem with multi-objective evolutionary algorithms. In *International Conference on Evolutionary Multi-Criterion Optimization*, pages 46–60. Springer.
- Bloch-Hansen, A., Page, D. R., and Solis-Oba, R. (2023). A polynomial-time approximation scheme for thief orienteering on directed acyclic graphs. In *International Workshop on Combinatorial Algorithms*, pages 87–98. Springer.
- Bloch-Hansen, A. and Solis-Oba, R. (2024). The thief orienteering problem on series-parallel graphs. In *International Symposium on Combinatorial Optimization*, pages 248–262. Springer.
- Bloch-Hansen, A., Solis-Oba, R., and Page, D. R. (2025). Algorithms for the thief orienteering problem on directed acyclic graphs. *Theoretical Computer Science*, 1023:114900.
- Bonyadi, M. R., Michalewicz, Z., and Barone, L. (2013). The travelling thief problem: The first step in the transition from theoretical problems to realistic problems. In *2013 IEEE Congress on Evolutionary Computation*, pages 1037–1044. IEEE.

- Bonyadi, M. R., Michalewicz, Z., Przybylek, M. R., and Wierzbicki, A. (2014). Socially inspired algorithms for the travelling thief problem. In *Proceedings of the 2014 Annual Conference on Genetic and Evolutionary Computation*, pages 421–428.
- Bossek, J. and Wagner, M. (2021). Generating instances with performance differences for more than just two algorithms. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, pages 1423–1432.
- Chagas, J. B., Blank, J., Wagner, M., Souza, M. J., and Deb, K. (2021). A non-dominated sorting based customized random-key genetic algorithm for the bi-objective traveling thief problem. *Journal of Heuristics*, 27(3):267–301.
- Chagas, J. B. and Wagner, M. (2020). Ants can orienteer a thief in their robbery. *Operations Research Letters*, 48(6):708–714.
- Chagas, J. B. and Wagner, M. (2022a). Efficiently solving the thief orienteering problem with a max–min ant colony optimization approach. *Optimization Letters*, 16(8):2313–2331.
- Chagas, J. B. and Wagner, M. (2022b). A weighted-sum method for solving the bi-objective traveling thief problem. *Computers & Operations Research*, 138:105560.
- Chand, S. and Wagner, M. (2016). Fast heuristics for the multiple traveling thieves problem. In *Proceedings of the Genetic and Evolutionary Computation Conference 2016*, pages 293–300.
- El Yafrani, M. and Ahiod, B. (2015). Cosolver2b: an efficient local search heuristic for the travelling thief problem. In *2015 IEEE/ACS 12th International Conference of Computer Systems and Applications (AICCSA)*, pages 1–5. IEEE.
- El Yafrani, M. and Ahiod, B. (2016). Population-based vs. single-solution heuristics for the travelling thief problem. In *Proceedings of the Genetic and Evolutionary Computation Conference 2016*, pages 317–324.
- El Yafrani, M. and Ahiod, B. (2017). A local search based approach for solving the travelling thief problem: The pros and cons. *Applied Soft Computing*, 52:795–804.
- El Yafrani, M. and Ahiod, B. (2018). Efficiently solving the traveling thief problem using hill climbing and simulated annealing. *Information Sciences*, 432:231–244.
- El Yafrani, M., Chand, S., Neumann, A., Ahiod, B., and Wagner, M. (2017). Multi-objectiveness in the single-objective traveling thief problem. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, pages 107–108.
- El Yafrani, M., Martins, M., Wagner, M., Ahiod, B., Delgado, M., and Lüders, R. (2018). A hyperheuristic approach based on low-level heuristics for the travelling thief problem. *Genetic Programming and Evolvable Machines*, 19(1-2):121–150.
- Erdoğan, K. (2022). Comparing alns and nsga-ii on bi-objective travelling thief problem. In *Proceedings of the International Conference on Industrial Engineering and Operations Management*, pages 1426–1435, Istanbul, Turkey.
- Eube, J., Luo, K., Neumann, A., Neumann, F., and Röglin, H. (2026a). Effective traveling for metric instances of the traveling thief problem. In *International Conference on Parallel Problem Solving from Nature*, pages 282–297. Springer.

- Eube, J., Luo, K., Röglin, H., and Sturm, S. (2026b). Approximation algorithms for the traveling thief problem. In *Leibniz International Proceedings in Informatics (LIPIcs), Proceedings of the European Symposium on Algorithms (ESA)*, volume 388.
- Faêda, L. M. and Santos, A. G. (2020). A genetic algorithm for the thief orienteering problem. In *2020 IEEE Congress on Evolutionary Computation (CEC)*, pages 1–8. IEEE.
- Faulkner, H., Polyakovskiy, S., Schultz, T., and Wagner, M. (2015). Approximate approaches to the traveling thief problem. In *Proceedings of the 2015 Annual Conference on Genetic and Evolutionary Computation*, pages 385–392.
- Garbaruk, J., Logofătu, D., and Leon, F. (2022). Comparative study by using a greedy approach and advanced bio-inspired strategies in the context of the traveling thief problem. In *Artificial Intelligence Applications and Innovations: 18th IFIP WG 12.5 International Conference, AIAI 2022, Hersonissos, Crete, Greece, June 17–20, 2022, Proceedings, Part I*, pages 383–393. Springer.
- Gupta, B. C. and Prakash, V. P. (2015). Greedy heuristics for the travelling thief problem. In *2015 39th National Systems Conference (NSC)*, pages 1–5. IEEE.
- Herring, D., Kirley, M., and Yao, X. (2020a). Dynamic multi-objective optimization of the travelling thief problem. *arXiv preprint arXiv:2002.02636*.
- Herring, D., Kirley, M., and Yao, X. (2020b). Responsive multi-population models for the dynamic travelling thief problem. In *2020 IEEE Symposium Series on Computational Intelligence (SSCI)*, pages 297–304. IEEE.
- Huynh, V., Le, T. V., and Luong, N. H. (2023). Self-adaptive ant system with hierarchical clustering for the thief orienteering problem. In *Proceedings of the 12th International Symposium on Information and Communication Technology*, pages 723–730.
- Kapancioglu, T., Bernardino, R., and Mourão, M. C. (2026). Formulations and an exact algorithm for the traveling thief problem. *European Journal of Operational Research*.
- Karder, J., Beham, A., Wagner, S., and Affenzeller, M. (2017). Solving the traveling thief problem using orchestration in optimization networks. In *International Conference on Computer Aided Systems Theory*, pages 307–315. Springer.
- Kumari, R. and Srivastava, K. (2020). Variable neighbourhood search for bi-objective travelling thief problem. In *2020 8th International Conference on Reliability, Infocom Technologies and Optimization (Trends and Future Directions)(ICRITO)*, pages 47–51. IEEE.
- Kumari, R. and Srivastava, K. (2023). Nsgaii for travelling thief problem with dropping rate. In *2023 7th International Conference on Intelligent Computing and Control Systems (ICICCS)*, pages 1647–1653. IEEE.
- Laszczyk, M. and Myszkowski, P. B. (2019). A specialized evolutionary approach to the bi-objective travelling thief problem. In *2019 Federated Conference on Computer Science and Information Systems (FedCSIS)*, pages 47–56. IEEE.
- Lourenço, N., Pereira, F. B., and Costa, E. (2016). An evolutionary approach to the full optimization of the traveling thief problem. In *Evolutionary Computation in Combinatorial Optimization*, pages 34–45. Springer.

- Maity, A. and Das, S. (2020). Efficient hybrid local search heuristics for solving the travelling thief problem. *Applied Soft Computing*, page 106284.
- Martins, M. S., El Yafrani, M., Delgado, M. R., Wagner, M., Ahiod, B., and Lüders, R. (2017). Hseda: a heuristic selection approach based on estimation of distribution algorithm for the travelling thief problem. In *Proceedings of the Genetic and Evolutionary Computation Conference*, pages 361–368.
- Mei, Y., Li, X., Salim, F., and Yao, X. (2015). Heuristic evolution with genetic programming for traveling thief problem. In *2015 IEEE Congress on Evolutionary Computation (CEC)*, pages 2753–2760. IEEE.
- Mei, Y., Li, X., and Yao, X. (2014). Improving efficiency of heuristics for the large scale traveling thief problem. In *Asia-Pacific Conference on Simulated Evolution and Learning*, pages 631–643, Cham. Springer, Springer International Publishing.
- Mei, Y., Li, X., and Yao, X. (2016). On investigation of interdependence between sub-problems of the travelling thief problem. *Soft Computing*, 20(1):157–172.
- Misir, M. (2021). Benchmark set reduction for cheap empirical algorithmic studies. In *2021 IEEE Congress on Evolutionary Computation (CEC)*, pages 871–877. IEEE.
- Moeini, M., Schermer, D., and Wendt, O. (2017). A hybrid evolutionary approach for solving the traveling thief problem. In *International Conference on Computational Science and Its Applications*, pages 652–668. Springer.
- Murjani, K. and Sobhanan, A. (2026). Drive, pack, fly: The travelling thief problem with drone. *arXiv preprint arXiv:2608.16435*.
- Myszkowski, P. B. and Laszczyk, M. (2021). Diversity based selection for many-objective evolutionary optimisation problems with constraints. *Information Sciences*, 546:665–700.
- Namazi, M., Newton, M., Sattar, A., and Sanderson, C. (2019a). A profit guided coordination heuristic for travelling thief problems. In *Proceedings of the International Symposium on Combinatorial Search*, volume 10, pages 140–144.
- Namazi, M., Newton, M. H., Sanderson, C., and Sattar, A. (2023). Solving travelling thief problems using coordination based methods. *Journal of Heuristics*, 29(4):487–544.
- Namazi, M., Sanderson, C., Newton, M., and Sattar, A. (2019b). A cooperative coordination solver for travelling thief problems. *arXiv preprint arXiv:1911.03124*.
- Namazi, M., Sanderson, C., Newton, M., and Sattar, A. (2020). Surrogate assisted optimisation for travelling thief problems. In *Proceedings of the International Symposium on Combinatorial Search*, volume 11, pages 111–115.
- Naumov, V. and Pawluś, M. (2021). Identifying the optimal packing and routing to improve last-mile delivery using cargo bicycles. *Energies*, 14(14):4132.
- Neumann, F., Polyakovskiy, S., Skutella, M., Stougie, L., and Wu, J. (2019). A fully polynomial time approximation scheme for packing while traveling. In Disser, Y. and Verykios, V. S., editors, *Algorithmic Aspects of Cloud Computin*, pages 59–72. Springer International Publishing.

- Nguyen, H., Le, N., Tran, K., and Luong, N. H. (2023). Simulated annealing with dynamic programming-based vertex insertion for efficiently solving the traveling thief problem. In *Proceedings of the 12th International Symposium on Information and Communication Technology*, pages 165–172.
- Ni, A. and Spector, L. (2024). Leveraging symbolic regression for heuristic design in the traveling thief problem. *arXiv preprint arXiv:2404.12750*.
- Nieto-Fuentes, R., Segura, C., and Valdez, S. I. (2018). A guided local search approach for the travelling thief problem. In *2018 IEEE Congress on Evolutionary Computation (CEC)*, pages 1–8. IEEE.
- Nikfarjam, A., Neumann, A., Bossek, J., and Neumann, F. (2022a). Co-evolutionary diversity optimisation for the traveling thief problem. In *Parallel Problem Solving from Nature – PPSN XVII*, pages 237–249. Springer International Publishing.
- Nikfarjam, A., Neumann, A., and Neumann, F. (2022b). Evolutionary diversity optimisation for the traveling thief problem. In *Proceedings of the Genetic and Evolutionary Computation Conference*, pages 749–756.
- Nikfarjam, A., Neumann, A., and Neumann, F. (2024). On the use of quality diversity algorithms for the travelling thief problem. *ACM Transactions on Evolutionary Learning and Optimization*, 4(2):1–22.
- Pathirage Don, T., Neumann, A., and Neumann, F. (2024). The chance constrained travelling thief problem: Problem formulations and algorithms. In *Proceedings of the Genetic and Evolutionary Computation Conference*, pages 214–222.
- Pathirage Don, T., Neumann, A., and Neumann, F. (2025a). Evolutionary multitasking for the scenario-based travelling thief problem. In *Proceedings of the Genetic and Evolutionary Computation Conference*, pages 809–817.
- Pathirage Don, T., Neumann, A., and Neumann, F. (2025b). Weighted-scenario optimisation for the chance constrained travelling thief problem. In *IEEE Congress on Evolutionary Computation*, pages 1–8.
- Pathirage Don, T., Neumann, A., and Neumann, F. (2026). Greedy approaches for packing while travelling with deterministic and stochastic constraints. *arXiv preprint arXiv:2604.13469*.
- Polyakovskiy, S., Bonyadi, M. R., Wagner, M., Michalewicz, Z., and Neumann, F. (2014). A comprehensive benchmark set and heuristics for the traveling thief problem. In *Proceedings of the 2014 Annual Conference on Genetic and Evolutionary Computation*, pages 1–6.
- Polyakovskiy, S. and Neumann, F. (2015). Packing while traveling: Mixed integer programming for a class of nonlinear knapsack problems. In *International Conference on AI and OR Techniques in Constraint Programming for Combinatorial Optimization Problems*, pages 332–346. Springer.
- Polyakovskiy, S. and Neumann, F. (2017). The packing while traveling problem. *European Journal of Operational Research*, 258(2):424–439.
- Przybyłek, M. R., Wierzbicki, A., and Michalewicz, Z. (2018). Decomposition algorithms for a multi-hard problem. *Evolutionary computation*, 26(3):507–533.

- Rodríguez, D., Cruz-Duarte, J. M., Ortiz-Bayliss, J. C., and Amaya, I. (2023). A sequence-based hyper-heuristic for traveling thieves. *Applied Sciences*, 13(1):56.
- Sachdeva, R., Neumann, F., and Wagner, M. (2020). The dynamic travelling thief problem: Benchmarks and performance of evolutionary algorithms. In *International Conference on Neural Information Processing*, pages 220–228. Springer.
- Santana, R. and Shakya, S. (2020). Dynamic programming operators for the bi-objective traveling thief problem. In *2020 IEEE Congress on Evolutionary Computation (CEC)*, pages 1–8. IEEE.
- Santiyuda, G., Wardoyo, R., and Pulungan, R. (2024). Solving biobjective traveling thief problems with multiobjective reinforcement learning. *Applied Soft Computing*, 161:111751.
- Santos, A. G. and Chagas, J. B. (2018). The thief orienteering problem: Formulation and heuristic approaches. In *2018 IEEE Congress on Evolutionary Computation (CEC)*, pages 1–9. IEEE.
- Sarkar, T. and Rajendran, C. (2024). Travelling thief problem: a survey of recent variants, solution approaches and future directions. *International Journal of Systems Science: Operations & Logistics*, 11(1):2424200.
- Strzeżek, A., Trammer, L., and Sydow, M. (2015). Divergene: Experiments on controlling population diversity in genetic algorithm with a dispersion operator. In *2015 Federated Conference on Computer Science and Information Systems (FedCSIS)*, pages 155–162. IEEE.
- Tran, K. and Luong, N. H. (2026). Integrating ant colony optimization and simulated annealing with restart for the traveling thief problem. In *Proceedings of the 2026 11th International Conference on Intelligent Information Technology*, pages 357–361.
- Vieira, D. K., Soares, G. L., Vasconcelos, J. A., and Mendes, M. H. (2017). A genetic algorithm for multi-component optimization problems: the case of the travelling thief problem. In *European Conference on Evolutionary Computation in Combinatorial Optimization*, pages 18–29. Springer.
- Viet, N. H., Tung, N. X., Van Chien, T., and Hwang, W.-J. (2026). Advanced quantum annealing for the bi-objective traveling thief problem: An ϵ -constraint-based approach. *IEEE Transactions on Quantum Engineering*.
- Wagner, M. (2016). Stealing items more efficiently with ants: a swarm intelligence approach to the travelling thief problem. In *International Conference on Swarm Intelligence*, pages 273–281. Springer.
- Wagner, M., Lindauer, M., Mısı, M., Nallaperuma, S., and Hutter, F. (2018). A case study of algorithm selection for the traveling thief problem. *Journal of Heuristics*, 24(3):295–320.
- Wu, J., Polyakovskiy, S., and Neumann, F. (2016). On the impact of the renting rate for the unconstrained nonlinear knapsack problem. In *Proceedings of the Genetic and Evolutionary Computation Conference 2016*, pages 413–419.
- Wu, J., Polyakovskiy, S., Wagner, M., and Neumann, F. (2018). Evolutionary computation plus dynamic programming for the bi-objective travelling thief problem. In *Proceedings of the Genetic and Evolutionary Computation Conference*, pages 777–784.
- Wu, J., Wagner, M., Polyakovskiy, S., and Neumann, F. (2017). Exact approaches for the travelling thief problem. In *Asia-Pacific Conference on Simulated Evolution and Learning*, pages 110–121. Springer.

- Wu, T., Cui, H., Guan, T., Wang, Y., and Jin, Y. (2024). Reinforcens: reinforcement learning-based multi-start neighborhood search for solving the traveling thief problem. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*, pages 7038–7046.
- Xiang, Y., Guo, J., Jiang, C., Ma, H., and Liu, M. (2024a). Multi-objective five-element cycle optimization algorithm based on multi-strategy fusion for the bi-objective traveling thief problem. *Applied Sciences* (2076-3417), 14(17).
- Xiang, Y., Guo, J., Mao, Z., Jiang, C., and Liu, M. (2024b). Five-element cycle optimization algorithm based on an integrated mutation operator for the traveling thief problem. *Symmetry*, 16(9):1153.
- Yang, L., Jia, X., Xu, R., and Cao, J. (2021). An moea/d-aco algorithm with finite pheromone weights for bi-objective ttp. In *International Conference on Data Mining and Big Data*, pages 468–482. Springer.
- Zhang, Z., Xia, X.-Y., Wang, Z.-J., Jin, Y.-Z., Liao, W.-Z., and Zhang, J. (2025). A two-stage algorithm based on greedy ant colony optimization for travelling thief problem. *Complex & Intelligent Systems*, 11(5):1–15.
- Zhang, Z., Yang, L., Kang, P., Jia, X., and Zhang, W. (2021). Solving the traveling thief problem based on item selection weight and reverse-order allocation. *IEEE Access*, 9:54056–54066.
- Zouari, W., Alaya, I., and Tagina, M. (2019). A new hybrid ant colony algorithms for the traveling thief problem. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, pages 95–96.